M5Stack Tab5 QMX+ Shared Over Network

Sharing a QRP Labs QMX/QMX+ over the network with the M5Stack Tab5 QMX-Panadapter: Synology NAS relay, Raspberry Pi relay, Tab5 firmware and Mac tools

Item	Value
Firmware base	Upstream SteffenLav/qmx-panadapter **v1.18.0**, plus the NAS patch series 0001-0019 (reports itself as **v1.18.0-NAS**)
Folders (Mac)	~/src/QMX-Panadapter-NAS (newest upstream + NAS patches; build, flash and test here); ~/src/QMX-Panadapter-Shared (last version tested on the hardware); ~/src/qmx-panadapter-v1.16/tools/nas-kit (the patch series and the update kit)
Relay	tools/nas-relay/qmx_nas_relay.py (Python 3). Synology DSM: native, started by Task Scheduler. Raspberry Pi 4/5: the same file as a systemd service (tools/pi-relay)
Network	TCP 7355 QMXR/1 to the Tab5 (I/Q, CAT, Radio menus, relay-timed transmit); TCP 7356 QMX serial port 3 for PC software; UDP 7355 discovery (QMXR? / QMXR!)
Tested on	Synology DS416j, DSM 7.4.1, NAS at 10.0.0.137, with a QMX+ (firmware 1_03_002) and an M5Stack Tab5
Toolchain	ESP-IDF v5.4.4, target ESP32-P4; built and flashed from a Mac with tools/mac-build-flash.sh
Document date	October 14, 2026

Contents

1. Summary of outcome 3
2. Architecture 4
3. Why the data goes through a relay 4
4. QMXR/1 wire protocol 5
5. Tab5 firmware changes 6
6. Synology relay: installation, operation and maintenance 7
7. Raspberry Pi relay: installation, operation and maintenance 8
8. Mac tools and the update kit 8
9. FT8 burst timing comparisons 9
10. Verification and open items 10
11. Sources (references) 12
12. Driver listings (Tab5 firmware) 14
13. Application listings 44

1. Summary of outcome

The QMX-Panadapter firmware on the M5Stack Tab5 can use a QMX or QMX+ that is plugged into another machine on the network, a Synology NAS or a Raspberry Pi, instead of the Tab5's own USB port. A small relay program on that machine owns the radio over USB and serves it to the Tab5 over Wi-Fi. Everything the Tab5 does with a USB-connected radio works over the network: the full-rate spectrum and waterfall, FT8/FT4 and WSPR receive and transmit, CAT control, RX audio and the radio's own menus.

- **Synology relay**: runs natively on DSM 7 (no Container Manager or DSM drivers needed), started at boot by Task Scheduler and kept running by a supervisor script. Tested on a DS416j.
- **Raspberry Pi relay**: the same relay file run as a systemd service on a Pi 4/5, with a control command, a log and limited automatic restarts. Written and checked against the Pi's kernel interfaces; not yet run on a Pi.
- **Tab5 firmware**: a network-radio driver (QMXR/1 client), Radio source in the touchscreen drawer and on the web page, Find NAS discovery, and Wi-Fi receive fixes that let the Tab5 take the full 48 kHz I/Q stream.
- **Relay-timed transmit**: the Tab5 hands each FT8/FT4 burst to the relay in one message and the relay plays every tone to the radio on its own clock. In 10 of 10 test bursts every tone reached the radio within 2 ms of schedule (section 9).
- **Operations**: CAT log of every command and reply, a power-off (PS0;) guard, a transmit log for every burst, and flag files to change behaviour without a restart.
- **PC and Mac**: the QMX's third serial port is offered on TCP 7356 for PC software; a Mac serial bridge presents it as a serial device (written; connection still to be finished).
- **Keeping current**: an update kit rebuilds the NAS version from every new upstream release and only promotes it to the Shared folder after it has been tested on the hardware.

Change history (patch series)

#	Date	Change	Tested
0001	2026-10-04	Network radio: use a QMX plugged into a Synology NAS over TCP/IP	Hardware
0002	2026-10-04	nas-relay: serve QMX serial port 3 to PC software on TCP 7356	Hardware
0003	2026-10-04	nas-relay: run natively on DSM with a usbfs backend; tested on real hardware	Hardware
0004	2026-10-04	network-radio: full-rate NAS stream on the Tab5 - WiFi RX fixes, build on macOS	Hardware
0005	2026-10-04	network-radio: Radio source on the touchscreen; fix network freeze and DMA RAM drain	Hardware
0006	2026-10-04	webserver: raise the URI handler limit so the live spectrum socket registers	Hardware
0007	2026-10-04	nas-kit: rebuild the NAS version from each new upstream release; PDF QMX-Panadapter-Shared	Kit tests
0008	2026-10-08	cat: route upstream v1.16.12's GP/TM probes through cat_tx, so they work on a network radio	Hardware
0009	2026-10-08	network radio: Find NAS - the Tab5 asks the network for the relay's address	Hardware
0010	2026-10-08	mac-serial-bridge: the NAS-shared QMX as a serial port on a Mac	Bench only
0011	2026-10-08	pi-relay: run the QMX relay on a Raspberry Pi 4/5 as a systemd service	Not on a Pi yet
0012	2026-10-08	patches: apply_fatfs_dma_aligned_alloc.ps1 matches on macOS/Linux too	Build
0013	2026-10-09	README: what the network-shared version adds, and how far each part is tested	-
0014	2026-10-14	cat: route upstream v1.18.0's KS keyer-speed commands through cat_tx, so they work on a network radio	Hardware
0015	2026-10-14	tx-test: log each transmit burst on the relay and compare it with the Tab5 console	Hardware
0016	2026-10-14	web: 10-burst test button on the FT8 page	Hardware
0017	2026-10-14	nas-relay: CAT log of every command and reply; PS0 (power off) blocked unless flag file allow-ps0	Hardware
0018	2026-10-14	nas-relay: pause I/Q to the Tab5 from TX; to RX;	Hardware
0019	2026-10-14	FT8/FT4 over the NAS: the relay times the transmit burst itself	Hardware

Patches 0001-0013 were first written on upstream v1.16.11/v1.16.12 and ported to v1.18.0 on 2026-10-14; the ported series was built, flashed and tested on the Tab5 (Find NAS, connect by address, spectrum, reconnection).

2. Architecture

```
QMX / QMX+ --USB--> Synology NAS or Raspberry Pi 4/5   (qmx_nas_relay.py)
                    |
                    |  TCP 7355  QMXR/1: I/Q 48 kHz, CAT, Radio menus, transmit bursts
                    +----------------------------> M5Stack Tab5 QMX-Panadapter
                    |                                (Radio source: NAS)
                    |  TCP 7356  QMX serial port 3, raw bytes
                    +----------------------------> PC software / Mac serial bridge
                    |
                    +  UDP 7355  answers "QMXR?" with "QMXR! <ip> <port>" (Find NAS)
```

The QMX presents itself over USB as a USB Audio Class device (the I/Q stream: 48 kHz, stereo, 24-bit) plus up to three CDC-ACM serial ports (System config → GPS & Ser. ports → USB serial ports). The relay uses them as follows:

QMX port	Used for	Needs "USB serial ports"
Audio (UAC)	I/Q to the Tab5: converted to 16-bit exactly as the Tab5's own USB path does (s24 >> 8)	any
Serial 1	CAT for the Tab5: panadapter, FT8/FT4, WSPR, power/SWR	1 or more
Serial 2	The Tab5's Radio menus (the QMX's own menu terminal), never mixed with CAT	2 or 3
Serial 3	PC software over TCP 7356 (WSJT-X, rigctld, loggers, the Mac serial bridge)	3

Components

Component	Where it runs	What it does
qmx_nas_relay.py	Synology NAS (DSM 7) or Raspberry Pi	Owns the QMX through Linux usbfs (or libusb), serves QMXR/1 to the Tab5, port 3 to PCs, answers discovery, keeps the CAT and transmit logs, times transmit bursts
Tab5 firmware (v1.18.0-NAS)	M5Stack Tab5 (ESP32-P4)	QMXR/1 client (radio_relay.c) feeding the same audio ring and CAT parser as the USB drivers; Radio source UI; Wi-Fi receive fixes; relay-timed FT8/FT4 transmit
Synology scripts	NAS	start-relay.sh (supervisor), stop-relay.sh, check.sh (diagnostics, restart.request), env.sh, nettest.py, get_libusb1.py, Docker files
Pi scripts	Raspberry Pi	install-pi.sh / uninstall-pi.sh, qmx-relay-ctl, systemd service and timer, event logger, udev rule, logrotate
Mac tools	Mac	qmx_serial_bridge.py (port 3 as a serial device), mac-build-flash.sh, mac-capture-serial.sh
Test tools	Mac	tx-test/tx_compare.py, capture-tab5-console.sh; the web page's 10-burst test button
Update kit	Mac	make-qmx-panadapter-nas.sh (rebuild from a new upstream release), push-to-github.sh

An open TCP session means "radio present". When the relay loses the QMX (powered off or unplugged) it closes the session, and the Tab5 treats it exactly like a USB unplug: it retries every ~2 s and resumes on its own when the radio comes back. One Tab5 and one port-3 client can use the radio at the same time; a new connection of the same kind replaces a stale one.

3. Why the data goes through a relay

Why a relay at all

The QMX is a USB device: its I/Q audio and its serial ports exist only on the USB bus of the machine it is plugged into. To use it from anywhere else, something on that machine has to own the USB interfaces and carry their data over the network. The relay is that program, and it runs on the machine the radio is already plugged into (the NAS, or a Pi), so no extra computer sits in the path.

- **No drivers on the host.** DSM ships no USB audio driver, and the DS416j has no libusb. The relay talks to the QMX directly from userspace through Linux usbfs ioctls, so it needs only the Python 3 that DSM already has, and runs unchanged on a Pi.
- **Same data as USB.** The relay converts the I/Q exactly as the Tab5's USB driver does and the Tab5 pushes it into the same sample ring, and CAT bytes go to the same parser. FFT, waterfall, FT8 decoding and the UI

cannot tell the difference.

- **One connection, several streams.** I/Q, CAT and the menu terminal share one TCP connection, framed by QMXR/1 so each stream stays separate (the menu port can never disturb CAT).
- **Control ahead of bulk data.** The relay sends CAT replies before any queued I/Q, keeps only 3 I/Q frames (30 ms) queued, drops I/Q rather than build up delay, and keeps the kernel send buffer small, so a radio reply never waits behind seconds of spectrum data.
- **Shared access.** Because the relay owns the radio, it can serve the Tab5 and PC software at once, each on its own serial port, and can apply rules for everyone: the CAT log and the power-off guard.

Why the whole transmit burst is sent to the relay

An FT8 transmission is 79 tones, each held for exactly 160 ms. Over USB the Tab5 sends one tone-change command (TA) every 160 ms and the radio follows it within a millisecond or two. Over the network, each of those 79 commands had to cross Wi-Fi on time. Most did, but the Tab5 only knows when it *handed* a command to its network stack, not when it reached the radio. The relay's transmit log showed what the Tab5 could not see:

- 16:42:30: the Tab5's send blocked while its Wi-Fi chip's SDIO link was draining an oversize receive; the last symbols reached the radio 1.3 s late (RF on-air 13,643 ms instead of 12,640).
- 17:54:15, with I/Q already paused during transmit: one gap of 1,316 ms in transit (most likely a TCP retransmission), while the Tab5's own figures were normal.

Pausing I/Q during transmit (patch 0018) removed one cause but not the other: one stall in ten bursts either way. The remedy is to stop depending on Wi-Fi during the burst. Since patch 0019 the Tab5 sends all 79 (FT8) or 105 (FT4) tone frequencies and the symbol period to the relay in one TX_SCHED message at key-down. The relay keys the QMX, sends each tone at key-down + i × 160 ms on the NAS's own clock over its wired USB link, sends the key-up tone at the end and reports back (TX_DONE). A Wi-Fi hiccup during the burst can no longer delay a symbol; it could at most delay the start, which shows up in the key-down offset.

The Tab5 still runs its symbol loop on its own clock for the progress display, the live power/SWR reading, SWR protection and abort (an abort sends TX_ABORT and the relay keys up at once). It still sends the post-burst power/SWR query and the final RX;. If the hand-off fails the Tab5 sends the tones itself as before, and if no TX_DONE arrives it sends the key-up tone itself; if the Tab5 disconnects mid-burst the relay keys up and returns the radio to receive.

Why I/Q is paused while transmitting

While the QMX transmits its I/Q is of no use, yet it is ~190 KB/s of traffic into the Tab5, the direction that overloads the Tab5's Wi-Fi (ESP32-C6 over SDIO). Pausing it leaves the link nearly idle during the burst. Pings continue, so the Tab5's 3 s idle timeout is not tripped; the pause is capped at 180 s in case RX; never comes. The spectrum and waterfall freeze for each burst.

Why PS0; is blocked

While the radio was being debugged it was found switched off more than once. CAT can switch a QMX off (PS0;) but never on: once off, it is not on the USB bus at all. (The cause turned out to be a 12 V current limiter in the supply, not CAT.) The relay therefore blocks PS0; from the Tab5 and from PC software alike, logs it as BLOCKED, and lets it through only while a flag file allow-ps0 is present.

Why discovery

The Tab5's Find NAS and the Mac bridge broadcast QMXR? on UDP 7355; every relay answers with its address and port. No address needs to be typed, and a relay that gets a new address is found again. The relay sends nothing unless asked.

4. QMXR/1 wire protocol

Every frame is a 4-byte header followed by the payload: **u8 type, u8 flags (0), u16 length (little-endian)**. The Tab5 opens with HELLO; the relay answers HELLO only after it has opened the QMX, announcing its capabilities, e.g. `QMXR/1 rate=48000 ch=2 fmt=s16le src=nas term=1 txsched=1`.

Type	Name	Direction	Payload
1	HELLO	both	ASCII "QMXR/1 ..." with capability tokens
2	IQ	relay → Tab5	Interleaved int16 LE I/Q pairs, 48 kHz (480 pairs per frame)
3	CAT	both	Raw CAT bytes for QMX serial port 1
4	PING	relay → Tab5	empty; once a second (the Tab5 treats 3 s of silence as a lost link)
5	TERM_OPEN	Tab5 → relay	Open serial port 2 (Radio menus)
6	TERM_STATUS	relay → Tab5	u8 ok (1/0) [+ reason text]; also unsolicited if the port goes away
7	TERM_DATA	both	Bytes on serial port 2
8	TERM_CLOSE	Tab5 → relay	Close serial port 2
9	TX_SCHED	Tab5 → relay	u8 version (1), u8 flags (0), u16 n, u32 symbol period (µs), then n × u32 tone frequency in centi-Hz. Relay keys the QMX at once.
10	TX_ABORT	Tab5 → relay	empty; stop now and send the key-up tone
11	TX_DONE	relay → Tab5	u8 status (0 complete, 1 aborted, 2 failed), u16 tones sent, u32 ms TX; → key-up, i32 ms latest symbol vs schedule

Tone frequencies are sent in centi-Hz so the relay prints exactly the TA*nnnn.nn*; command the Tab5 would have sent. Types 9-11 are used only when the relay announces `txsched=1`; with an older relay, or with the flag file tab5-timed-tx present, the Tab5 sends every tone itself.

Discovery (UDP 7355)

```
Tab5 / Mac  -> broadcast   QMXR?                          (3 times over 1.5 s)
each relay  -> direct      QMXR! <relay IP> <TCP port>   e.g.  QMXR! 10.0.0.137 7355
```

5. Tab5 firmware changes

Driver side

- **main/net/radio_relay.c/.h (new)**: the QMXR/1 client. Connect and handshake, a receive task that feeds I/Q into the audio ring and CAT into the CAT parser, the Radio menus channel, UDP discovery, relay-timed transmit (radio_relay_tx_schedule / tx_abort / tx_wait_done), and diagnostics for /api/relay. Refuses to send from lwIP's own thread instead of deadlocking it.
- **main/cat/cat.c/.h**: a transport layer (USB or network) under the existing CAT code; every write goes through cat_tx so upstream's GP/TM probes and KS keyer-speed commands also work over the network; cat_get_transport() reports "usb", "net" or "none".
- **main/audio/audio.c/.h**: an entry point for network I/Q into the same ring the USB audio driver fills.
- **main/wifi/wifi.c**: the QMX clock update after network time runs on a worker task, not the lwIP thread (it froze every connection 20-50 s after boot).
- **main/qmx_term.c**: Radio menus over the network on serial port 2.
- **Wi-Fi receive patches** (tools/patches): esp_hosted SDIO receive keeps every frame of a bundled transfer, copies frames to PSRAM instead of internal RAM, and returns pool blocks at once; with the TCP window at 11520 the Tab5 receives ~47,700 of 48,000 I/Q pairs a second. tools/check_patches.py refuses a build without them.

Application side

- **main/ui/ui.c**: Radio source in the settings drawer: shows USB or NAS and whether it is connected, with Find NAS, Use NAS and Use USB; the last NAS address is remembered.
- **main/net/webserver.c, www/index.html**: Radio source on the web page and /api/relay; the URI handler limit raised so the live-spectrum socket registers; the 10-burst test button (shown only at http://qmx.local/?test).
- **main/ft8_tx.c**: hands each FT8/FT4 burst to the relay when it is connected over the network and the relay supports it; logs the relay's TX_DONE report.
- **main/storage/settings.c/.h, config_io.c, main.c**: relay_host / relay_port settings, the config-file keys, and start-up.
- **Version**: builds report the upstream version with -NAS appended, e.g. v1.18.0-NAS.

6. Synology relay: installation, operation and maintenance

Installation

- Copy the tools/nas-relay folder to a shared folder on the NAS with File Station, e.g. `/volume1/File Share/qmx-relay`.
- **Control Panel → Task Scheduler → Create → Triggered Task → User-defined script.** Task *QMX relay*, user *root*, event *Boot-up*, run command `sh "/volume1/File Share/qmx-relay/start-relay.sh"`. DSM asks for your password because the task runs as root (needed to open the USB device).
- Select the task and click **Run** (it also starts on every boot). start-relay.sh keeps the relay running and restarts it if it exits.
- Create a second user-defined task, *QMX relay check* (root, a scheduled task set to run once is fine), with `sh "/volume1/File Share/qmx-relay/check.sh"`.
- Open qmx-relay.log in the folder: expect SELFTEST PASSED and "listening on 0.0.0.0:7355" and ":7356".
- On the Tab5: settings drawer → Radio source → **Find NAS**, then **Use NAS**.
- On the radio: USB serial ports = 2 for Radio menus, 3 to also serve PC software on TCP 7356. If the DSM firewall is on, allow TCP 7355, TCP 7356 and UDP 7355.

Operation

Script / task	What it does
start-relay.sh (task QMX relay, boot)	Supervisor: runs the relay with --selftest, restarts it 5 s after it exits, writes qmx-relay.log
stop-relay.sh	Stops the supervisor and the relay
check.sh (task QMX relay check)	Writes check.log: system info, USB devices, a probe of the QMX's interfaces; then nettest.py against the running relay (NETTEST PASSED) or a one-off self-test. Also handles restart.request (see Maintenance)
nettest.py	Connects like a Tab5 (handshake, I/Q rate, CAT, Radio menus) and like a PC on port 3; checks discovery. Read-only: never transmits
env.sh, get_libusb1.py	Environment for the scripts; fetches python-libusb1 (SHA-256 checked) where libusb-1.0 exists. Not needed on the DS416j (usbfs backend)
Dockerfile, docker-compose.yml	Alternative for x86 models with Container Manager
qmx_nas_relay.py --fake	Bench test without a radio: synthetic tone, CAT emulator, menu terminal

Flag files (in the relay folder, re-checked every 2 s, no restart needed)

File	Effect
restart.request	Run the QMX relay check task: the relay is stopped, the supervisor starts the new version 5 s later, the file is deleted and the network test runs (check.log about 30 s later)
catlog.off	Pauses the CAT log (catlog/catlog-<date>.log, 40 MB a day cap)
allow-ps0	Lets the power-off command PS0; through (blocked by default)
iq-during-tx	Keeps I/Q streaming to the Tab5 while transmitting (paused by default)
tab5-timed-tx	The Tab5 times each tone itself instead of the relay (takes effect when the Tab5 next connects)

Logs

Log	Contents
qmx-relay.log	Self-test, connections, warnings, blocked PS0;, transmit summaries
check.log	Output of the QMX relay check task
catlog/catlog-<date>.log	Every CAT command and reply, timestamped; labels T5>QMX, QMX>T5, PC>QMX, QMX>PC, T5 menu, RELAY>QMX
txlog/txlog-<date>.log	One entry per transmit burst: every command handed to the QMX on the NAS clock, RF on-air, tone gaps, symbol timing, I/Q mode, who timed the tones. Off with --txlog-dir ''

Maintenance

- **Updating the relay**: copy the new files over the old ones with File Station, create an empty file restart.request in the folder and Run the QMX relay check task. No new root task or password is needed.

- **QMX not found** in check.log: check that the radio is on and powered, and try another USB cable (a charge-only cable powers the QMX but shows no device). The radio needs its own supply; the relay keeps running while it is off and the Tab5 reconnects when it returns.
- **QMX is in use by another program**: the relay must be the only program using the radio. Never run a NAS relay and a Pi relay on the same radio.
- **Touch dead on the Tab5** after a restart without power-off: unplug its USB-C and battery for ~15 s.

7. Raspberry Pi relay: installation, operation and maintenance

The Pi runs the same qmx_nas_relay.py, unchanged: it reaches the QMX through Linux usbfs, which the Pi has. Its USB structures and ioctl numbers were checked against the 64-bit ARM kernel headers; on the Pi it detaches the kernel's serial and audio drivers from the QMX, as on any Linux. Status: written, not yet run on a Pi.

Installation

```
# on the Mac: copy the relay and the Pi scripts to the Pi (Raspberry Pi OS Bookworm, 64-bit)
rsync -a ~/src/QMX-Panadapter-NAS/tools/{nas-relay,pi-relay} pi@qmxpi.local:qmx/tools/

# on the Pi
cd ~/qmx/tools/pi-relay
sudo bash install-pi.sh              # install and start now
sudo bash install-pi.sh --no-start   # install; start 60 s after the next boot
```

Then on the Tab5: Radio source → Find NAS finds the Pi. Set USB serial ports = 3 on the radio for PC software on TCP 7356.

How it starts and restarts

Situation	Behaviour
After a reboot	qmx-relay.timer starts the relay 60 s after boot (OnBootSec=60s)
If it fails	Restart=on-failure, 10 s apart, up to 3 restarts (StartLimitBurst=4)
After 3 restarts	Stays stopped until the next reboot or sudo qmx-relay-ctl restart; qmx-relay-gaveup.service writes GAVE UP in the log
Stopped by hand	sudo qmx-relay-ctl stop is not counted as a failure

Operation and maintenance commands

```
qmx-relay-ctl status         # service, timer, restarts used, ports, QMX on USB, last log lines
qmx-relay-ctl log 100        # last 100 log lines
qmx-relay-ctl follow         # watch the log
qmx-relay-ctl test           # network test (read-only, never transmits)
sudo qmx-relay-ctl probe     # USB devices and the QMX's interfaces
sudo qmx-relay-ctl restart   # start again now, with a fresh 3 restarts
sudo qmx-relay-ctl stop      # stop (starts again 60 s after the next boot)
sudo bash uninstall-pi.sh    # remove (--purge also removes the log and settings)
```

File	Purpose
install-pi.sh / uninstall-pi.sh	Install or remove the relay, units, rules, settings and log rotation
qmx-relay-ctl	Everyday control command (above)
qmx-relay-event.sh	Writes SERVICE lines (boot, start, failure, restart count, GAVE UP) into the log
/etc/default/qmx-relay	RELAY_ARGS and QMX_BACKEND; then sudo qmx-relay-ctl restart
systemd/qmx-relay.service, .timer, -boot, -gaveup	Service with hardening (NoNewPrivileges, ProtectSystem, ProtectHome, PrivateTmp), the 60 s boot timer and the event hooks
udev/70-qmx-relay.rules	Keeps ModemManager and PipeWire/PulseAudio away from the QMX; no USB autosuspend
logrotate/qmx-relay	/var/log/qmx-relay/qmx-relay.log rotated weekly, 8 kept

The flag files of section 6 work the same way in the Pi relay's folder.

8. Mac tools and the update kit

- **Mac serial bridge** (tools/mac-serial-bridge): presents the QMX's port 3 as ~/cu.QMX-NAS for WSJT-X, JTDX, fldigi, rigctld and loggers; finds the relay by discovery; installed as a login item by install-mac.sh. Status: written, with bench-test helpers; connecting to the real relay is still to be finished.
- **Build and flash**: `bash tools/mac-build-flash.sh` (ESP-IDF v5.4.4 in ~/esp/esp-idf, pwsh for the standing patches; applies and checks all patches, builds, asks before a normal flash). Nothing may be plugged into the Tab5's USB-A port while flashing; power the Tab5 off and on afterwards.
- **Serial log**: `bash tools/mac-capture-serial.sh 90`. Opening the port restarts the Tab5, so for timing tests use the Tab5's own log at http://qmx.local/api/log instead.
- **Update kit**: `bash make-qmx-panadapter-nas.sh` clones upstream, checks out the newest release and applies patches 0001-0019 with a 3-way merge into ~/src/QMX-Panadapter-NAS; `--mark-tested` copies it to ~/src/QMX-Panadapter-Shared after a hardware test; replaced folders go to ~/src/.qmx-panadapter-archive. push-to-github.sh publishes both folders to jpwatters/QMX-PanadapterShared.

9. FT8 burst timing comparisons

The FT8 burst

An FT8 transmission carries a 77-bit message plus a 14-bit CRC (91 bits), LDPC-encoded to 174 bits and sent as 58 data symbols of 3 bits each, with three 7-symbol Costas synchronisation arrays at symbols 0-6, 36-42 and 72-78: 79 symbols in all. Each symbol is one of 8 tones 6.25 Hz apart, held for 160 ms, so a correct burst is 79 × 160 ms = **12,640 ms** of RF and occupies about 50 Hz. The transmitter changes tone with one TA command per symbol, then keys up with TA0;. FT4 uses 105 symbols of 48 ms.

How it was measured

- The relay writes every burst to its transmit log with each command's time on the NAS clock; the Tab5 logs its own key-down offset and on-air time on its own clock. tx_compare.py pairs bursts by FT8 slot.
- **One clock per number.** Every interval compared has both ends on the same clock. The NAS and Tab5 clocks are used only to identify the slot; a Tab5 time is never subtracted from a NAS time.
- A **stall** is a gap between two tone changes at the radio of more than 300 ms, or RF on-air more than 20 ms from 12,640 ms. Ten consecutive CQs were sent with the web page's 10-burst test button into a dummy load.

Summary by run (2026-10-14, local time)

Run	Time	Tones timed by	I/Q in TX	Bursts	Stalls	Longest gap (ms)	Worst symbol (ms late)	RF on-air min-max (ms)	Relay key-down mean (+ms)
1	10:54-10:58	Tab5	streamed	8	0	236	+69	12624-12656	+290
2	16:42-16:46	Tab5	streamed	10	1	1505	+1342	12627-13643	+610
3	17:53-17:58	Tab5	paused	10	1	1316	+1145	12620-12638	+408
4	18:45-18:50	relay	paused	10	0	161	+2	12640-12641	+209

Ideal: longest gap 160 ms, symbol timing 0 ms, RF on-air 12,640 ms. Run 1 used the relay before the I/Q pause existed. "Relay key-down" is when the slot's first command reached the radio, NAS clock vs UTC; it includes the Tab5's own start-up delay.

What the runs show

- **Runs 1-3 (Tab5 timing)**: most bursts were close to ideal, but one burst in ten stalled in runs 2 and 3 (1.5 s and 1.3 s gaps). In run 1, tone gaps of 175-236 ms and up to 13 tone changes per burst more than 20 ms off show Wi-Fi jitter even without a stall.
- **Pausing I/Q (run 3)** did not remove stalls: the 17:54:15 stall happened in transit with the link nearly idle.
- **Relay timing (run 4)**: every burst 12,640-12,641 ms, longest gap 160-161 ms, no tone change more than 20 ms off, worst symbol +2 ms. Ten bursts cannot prove stalls are gone for good (they came about once in 10-18 bursts), but a Wi-Fi delay during the burst can no longer reach the radio.
- **Late key-down** (17:53:45, +1,647 ms) is unrelated to the network: the CQ waited for the previous slot's decode before transmitting.

Every burst

Relay columns are on the NAS clock, Tab5 columns on the Tab5 clock (Tab5 logs were kept for run 4 only). Stalled bursts are marked.

Run	Slot (local)	Relay key (+ms)	RF on-air (ms)	TX;→RX; (ms)	Longest gap (ms)	Gaps >20 ms off	Symbol timing (ms)	Tab5 key (+ms)	Tab5 on-air (ms)	Result
1	10:54:45	+292	12628	12681	185	4	-13..+16			ok
1	10:55:00	+289	12631	12668	200	13	-11..+34			ok
1	10:55:30	+287	12650	12675	175	0	-10..+11			ok
1	10:56:00	+292	12634	12667	201	2	-14..+28			ok
1	10:56:30	+289	12630	12670	192	7	-13..+20			ok
1	10:57:00	+288	12634	12673	205	4	-11..+41			ok
1	10:57:30	+286	12656	12693	236	6	-10..+69			ok
1	10:58:30	+293	12624	12660	184	4	-17..+10			ok
2	16:42:00	+522	12641	12682	201	9	-10..+35			ok
2	16:42:30	+525	13643	13680	1505	11	-13..+1342			**STALL**
2	16:43:00	+638	12628	12665	199	4	-12..+32			ok
2	16:43:30	+579	12637	12673	222	11	-8..+57			ok
2	16:44:00	+637	12634	12669	210	6	-9..+42			ok
2	16:44:30	+525	12630	12667	201	6	-12..+29			ok
2	16:45:00	+612	12627	12664	255	4	-13..+84			ok
2	16:45:30	+595	12632	12670	184	5	-12..+13			ok
2	16:46:00	+944	12630	12670	193	4	-11..+27			ok
2	16:46:30	+518	12649	12686	255	12	-11..+86			ok
3	17:53:45	+1647	12620	12655	196	4	-28..+13			ok
3	17:54:15	+263	12630	12681	1316	17	-11..+1145			**STALL**
3	17:54:45	+263	12638	12671	244	3	-11..+82			ok
3	17:55:15	+271	12622	12658	183	1	-17..+6			ok
3	17:55:45	+265	12631	12664	209	4	-13..+56			ok
3	17:56:15	+276	12625	12679	197	3	-14..+24			ok
3	17:56:45	+262	12631	12669	253	2	-12..+91			ok
3	17:57:15	+262	12631	12673	205	7	-9..+75			ok
3	17:57:45	+299	12621	12666	204	4	-19..+30			ok
3	17:58:15	+268	12633	12672	194	3	-11..+25			ok
4	18:45:30	+171	12640	12682	160	0	+0..+1	+24	12725	ok
4	18:46:00	+180	12641	12689	161	0	+1..+2	+24	12740	ok
4	18:46:30	+177	12641	12678	160	0	+0..+1	+25	12728	ok
4	18:47:00	+194	12640	12685	161	0	+0..+1	+78	12690	ok
4	18:47:30	+215	12640	12677	160	0	+0..+1	+87	12704	ok
4	18:48:00	+335	12641	12687	160	0	+0..+1	+217	12704	ok
4	18:48:30	+233	12641	12691	160	0	+0..+1	+121	12688	ok
4	18:49:00	+204	12641	12681	160	0	+0..+1	+86	12694	ok
4	18:49:30	+229	12640	12682	160	0	+0..+1	+92	12717	ok
4	18:50:00	+148	12641	12679	161	0	+0..+1	+24	12702	ok

Run 4 Tab5 summary: key-down +24 to +217 ms (median +82), on-air 12,688-12,740 ms (median 12,704). The Tab5's on-air figure runs from key-down until RX; has been sent, so it includes the post-burst power/SWR query and, for relay-timed bursts, waiting for TX_DONE; it is not a measure of the RF.

The same data is in the workbook QMX-NAS-TX-Timing-2026-10-14.xlsx.

10. Verification and open items

Area	Status
Synology relay (DS416j, DSM 7.4.1, QMX+)	Tested on the hardware: self-test, network test, I/Q ~47,700 pairs/s on the Tab5, CAT, Radio menus, port 3, discovery, QMX power-cycle recovery

Area	Status
Tab5 v1.18.0-NAS	Built and flashed from the Mac; Find NAS, connect by address, spectrum, reconnection, Radio source switching, web live spectrum tested
CAT log, PS0 guard, I/Q pause	Running on the NAS during the timing runs
Relay-timed transmit	Bench-tested against the fake radio (complete, abort, disconnect mid-burst), firmware compile-checked, then 10 of 10 FT8 bursts on the hardware
Raspberry Pi relay	Written; not yet run on a Pi
Mac serial bridge	Written, with bench-test helpers; connection to the real relay still to be finished
FT4 relay-timed bursts	Supported by the same path; not yet measured on the air
Direct-USB baseline	Optional comparison run (tx_compare.py --usb) not yet made

11. Sources (references)

"Verified" means it was read or executed during this work; "background knowledge" means it was relied on but not re-checked here. No source is split across pages.

[1] Upstream firmware: SteffenLav/qmx-panadapter v1.18.0
Location: https://github.com/SteffenLav/qmx-panadapter (tag v1.18.0; earlier work on v1.16.11 and v1.16.12)
Used for: Base for the port. cat.c, audio.c, qmx_term.c, ft8_tx.c, webserver.c, ui.c, settings, config_io, main.c, www/index.html, CMake files and the QMX USB descriptor notes were read and changed. Verified.

[2] QMX second and third serial ports
Location: Upstream main/qmx_term.h and cat.c (cat_probe_extra_cdc_ports)
Used for: Second CDC function at interface 5 when USB serial ports = 2; CAT stays on interface 0; terminal entered with a bare CR. Verified in code (measurements by the upstream authors).

[3] QRP Labs QMX / QMX+ operating and CAT manuals
Location: https://qrp-labs.com/qmx
Used for: USB serial ports setting; CAT commands TX;, RX;, TA (tone frequency; a value below 10 Hz keys up), PC; and SW; (power and SWR), PS0; (power off). Referenced via the upstream code and used on the radio.

[4] FT8 and FT4 protocol
Location: S. Franke, B. Somerville, J. Taylor, "The FT4 and FT8 Communication Protocols", QEX, July/August 2020; WSJT-X source and user guide, https://wsjt.sourceforge.io
Used for: 79 symbols of 160 ms (FT8), 105 of 48 ms (FT4), 8-FSK at 6.25 Hz, 77+14 bits, LDPC(174,91), Costas arrays at symbols 0-6, 36-42, 72-78. Background knowledge; burst length confirmed by the relay’s transmit log.

[5] ft8_lib (FT8/FT4 encoder used by the firmware)
Location: https://github.com/kgoba/ft8_lib (vendored in components/ft8_lib)
Used for: Symbol counts and tone spacing constants (FT8_NN, FT4_NN, symbol periods) used for scheduling. Verified in code.

[6] Espressif ESP-IDF v5.4.4
Location: https://github.com/espressif/esp-idf (tag v5.4.4)
Used for: Toolchain and headers for ESP32-P4; FreeRTOS semaphores, esp_timer, heap_caps, lwIP sockets. Used to build and compile-check the firmware. Verified.

[7] Espressif esp_hosted (ESP-Hosted-MCU) host driver
Location: https://github.com/espressif/esp-hosted-mcu (host/drivers/transport/sdio)
Used for: SDIO receive path the Wi-Fi patches change (bundled frames, per-frame copy, mempool). Verified (read, compiled, tested on the Tab5).

[8] lwIP TCP receive window and thread model
Location: lwIP / ESP-IDF lwipopts.h; ESP-IDF networking guide
Used for: TCP_WND sizing for the stream; sockets must not be used from the tcpip thread. Verified on the Tab5.

[9] Espressif esp-usb, esp-bsp, esp-dsp; LVGL
Location: https://github.com/espressif/esp-usb ; esp-bsp ; esp-dsp ; https://github.com/lvgl/lvgl
Used for: CDC-ACM host API, LVGL port, touch and DSP headers needed to compile the changed files. Verified.

[10] Linux usbfs ioctl interface
Location: Linux include/uapi/linux/usbdevice_fs.h; Documentation/driver-api/usb/usb.rst
Used for: SUBMITURB / REAPURBNDELAY / DISCARDURB, CONTROL, SETINTERFACE, CLAIM/RELEASEINTERFACE, DISCONNECT/CONNECT; struct layouts on 32-bit ARM (verified on the DS416j, kernel 3.10.108) and aarch64 headers (Pi).

[11] USB CDC PSTN and USB Audio Class 1.0
Location: USB-IF Class Definitions for Communications Devices 1.2; Audio Devices 1.0
Used for: SET_LINE_CODING / SET_CONTROL_LINE_STATE on each serial port; Format Type I parsing and the 48 kHz SET_CUR request. Background knowledge.

[12] libusb-1.0 and python-libusb1
Location: https://libusb.info ; https://pypi.org/project/libusb1/
Used for: Alternative relay backend where libusb exists. Verified against a mock device; not present on the DS416j.

[13] Synology DSM 7.4: Task Scheduler and File Station
Location: https://www.synology.com ; DSM Control Panel → Task Scheduler
Used for: Root boot-up task for the relay, the check task, file deployment and log retrieval. Verified on the NAS.

[14] Raspberry Pi OS and systemd
Location: https://www.raspberrypi.com/documentation ; systemd.service(5), systemd.timer(5), systemd.exec(5)
Used for: OnBootSec timer, Restart=on-failure with StartLimitBurst, OnFailure hooks, sandboxing options. Background knowledge.

[15] udev, ModemManager and PipeWire device rules
Location: udev(7); ModemManager ID_MM_DEVICE_IGNORE; PipeWire/PulseAudio device properties
Used for: Keeping other services away from the QMX on the Pi. Background knowledge.

[16] Python 3 standard library
Location: https://docs.python.org/3/ (socket, struct, threading, ctypes, fcntl)
Used for: The relay, network test, serial bridge and tx_compare. Verified by running them.

[17] Git: format-patch, am --3way, mailinfo, commit-tree
Location: https://git-scm.com/docs
Used for: Exporting the patch series and re-applying it to each new upstream release. Verified by the kit.

[18] Project measurements, 2026-10-14
Location: Relay txlog-2026-10-14.log (NAS) and the Tab5 log from http://qmx.local/api/log
Used for: All timing figures in section 9. Verified.

12. Driver listings (Tab5 firmware)

The network-radio driver is listed in full; changes to upstream driver files are unified diffs against tag v1.18.0. Long lines are wrapped (↪). Identical in ~/src/QMX-Panadapter-NAS and the kit.

main/net/radio_relay.h

new file, 115 lines

```
#pragma once

/*
 * radio_relay - "Network radio" source for the QMX panadapter.
 *
 * Lets the Tab5 use a QMX that is NOT plugged into its own USB port but into
 * a Synology NAS. tools/nas-relay/qmx_nas_relay.py runs on the NAS (Docker /
 * Container Manager), owns the QMX through libusb, and streams the QMX's I/Q
 * audio and CAT bytes over one TCP connection using the small QMXR/1 framing:
 *
 *     u8 type | u8 flags | u16 len (LE) | payload
 *       1=HELLO 2=IQ 3=CAT 4=PING
 *       5=TERM_OPEN 6=TERM_STATUS 7=TERM_DATA 8=TERM_CLOSE   (relay says "term=1")
 *
 * IQ payloads are interleaved int16 LE I/Q pairs at 48 kHz, already scaled
 * exactly like the USB path's (s24 >> 8), and are pushed into the same audio
 * ring buffer the UAC driver feeds, so DSP/FFT/FT8 see no difference.
 * CAT payloads go to/from the same Kenwood parser cat.c uses for USB CDC.
 *
 * An open TCP session == "radio present": when the NAS loses the QMX it
 * closes the socket and cat.c handles it exactly like a USB unplug.
 *
 * TERM_* carry the QMX's SECOND serial port (the one qmx_term.c uses for the
 * radio's own menus, interface 5 on USB) as a separate byte stream, so a
 * terminal session over the network still never touches the CAT port.
 */

#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include "esp_err.h"

#define RADIO_RELAY_DEFAULT_PORT 7355
#define RADIO_RELAY_HOST_LEN     64

/* CAT bytes from the radio. Same signature as the CDC-ACM data callback. */
typedef bool (*radio_relay_cat_rx_cb_t)(const uint8_t *data, size_t len, void *arg);
/* Called (from the relay RX task) when an established link drops on its own. */
typedef void (*radio_relay_gone_cb_t)(void);

/* Set the runtime relay target. Empty/NULL host disables the network source
 * (USB is used). Does not by itself drop an open link - see cat_reconnect(). */
void radio_relay_set_target(const char *host, uint16_t port);
void radio_relay_get_target(char *host, size_t cap, uint16_t *port);
bool radio_relay_enabled(void);

/* Connect + QMXR/1 handshake. Starts the RX task on success. */
esp_err_t radio_relay_connect(radio_relay_cat_rx_cb_t cat_cb,
                              radio_relay_gone_cb_t gone_cb,
                              uint32_t timeout_ms);
bool radio_relay_connected(void);

/* Send CAT bytes to the radio (thread-safe). */
esp_err_t radio_relay_send_cat(const uint8_t *data, size_t len, uint32_t timeout_ms);

/* Tear down the link (does NOT invoke gone_cb). Safe if not connected. */
void radio_relay_close(void);

/* ---- QMX second serial port (terminal) over the relay -------------------
 * true when the connected relay advertised "term=1" in its HELLO. */
bool radio_relay_term_supported(void);

/* Ask the relay to open the QMX's second serial port. data_cb gets the bytes
 * the radio sends on it (relay RX task context - must not block); gone_cb fires
 * if the port or the whole link goes away while open. Blocks up to timeout_ms
 * for the relay's answer. ESP_ERR_NOT_FOUND = the radio has no second port
 * (set to 1 USB serial port); radio_relay_term_error() has the relay's reason. */
esp_err_t radio_relay_term_open(radio_relay_cat_rx_cb_t data_cb,
                                radio_relay_gone_cb_t gone_cb,
                                uint32_t timeout_ms);
esp_err_t radio_relay_term_send(const uint8_t *data, size_t len, uint32_t timeout_ms);
/* Close the second port on the relay. Does NOT invoke gone_cb. */
void radio_relay_term_close(void);
const char *radio_relay_term_error(void);

/* ---- Discovery ----------------------------------------------------------
 * Broadcast "QMXR?" on UDP RADIO_RELAY_DEFAULT_PORT; every relay on this
```

```
 * network answers "QMXR! <ip> <tcp port>". Asks 3 times over timeout_ms and
 * collects distinct answers. BLOCKS for timeout_ms - never call it on the
 * LVGL task. Returns how many relays answered (0..max), or -1 if WiFi is down
 * or no socket could be opened. Broadcasts do not cross routers: the relay
 * must be on the Tab5's own network. */
typedef struct {
    char     ip[16];
    uint16_t port;
} radio_relay_found_t;
int radio_relay_discover(radio_relay_found_t *out, int max, uint32_t timeout_ms);

/* ---- Relay-timed transmit bursts (QMXR/1 "txsched=1") ----------------------
 * Over the network every tone change used to cross WiFi on its own, 160 ms
 * apart, and a lost packet or a stalled WiFi link (2026-10-14: 1.1-1.5 s)
 * delivered symbols late. Instead the whole burst goes to the relay in ONE
 * frame at key-down; the relay keys the QMX, sends each tone on the NAS's own
 * timer over its wired link and USB, sends the key-up tone, and reports back.
 * Only relays that announce "txsched=1" in their HELLO accept it. */
typedef struct {
    bool     done;            // a TX_DONE arrived (false: timed out / link lost)
    uint8_t  status;          // 0 complete, 1 aborted, 2 failed
    uint16_t tones_sent;
    uint32_t duration_ms;     // TX; -> key-up, NAS clock
    int32_t  latest_ms;       // latest symbol vs its schedule, NAS clock
} radio_relay_tx_result_t;

bool      radio_relay_txsched_supported(void);
/* Hand the burst to the relay: n tone frequencies (Hz), one every symbol_us.
 * The relay keys the radio on receipt. */
esp_err_t radio_relay_tx_schedule(const float *freq_hz, int n, uint32_t symbol_us);
/* Stop a relay-timed burst now (the relay sends the key-up tone). */
esp_err_t radio_relay_tx_abort(void);
/* Wait for the relay's report that the burst (including key-up) is done. */
bool      radio_relay_tx_wait_done(uint32_t timeout_ms, radio_relay_tx_result_t *out);

/* Diagnostics for /api/relay. */
uint32_t radio_relay_iq_pairs_total(void);
const char *radio_relay_last_error(void);
```

main/net/radio_relay.c

new file, 569 lines

```
#include "radio_relay.h"

#include <string.h>
#include <stdio.h>
#include <errno.h>
#include <fcntl.h>

#include "esp_log.h"
#include "esp_timer.h"
#include "esp_heap_caps.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "lwip/sockets.h"
#include "lwip/netdb.h"

#include "audio.h"
#include "wifi.h"           // wifi_is_connected(): no socket calls before the stack is up

static const char *TAG = "relay";

#define PROTO_HELLO 1
#define PROTO_IQ    2
#define PROTO_CAT   3
#define PROTO_PING  4
#define PROTO_TERM_OPEN   5   // Tab5 -> relay: open the QMX's second serial port
#define PROTO_TERM_STATUS 6   // relay -> Tab5: u8 ok (1/0) [+ reason text]
#define PROTO_TERM_DATA   7   // both ways: bytes on the second serial port
#define PROTO_TERM_CLOSE  8   // Tab5 -> relay: close it
#define PROTO_TX_SCHED    9   // Tab5 -> relay: a whole transmit burst (see radio_relay.h)
#define PROTO_TX_ABORT   10   // Tab5 -> relay: stop the burst now
#define PROTO_TX_DONE    11   // relay -> Tab5: u8 status, u16 tones, u32 ms, i32 latest ms

#define RX_PAYLOAD_MAX   8192     // NAS relay sends <=1920-byte IQ frames; anything larger is skipped
#define RX_IDLE_TIMEOUT_S   3     // relay streams I/Q + 1 s PINGs; 3 s of silence == gone

static char     s_host[RADIO_RELAY_HOST_LEN] = {0};
static uint16_t s_port = RADIO_RELAY_DEFAULT_PORT;
static portMUX_TYPE s_cfg_mux = portMUX_INITIALIZER_UNLOCKED;

static int s_fd = -1;
static SemaphoreHandle_t s_tx_mutex = NULL;
static TaskHandle_t s_rx_task = NULL;
static volatile bool s_connected = false;
static volatile bool s_closing = false;
```

```
static radio_relay_cat_rx_cb_t s_cat_cb = NULL;
static radio_relay_gone_cb_t   s_gone_cb = NULL;
static volatile uint32_t s_iq_pairs_total = 0;
static char s_last_err[64] = "not configured";

// Second serial port (terminal) state.
static volatile bool s_term_supported = false;
static radio_relay_cat_rx_cb_t s_term_cb = NULL;
static radio_relay_gone_cb_t   s_term_gone_cb = NULL;
static SemaphoreHandle_t       s_term_sem = NULL;
static volatile int            s_term_result = 0;   // -1 waiting, 0 failed, 1 open
static char                    s_term_err[64] = "";
static int64_t s_last_fail_log_us = 0;

// Relay-timed transmit bursts.
static volatile bool s_txsched_supported = false;
static SemaphoreHandle_t s_txdone_sem = NULL;
static radio_relay_tx_result_t s_txdone;

// ---- config ------------------------------------------------------------

void radio_relay_set_target(const char *host, uint16_t port)
{
    portENTER_CRITICAL(&s_cfg_mux);
    // snprintf, not strncpy: IDF builds with -Werror=stringop-truncation.
    snprintf(s_host, sizeof(s_host), "%s", host ? host : "");
    s_port = port ? port : RADIO_RELAY_DEFAULT_PORT;
    portEXIT_CRITICAL(&s_cfg_mux);
    ESP_LOGI(TAG, "target: %s", s_host[0] ? s_host : "(none - USB)");
}

void radio_relay_get_target(char *host, size_t cap, uint16_t *port)
{
    portENTER_CRITICAL(&s_cfg_mux);
    if (host && cap) snprintf(host, cap, "%s", s_host);
    if (port) *port = s_port;
    portEXIT_CRITICAL(&s_cfg_mux);
}

bool radio_relay_enabled(void) { return s_host[0] != '\0'; }
bool radio_relay_connected(void) { return s_connected; }
uint32_t radio_relay_iq_pairs_total(void) { return s_iq_pairs_total; }
const char *radio_relay_last_error(void) { return s_last_err; }

// ---- socket helpers ----------------------------------------------------

static int recv_exact(int fd, void *buf, size_t n)
{
    uint8_t *p = (uint8_t *)buf;
    size_t got = 0;
    while (got < n) {
        int r = recv(fd, p + got, n - got, 0);
        if (r <= 0) return -1;
        got += (size_t)r;
    }
    return 0;
}

static esp_err_t send_frame(uint8_t type, const uint8_t *payload, size_t len, uint32_t timeout_ms)
{
    if (len > 0xFFFF) return ESP_ERR_INVALID_SIZE;
    /* (!) Never from lwIP's own thread ("tiT"): a socket send there waits on the
     * tcpip thread, i.e. on itself, and the whole network stack stops for good
     * (the SNTP callback did exactly this with the QMX time push, 2026-10-04 -
     * see wifi.c sntp_sync_cb). Refuse loudly instead of hanging silently. */
    if (strcmp(pcTaskGetName(NULL), "tiT") == 0) {
        ESP_LOGE(TAG, "send from the tcpip thread refused (would deadlock lwIP)");
        return ESP_ERR_INVALID_STATE;
    }
    if (!s_tx_mutex || xSemaphoreTake(s_tx_mutex, pdMS_TO_TICKS(timeout_ms)) != pdTRUE) {
        return ESP_ERR_TIMEOUT;
    }
    esp_err_t ret = ESP_OK;
    int fd = s_fd;
    if (fd < 0 || s_closing) {
        ret = ESP_ERR_INVALID_STATE;
    } else {
        uint8_t hdr[4] = { type, 0, (uint8_t)(len & 0xFF), (uint8_t)(len >> 8) };
        const uint8_t *parts[2] = { hdr, payload };
        size_t lens[2] = { sizeof(hdr), len };
        for (int i = 0; i < 2 && ret == ESP_OK; i++) {
            size_t off = 0;
            while (off < lens[i]) {
                int n = send(fd, parts[i] + off, lens[i] - off, 0);   // SO_SNDTIMEO bounds each
                if (n <= 0) { ret = (errno == EAGAIN) ? ESP_ERR_TIMEOUT : ESP_FAIL; break; }
                off += (size_t)n;
            }
```

```
        }
    }
    xSemaphoreGive(s_tx_mutex);
    return ret;
}

esp_err_t radio_relay_send_cat(const uint8_t *data, size_t len, uint32_t timeout_ms)
{
    if (!s_connected) return ESP_ERR_INVALID_STATE;
    return send_frame(PROTO_CAT, data, len, timeout_ms ? timeout_ms : 200);
}

// ---- RX task ---------------------------------------------------------------

static void rx_task(void *arg)
{
    (void)arg;
    // int16-aligned so IQ payloads can be handed to the audio ring directly.
    static int16_t payload16[RX_PAYLOAD_MAX / 2];
    uint8_t *payload = (uint8_t *)payload16;
    int fd = s_fd;

    while (!s_closing) {
        uint8_t hdr[4];
        if (recv_exact(fd, hdr, sizeof(hdr)) < 0) break;
        uint8_t type = hdr[0];
        size_t len = (size_t)hdr[2] | ((size_t)hdr[3] << 8);

        if (len > RX_PAYLOAD_MAX) {
            // Unknown/oversized frame: skip it in chunks to stay in sync.
            size_t left = len;
            bool ok = true;
            while (left && ok) {
                size_t n = left > RX_PAYLOAD_MAX ? RX_PAYLOAD_MAX : left;
                ok = recv_exact(fd, payload, n) == 0;
                left -= n;
            }
            if (!ok) break;
            continue;
        }
        if (len && recv_exact(fd, payload, len) < 0) break;

        switch (type) {
        case PROTO_IQ: {
            size_t pairs = len / 4;
            if (pairs) {
                audio_push_net_iq(payload16, pairs);
                s_iq_pairs_total += pairs;
            }
            break;
        }
        case PROTO_CAT:
            if (s_cat_cb && len) s_cat_cb(payload, len, NULL);
            break;
        case PROTO_TERM_DATA:
            if (s_term_cb && len) s_term_cb(payload, len, NULL);
            break;
        case PROTO_TERM_STATUS: {
            bool ok = len >= 1 && payload[0] == 1;
            if (len > 1) {
                size_t n = len - 1 < sizeof(s_term_err) - 1 ? len - 1 : sizeof(s_term_err) - 1;
                memcpy(s_term_err, payload + 1, n);
                s_term_err[n] = '\0';
            } else if (ok) {
                s_term_err[0] = '\0';
            }
            if (s_term_result == -1) {
                // The answer to our TERM_OPEN.
                s_term_result = ok ? 1 : 0;
                if (s_term_sem) xSemaphoreGive(s_term_sem);
            } else if (!ok && s_term_cb) {
                // Unsolicited "port gone" while a session is open.
                radio_relay_gone_cb_t g = s_term_gone_cb;
                s_term_cb = NULL;
                s_term_gone_cb = NULL;
                ESP_LOGW(TAG, "relay closed the terminal port: %s", s_term_err);
                if (g) g();
            }
            break;
        }
        case PROTO_TX_DONE:
            if (len >= 11) {
                s_txdone.done        = true;
                s_txdone.status      = payload[0];
                s_txdone.tones_sent  = (uint16_t)(payload[1] | (payload[2] << 8));
                s_txdone.duration_ms = (uint32_t)payload[3] | ((uint32_t)payload[4] << 8) |
                                       ((uint32_t)payload[5] << 16) | ((uint32_t)payload[6] << 24);
```

```
                s_txdone.latest_ms   = (int32_t)((uint32_t)payload[7] | ((uint32_t)payload[8] << 8) |
                                        ((uint32_t)payload[9] << 16) | ((uint32_t)payload[10] << 24));
                if (s_txdone_sem) xSemaphoreGive(s_txdone_sem);
            }
            break;
        default:   // HELLO / PING / unknown: nothing to do
            break;
        }
    }
    if (s_txdone_sem) xSemaphoreGive(s_txdone_sem);   // a waiting burst must not wait for a dead link

    bool unexpected = !s_closing;
    s_connected = false;
    if (s_term_result == -1 && s_term_sem) {        // a TERM_OPEN still waiting
        s_term_result = 0;
        snprintf(s_term_err, sizeof(s_term_err), "network link dropped");
        xSemaphoreGive(s_term_sem);
    }
    if (s_term_cb) {                                // an open terminal loses its port too
        radio_relay_gone_cb_t g = s_term_gone_cb;
        s_term_cb = NULL;
        s_term_gone_cb = NULL;
        if (g) g();
    }
    if (unexpected) {
        snprintf(s_last_err, sizeof(s_last_err), "link dropped (errno %d)", errno);
        ESP_LOGW(TAG, "relay link dropped (errno %d)", errno);
    }
    s_rx_task = NULL;
    if (unexpected && s_gone_cb) s_gone_cb();
    vTaskDelete(NULL);
}

// ---- connect / close ------------------------------------------------------

static void note_fail(const char *what, int code)
{
    snprintf(s_last_err, sizeof(s_last_err), "%s (%d)", what, code);
    // link_task retries every ~2 s; don't flood the log.
    int64_t now = esp_timer_get_time();
    if (now - s_last_fail_log_us > 30LL * 1000 * 1000) {
        s_last_fail_log_us = now;
        ESP_LOGW(TAG, "connect to %s:%u failed: %s", s_host, s_port, s_last_err);
    }
}

esp_err_t radio_relay_connect(radio_relay_cat_rx_cb_t cat_cb,
                              radio_relay_gone_cb_t gone_cb,
                              uint32_t timeout_ms)
{
    if (s_connected || s_rx_task) return ESP_ERR_INVALID_STATE;
    if (!s_tx_mutex) {
        s_tx_mutex = xSemaphoreCreateMutex();
        if (!s_tx_mutex) return ESP_ERR_NO_MEM;
    }

    char host[RADIO_RELAY_HOST_LEN];
    uint16_t port;
    radio_relay_get_target(host, sizeof(host), &port);
    if (!host[0]) return ESP_ERR_INVALID_STATE;

    /* (!) No lwIP call until WiFi is up. link_task starts inside cat_init(),
     * seconds before wifi_init() has created the tcpip thread, and getaddrinfo()
     * on a stack that does not exist yet asserts in tcpip_send_msg_wait_sem
     * ("Invalid mbox") - a boot loop as soon as a NAS address is saved
     * (hardware-observed 2026-10-04, cat_link at 4.0 s uptime). Not connected
     * yet is an ordinary "try again": link_task retries every ~2 s. */
    if (!wifi_is_connected()) {
        note_fail("waiting for WiFi", 0);
        return ESP_ERR_INVALID_STATE;
    }

    char port_str[8];
    snprintf(port_str, sizeof(port_str), "%u", port);
    struct addrinfo hints = { .ai_family = AF_INET, .ai_socktype = SOCK_STREAM };
    struct addrinfo *res = NULL;
    int gai = getaddrinfo(host, port_str, &hints, &res);
    if (gai != 0 || !res) {
        note_fail("DNS/WiFi not ready", gai);
        if (res) freeaddrinfo(res);
        return ESP_ERR_NOT_FOUND;
    }

    int fd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
    if (fd < 0) {
        freeaddrinfo(res);
        note_fail("no free socket", errno);
```

```
        return ESP_ERR_NO_MEM;
    }

    // Non-blocking connect bounded by timeout_ms.
    int fl = fcntl(fd, F_GETFL, 0);
    fcntl(fd, F_SETFL, fl | O_NONBLOCK);
    int rc = connect(fd, res->ai_addr, res->ai_addrlen);
    freeaddrinfo(res);
    if (rc != 0 && errno != EINPROGRESS) {
        note_fail("connect", errno);
        close(fd);
        return ESP_FAIL;
    }
    if (rc != 0) {
        fd_set wfds;
        FD_ZERO(&wfds);
        FD_SET(fd, &wfds);
        struct timeval tv = { .tv_sec = timeout_ms / 1000, .tv_usec = (timeout_ms % 1000) * 1000 };
        int sel = select(fd + 1, NULL, &wfds, NULL, &tv);
        int soerr = 0;
        socklen_t sl = sizeof(soerr);
        if (sel <= 0 || getsockopt(fd, SOL_SOCKET, SO_ERROR, &soerr, &sl) != 0 || soerr != 0) {
            note_fail(sel <= 0 ? "connect timeout" : "connect refused", soerr);
            close(fd);
            return ESP_ERR_TIMEOUT;
        }
    }
    fcntl(fd, F_SETFL, fl & ~O_NONBLOCK);

    int one = 1;
    setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &one, sizeof(one));
    setsockopt(fd, SOL_SOCKET, SO_KEEPALIVE, &one, sizeof(one));
    struct timeval rto = { .tv_sec = RX_IDLE_TIMEOUT_S, .tv_usec = 0 };
    setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &rto, sizeof(rto));
    struct timeval sto = { .tv_sec = 0, .tv_usec = 200 * 1000 };
    setsockopt(fd, SOL_SOCKET, SO_SNDTIMEO, &sto, sizeof(sto));

    s_fd = fd;
    s_closing = false;

    // Handshake: say hello, expect "QMXR/1 ..." back as the first frame. The
    // NAS relay only answers once it has actually opened the QMX, so a
    // completed handshake means the radio is there.
    static const char hello[] = "QMXR/1 tab5-qmx-panadapter";
    uint8_t hdr[4];
    char resp[96];
    size_t len;
    if (send_frame(PROTO_HELLO, (const uint8_t *)hello, sizeof(hello) - 1, 500) != ESP_OK) {
        note_fail("hello send", errno);
        goto fail;
    }
    if (recv_exact(fd, hdr, sizeof(hdr)) < 0) { note_fail("no hello (QMX not on NAS?)", errno); goto fail; }
    len = (size_t)hdr[2] | ((size_t)hdr[3] << 8);
    if (hdr[0] != PROTO_HELLO || len == 0 || len >= sizeof(resp)) {
        note_fail("not a QMXR/1 relay", hdr[0]);
        goto fail;
    }
    if (recv_exact(fd, resp, len) < 0) { note_fail("hello recv", errno); goto fail; }
    resp[len] = '\0';
    if (strncmp(resp, "QMXR/1", 6) != 0) { note_fail("bad protocol", 0); goto fail; }
    s_term_supported = strstr(resp, "term=1") != NULL;
    s_txsched_supported = strstr(resp, "txsched=1") != NULL;
    if (!strstr(resp, "rate=48000")) {
        ESP_LOGW(TAG, "relay rate is not 48000 Hz (%s) - spectrum scale will be off", resp);
    }

    s_cat_cb = cat_cb;
    s_gone_cb = gone_cb;
    s_connected = true;
    if (xTaskCreatePinnedToCore(rx_task, "relay_rx", 4096, NULL, 5, &s_rx_task, 1) != pdPASS) {
        s_connected = false;
        note_fail("rx task", 0);
        goto fail;
    }
    audio_request_reset();
    snprintf(s_last_err, sizeof(s_last_err), "ok");
    ESP_LOGI(TAG, "connected to %s:%u (%s)", host, port, resp);
    return ESP_OK;

fail:
    s_fd = -1;
    close(fd);
    return ESP_FAIL;
}

void radio_relay_close(void)
{
```

```
    s_closing = true;
    int fd = s_fd;
    if (fd >= 0) shutdown(fd, SHUT_RDWR);
    // Let the RX task notice and exit before the fd is released.
    for (int i = 0; i < 100 && s_rx_task != NULL; i++) vTaskDelay(pdMS_TO_TICKS(20));
    if (s_tx_mutex) xSemaphoreTake(s_tx_mutex, pdMS_TO_TICKS(500));
    if (s_fd >= 0) { close(s_fd); s_fd = -1; }
    if (s_tx_mutex) xSemaphoreGive(s_tx_mutex);
    s_connected = false;
    s_cat_cb = NULL;
    s_gone_cb = NULL;
    s_term_cb = NULL;
    s_term_gone_cb = NULL;
    s_term_supported = false;
    s_txsched_supported = false;
    ESP_LOGI(TAG, "relay link closed");
}

// ---- second serial port (terminal) ------------------------------------------

bool radio_relay_term_supported(void) { return s_connected && s_term_supported; }
const char *radio_relay_term_error(void) { return s_term_err; }

esp_err_t radio_relay_term_open(radio_relay_cat_rx_cb_t data_cb,
                                radio_relay_gone_cb_t gone_cb,
                                uint32_t timeout_ms)
{
    if (!s_connected) return ESP_ERR_INVALID_STATE;
    if (!s_term_supported) {
        snprintf(s_term_err, sizeof(s_term_err), "NAS relay too old (no terminal support)");
        return ESP_ERR_NOT_SUPPORTED;
    }
    if (!s_term_sem) {
        s_term_sem = xSemaphoreCreateBinary();
        if (!s_term_sem) return ESP_ERR_NO_MEM;
    }
    xSemaphoreTake(s_term_sem, 0);          // clear any stale give
    // Install the data path FIRST: the radio repaints as soon as it is open.
    s_term_cb = data_cb;
    s_term_gone_cb = gone_cb;
    s_term_result = -1;
    esp_err_t e = send_frame(PROTO_TERM_OPEN, NULL, 0, 500);
    if (e == ESP_OK && xSemaphoreTake(s_term_sem, pdMS_TO_TICKS(timeout_ms)) != pdTRUE) {
        snprintf(s_term_err, sizeof(s_term_err), "no answer from the NAS relay");
        e = ESP_ERR_TIMEOUT;
    }
    if (e == ESP_OK && s_term_result != 1) e = ESP_ERR_NOT_FOUND;
    if (e != ESP_OK) {
        s_term_cb = NULL;
        s_term_gone_cb = NULL;
        if (s_term_result == -1) s_term_result = 0;
        ESP_LOGW(TAG, "terminal port over the network not available: %s", s_term_err);
        return e;
    }
    ESP_LOGI(TAG, "QMX second serial port open on the NAS relay");
    return ESP_OK;
}

esp_err_t radio_relay_term_send(const uint8_t *data, size_t len, uint32_t timeout_ms)
{
    if (!s_connected || !s_term_cb) return ESP_ERR_INVALID_STATE;
    return send_frame(PROTO_TERM_DATA, data, len, timeout_ms ? timeout_ms : 300);
}

void radio_relay_term_close(void)
{
    bool was_open = s_term_cb != NULL;
    s_term_cb = NULL;
    s_term_gone_cb = NULL;
    if (was_open && s_connected) send_frame(PROTO_TERM_CLOSE, NULL, 0, 300);
}

// ---- Discovery --------------------------------------------------------------
// See radio_relay.h. One UDP socket, three broadcasts ~a third of the timeout
// apart (a single datagram on WiFi can be lost), replies collected until the
// timeout. The reply carries the relay's own idea of its address; the packet's
// source address is used if that does not parse.

int radio_relay_discover(radio_relay_found_t *out, int max, uint32_t timeout_ms)
{
    if (!out || max <= 0) return 0;
    if (!wifi_is_connected()) return -1;

    int s = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (s < 0) return -1;
    int one = 1;
    setsockopt(s, SOL_SOCKET, SO_BROADCAST, &one, sizeof(one));
```

```
    struct timeval tv = { .tv_sec = 0, .tv_usec = 100 * 1000 };
    setsockopt(s, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

    struct sockaddr_in bc = { 0 };
    bc.sin_family      = AF_INET;
    bc.sin_port        = htons(RADIO_RELAY_DEFAULT_PORT);
    bc.sin_addr.s_addr = htonl(INADDR_BROADCAST);

    int n = 0, sent = 0;
    const int64_t start = esp_timer_get_time();
    const int64_t total = (int64_t)timeout_ms * 1000;
    while (esp_timer_get_time() - start < total) {
        if (sent < 3 && esp_timer_get_time() - start >= sent * total / 3) {
            if (sendto(s, "QMXR?", 5, 0, (struct sockaddr *)&bc, sizeof(bc)) < 0)
                ESP_LOGW(TAG, "discovery: broadcast failed (errno %d)", errno);
            sent++;
        }
        char buf[64];
        struct sockaddr_in from;
        socklen_t fl = sizeof(from);
        int r = recvfrom(s, buf, sizeof(buf) - 1, 0, (struct sockaddr *)&from, &fl);
        if (r <= 0) continue;
        buf[r] = '\0';
        if (strncmp(buf, "QMXR! ", 6) != 0) continue;

        char ip[16] = "";
        unsigned port = RADIO_RELAY_DEFAULT_PORT;
        struct in_addr chk;
        if (sscanf(buf + 6, "%15s %u", ip, &port) < 1 || inet_aton(ip, &chk) == 0)
            inet_ntoa_r(from.sin_addr, ip, sizeof(ip));
        if (port == 0 || port > 65535) port = RADIO_RELAY_DEFAULT_PORT;

        bool dup = false;
        for (int i = 0; i < n; i++)
            if (strcmp(out[i].ip, ip) == 0 && out[i].port == port) dup = true;
        if (!dup && n < max) {
            snprintf(out[n].ip, sizeof(out[n].ip), "%s", ip);
            out[n].port = (uint16_t)port;
            n++;
        }
    }
    close(s);
    ESP_LOGI(TAG, "discovery: %d relay(s) answered", n);
    return n;
}

// ---- relay-timed transmit bursts ---------------------------------------------

bool radio_relay_txsched_supported(void) { return s_connected && s_txsched_supported; }

esp_err_t radio_relay_tx_schedule(const float *freq_hz, int n, uint32_t symbol_us)
{
    if (!radio_relay_txsched_supported()) return ESP_ERR_NOT_SUPPORTED;
    if (!freq_hz || n <= 0 || n > 400) return ESP_ERR_INVALID_ARG;
    if (!s_txdone_sem) {
        s_txdone_sem = xSemaphoreCreateBinary();
        if (!s_txdone_sem) return ESP_ERR_NO_MEM;
    }
    xSemaphoreTake(s_txdone_sem, 0);              // drop a stale signal
    memset(&s_txdone, 0, sizeof(s_txdone));
    /* u8 version, u8 flags, u16 n, u32 symbol_us, then n x u32 centi-Hz -
     * centi-Hz so the relay can print exactly the "TA%.2f;" the Tab5 would. */
    size_t len = 8 + (size_t)n * 4;
    uint8_t *p = heap_caps_malloc(len, MALLOC_CAP_8BIT);
    if (!p) return ESP_ERR_NO_MEM;
    p[0] = 1; p[1] = 0; p[2] = (uint8_t)n; p[3] = (uint8_t)(n >> 8);
    for (int b = 0; b < 4; b++) p[4 + b] = (uint8_t)(symbol_us >> (8 * b));
    for (int i = 0; i < n; i++) {
        uint32_t c = (uint32_t)(freq_hz[i] * 100.0f + 0.5f);
        for (int b = 0; b < 4; b++) p[8 + i * 4 + b] = (uint8_t)(c >> (8 * b));
    }
    esp_err_t e = send_frame(PROTO_TX_SCHED, p, len, 500);
    heap_caps_free(p);
    return e;
}

esp_err_t radio_relay_tx_abort(void)
{
    if (!s_connected) return ESP_ERR_INVALID_STATE;
    return send_frame(PROTO_TX_ABORT, NULL, 0, 500);
}

bool radio_relay_tx_wait_done(uint32_t timeout_ms, radio_relay_tx_result_t *out)
{
    bool got = s_txdone_sem && xSemaphoreTake(s_txdone_sem, pdMS_TO_TICKS(timeout_ms)) == pdTRUE
               && s_txdone.done;
    if (out) { *out = s_txdone; out->done = got; }
```

```
    return got;
}
```

main/cat/cat.h

changes against upstream v1.18.0 (unified diff)

```
@@ -660,3 +660,15 @@ bool cat_force_rx_pending(void);
  * Reached from POST /api/cmd {"action":"band_scan_restore_test", ...}. Nothing
  * calls it in normal operation; it sends no TX. */
 void cat_band_scan_restore_test(uint32_t pre_freq, char pre_mode_digit);
+
+/**
+ * @brief Drop the current CAT/IQ link so link_task re-selects the transport.
+ *
+ * Call after changing the network-radio target (radio_relay_set_target()):
+ * link_task uses the QMX on USB when no NAS host is set, otherwise the QMX on
+ * the NAS (tools/nas-relay). No-op when no link is up.
+ */
+void cat_reconnect(void);
+
+/** @brief "usb", "net" or "none" - which transport the CAT link is using. */
+const char *cat_get_transport(void);
```

main/cat/cat.c

changes against upstream v1.18.0 (unified diff)

```
@@ -19,6 +19,8 @@

 #include "usb/usb_host.h"
 #include "usb/cdc_acm_host.h"
+#include "radio_relay.h"
+#include "audio.h"          // audio_request_reset - network link teardown
 #include "bsp/m5stack_tab5.h"

 #include "wspr_rx.h"   // wspr_pa_guard_release_pending - see the VN; handler
@@ -55,6 +57,21 @@ static const char *TAG = "cat";
 static TaskHandle_t s_poll_task = NULL;
 static EventGroupHandle_t s_evt_group = NULL;
 static cdc_acm_dev_hdl_t s_cdc_dev = NULL;
+// Network radio source (QMX plugged into a NAS, QMXR/1 relay - see
+// net/radio_relay.h and tools/nas-relay). When true the CAT link is a TCP
+// session to the NAS instead of the USB CDC-ACM device; every command path
+// goes through cat_tx() so it doesn't care which. The QMX's optional SECOND
+// serial port (terminal, qmx_term.c) is USB-only and simply isn't available
+// over the network.
+static volatile bool s_net_link = false;
+#define CAT_LINK_UP() (s_cdc_dev != NULL || s_net_link)
+static esp_err_t cat_tx(const uint8_t *buf, size_t len, uint32_t timeout_ms)
+{
+    if (s_net_link) return radio_relay_send_cat(buf, len, timeout_ms);
+    cdc_acm_dev_hdl_t dev = s_cdc_dev;
+    if (!dev) return ESP_ERR_INVALID_STATE;
+    return cdc_acm_host_data_tx_blocking(dev, buf, len, timeout_ms);
+}
 static volatile bool s_cat_ready = false;

 bool cat_is_ready(void)
@@ -584,7 +601,7 @@ static volatile bool s_tune_poll_active = false;
 void cat_tune_poll_set_active(bool active) { s_tune_poll_active = active; }
 esp_err_t cat_send_raw_cmd(const char *fmt, ...)
 {
-    if (!s_cdc_dev) return ESP_ERR_INVALID_STATE;
+    if (!CAT_LINK_UP()) return ESP_ERR_INVALID_STATE;
     char buf[64];
     va_list ap;
     va_start(ap, fmt);
@@ -599,17 +616,17 @@ esp_err_t cat_send_raw_cmd(const char *fmt, ...)
      * 2026-08-29: a burst at a hand-set 7.5 V announced PA=15.0 V. A label that
      * can be stale is worse than one that admits it does not know. */
     if (strstr(buf, "Max. PA voltage")) s_pa_voltage_x10 = -1;
-    return cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)buf, len, 200);
+    return cat_tx((const uint8_t *)buf, len, 200);
 }

 esp_err_t cat_query_power_swr(float *power_w, float *swr)
 {
-    if (!s_cdc_dev) return ESP_ERR_INVALID_STATE;
+    if (!CAT_LINK_UP()) return ESP_ERR_INVALID_STATE;

     s_pc_resp_len = 0;
     s_sw_resp_len = 0;

-    esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"PC;SW;", 6, 200);
+    esp_err_t err = cat_tx((const uint8_t *)"PC;SW;", 6, 200);
     if (err != ESP_OK) return err;
     for (int wi = 0; wi < 20 && (s_pc_resp_len == 0 || s_sw_resp_len == 0); wi++) {
@@ -648,10 +665,10 @@ esp_err_t cat_query_power_swr(float *power_w, float *swr)
 // keyed, same as the unchanged end-of-burst query.
```

```
 esp_err_t cat_pwr_swr_async_send(void)
 {
-    if (!s_cdc_dev) return ESP_ERR_INVALID_STATE;
+    if (!CAT_LINK_UP()) return ESP_ERR_INVALID_STATE;
     s_pc_resp_len = 0;
     s_sw_resp_len = 0;
-    return cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"PC;SW;", 6, 50);
+    return cat_tx((const uint8_t *)"PC;SW;", 6, 50);
 }

 // Parse whatever response has arrived since the last _send(). Returns ESP_OK
@@ -1431,7 +1448,7 @@ static esp_err_t try_open_qmx(void)
     cdc_acm_host_set_control_line_state(s_cdc_dev, true, true);
     ESP_LOGI(TAG, "QMX configured: %d baud, 8N1", CAT_BAUD_RATE);

-    cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"ID;", 3, 500);
+    cat_tx((const uint8_t *)"ID;", 3, 500);

     // Phase 3.1 — dump audio descriptors ONCE per session
     if (!s_audio_dumped) {
@@ -1576,7 +1593,7 @@ static void cw_filters_read(void)
         char q[40];
         int  n = snprintf(q, sizeof(q), "MMCW|Choose filters|%d;", i);
         s_mm_resp_len = 0;
-        if (n <= 0 || cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)q,
+        if (n <= 0 || cat_tx((const uint8_t *)q,
                                                     (size_t)n, 200) != ESP_OK) {
             ESP_LOGW(TAG, "CW filters: write failed at index %d - offering all widths", i);
             s_cw_filter_mask = 0;
@@ -1613,7 +1630,7 @@ static bool iq_mode_handshake(int max_attempts);

 bool cat_apply_cw_profile(uint16_t centre_hz, uint8_t mask)
 {
-    if (!s_cdc_dev || !centre_hz) return false;
+    if (!CAT_LINK_UP() || !centre_hz) return false;
     s_pending_prof_mask   = mask;
     s_pending_prof_centre = centre_hz;   /* set LAST - it is the "go" flag */
     return true;
@@ -1644,12 +1661,12 @@ static bool cw_center_write_confirmed(uint16_t centre_hz)
         char cmd[40];
         int n = snprintf(cmd, sizeof(cmd), "MMCW|CW center=%u;", (unsigned)centre_hz);
         if (n > 0)
-            cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, (size_t)n, 200);
+            cat_tx((const uint8_t *)cmd, (size_t)n, 200);
         vTaskDelay(pdMS_TO_TICKS(200));      /* let the menu redraw finish */

         s_mm_resp_len = 0;
         const char *q = "MMCW|CW center;";
-        if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)q,
+        if (cat_tx((const uint8_t *)q,
                                           strlen(q), 200) != ESP_OK) continue;
         for (int wi = 0; wi < 30 && s_mm_resp_len == 0; wi++) vTaskDelay(pdMS_TO_TICKS(10));
         if (s_mm_resp_len >= 4 && strncmp(s_mm_resp, "MM", 2) == 0 &&
@@ -1692,7 +1709,7 @@ static bool cw_profile_apply_pending(void)
         int n = snprintf(cmd, sizeof(cmd), "MMCW|Choose filters|%d=%s;",
                          i, (mask & (1u << i)) ? "ENABLED" : "DISABLED");
         if (n > 0)
-            cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, (size_t)n, 200);
+            cat_tx((const uint8_t *)cmd, (size_t)n, 200);
         /* 120 ms, not 40: an MM write sprays a menu redraw back at us and the
          * next command must not arrive into it. 40 ms was measured refusing the
          * write that followed the eighth row. */
@@ -1707,7 +1724,7 @@ static bool cw_profile_apply_pending(void)
      * on the WSPR PA guard, where the read-back agreed and the radio still ran
      * at full power. */
     const char *mu = "MU;";
-    cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)mu, 3, 200);
+    cat_tx((const uint8_t *)mu, 3, 200);
     vTaskDelay(pdMS_TO_TICKS(250));

     /* (!) AND MU; DROPS IQ MODE. Q9 is session state, so the reload leaves the
@@ -1742,7 +1759,7 @@ static bool cw_offset_refresh(void)

     const char *q = "MMCW|CW offset;";
     s_mm_resp_len = 0;
-    if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)q, strlen(q), 200) != ESP_OK) {
+    if (cat_tx((const uint8_t *)q, strlen(q), 200) != ESP_OK) {
         return true;   // transient; the poll's own error handling owns the link
     }
     for (int wi = 0; wi < 10 && s_mm_resp_len == 0; wi++) vTaskDelay(pdMS_TO_TICKS(10));
@@ -1810,11 +1827,11 @@ static bool cw_split_maintain(void)
         if (base != 0) {
             char fb[20];
             int n = snprintf(fb, sizeof fb, "FB%011lu;", (unsigned long)base);
-            efb = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)fb, (size_t)n, 200);
+            efb = cat_tx((const uint8_t *)fb, (size_t)n, 200);
             vTaskDelay(pdMS_TO_TICKS(30));
```

```
         }
         // Now stand down: back to simplex, transmitting where we listen.
-        esp_err_t e = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"SP0;", 4, 200);
+        esp_err_t e = cat_tx((const uint8_t *)"SP0;", 4, 200);
         // ⚠ AND PUT THE VFO MODE BACK, which SP0; does NOT do. The QMX has a single
         // three-state VFO Mode - A / B / Split - and the CAT manual is explicit that
         // it is FR/FT that select it: "0, 1, 2 correspond to VFO A, VFO B or Split
@@ -1825,7 +1842,7 @@ static bool cw_split_maintain(void)
         // been "fixed" twice: B matched A, split read off, and the display was still
         // A/B. FR0; is what actually returns the radio to plain VFO A.
         vTaskDelay(pdMS_TO_TICKS(30));
-        esp_err_t efr = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"FR0;", 4, 200);
+        esp_err_t efr = cat_tx((const uint8_t *)"FR0;", 4, 200);
         // The manual says FR and FT both select the same three-state VFO Mode, so send
         // BOTH - and then ASK, because FR0; alone demonstrably did not clear the A/B
         // display and I am not going to guess a fourth time. The answers arrive as
@@ -1833,16 +1850,16 @@ static bool cw_split_maintain(void)
         // for "FA"), which is enough to tell whether the mode really is 0 and the
         // display is driven by something else, or the write is simply not landing.
         vTaskDelay(pdMS_TO_TICKS(30));
-        cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"FT0;", 4, 200);
+        cat_tx((const uint8_t *)"FT0;", 4, 200);
         vTaskDelay(pdMS_TO_TICKS(30));
-        cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"FR;", 3, 200);
+        cat_tx((const uint8_t *)"FR;", 3, 200);
         vTaskDelay(pdMS_TO_TICKS(30));
-        cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"FT;", 3, 200);
+        cat_tx((const uint8_t *)"FT;", 3, 200);
         // Ask the radio to confirm simplex, judged on a later cycle above.
         vTaskDelay(pdMS_TO_TICKS(30));
         s_split_readback = -1;
         s_split_verify_pending = true;
-        cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"SP;", 3, 200);
+        cat_tx((const uint8_t *)"SP;", 3, 200);
         s_split_engaged = false;
         s_split_warned = false;
         ESP_LOGI(TAG, "CW TX offset off - VFO B -> %lu (%s), split off (%s), VFO mode A (%s)",
@@ -1871,20 +1888,20 @@ static bool cw_split_maintain(void)
     if (tx < 0) tx = 0;
     char cmd[20];
     int n = snprintf(cmd, sizeof cmd, "FB%011lld;", (long long)tx);
-    esp_err_t e1 = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, (size_t)n, 200);
+    esp_err_t e1 = cat_tx((const uint8_t *)cmd, (size_t)n, 200);
     vTaskDelay(pdMS_TO_TICKS(30));
     // SP1 is re-sent with every FB, not just on the first one: it is one short
     // command, and it is the difference between "the offset is applied" and
     // "we quietly transmitted on top of the station" if the radio dropped split
     // while we were not looking.
-    esp_err_t e2 = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"SP1;", 4, 200);
+    esp_err_t e2 = cat_tx((const uint8_t *)"SP1;", 4, 200);
     // Ask what actually happened. The answer lands asynchronously in
     // s_split_readback and is judged on a later cycle, so this costs one short
     // write and no waiting.
     vTaskDelay(pdMS_TO_TICKS(30));
     s_split_readback = -1;
     s_split_warned = false;
-    cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"SP;", 3, 200);
+    cat_tx((const uint8_t *)"SP;", 3, 200);
     s_split_engaged = true;
     s_split_base_hz = base;
     s_split_last_us = now;
@@ -1911,8 +1928,7 @@ static bool iq_mode_handshake(int max_attempts)
     s_iq_mode_confirmed = false;
     for (int attempt = 1; attempt <= max_attempts; attempt++) {
         const char *iq_on = "Q9 1;";
-        esp_err_t terr = cdc_acm_host_data_tx_blocking(
-            s_cdc_dev, (const uint8_t *)iq_on, 5, 200);
+        esp_err_t terr = cat_tx((const uint8_t *)iq_on, 5, 200);
         if (terr == ESP_OK) {
             ESP_LOGI(TAG, "QMX IQ mode enabled (Q9 1;) attempt %d/%d",
                      attempt, max_attempts);
@@ -1934,8 +1950,7 @@ static bool iq_mode_handshake(int max_attempts)
         vTaskDelay(pdMS_TO_TICKS(150));
         s_q9_resp_len = 0;
         const char *iq_q = "Q9;";
-        esp_err_t qerr = cdc_acm_host_data_tx_blocking(
-            s_cdc_dev, (const uint8_t *)iq_q, strlen(iq_q), 200);
+        esp_err_t qerr = cat_tx((const uint8_t *)iq_q, strlen(iq_q), 200);
         if (qerr == ESP_OK) {
             for (int wi = 0; wi < 20 && s_q9_resp_len == 0; wi++) {
                 vTaskDelay(pdMS_TO_TICKS(20));
@@ -2001,7 +2016,7 @@ static void poll_task(void *arg)

     int phase = 0;
     int poll_fail = 0;   // consecutive poll-TX failures; one transient timeout must not kill the poll
-    while (s_cdc_dev != NULL) {
+    while (CAT_LINK_UP()) {
```

```
         /* RX watchdog - see the note above the declaration. Checked once a
          * second so the arithmetic costs nothing at the 50 ms poll rate, and
          * only while we are actually polling: a paused link (FT8 burst, the
@@ -2085,7 +2100,7 @@ static void poll_task(void *arg)
             s_pending_mode_digit = 0;
             char cmd[8];
             cmd[0] = 'M'; cmd[1] = 'D'; cmd[2] = md; cmd[3] = ';'; cmd[4] = 0;
-            esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, 4, 200);
+            esp_err_t err = cat_tx((const uint8_t *)cmd, 4, 200);
             ESP_LOGI(TAG, "mode -> MD%c; (%s)", md, err == ESP_OK ? "ok" : "fail");
             vTaskDelay(pdMS_TO_TICKS(CAT_POLL_INTERVAL_MS));
             continue;
@@ -2093,7 +2108,7 @@ static void poll_task(void *arg)
         if (s_af_gain_query_pending) {
             s_af_gain_query_pending = false;
             static const char q[] = "AG;";
-            esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)q, 3, 200);
+            esp_err_t err = cat_tx((const uint8_t *)q, 3, 200);
             ESP_LOGI(TAG, "AF gain query AG; (%s)", err == ESP_OK ? "sent" : "fail");
             vTaskDelay(pdMS_TO_TICKS(CAT_POLL_INTERVAL_MS));
             continue;
@@ -2108,7 +2123,7 @@ static void poll_task(void *arg)
             // is in 0.25 dB steps, range 0-799 per the manual.
             char cmd[16];
             int n = snprintf(cmd, sizeof cmd, "AG0%03u;", ag);
-            esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd,
+            esp_err_t err = cat_tx((const uint8_t *)cmd,
                                                          (size_t)n, 200);
             ESP_LOGI(TAG, "AF gain -> %s (%.2f dB) (%s)", cmd, ag * 0.25,
                     err == ESP_OK ? "ok" : "fail");
@@ -2119,7 +2134,7 @@ static void poll_task(void *arg)
         if (s_keyer_query_pending) {
             s_keyer_query_pending = false;
             static const char q[] = "KS;";
-            esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)q, 3, 200);
+            esp_err_t err = cat_tx((const uint8_t *)q, 3, 200);   /* network radio: cat_tx, not s_cdc_dev */
             ESP_LOGI(TAG, "Keyer speed query KS; (%s)", err == ESP_OK ? "sent" : "fail");
             vTaskDelay(pdMS_TO_TICKS(CAT_POLL_INTERVAL_MS));
             continue;
@@ -2132,8 +2147,7 @@ static void poll_task(void *arg)
              * same shape its AG/RG commands use. */
             char cmd[16];
             int n = snprintf(cmd, sizeof cmd, "KS%03u;", ks);
-            esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd,
-                                                         (size_t)n, 200);
+            esp_err_t err = cat_tx((const uint8_t *)cmd, (size_t)n, 200);   /* network radio */
             ESP_LOGI(TAG, "Keyer speed -> %s (%s)", cmd, err == ESP_OK ? "ok" : "fail");
             /* (!) ALWAYS READ BACK. A successful CDC write only proves the
              * bytes reached the radio - the lesson Q9/IQ mode taught the hard
@@ -2146,7 +2160,7 @@ static void poll_task(void *arg)
         if (s_rf_gain_query_pending) {
             s_rf_gain_query_pending = false;
             static const char q[] = "RG;";
-            esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)q, 3, 200);
+            esp_err_t err = cat_tx((const uint8_t *)q, 3, 200);
             ESP_LOGI(TAG, "RF gain query RG; (%s)", err == ESP_OK ? "sent" : "fail");
             vTaskDelay(pdMS_TO_TICKS(CAT_POLL_INTERVAL_MS));
             continue;
@@ -2158,21 +2172,21 @@ static void poll_task(void *arg)
             // covers hz == 0 on its own, which is how RIT gets switched off.
             char cmd[24];
             int n = snprintf(cmd, sizeof cmd, "RC;");
-            esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd,
+            esp_err_t err = cat_tx((const uint8_t *)cmd,
                                                          (size_t)n, 200);
             if (err == ESP_OK && hz != 0) {
                 vTaskDelay(pdMS_TO_TICKS(20));
                 unsigned mag = (unsigned)(hz < 0 ? -hz : hz);
                 n = snprintf(cmd, sizeof cmd, "%s%03u;", hz > 0 ? "RU" : "RD", mag);
-                err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd,
+                err = cat_tx((const uint8_t *)cmd,
                                                     (size_t)n, 200);
                 vTaskDelay(pdMS_TO_TICKS(20));
                 n = snprintf(cmd, sizeof cmd, "RT1;");
-                cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, (size_t)n, 200);
+                cat_tx((const uint8_t *)cmd, (size_t)n, 200);
             } else if (err == ESP_OK) {
                 vTaskDelay(pdMS_TO_TICKS(20));
                 n = snprintf(cmd, sizeof cmd, "RT0;");
-                cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, (size_t)n, 200);
+                cat_tx((const uint8_t *)cmd, (size_t)n, 200);
             }
             if (err == ESP_OK) s_rit_hz = hz;
             ESP_LOGI(TAG, "RIT -> %+d Hz (%s)", hz, err == ESP_OK ? "ok" : "fail");
@@ -2190,7 +2204,7 @@ static void poll_task(void *arg)
             // re-reads on open rather than replaying a stored number).
             char cmd[16];
```

```
             int n = snprintf(cmd, sizeof cmd, "RG%03u;", rg);
-            esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd,
+            esp_err_t err = cat_tx((const uint8_t *)cmd,
                                                           (size_t)n, 200);
             ESP_LOGI(TAG, "RF gain -> %s (%s)", cmd, err == ESP_OK ? "ok" : "fail");
             // Confirm it from HERE, after the value has gone out. Queued from the
@@ -2211,7 +2225,7 @@ static void poll_task(void *arg)
             s_split_query_pending = false;
             if (!s_split_engaged) {
                 s_split_readback = -1;
-                cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"SP;", 3, 200);
+                cat_tx((const uint8_t *)"SP;", 3, 200);
                 vTaskDelay(pdMS_TO_TICKS(CAT_POLL_INTERVAL_MS));
                 continue;
             }
@@ -2220,7 +2234,7 @@ static void poll_task(void *arg)
             s_pa_query_pending = false;
             const char *q = "MMProtection|Max. PA voltage;";
             s_pa_awaiting_reply = true;
-            if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)q,
+            if (cat_tx((const uint8_t *)q,
                                               strlen(q), 200) != ESP_OK) {
                 s_pa_awaiting_reply = false;   /* nothing is coming */
             }
@@ -2233,7 +2247,7 @@ static void poll_task(void *arg)
             int n = snprintf(mm, sizeof(mm), "MMProtection|Max. PA voltage=%u.%u;",
                              (unsigned)(pav / 10), (unsigned)(pav % 10));
-            esp_err_t e = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)mm, n, 200);
+            esp_err_t e = cat_tx((const uint8_t *)mm, n, 200);
             ESP_LOGW(TAG, "PA voltage -> %u.%u V (%s)", (unsigned)(pav / 10),
                     (unsigned)(pav % 10), e == ESP_OK ? "ok" : "fail");
             /* (*) AND NOW MAKE THE RADIO ACTUALLY USE IT.
@@ -2258,7 +2272,7 @@ static void poll_task(void *arg)
             if (e == ESP_OK) {
                 vTaskDelay(pdMS_TO_TICKS(120));
                 const char *mu = "MU;";
-                if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)mu, 3, 200) == ESP_OK) {
+                if (cat_tx((const uint8_t *)mu, 3, 200) == ESP_OK) {
                     ESP_LOGW(TAG, "PA voltage: MU; sent - config reloaded so the "
                                   "new limit takes effect (MM Effect may be 'On demand')");
                     vTaskDelay(pdMS_TO_TICKS(250));
@@ -2285,10 +2299,10 @@ static void poll_task(void *arg)
             // Write both to the same value: live applies AND there's nothing
             // for the FW; poll to revert to. Result sticks and persists.
             int n = snprintf(mm, sizeof(mm), "MMSSB|Filter RX=%lu;", (unsigned long)bw);
-            esp_err_t e1 = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)mm, n, 200);
+            esp_err_t e1 = cat_tx((const uint8_t *)mm, n, 200);
             vTaskDelay(pdMS_TO_TICKS(40));
             n = snprintf(mm, sizeof(mm), "MMSSB|Bandwidth=%lu;", (unsigned long)bw);
-            esp_err_t e2 = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)mm, n, 200);
+            esp_err_t e2 = cat_tx((const uint8_t *)mm, n, 200);
             ESP_LOGI(TAG, "SSB filter -> %lu Hz (RX=%s, BW=%s)", (unsigned long)bw,
                     e1 == ESP_OK ? "ok" : "fail", e2 == ESP_OK ? "ok" : "fail");
             vTaskDelay(pdMS_TO_TICKS(CAT_POLL_INTERVAL_MS));
@@ -2299,7 +2313,7 @@ static void poll_task(void *arg)
             s_pending_cw_passband = 0;
             char mm[32];
             int n = snprintf(mm, sizeof(mm), "MMCW|CW passband=%lu;", (unsigned long)cwbw);
-            esp_err_t e = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)mm, n, 200);
+            esp_err_t e = cat_tx((const uint8_t *)mm, n, 200);
             ESP_LOGI(TAG, "CW passband -> %lu Hz (%s)", (unsigned long)cwbw,
                     e == ESP_OK ? "ok" : "fail");
             vTaskDelay(pdMS_TO_TICKS(CAT_POLL_INTERVAL_MS));
@@ -2388,8 +2402,7 @@ static void poll_task(void *arg)
                                 CAT_POLL_INTERVAL_MS, (unsigned long)s_last_freq_hz, cat_get_mode_str());
                 }
             }
-            esp_err_t err = cdc_acm_host_data_tx_blocking(
-                s_cdc_dev, (const uint8_t *)cmd, cmd_len, 200);
+            esp_err_t err = cat_tx((const uint8_t *)cmd, cmd_len, 200);
             if (err != ESP_OK) {
                 // A CDC TX failure burst must NEVER permanently kill the poll
                 // task. Root-caused 2026-07-10 (serial-log proven): opening any
@@ -2484,7 +2497,7 @@ static void band_scan_restore(uint32_t pre_freq, char pre_mode_digit)
         snprintf(fa, sizeof(fa), "FA%011lu;", (unsigned long)plan.freq_hz);
         ESP_LOGW(TAG, "the band scan left the radio on %lu Hz - restoring %lu Hz",
                  (unsigned long)s_last_freq_hz, (unsigned long)plan.freq_hz);
-        if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)fa,
+        if (cat_tx((const uint8_t *)fa,
                                           strlen(fa), 200) == ESP_OK) {
             s_last_freq_hz = plan.freq_hz;
             ui_update_frequency(plan.freq_hz);
@@ -2498,7 +2511,7 @@ static void band_scan_restore(uint32_t pre_freq, char pre_mode_digit)
         snprintf(md, sizeof(md), "MD%c;", plan.mode_digit);
         ESP_LOGW(TAG, "the band scan left the radio in mode '%c' - restoring '%c'",
                  s_last_mode_digit ? s_last_mode_digit : '?', plan.mode_digit);
```

```
-        if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)md,
+        if (cat_tx((const uint8_t *)md,
                                           strlen(md), 200) != ESP_OK)
             ESP_LOGW(TAG, "restore of mode '%c' could not be sent", plan.mode_digit);
     }
@@ -2518,7 +2531,7 @@ static void band_scan_restore(uint32_t pre_freq, char pre_mode_digit)
   * and the mode. */
 void cat_band_scan_restore_test(uint32_t pre_freq, char pre_mode_digit)
 {
-    if (!s_cdc_dev) {
+    if (!CAT_LINK_UP()) {
         ESP_LOGW(TAG, "restore test: no CAT link");
         return;
     }
@@ -2531,10 +2544,44 @@ void cat_band_scan_restore_test(uint32_t pre_freq, char pre_mode_digit)
     band_scan_restore(pre_freq, pre_mode_digit);
 }

+static void net_gone_cb(void)
+{
+    ESP_LOGW(TAG, "QMX network link dropped");
+    xEventGroupSetBits(s_evt_group, EVT_DEV_GONE);
+}
+
+// Network counterpart of try_open_qmx(): connect to the QMXR/1 relay running
+// on the NAS the QMX is plugged into. CAT bytes go through the same handle_rx()
+// queue the CDC driver feeds, so cat_rx_task parses them identically.
+static esp_err_t try_open_net(void)
+{
+    xEventGroupClearBits(s_evt_group, EVT_DEV_GONE);
+    esp_err_t err = radio_relay_connect(handle_rx, net_gone_cb, 3000);
+    if (err != ESP_OK) return err;
+    s_net_link = true;
+    ESP_LOGI(TAG, "QMX CAT link via network (NAS relay)");
+    cat_tx((const uint8_t *)"ID;", 3, 500);
+    return ESP_OK;
+}
+
+void cat_reconnect(void)
+{
+    // Only meaningful while a link is up; a stale GONE bit would otherwise
+    // kill the next session the moment it opened.
+    if (s_evt_group && CAT_LINK_UP()) xEventGroupSetBits(s_evt_group, EVT_DEV_GONE);
+}
+
+const char *cat_get_transport(void)
+{
+    if (s_net_link) return "net";
+    if (s_cdc_dev)  return "usb";
+    return "none";
+}
+
 static void link_task(void *arg)
 {
     while (1) {
-        esp_err_t err = try_open_qmx();
+        esp_err_t err = radio_relay_enabled() ? try_open_net() : try_open_qmx();
         if (err == ESP_OK) {
             s_rx_len = 0;
             s_last_freq_hz = 0;
@@ -2609,8 +2656,7 @@ static void link_task(void *arg)
                 const int VOX_MAX_ATTEMPTS = 3;
                 for (int attempt = 1; attempt <= VOX_MAX_ATTEMPTS; attempt++) {
                     const char *vox_off = "Q3 0;";
-                    esp_err_t terr = cdc_acm_host_data_tx_blocking(
-                        s_cdc_dev, (const uint8_t *)vox_off, 5, 200);
+                    esp_err_t terr = cat_tx((const uint8_t *)vox_off, 5, 200);
                     if (terr != ESP_OK) {
                         ESP_LOGW(TAG, "Failed to send VOX-off (Q3 0;) attempt %d/%d: 0x%x",
                                  attempt, VOX_MAX_ATTEMPTS, terr);
@@ -2622,8 +2668,7 @@ static void link_task(void *arg)
                     vTaskDelay(pdMS_TO_TICKS(150));
                     s_q3_resp_len = 0;
                     const char *vox_q = "Q3;";
-                    esp_err_t qerr = cdc_acm_host_data_tx_blocking(
-                        s_cdc_dev, (const uint8_t *)vox_q, strlen(vox_q), 200);
+                    esp_err_t qerr = cat_tx((const uint8_t *)vox_q, strlen(vox_q), 200);
                     if (qerr == ESP_OK) {
                         for (int w = 0; w < 20 && s_q3_resp_len == 0; w++) {
                             vTaskDelay(pdMS_TO_TICKS(20));
@@ -2663,8 +2708,7 @@ static void link_task(void *arg)
                 static const char *const warmup_cmds[] = { "FA;", "MD;", "FW;" };
                 for (size_t wi = 0; wi < sizeof(warmup_cmds) / sizeof(warmup_cmds[0]); wi++) {
                     const char *cmd = warmup_cmds[wi];
-                    esp_err_t werr = cdc_acm_host_data_tx_blocking(
-                        s_cdc_dev, (const uint8_t *)cmd, strlen(cmd), 200);
+                    esp_err_t werr = cat_tx((const uint8_t *)cmd, strlen(cmd), 200);
```

```
                    if (werr != ESP_OK) {
                        ESP_LOGW(TAG, "Warmup %s send failed: 0x%x", cmd, werr);
                        break;
@@ -2678,8 +2722,7 @@ static void link_task(void *arg)
             // differently (IQ output, TA TX, MMSSB filter tokens).
             {
                 const char *vn_q = "VN;";
-                esp_err_t verr = cdc_acm_host_data_tx_blocking(
-                    s_cdc_dev, (const uint8_t *)vn_q, strlen(vn_q), 200);
+                esp_err_t verr = cat_tx((const uint8_t *)vn_q, strlen(vn_q), 200);
                 if (verr == ESP_OK) {
                     vTaskDelay(pdMS_TO_TICKS(100));  // response handled in process_cat_message
                 } else {
@@ -2699,8 +2742,7 @@ static void link_task(void *arg)
              * - an unidentifiable radio keeps today's behaviour. */
             if (cat_qmx_fw_at_least(1, 4, 3)) {
                 const char *ui_q = "UI;";
-                esp_err_t uerr = cdc_acm_host_data_tx_blocking(
-                    s_cdc_dev, (const uint8_t *)ui_q, strlen(ui_q), 200);
+                esp_err_t uerr = cat_tx((const uint8_t *)ui_q, strlen(ui_q), 200);
                 if (uerr == ESP_OK) vTaskDelay(pdMS_TO_TICKS(100));  /* handled in process_cat_message */
                 else ESP_LOGW(TAG, "Failed to query unique id (UI): 0x%x", uerr);
             } else {
@@ -2711,8 +2753,7 @@ static void link_task(void *arg)
             // Read CW offset from QMX menu (session value, EEPROM-persisted on QMX side)
             {
                 const char *cw_q = "MMCW|CW offset;";
-                esp_err_t cerr = cdc_acm_host_data_tx_blocking(
-                    s_cdc_dev, (const uint8_t *)cw_q, strlen(cw_q), 200);
+                esp_err_t cerr = cat_tx((const uint8_t *)cw_q, strlen(cw_q), 200);
                 if (cerr == ESP_OK) {
                     vTaskDelay(pdMS_TO_TICKS(100));
                     // Response is in s_mm_resp — parse MMnnn;
@@ -2771,7 +2812,7 @@ static void link_task(void *arg)
              * undo a split we did not engage. */
             {
                 s_rx_vfo_mode = -1;
-                cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"FR;", 3, 200);
+                cat_tx((const uint8_t *)"FR;", 3, 200);
                 for (int wi = 0; wi < 10 && s_rx_vfo_mode < 0; wi++) {
                     vTaskDelay(pdMS_TO_TICKS(20));
                 }
@@ -2784,10 +2825,10 @@ static void link_task(void *arg)
                     ESP_LOGW(TAG, "VFO mode: radio was receiving on %s - the Tab5 only "
                                   "reads and writes VFO A, so nothing it did would have "
                                   "been audible. Switching to VFO A.", was);
-                    cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"FR0;", 4, 200);
+                    cat_tx((const uint8_t *)"FR0;", 4, 200);
                     vTaskDelay(pdMS_TO_TICKS(60));
                     s_rx_vfo_mode = -1;
-                    cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"FR;", 3, 200);
+                    cat_tx((const uint8_t *)"FR;", 3, 200);
                     for (int wi = 0; wi < 10 && s_rx_vfo_mode < 0; wi++) {
                         vTaskDelay(pdMS_TO_TICKS(20));
                     }
@@ -2834,8 +2875,7 @@ static void link_task(void *arg)
                  * value. */
                 const char *gq = "MMSystem config|GPS & Ser. ports|GPS source;";
                 s_mm_resp_len = 0;
-                esp_err_t gerr = cdc_acm_host_data_tx_blocking(
-                    s_cdc_dev, (const uint8_t *)gq, strlen(gq), 200);
+                esp_err_t gerr = cat_tx((const uint8_t *)gq, strlen(gq), 200);
                 if (gerr == ESP_OK) {
                     for (int wi = 0; wi < 15 && s_mm_resp_len == 0; wi++) {
                         vTaskDelay(pdMS_TO_TICKS(20));
@@ -2905,7 +2945,7 @@ static void link_task(void *arg)
             char     pre_mode_digit = 0;
             {
                 const char *q = "FA;MD;";
-                if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)q,
+                if (cat_tx((const uint8_t *)q,
                                                   strlen(q), 200) == ESP_OK) {
                     for (int wi = 0; wi < 25 && !s_last_freq_hz; wi++)
                         vTaskDelay(pdMS_TO_TICKS(20));
@@ -2943,8 +2983,7 @@ static void link_task(void *arg)
                         snprintf(qname, sizeof(qname), "MMBand config.|Band name (m)[%d];", bi);
                         s_rx_len = 0;
                         s_mm_resp_len = 0;
-                        esp_err_t be = cdc_acm_host_data_tx_blocking(
-                            s_cdc_dev, (const uint8_t *)qname, strlen(qname), 200);
+                        esp_err_t be = cat_tx((const uint8_t *)qname, strlen(qname), 200);
                         if (be != ESP_OK) { transport_error = true; break; }
                         for (int wi = 0; wi < 20 && s_mm_resp_len == 0; wi++) {
                             vTaskDelay(pdMS_TO_TICKS(20));
@@ -2967,7 +3006,7 @@ static void link_task(void *arg)
                         // Query center frequency
                         char qfreq[56];
```

```
                        snprintf(qfreq, sizeof(qfreq), "MMBand config.|Frequency center[%d];", bi);
-                       be = cdc_acm_host_data_tx_blocking(
-                           s_cdc_dev, (const uint8_t *)qfreq, strlen(qfreq), 200);
+                       be = cat_tx((const uint8_t *)qfreq, strlen(qfreq), 200);
                        if (be != ESP_OK) { transport_error = true; break; }
                        for (int wi = 0; wi < 20 && s_mm_resp_len == 0; wi++) {
                            vTaskDelay(pdMS_TO_TICKS(20));
@@ -3020,6 +3058,19 @@ static void link_task(void *arg)
             xEventGroupWaitBits(s_evt_group, EVT_DEV_GONE,
                                 pdTRUE, pdFALSE, portMAX_DELAY);
             ESP_LOGW(TAG, "QMX gone, cleaning up");
+            if (s_net_link) {
+                // Same ordering rules as the USB branch below: drop the link
+                // flag first so poll_task exits, wait for it, then close.
+                s_net_link = false;
+                s_cat_ready = false;
+                int wait_ms = 0;
+                while (s_poll_task != NULL && wait_ms < 4000) {
+                    vTaskDelay(pdMS_TO_TICKS(20));
+                    wait_ms += 20;
+                }
+                radio_relay_close();
+                audio_request_reset();
+            }
             if (s_cdc_dev) {
                 cdc_acm_dev_hdl_t dev = s_cdc_dev;
                 // Clear the handle FIRST so poll_task's "while (s_cdc_dev !=
@@ -3057,7 +3108,7 @@ static void link_task(void *arg)

 esp_err_t cat_set_frequency(uint32_t freq_hz)
 {
-    if (s_cdc_dev == NULL) {
+    if (!CAT_LINK_UP()) {
         return ESP_ERR_INVALID_STATE;  // QMX not connected
     }

@@ -3116,7 +3167,7 @@ esp_err_t cat_set_frequency(uint32_t freq_hz)
         ESP_LOGW(TAG, "cat_set_frequency: snprintf produced %d chars (expected 14)", n);
         return ESP_FAIL;
     }
-    esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, 14, 200);
+    esp_err_t err = cat_tx((const uint8_t *)cmd, 14, 200);
     if (err != ESP_OK) {
         ESP_LOGW(TAG, "FA TX failed: 0x%x", err);
         return err;
@@ -3237,7 +3288,7 @@ esp_err_t cat_set_mode(const char *mode)
         ESP_LOGW(TAG, "cat_set_mode: unknown mode '%s'", mode ? mode : "(null)");
         return ESP_ERR_INVALID_ARG;
     }
-    if (s_cdc_dev == NULL) return ESP_ERR_INVALID_STATE;
+    if (!CAT_LINK_UP()) return ESP_ERR_INVALID_STATE;
     // (!) WAS THE ONE BLOCKING CDC WRITER IN THIS FILE WITH NO s_poll_paused
     // GUARD (found 2026-09-16, chasing Randy N4OPI's "web Apply hangs and
     // loses the QMX, only during an active QSO/CQ exchange, needs a power
@@ -3260,7 +3311,7 @@ esp_err_t cat_set_mode(const char *mode)

     char cmd[8];
     cmd[0] = 'M'; cmd[1] = 'D'; cmd[2] = digit; cmd[3] = ';'; cmd[4] = 0;
-    esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, 4, 200);
+    esp_err_t err = cat_tx((const uint8_t *)cmd, 4, 200);
     if (err != ESP_OK) {
         ESP_LOGW(TAG, "MD TX failed: 0x%x", err);
         return err;
@@ -3272,7 +3323,7 @@ esp_err_t cat_set_mode(const char *mode)
 esp_err_t cat_set_passband_hz(uint32_t hz)
 {
     if (hz < 50 || hz > 9999) return ESP_ERR_INVALID_ARG;
-    if (s_cdc_dev == NULL) return ESP_ERR_INVALID_STATE;
+    if (!CAT_LINK_UP()) return ESP_ERR_INVALID_STATE;

     uint64_t now = esp_timer_get_time();
     if (now - s_last_tx_us < 200000) return ESP_ERR_TIMEOUT;
@@ -3285,7 +3336,7 @@ esp_err_t cat_set_passband_hz(uint32_t hz)
         ESP_LOGW(TAG, "cat_set_passband_hz: snprintf produced %d chars (expected 7)", n);
         return ESP_FAIL;
     }
-    esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, 7, 200);
+    esp_err_t err = cat_tx((const uint8_t *)cmd, 7, 200);
     if (err != ESP_OK) {
         ESP_LOGW(TAG, "FW TX failed: 0x%x", err);
         return err;
@@ -3304,7 +3355,7 @@ esp_err_t cat_set_qmx_time(int hour, int min, int sec)

 esp_err_t cat_query_qmx_time(int *out_hour, int *out_min, int *out_sec)
 {
-    if (!s_cdc_dev || !s_cat_ready) return ESP_ERR_INVALID_STATE;
+    if (!CAT_LINK_UP() || !s_cat_ready) return ESP_ERR_INVALID_STATE;
```

```
     /* (!) NOT WHILE A BURST IS KEYED. This was missing, and it is what let the
      * RTC read clear a WSPR burst's hold 30 s into a 110.6 s transmission -
@@ -3321,7 +3372,7 @@ esp_err_t cat_query_qmx_time(int *out_hour, int *out_min, int *out_sec)

     cat_poll_hold(CAT_HOLD_TIME_SYNC);
     s_tm_resp_len = 0;
-    esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"TM;", 3, 200);
+    esp_err_t err = cat_tx((const uint8_t *)"TM;", 3, 200);
     if (err == ESP_OK) {
         for (int i = 0; i < 10 && s_tm_resp_len == 0; i++) {
             vTaskDelay(pdMS_TO_TICKS(20));
@@ -3484,7 +3535,7 @@ bool cat_qmx_gps_position(double *lat, double *lon)
 esp_err_t cat_gps_gp_sync(int *out_y, int *out_mo, int *out_d,
                           int *out_h, int *out_mi, int *out_s, int64_t *out_flip_us)
 {
-    if (!s_cdc_dev || !s_cat_ready) return ESP_ERR_INVALID_STATE;
+    if (!CAT_LINK_UP() || !s_cat_ready) return ESP_ERR_INVALID_STATE;   /* USB or network radio */
     /* GP arrived in 1.04_004. Older radios answer with an error, and asking
      * them repeatedly in a 1.3 s loop is not free. */
     if (!cat_qmx_fw_at_least(1, 4, 4)) return ESP_ERR_NOT_SUPPORTED;
@@ -3504,7 +3555,7 @@ esp_err_t cat_gps_gp_sync(int *out_y, int *out_mo, int *out_d,

     while (esp_timer_get_time() - start < 1300000) {
         s_gp_resp_len = 0;
-        if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"GP;", 3, 100) != ESP_OK)
+        if (cat_tx((const uint8_t *)"GP;", 3, 100) != ESP_OK)   /* USB or network radio */
             break;
         polls++;
         for (int i = 0; i < 25 && s_gp_resp_len == 0; i++) vTaskDelay(pdMS_TO_TICKS(2));
@@ -3601,7 +3652,7 @@ esp_err_t cat_gps_gp_probe(int secs, int gap_ms, bool use_tm, cat_gp_probe_t *ou
 {
     if (!out) return ESP_ERR_INVALID_ARG;
     out->n = 0; out->polls = 0; out->nflip = 0;
-    if (!s_cdc_dev || !s_cat_ready) return ESP_ERR_INVALID_STATE;
+    if (!CAT_LINK_UP() || !s_cat_ready) return ESP_ERR_INVALID_STATE;   /* USB or network radio */
     if (!use_tm && !cat_qmx_fw_at_least(1, 4, 4)) return ESP_ERR_NOT_SUPPORTED;
     if (cat_poll_held_by_other(CAT_HOLD_TIME_SYNC)) return ESP_ERR_INVALID_STATE;
     if (secs < 1) secs = 1;
@@ -3618,8 +3669,7 @@ esp_err_t cat_gps_gp_probe(int secs, int gap_ms, bool use_tm, cat_gp_probe_t *ou
     while (esp_timer_get_time() - start < (int64_t)secs * 1000000LL) {
         if (gap_ms > 0) vTaskDelay(pdMS_TO_TICKS(gap_ms));
         if (use_tm) s_tm_resp_len = 0; else s_gp_resp_len = 0;
-        if (cdc_acm_host_data_tx_blocking(s_cdc_dev,
-                (const uint8_t *)(use_tm ? "TM;" : "GP;"), 3, 100) != ESP_OK)
+        if (cat_tx((const uint8_t *)(use_tm ? "TM;" : "GP;"), 3, 100) != ESP_OK)   /* USB or network */
             break;
         out->polls++;
         for (int i = 0; i < 25 && (use_tm ? s_tm_resp_len : s_gp_resp_len) == 0; i++)
@@ -3687,7 +3737,7 @@ esp_err_t cat_gps_gp_probe(int secs, int gap_ms, bool use_tm, cat_gp_probe_t *ou
 // already owns the CDC pipe (FT8 TX pauses the same poll flag).
 esp_err_t cat_gps_tick_sync(int *out_hour, int *out_min, int *out_sec, int64_t *out_flip_us)
 {
-    if (!s_cdc_dev || !s_cat_ready) return ESP_ERR_INVALID_STATE;
+    if (!CAT_LINK_UP() || !s_cat_ready) return ESP_ERR_INVALID_STATE;
     if (cat_poll_held_by_other(CAT_HOLD_TIME_SYNC)) {
         ESP_LOGI(TAG, "GPS tick skipped - %s holds the link", hold_name(s_poll_holds));
         return ESP_ERR_INVALID_STATE;   // a burst owns the pipe
@@ -3714,7 +3764,7 @@ esp_err_t cat_gps_tick_sync(int *out_hour, int *out_min, int *out_sec, int64_t *

     while (esp_timer_get_time() - start < 1300000) {   // ~1.3 s cap: spans any 1 s boundary
         s_tm_resp_len = 0;
-        if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"TM;", 3, 100) != ESP_OK)
+        if (cat_tx((const uint8_t *)"TM;", 3, 100) != ESP_OK)
             break;
         // Wait briefly for the async RX handler to stamp + fill the response.
         for (int i = 0; i < 25 && s_tm_resp_len == 0; i++) vTaskDelay(pdMS_TO_TICKS(2));
@@ -3798,6 +3848,20 @@ esp_err_t cat_gps_tick_sync(int *out_hour, int *out_min, int *out_sec, int64_t *
 // `while (s_cdc_dev != NULL)` lets it exit, wait for it, and only then close.
 void cat_usb_shutdown(void)
 {
+    if (s_net_link) {
+        ESP_LOGI(TAG, "shutdown: returning the networked radio to RX");
+        const char *rx_net = "TA0;RX;";
+        radio_relay_send_cat((const uint8_t *)rx_net, strlen(rx_net), 200);
+        s_net_link = false;
+        s_cat_ready = false;
+        int wait_ms = 0;
+        while (s_poll_task != NULL && wait_ms < 1500) {
+            vTaskDelay(pdMS_TO_TICKS(20));
+            wait_ms += 20;
+        }
+        radio_relay_close();
+        return;
+    }
     cdc_acm_dev_hdl_t dev = s_cdc_dev;
     if (!dev) {
         ESP_LOGI(TAG, "shutdown: no CAT device open");
```

main/audio/audio.h

changes against upstream v1.18.0 (unified diff)

```
@@ -58,6 +58,18 @@ bool audio_uac_active(void);
  */
 void audio_request_reset(void);

+/**
+ * @brief Push already-decoded I/Q pairs from the network radio source
+ *        (QMX on a NAS, see net/radio_relay.h).
+ *
+ * Input is interleaved int16 I/Q, scaled exactly like the USB path's
+ * (s24 >> 8). Applies IQ balance and feeds the same ring buffer (and the RX
+ * audio demodulator) the UAC path feeds, so FFT/FT8/WSPR/RX audio can't tell
+ * the difference. Never blocks: drops on ring-full like the USB path.
+ * Called from the relay RX task.
+ */
+void audio_push_net_iq(const int16_t *iq, size_t pairs);
+
 // Drop any partial 6-byte I/Q frame held from a previous UAC session. Called
 // when a session starts or ends - a new stream begins on a frame boundary, and
 // carrying a fragment across would misalign the interleave from the first
```

main/audio/audio.c

changes against upstream v1.18.0 (unified diff)

```
@@ -20,6 +20,7 @@
 #include "psram_task.h"
 #include "ui.h"
 #include "storage/settings.h"   // rx_source_is_real: which producer feeds the ring
+#include "radio_relay.h"

 static const char *TAG = "audio";

@@ -712,6 +713,48 @@ static bool process_rx(void)
     }
 }

+// Network radio source (net/radio_relay.c): the same post-decode path as
+// process_rx(), minus the 24-bit unpack the NAS relay already did.
+void audio_push_net_iq(const int16_t *iq, size_t pairs)
+{
+    static int16_t buf[2048 * 2];
+    if (!s_ring || !iq || pairs == 0) return;
+
+    if (s_flat_reset_pending) {
+        s_flat_reset_pending = false;
+        ui_flat_mode_reset();
+    }
+
+    int16_t local_peak_L = s_peak_left;
+    int16_t local_peak_R = s_peak_right;
+    while (pairs) {
+        size_t n = pairs > 2048 ? 2048 : pairs;
+        for (size_t i = 0; i < n; i++) {
+            int16_t Is = iq[2*i];
+            int16_t Qs = iq[2*i + 1];
+            int16_t aL = (Is < 0) ? (int16_t)(Is == -32768 ? 32767 : -Is) : Is;
+            int16_t aR = (Qs < 0) ? (int16_t)(Qs == -32768 ? 32767 : -Qs) : Qs;
+            if (aL > local_peak_L) local_peak_L = aL;
+            if (aR > local_peak_R) local_peak_R = aR;
+            iq_balance_apply(&Is, &Qs);
+            buf[2*i]     = Is;
+            buf[2*i + 1] = Qs;
+        }
+        s_samples_this_period += n;
+        if (xRingbufferSend(s_ring, buf, n * sizeof(int16_t) * 2, 0) == pdTRUE) {
+            s_pairs_written += n;
+        } else {
+            s_dropped_this_period += n;
+            s_dropped_total += n;
+        }
+        dsp_rxaudio_forward(buf, n);   // RX audio demodulator, as process_rx() does
+        iq += 2 * n;
+        pairs -= n;
+    }
+    s_peak_left = local_peak_L;
+    s_peak_right = local_peak_R;
+}
+
 static esp_err_t open_and_start(uint8_t addr, uint8_t iface_num)
 {
     const uac_host_device_config_t dev_cfg = {
@@ -783,7 +826,11 @@ static void audio_task(void *arg)
         if (xQueueReceive(s_evt_queue, &e, 0) == pdTRUE) {
             switch (e.kind) {
             case AE_RX_CONNECTED:
-                if (s_uac_dev == NULL) {
+                if (radio_relay_enabled()) {
+                    // Network radio (QMX on the NAS) selected: a locally plugged
+                    // QMX would double-feed the sample ring, so leave it idle.
+                    ESP_LOGW(TAG, "USB audio device ignored - network radio source is selected");
+                } else if (s_uac_dev == NULL) {
                     // A new UAC session starts on a frame boundary; anything
                     // held from the old one belongs to a stream that is gone.
                     audio_reset_frame_alignment();
```

main/wifi/wifi.c

changes against upstream v1.18.0 (unified diff)

```
@@ -138,6 +138,18 @@ static void set_sdio_gpio_drive(void)
 }

 // SNTP callback ------------------------------------------------------------
+static TaskHandle_t   s_sntp_task;
+static volatile time_t s_sntp_pending;
+
+static void sntp_apply_task(void *arg)
+{
+    (void)arg;
+    for (;;) {
+        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
+        time_sync_notify_sntp(s_sntp_pending);
+    }
+}
+
 static void sntp_sync_cb(struct timeval *tv)
 {
     struct tm tm_utc;
@@ -148,8 +160,18 @@ static void sntp_sync_cb(struct timeval *tv)
     xEventGroupSetBits(s_events, BIT_TIME_OK);

     // Write to the Tab5 supercap RTC, persist NVS anchor, update system clock,
-    // and push to QMX (unless QMX GPS flag is set).
-    time_sync_notify_sntp(tv->tv_sec);
+    // and push to QMX (unless QMX GPS flag is set) - on sntp_apply_task, NOT
+    // here. (!) This callback runs ON lwIP's tcpip thread. The QMX time push is
+    // a CAT write, and with the network radio (QMX on a NAS) a CAT write is a
+    // lwIP socket send(), which posts to the tcpip thread and waits for it -
+    // i.e. for itself. Measured 2026-10-04: every boot on the NAS radio, the
+    // whole network stack (radio stream, web page, TLS feeds) died within a
+    // second of "SNTP sync", for good. Over USB the push never touched lwIP,
+    // which is why this was invisible until now. The RTC (I2C) and NVS writes
+    // do not belong on the tcpip thread either.
+    s_sntp_pending = tv->tv_sec;
+    if (s_sntp_task) xTaskNotifyGive(s_sntp_task);
+    else time_sync_notify_sntp(tv->tv_sec);   // worker not up: old behaviour
 }

 // Replicates the work IDF's default WIFI_EVENT_STA_START handler (wifi_start in
@@ -732,6 +754,11 @@ static void on_ip_event(void *arg, esp_event_base_t base,
             sntp_started = true;
             esp_sntp_config_t cfg = ESP_NETIF_SNTP_DEFAULT_CONFIG("pool.ntp.org");
             cfg.sync_cb = sntp_sync_cb;
+            // Same stack/priority as time_sync_task, which runs this same
+            // time_sync code for its other sources.
+            if (!s_sntp_task)
+                s_sntp_task = psram_task_create(sntp_apply_task, "sntp_apply", 8192,
+                                                NULL, 4, tskNO_AFFINITY);
             esp_netif_sntp_init(&cfg);
             ESP_LOGI(TAG, "SNTP started (pool.ntp.org)");
         }
```

main/qmx_term.c

changes against upstream v1.18.0 (unified diff)

```
@@ -9,6 +9,7 @@
 #include "freertos/semphr.h"
 #include "util/psram_task.h"
 #include "cat.h"          /* cat_request_iq_reassert - see close_locked */
+#include "net/radio_relay.h" /* the second port over the network (QMX on a NAS) */
 #include "ui.h"           /* ui_flat_mode_reset                         */
 #include <string.h>
 #include <stdio.h>
@@ -58,6 +59,10 @@ static uint32_t          s_pre_freq_hz;
 static char              s_pre_mode[12];

 static cdc_acm_dev_hdl_t s_dev;
+/* True when the session runs over the NAS relay instead of USB: the QMX is
+ * plugged into a NAS (tools/nas-relay), and the relay opens the radio's second
+ * serial port for us and carries it as its own stream, separate from CAT. */
+static volatile bool     s_net;
 static ansi_term_t      *s_scr;          /* PSRAM: ~4 KB of cells */
 static bool              s_open;
 static int64_t           s_last_activity_us;
@@ -111,6 +116,7 @@ static bool on_rx(const uint8_t *data, size_t len, void *arg)

 static bool tx(const char *s, int len)
 {
+    if (s_net) return radio_relay_term_send((const uint8_t *)s, (size_t)len, 300) == ESP_OK;
     if (!s_dev) return false;
     return cdc_acm_host_data_tx_blocking(s_dev, (const uint8_t *)s, len, 300) == ESP_OK;
 }
@@ -264,6 +270,7 @@ static void close_locked(void)
```

```
     } else {
         s_open = false;
     }
+    if (s_net) { radio_relay_term_close(); s_net = false; }
     if (s_dev) { cdc_acm_host_close(s_dev); s_dev = NULL; }

     /* (*) Hand the radio back the way the operator-pause resume does, because a
@@ -342,6 +349,15 @@ static void on_cdc_event(const cdc_acm_host_dev_event_data_t *ev, void *arg)
     if (s_dev) { cdc_acm_host_close(s_dev); s_dev = NULL; }
 }

+/* The network counterpart of the DISCONNECTED event: the relay lost the port or
+ * the whole link. Runs on the relay RX task - flags only, nothing that blocks. */
+static void on_net_gone(void)
+{
+    ESP_LOGW(TAG, "the network radio went away with a terminal session open");
+    s_open = false;
+    s_net = false;
+}
+
 static void idle_task(void *arg)
 {
     (void)arg;
@@ -390,6 +406,26 @@ bool qmx_term_open(void)
         snprintf(s_pre_mode, sizeof(s_pre_mode), "%s", (m && m[0]) ? m : "");
     }

+    const char *transport = cat_get_transport();
+    if (transport && strcmp(transport, "net") == 0) {
+        /* The QMX is on a NAS: the relay owns the USB device and opens port 2
+         * for us. Same retry reasoning as below does not apply - the relay
+         * answers definitively (open, or "the radio has one serial port"). */
+        if (!radio_relay_term_supported()) {
+            ESP_LOGW(TAG, "network radio: the NAS relay is too old to carry the "
+                          "terminal port - update tools/nas-relay on the NAS");
+            xSemaphoreGive(s_lock);
+            return false;
+        }
+        esp_err_t ne = radio_relay_term_open(on_rx, on_net_gone, 3000);
+        if (ne != ESP_OK) {
+            ESP_LOGW(TAG, "network radio: port 2 would not open: %s - is the radio "
+                          "set to 2 USB serial ports?", radio_relay_term_error());
+            xSemaphoreGive(s_lock);
+            return false;
+        }
+        s_net = true;
+    } else {
     const cdc_acm_host_device_config_t cfg = {
         .connection_timeout_ms = 1000,
         .out_buffer_size = 64,
@@ -441,6 +477,7 @@ bool qmx_term_open(void)
     };
     cdc_acm_host_line_coding_set(s_dev, &lc);
     cdc_acm_host_set_control_line_state(s_dev, true, true);
+    }   /* USB */

     s_open = true;
     s_last_activity_us = esp_timer_get_time();
@@ -506,7 +543,8 @@ bool qmx_term_open(void)
         s_idle_task = psram_task_create(idle_task, "qmx_term_idle", 3072, NULL, 2,
                                         tskNO_AFFINITY);
     }
-    ESP_LOGI(TAG, "terminal session open on interface %d", QMX_TERM_INTERFACE);
+    if (s_net) ESP_LOGI(TAG, "terminal session open on port 2 via the NAS relay");
+    else       ESP_LOGI(TAG, "terminal session open on interface %d", QMX_TERM_INTERFACE);
     xSemaphoreGive(s_lock);
     return true;
 }
```

tools/patches/apply_esp_hosted_sdio_split_bundles.ps1

new file, 274 lines

```
<#
.SYNOPSIS
    esp_hosted SDIO RX: deliver every frame of a multi-frame read instead of
    keeping only the first (normal reads) or discarding all of them (oversize
    bursts). Marker: QMX_SDIO_RX_SPLIT.

.DESCRIPTION
    The Tab5's ESP32-C6 runs its SDIO send path in STREAMING mode: when several
    frames are waiting it exposes them back to back, each with its own
    esp_payload_header, and the pending-length register covers all of them.
    This host is built for PACKET mode (RX_NONE + 1536-byte mempool buffers,
    v0.18.2), so the stock read path only ever looked at the first frame:

      - a normal read holding two small frames lost the second one;
      - a burst larger than 1536 bytes was drained and DISCARDED by the
        oversize recovery (apply_esp_hosted_sdio_recovery.ps1) - every frame
        in it lost.

    Invisible under light traffic, fatal for a sustained inbound stream:
    measured 2026-10-04 with the NAS radio relay (1.5 Mbit/s of I/Q) the
    oversize drain fired several times a second, TCP retransmitted the lost
    frames, the stream collapsed to 10-20k of 48k pairs/s and the Tab5 was
    too busy to take touch input until rebooted.

    The patch walks such a read, copies each frame into its own mempool buffer
    (no DMA allocation - the v0.18.2 rule stands) and queues it exactly like a
    single-frame read. The oversize drain keeps its chunked reads and its
    exact byte accounting unchanged; the drained bytes are also copied into a
    96 KB PSRAM buffer and split after the bus is released. A header that
    does not parse, or an empty mempool, ends the walk: frames already
    delivered stay delivered, the rest is dropped, nothing aborts. The
    "SDIO RX oversize" warning (the recovery patch's check_patches marker) is
    kept, rate-limited to one line per 5 s with the split counters.

    Requires apply_esp_hosted_sdio_recovery.ps1 first (its drain code is an
    anchor here) - this file name sorts after it.

    Edits managed_components/ - wiped by fullclean or a dependency refresh;
    re-run like the other apply_*.ps1 patches. Idempotent.

.HOW TO USE
    powershell -ExecutionPolicy Bypass -File tools/patches/apply_esp_hosted_sdio_split_bundles.ps1
#>

$ErrorActionPreference = "Stop"

$repo   = Split-Path -Parent (Split-Path -Parent $PSScriptRoot)
$target = Join-Path $repo "managed_components/espressif__esp_hosted/host/drivers/transport/sdio/sdio_drv.c"

if (-not (Test-Path $target)) {
    Write-Host "sdio_drv.c not found at:" -ForegroundColor Red
    Write-Host "  $target"
    Write-Host "Build once so the component is fetched, then re-run."
    exit 1
}

$content = (Get-Content -Raw -Path $target) -replace "`r`n", "`n"

if ($content.Contains("QMX_SDIO_RX_SPLIT")) {
    Write-Host "Already patched (SDIO RX split of multi-frame reads) - nothing to do." -ForegroundColor Green
    exit 0
}
if (-not $content.Contains("SDIO RX oversize")) {
    Write-Host "apply_esp_hosted_sdio_recovery.ps1 has not been applied yet - run it first." -ForegroundColor Red
    exit 1
}

$pairs = @(
    @{ Old = @'
    g_h.funcs->_h_queue_item(from_slave_queue[pkt_prio], &buf_handle, HOSTED_BLOCK_MAX);
    g_h.funcs->_h_post_semaphore(sem_from_slave_queue);

    return ESP_OK;
}

'@; New = @'
    g_h.funcs->_h_queue_item(from_slave_queue[pkt_prio], &buf_handle, HOSTED_BLOCK_MAX);
    g_h.funcs->_h_post_semaphore(sem_from_slave_queue);

    return ESP_OK;
}

/* QMX_SDIO_RX_SPLIT (qmx-panadapter network-radio, 2026-10-04).
 *
```

```
 * The C6 slave runs its SDIO send path in STREAMING mode: when several frames
 * are waiting it exposes them back to back, each starting with its own
 * esp_payload_header, and the pending-length register covers all of them.
 * This host is built for PACKET mode (RX_NONE, fixed 1536-byte mempool
 * buffers - chosen in v0.18.2 because streaming mode re-allocates DMA memory
 * per burst), so stock code only ever looked at the FIRST frame of a read:
 *
 *   - a read <= 1536 bytes holding two small frames (two TCP ACKs, an ACK
 *     plus a CAT reply) silently lost the second one;
 *   - a read > 1536 bytes (a burst of full-size frames) was drained and
 *     DISCARDED by the oversize recovery - every frame in it lost.
 *
 * Harmless for light traffic, fatal for a sustained inbound stream: measured
 * 2026-10-04 with the NAS radio relay (1.5 Mbit/s of I/Q) - "SDIO RX oversize
 * ... draining" several times a second, TCP retransmitting the lost frames and
 * the stream collapsing to 10-20 k of the 48 k pairs/s.
 *
 * This walks such a byte stream, copies each frame into its own mempool
 * buffer (no DMA allocation, the v0.18.2 rule stands) and queues it exactly
 * as a single-frame read would be. A header that does not parse, or an empty
 * mempool, ends the walk: what was already delivered stays delivered, the
 * rest is dropped, and nothing aborts. */
static uint32_t qmx_split_bundles, qmx_split_frames, qmx_split_drops;

static void qmx_sdio_push_stream(const uint8_t *s, uint32_t s_len)
{
    while (s_len >= sizeof(struct esp_payload_header)) {
        const struct esp_payload_header *h = (const struct esp_payload_header *)s;
        uint16_t len = le16toh(h->len);
        uint16_t offset = le16toh(h->offset);
        uint32_t psize = (uint32_t)len + offset;
        uint16_t vlen = 0, voff = 0;
        uint8_t *p;

        if (!len && !offset)
            return;                         /* block padding after the last frame */
        if (!len || len > MAX_PAYLOAD_SIZE ||
            offset != sizeof(struct esp_payload_header) ||
            psize > s_len || psize > MAX_SDIO_BUFFER_SIZE) {
            qmx_split_drops++;
            return;
        }
        p = sdio_buffer_alloc(MEMSET_REQUIRED);
        if (!p) {
            qmx_split_drops++;
            return;
        }
        memcpy(p, s, psize);
        if (!is_valid_sdio_rx_packet(p, &vlen, &voff)) {
            sdio_buffer_free(p);
            qmx_split_drops++;
            return;
        }
        sdio_push_pkt_to_queue(p, vlen, voff);
        qmx_split_frames++;
        s += psize;
        s_len -= psize;
    }
}

'@ }
    ,@{ Old = @'
    uint16_t len = 0;
    uint16_t offset = 0;

    /* Drop packet if no processing needed */
    if (!is_valid_sdio_rx_packet(buf, &len, &offset)) {
'@; New = @'
    uint16_t len = 0;
    uint16_t offset = 0;

    /* QMX_SDIO_RX_SPLIT: a read can hold more than one frame (see
     * qmx_sdio_push_stream). Peek at the first header WITHOUT validating it
     * (validation may rewrite it): if another frame follows, copy every frame
     * out in order and release this buffer. */
    {
        const struct esp_payload_header *h0 = (const struct esp_payload_header *)buf;
        uint32_t first = (uint32_t)le16toh(h0->len) + le16toh(h0->offset);
        if (le16toh(h0->offset) == sizeof(struct esp_payload_header) &&
            le16toh(h0->len) && first <= buf_len &&
            buf_len - first >= sizeof(struct esp_payload_header)) {
            const struct esp_payload_header *h1 =
                (const struct esp_payload_header *)(buf + first);
            if (le16toh(h1->len) && le16toh(h1->offset) == sizeof(struct esp_payload_header)) {
                qmx_split_bundles++;
                qmx_sdio_push_stream(buf, buf_len);
                sdio_buffer_free(buf);
```

```
                return ESP_OK;
            }
        }
    }

    /* Drop packet if no processing needed */
    if (!is_valid_sdio_rx_packet(buf, &len, &offset)) {
'@ }
    ,@{ Old = @'
                if (sdio_oversize_drain_buf) {
                    uint32_t d_left = len_from_slave;
'@; New = @'
                /* QMX_SDIO_RX_SPLIT: keep what is drained, in PSRAM (never DMA). */
                if (!qmx_split_stream)
                    qmx_split_stream = (uint8_t *)heap_caps_malloc(
                        64u * (uint32_t)ESP_RX_BUFFER_SIZE,
                        MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT);
                if (sdio_oversize_drain_buf) {
                    uint32_t d_left = len_from_slave;
'@ }
    ,@{ Old = @'
                        d_left -= chunk;
                        drained += chunk;
'@; New = @'
                        if (qmx_split_stream)
                            memcpy(qmx_split_stream + drained, sdio_oversize_drain_buf, chunk);
                        d_left -= chunk;
                        drained += chunk;
'@ }
    ,@{ Old = @'
            sdio_rx_byte_count = (sdio_rx_byte_count + drained) % ESP_RX_BYTE_MAX;
            SDIO_DRV_UNLOCK();
            if (!drained)
                g_h.funcs->_h_msleep(1);
            continue;
'@; New = @'
            sdio_rx_byte_count = (sdio_rx_byte_count + drained) % ESP_RX_BYTE_MAX;
            SDIO_DRV_UNLOCK();
            /* QMX_SDIO_RX_SPLIT: a complete drain is a burst of whole frames -
             * deliver them instead of throwing them away. A partial drain (a
             * read failed part-way) is still discarded: its tail is unknown. */
            if (qmx_split_stream && drained && drained == len_from_slave) {
                qmx_split_bundles++;
                qmx_sdio_push_stream(qmx_split_stream, drained);
            } else if (drained) {
                qmx_split_drops++;
            }
            if (!drained)
                g_h.funcs->_h_msleep(1);
            continue;
'@ }
    ,@{ Old = @'
            ESP_LOGW(TAG, "SDIO RX oversize: len=%lu host_cnt=%lu slave_reg=%lu - draining to recover",
                    (unsigned long)len_from_slave, (unsigned long)sdio_rx_byte_count,
                    (unsigned long)slave_reg);
'@; New = @'
            /* QMX_SDIO_RX_SPLIT: with the burst now delivered this is routine
             * under a sustained stream, so report it at most every 5 s with
             * running totals rather than per event (it was flooding the log
             * several times a second). */
            {
                static int64_t qmx_last_oversize_log_us;
                static uint32_t qmx_oversize_since_log;
                int64_t now_us = esp_timer_get_time();
                qmx_oversize_since_log++;
                if (now_us - qmx_last_oversize_log_us >= 5000000LL) {
                    ESP_LOGW(TAG, "SDIO RX oversize: len=%lu host_cnt=%lu slave_reg=%lu - "
                            "draining (x%lu in 5 s); split: %lu bundles, %lu frames delivered, %lu dropped",
                            (unsigned long)len_from_slave, (unsigned long)sdio_rx_byte_count,
                            (unsigned long)slave_reg, (unsigned long)qmx_oversize_since_log,
                            (unsigned long)qmx_split_bundles, (unsigned long)qmx_split_frames,
                            (unsigned long)qmx_split_drops);
                    qmx_last_oversize_log_us = now_us;
                    qmx_oversize_since_log = 0;
                }
            }
'@ }
    ,@{ Old = @'
static uint8_t *sdio_oversize_drain_buf = NULL;

'@; New = @'
static uint8_t *sdio_oversize_drain_buf = NULL;
/* QMX_SDIO_RX_SPLIT: PSRAM copy of a drained burst, split into frames afterwards. */
static uint8_t *qmx_split_stream = NULL;

'@ }
)
```

```
foreach ($p in $pairs) {
    $old = $p.Old -replace "`r`n", "`n"
    $new = $p.New -replace "`r`n", "`n"
    $n = ([regex]::Matches($content, [regex]::Escape($old))).Count
    if ($n -ne 1) {
        Write-Host "Anchor found $n times (expected 1) - esp_hosted or the recovery patch changed; patch by hand and update this scr
  ↪ ipt:" -ForegroundColor Red
        Write-Host ($old.Substring(0, [Math]::Min(160, $old.Length)))
        exit 1
    }
    $content = $content.Replace($old, $new)
}

[System.IO.File]::WriteAllText($target, $content, (New-Object System.Text.UTF8Encoding $false))
Write-Host "Patched sdio_drv.c: multi-frame SDIO reads are split and delivered, not dropped." -ForegroundColor Green
Write-Host "  $target"
```

tools/patches/apply_esp_hosted_sdio_rx_copy_psram.ps1

new file, 98 lines

```
<#
.SYNOPSIS
    esp_hosted SDIO RX: copy each received WiFi frame into PSRAM instead of
    internal RAM, and drop a frame instead of asserting when no memory is
    left. Marker: QMX_SDIO_RX_COPY_PSRAM.

.DESCRIPTION
    sdio_process_rx_task() copies every received frame into a fresh buffer
    that lwIP then holds (zero-copy, L2_TO_L3_COPY off) until TCP has handed
    the data to the application. Stock code used _h_malloc -> malloc, and with
    CONFIG_SPIRAM_MALLOC_ALWAYSINTERNAL=16384 every frame-sized malloc lands in
    INTERNAL RAM, so internal RAM in use grows with the data in flight.

    Measured 2026-10-04 with the NAS radio relay's sustained 1.5 Mbit/s stream:
    TCP window 11520 -> internal low-water 3 KB and DMA-capable free 4 KB;
    touch input (I2C) stopped and the web server stopped answering until a
    reboot. Window 5760 -> the same. Window 2880 -> usable, but only 66% of
    the stream arrives.

    The copy now goes to PSRAM (it is released with free(), which handles
    either heap), falling back to the stock allocation only if PSRAM is
    exhausted. Stock also assert()ed when the allocation failed; the frame is
    now dropped (TCP retransmits) instead of rebooting the device.

    Edits managed_components/ - wiped by fullclean or a dependency refresh;
    re-run like the other apply_*.ps1 patches. Idempotent.

.HOW TO USE
    powershell -ExecutionPolicy Bypass -File tools/patches/apply_esp_hosted_sdio_rx_copy_psram.ps1
#>

$ErrorActionPreference = "Stop"

$repo   = Split-Path -Parent (Split-Path -Parent $PSScriptRoot)
$target = Join-Path $repo "managed_components/espressif__esp_hosted/host/drivers/transport/sdio/sdio_drv.c"

if (-not (Test-Path $target)) {
    Write-Host "sdio_drv.c not found at:" -ForegroundColor Red
    Write-Host "  $target"
    Write-Host "Build once so the component is fetched, then re-run."
    exit 1
}

$content = (Get-Content -Raw -Path $target) -replace "`r`n", "`n"

if ($content.Contains("QMX_SDIO_RX_COPY_PSRAM")) {
    Write-Host "Already patched (SDIO RX copy in PSRAM) - nothing to do." -ForegroundColor Green
    exit 0
}

$old = @'
                /* TODO : Need to abstract heap_caps_malloc */
                uint8_t * copy_payload = (uint8_t *)g_h.funcs->_h_malloc(buf_handle->payload_len);
                assert(copy_payload);
                assert(buf_handle->payload_len);
                assert(buf_handle->payload);
                memcpy(copy_payload, buf_handle->payload, buf_handle->payload_len);
'@
$new = @'
                /* QMX_SDIO_RX_COPY_PSRAM (qmx-panadapter network-radio, 2026-10-04).
                 *
                 * Every received WiFi frame is copied into a fresh buffer that
                 * lwIP holds (zero-copy, L2_TO_L3_COPY off) until TCP has
                 * delivered it to the application. Stock code used _h_malloc ->
                 * malloc, and with CONFIG_SPIRAM_MALLOC_ALWAYSINTERNAL=16384 every
                 * frame-sized malloc lands in INTERNAL RAM - so internal RAM in
                 * use grows with the data in flight. Measured with the NAS radio
```

```
                 * relay's 1.5 Mbit/s stream: TCP window 11520 -> internal low-water
                 * 3 KB, DMA free 4 KB, touch input (I2C) dead and the web server
                 * unresponsive until reboot; window 2880 -> usable but only 66% of
                 * the stream. The copy goes to PSRAM instead (freed with free(),
                 * which handles either heap); internal RAM only if PSRAM is full.
                 * Stock also assert()ed on allocation failure - now the frame is
                 * dropped (TCP retransmits) instead of rebooting the device. */
                uint8_t * copy_payload = (uint8_t *)heap_caps_malloc(buf_handle->payload_len,
                        MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT);
                if (!copy_payload)
                    copy_payload = (uint8_t *)g_h.funcs->_h_malloc(buf_handle->payload_len);
                if (!copy_payload || !buf_handle->payload_len || !buf_handle->payload) {
                    if (copy_payload)
                        HOSTED_FREE(copy_payload);
                    H_FREE_PTR_WITH_FUNC(buf_handle->free_buf_handle, buf_handle->priv_buffer_handle);
                    continue;
                }
                memcpy(copy_payload, buf_handle->payload, buf_handle->payload_len);
'@
$old = $old -replace "`r`n", "`n"
$new = $new -replace "`r`n", "`n"
$n = ([regex]::Matches($content, [regex]::Escape($old))).Count
if ($n -ne 1) {
    Write-Host "Anchor found $n times (expected 1) - esp_hosted changed; patch by hand and update this script." -ForegroundColor Red
    exit 1
}
$content = $content.Replace($old, $new)

[System.IO.File]::WriteAllText($target, $content, (New-Object System.Text.UTF8Encoding $false))
Write-Host "Patched sdio_drv.c: received WiFi frames are copied into PSRAM, not internal RAM." -ForegroundColor Green
Write-Host "  $target"
```

tools/patches/apply_esp_hosted_sdio_split_psram.ps1

new file, 171 lines

```
<#
.SYNOPSIS
    esp_hosted SDIO RX: frames split out of a multi-frame read go into PSRAM,
    not into the DMA mempool. Marker: QMX_SDIO_SPLIT_PSRAM.

.DESCRIPTION
    apply_esp_hosted_sdio_split_bundles.ps1 copies every frame of a bundled
    SDIO read into its own buffer from the SDIO mempool. That mempool NEVER
    gives memory back to the heap: a freed block goes onto its free list and
    stays there. So the pool grows to the largest number of frames ever in
    flight at once, in DMA-capable internal RAM, for the rest of the session.

    With the NAS radio's full-rate stream (~35 bundles/s, up to 7 frames
    each) that peak is 20-30 blocks of ~1.6 KB. Measured 2026-10-04: DMA free
    fell from 38 KB to 1.2 KB within ~20 s of the stream starting, internal
    free bottomed at 1 KB, the touch controller's I2C reads (and the PI4IO
    expander's) started timing out, SD reads failed - and the touchscreen
    stopped responding while the spectrum kept running.

    These frames are already copies; nothing DMAs into them. The patch takes
    them from PSRAM with heap_caps_malloc() and hands sdio_push_pkt_to_queue()
    heap_caps_free() as their free function, so they are really freed when the
    RX task is done with them. The single-frame path (the DMA read buffer
    itself) still uses the mempool, as before.

    STAGE 2 (marker QMX_SDIO_RXQ_PSRAM, added the same day): moving the split
    frames was not enough. A NORMAL single-frame read also parks its mempool
    block in the RX queue (depth 20) until sdio_process_rx_task gets to it, so
    under the stream the pool still grew to ~20 blocks - measured: DMA free
    691 B, touch still dead. So every received frame is now copied into PSRAM
    the moment it is queued and its mempool block goes straight back; the pool
    only ever holds the one or two blocks the SDIO read itself is using. Cost:
    one extra ~1.5 KB memcpy per frame (~1.5 MB/s at the full radio rate).
    If PSRAM is ever exhausted the frame is queued in its mempool block exactly
    as before. Stage 2 is applied on its own to a file that already has stage 1.

    Requires apply_esp_hosted_sdio_split_bundles.ps1 first - this file name
    sorts after it. Edits managed_components/; re-run like the other
    apply_*.ps1 patches. Idempotent.

.HOW TO USE
    pwsh -File tools/patches/apply_esp_hosted_sdio_split_psram.ps1
#>

$ErrorActionPreference = "Stop"

$repo   = Split-Path -Parent (Split-Path -Parent $PSScriptRoot)
$target = Join-Path $repo "managed_components/espressif__esp_hosted/host/drivers/transport/sdio/sdio_drv.c"

if (-not (Test-Path $target)) {
    Write-Host "sdio_drv.c not found at:" -ForegroundColor Red
    Write-Host "  $target"
```

```
    Write-Host "Build once so the component is fetched, then re-run."
    exit 1
}

$content = (Get-Content -Raw -Path $target) -replace "`r`n", "`n"

if ($content.Contains("QMX_SDIO_RXQ_PSRAM")) {
    Write-Host "Already patched (SDIO RX frames queued in PSRAM) - nothing to do." -ForegroundColor Green
    exit 0
}
if (-not $content.Contains("QMX_SDIO_RX_SPLIT")) {
    Write-Host "apply_esp_hosted_sdio_split_bundles.ps1 has not been applied yet - run it first." -ForegroundColor Red
    exit 1
}

$stage1 = @(
@{ Old = @'
    buf_handle.priv_buffer_handle = rxbuff;
    buf_handle.free_buf_handle    = sdio_buffer_free;
    buf_handle.payload_len        = len;
'@; New = @'
    buf_handle.priv_buffer_handle = rxbuff;
    /* QMX_SDIO_SPLIT_PSRAM: a frame split out of a bundle lives in PSRAM
     * and is released with heap_caps_free, not back into the mempool. */
    buf_handle.free_buf_handle    = qmx_pkt_free ? qmx_pkt_free : sdio_buffer_free;
    buf_handle.payload_len        = len;
'@ },
@{ Old = @'
static esp_err_t sdio_push_pkt_to_queue(uint8_t * rxbuff, uint16_t len, uint16_t offset)
{
'@; New = @'
/* QMX_SDIO_SPLIT_PSRAM: set only around qmx_sdio_push_stream()'s pushes. */
static void (*qmx_pkt_free)(void *);

static esp_err_t sdio_push_pkt_to_queue(uint8_t * rxbuff, uint16_t len, uint16_t offset)
{
'@ },
@{ Old = @'
        p = sdio_buffer_alloc(MEMSET_REQUIRED);
        if (!p) {
            qmx_split_drops++;
            return;
        }
        memcpy(p, s, psize);
        if (!is_valid_sdio_rx_packet(p, &vlen, &voff)) {
            sdio_buffer_free(p);
            qmx_split_drops++;
            return;
        }
        sdio_push_pkt_to_queue(p, vlen, voff);
'@; New = @'
        /* QMX_SDIO_SPLIT_PSRAM: PSRAM, not the never-shrinking DMA mempool. */
        p = heap_caps_malloc(psize, MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT);
        if (!p) {
            qmx_split_drops++;
            return;
        }
        memcpy(p, s, psize);
        if (!is_valid_sdio_rx_packet(p, &vlen, &voff)) {
            heap_caps_free(p);
            qmx_split_drops++;
            return;
        }
        qmx_pkt_free = heap_caps_free;
        sdio_push_pkt_to_queue(p, vlen, voff);
        qmx_pkt_free = NULL;
'@ }
)
# Stage 2 - anchors are stage 1's own output, so it always runs after it.
$stage2 = @(
@{ Old = @'
    buf_handle.free_buf_handle    = qmx_pkt_free ? qmx_pkt_free : sdio_buffer_free;
'@; New = @'
    buf_handle.free_buf_handle    = qmx_free ? qmx_free : sdio_buffer_free;
'@ },
@{ Old = @'
    interface_buffer_handle_t buf_handle;

    h = (struct esp_payload_header *)rxbuff;
'@; New = @'
    interface_buffer_handle_t buf_handle;
    /* QMX_SDIO_RXQ_PSRAM: queue a PSRAM copy and give the DMA mempool block
     * back at once - the pool never returns memory to the heap, so blocks
     * parked in the RX queue under a sustained stream ate all DMA RAM. */
    void (*qmx_free)(void *) = qmx_pkt_free;
    if (!qmx_free) {
        uint8_t *cp = heap_caps_malloc((size_t)offset + len, MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT);
```

```
        if (cp) {
            memcpy(cp, rxbuff, (size_t)offset + len);
            sdio_buffer_free(rxbuff);
            rxbuff = cp;
            qmx_free = heap_caps_free;
        }
    }

    h = (struct esp_payload_header *)rxbuff;
'@ }
)

$pairs = @()
if (-not $content.Contains("QMX_SDIO_SPLIT_PSRAM")) { $pairs += $stage1 }
$pairs += $stage2

foreach ($p in $pairs) {
    $old = $p.Old -replace "`r`n", "`n"
    $new = $p.New -replace "`r`n", "`n"
    $n = ([regex]::Matches($content, [regex]::Escape($old))).Count
    if ($n -ne 1) {
        Write-Host "Anchor found $n times (expected 1) - esp_hosted or the split patch changed; patch by hand and update this script
  ↪ :" -ForegroundColor Red
        Write-Host ($old.Substring(0, [Math]::Min(160, $old.Length)))
        exit 1
    }
    $content = $content.Replace($old, $new)
}

[System.IO.File]::WriteAllText($target, $content, (New-Object System.Text.UTF8Encoding $false))
Write-Host "Patched sdio_drv.c: received SDIO frames are queued in PSRAM, not in the DMA mempool." -ForegroundColor Green
Write-Host "  $target"
```

tools/patches/apply_cdc_acm_close_tolerant.ps1

changes against upstream v1.18.0 (unified diff)

```
@@ -149,13 +149,30 @@ $patched = @'
 $originalLf = $original -replace "`r`n", "`n"
 $contentLf  = $content  -replace "`r`n", "`n"

+$patchedLf = $patched -replace "`r`n", "`n"
+
+# usb_host_cdc_acm 2.4.1 (2026-08-25, "Fixed submitting transfer poll race
+# condition") brought the two polling flags back, so its close() still clears
+# them before the endpoint resets. Accept that layout too and keep the two
+# lines, so the upstream race fix stays intact; the patch is otherwise identical.
+$pollLines = "    cdc_dev->data.in_polling = false;`n    cdc_dev->notif.xfer_polling = false;`n"
+$firstIf   = "    if (cdc_dev->data.in_xfer) {`n"
+$originalPollLf = $originalLf.Replace("    // Cancel polling of BULK IN and INTERRUPT IN`n",
+                                      "    // Cancel polling of BULK IN and INTERRUPT IN`n" + $pollLines)
+if (-not $contentLf.Contains($originalLf) -and $contentLf.Contains($originalPollLf)) {
+    Write-Host "usb_host_cdc_acm with polling flags (2.4.1+) - keeping them." -ForegroundColor Yellow
+    $originalLf = $originalPollLf
+    $i = $patchedLf.IndexOf($firstIf)
+    $patchedLf = $patchedLf.Substring(0, $i) + $pollLines + $patchedLf.Substring($i)
+    $patchedLf = $patchedLf.Replace("    // unchanged.`n", "    // unchanged. 2026-10-04: usb_host_cdc_acm 2.4.1 restored both flag
  ↪ s (its`n    // poll race fix), so with 2.4.1+ they are cleared here again.`n")
+}
+
 if (-not $contentLf.Contains($originalLf)) {
     Write-Host "Could not find the stock cdc_acm_host_close() block - the component may have changed." -ForegroundColor Red
     Write-Host "Patch by hand and update this script."
     exit 1
 }

-$patchedLf = $patched -replace "`r`n", "`n"
 $result = $contentLf.Replace($originalLf, $patchedLf)
 [System.IO.File]::WriteAllText($target, $result, (New-Object System.Text.UTF8Encoding $false))
```

tools/patches/apply_fatfs_dma_aligned_alloc.ps1

changes against upstream v1.18.0 (unified diff)

```
@@ -109,6 +109,10 @@ $oldInc = @"
 #endif
 "@
 $newInc = '#include "esp_heap_caps.h"'
+# The here-strings carry this script's own line endings, which are CRLF when
+# git checks it out with eol=crlf (always, on macOS/Linux too) - normalise
+# them like $content, or nothing ever matches outside Windows. (2026-10-08)
+$oldInc = $oldInc.Replace("`r`n", "`n")

 if ($content -notmatch [regex]::Escape($oldInc)) {
     if ($content -notmatch [regex]::Escape($newInc)) {
@@ -130,6 +134,7 @@ $oldBody = @"
 #endif
 "@

+$oldBody = $oldBody.Replace("`r`n", "`n")
 if ($content -notmatch [regex]::Escape($oldBody)) {
     Write-Host "Could not find the expected ff_memalloc body - ffsystem.c has changed." -ForegroundColor Red
     Write-Host "Patch by hand and update this script."
@@ -165,6 +170,7 @@ $newBody = @"
 #endif
 "@

+$newBody = $newBody.Replace("`r`n", "`n")
 $content = $content.Replace($oldBody, $newBody)

 if ($crlf) { $content = $content.Replace("`n", "`r`n") }
```

tools/patches/apply_httpd_ws_dead_socket_close.ps1

changes against upstream v1.18.0 (unified diff)

```
@@ -67,6 +67,11 @@ $old = @"
         return ESP_OK;
 "@

+# Match the file's own line endings: this script is CRLF (so $old is too), an
+# IDF tree checked out on Windows is CRLF, but on macOS/Linux it is LF.
+$eol = if ($text.Contains("`r`n")) { "`r`n" } else { "`n" }
+$old = ($old -replace "`r`n", "`n") -replace "`n", $eol
+
 if ($text -notmatch [regex]::Escape($old)) {
     Write-Error "httpd_ws.c does not contain the expected block - IDF version drift? Patch NOT applied."
 }
@@ -87,6 +92,7 @@ $new = @"
         return ESP_FAIL;
 "@

+$new  = ($new -replace "`r`n", "`n") -replace "`n", $eol
 $text = $text.Replace($old, $new)
 Set-Content -LiteralPath $file -Value $text -NoNewline -Encoding UTF8
 Write-Output "Patched httpd_ws.c: a dead WS socket now closes instead of spinning."
```

tools/check_patches.py

changes against upstream v1.18.0 (unified diff)

```
@@ -77,6 +77,29 @@ PATCHES = [
     # Same FILE as the row above, different marker. sdio_drv.c now carries
     # several of our fixes and one apply script restores all of them, so a row
     # per marker is the only way a partially-restored copy shows up as missing.
+    ("apply_esp_hosted_sdio_rx_copy_psram.ps1", "repo",
+     "managed_components/espressif__esp_hosted/host/drivers/transport/sdio/sdio_drv.c",
+     "QMX_SDIO_RX_COPY_PSRAM",
+     "every received WiFi frame is copied into INTERNAL RAM (malloc under "
+     "SPIRAM_MALLOC_ALWAYSINTERNAL) and held until TCP delivers it, so a "
+     "sustained inbound stream exhausts internal/DMA RAM - touch (I2C) and the "
+     "web server stop until reboot - and an allocation failure asserts"),
+
+    ("apply_esp_hosted_sdio_split_bundles.ps1", "repo",
+     "managed_components/espressif__esp_hosted/host/drivers/transport/sdio/sdio_drv.c",
+     "QMX_SDIO_RX_SPLIT",
+     "the C6 streams several frames per SDIO read; without the split only the "
+     "first survives a normal read and a burst is drained and discarded whole - "
+     "a sustained inbound stream (the NAS radio relay) collapses to a fraction "
+     "of its rate under TCP retransmits and starves the UI"),
+
+    ("apply_esp_hosted_sdio_split_psram.ps1", "repo",
+     "managed_components/espressif__esp_hosted/host/drivers/transport/sdio/sdio_drv.c",
+     "QMX_SDIO_RXQ_PSRAM",
+     "received SDIO frames sat in the RX queue in DMA mempool blocks, and that "
+     "pool never returns memory to the heap - the NAS radio stream grew it until "
+     "DMA RAM was gone and the touchscreen's I2C reads failed (no tap, no swipe). "
+     "Stage 2 marker; stage 1 (QMX_SDIO_SPLIT_PSRAM) moved only the split frames."),
     ("apply_esp_hosted_sdio_recovery.ps1", "repo",
      "managed_components/espressif__esp_hosted/host/drivers/transport/sdio/sdio_drv.c",
      "QMX_SDIO_INIT_FAIL_TOLERANT",
```

sdkconfig

changes against upstream v1.18.0 (unified diff)

```
@@ -1999,8 +1999,8 @@ CONFIG_LWIP_TCP_TMR_INTERVAL=250
 CONFIG_LWIP_TCP_MSL=10000
 CONFIG_LWIP_TCP_FIN_WAIT_TIMEOUT=20000
 CONFIG_LWIP_TCP_SND_BUF_DEFAULT=2880
-CONFIG_LWIP_TCP_WND_DEFAULT=2880
-CONFIG_LWIP_TCP_RECVMBOX_SIZE=6
+CONFIG_LWIP_TCP_WND_DEFAULT=11520
+CONFIG_LWIP_TCP_RECVMBOX_SIZE=12
 CONFIG_LWIP_TCP_ACCEPTMBOX_SIZE=6
 CONFIG_LWIP_TCP_QUEUE_OOSEQ=y
 CONFIG_LWIP_TCP_OOSEQ_TIMEOUT=6
@@ -3406,8 +3406,8 @@ CONFIG_TCP_SYNMAXRTX=12
 CONFIG_TCP_MSS=1440
 CONFIG_TCP_MSL=10000
 CONFIG_TCP_SND_BUF_DEFAULT=2880
-CONFIG_TCP_WND_DEFAULT=2880
-CONFIG_TCP_RECVMBOX_SIZE=6
+CONFIG_TCP_WND_DEFAULT=11520
+CONFIG_TCP_RECVMBOX_SIZE=12
 CONFIG_TCP_QUEUE_OOSEQ=y
 CONFIG_TCP_OVERSIZE_MSS=y
 # CONFIG_TCP_OVERSIZE_QUARTER_MSS is not set
```

main/idf_component.yml

changes against upstream v1.18.0 (unified diff)

```
@@ -1,3 +1,8 @@
+# network-radio (2026-10-04): four components are pinned to the versions the
+# official v1.16.11 release was built with (identified from its firmware image).
+# A fresh fetch resolved newer ones (cdc_acm 2.4.1, usb_host_hid 1.2.1,
+# esp_codec_dev 1.6.2, mdns 1.14.0) and that build had NO working touch input on
+# the Tab5, while the official binary of the same source had it.
 dependencies:
   idf:
     version: ">=5.4.0"
@@ -5,19 +10,19 @@ dependencies:
   espressif/esp_lcd_ili9881c: "^1.1.0"
   espressif/esp_lcd_touch_gt911: "^1.1.1~2"
   espressif/esp_lcd_touch_st7123: "^1.0.0"
-  espressif/esp_codec_dev: "^1.3.0"
+  espressif/esp_codec_dev: "1.5.11"     # network-radio: pinned, see note above
   espressif/esp_lvgl_port: "~2.5.0"

-  espressif/usb_host_cdc_acm: "^2.0.0"
+  espressif/usb_host_cdc_acm: "2.4.0"   # network-radio: pinned
   espressif/usb_host_uac: "^1.4.0"
-  espressif/usb_host_hid: "^1.0.0"
+  espressif/usb_host_hid: "1.2.0"       # network-radio: pinned

   espressif/esp-dsp: "^1.6.0"

   # mDNS: reach the Tab5 as qmx.local instead of hunting an IP that changes with
   # the network. Added after an evening lost to exactly that (roamed onto a hotel
   # WiFi, new lease, no way to find it).
-  espressif/mdns: "^1.2.0"
+  espressif/mdns: "1.11.3"               # network-radio: pinned

   lvgl/lvgl: "~9.2.0"
   espressif/esp_wifi_remote:
```

main/CMakeLists.txt

changes against upstream v1.18.0 (unified diff)

```
@@ -103,6 +103,7 @@ idf_component_register(
         "net/filebrowser.c"
         "usb_hid_mouse.c"
         "net/rigctld_server.c"
+        "net/radio_relay.c"
         "net/reader_net.c"
         "net/manual_embed.c"
         "net/update_check.c"
```

13. Application listings

Tab5 application changes (diffs against v1.18.0), then the Synology relay and its scripts, the Raspberry Pi scripts, the Mac tools, the timing-test tools and the update kit, in full.

Tab5 firmware application

main/ft8_tx.c

changes against upstream v1.18.0 (unified diff)

```
@@ -29,6 +29,7 @@
 #include "ft8/js8_text.h"

 #include "cat/cat.h"
+#include "net/radio_relay.h"
 #include "storage/settings.h"

 static const char *TAG = "ft8_tx";
@@ -1298,7 +1299,36 @@ void ft8_tx_run(const ft8_tx_request_t *req)
             ESP_LOGI(TAG, "key down at +%lld ms into the slot",
                      (long long)(key_ms % per_ms));
         }
-        tx_cmd(t0, sim, "TX;");   // key down - radio's own envelope shaping
+        /* (*) RELAY-TIMED BURST (QMXR/1 "txsched=1").
+         * Over the NAS relay, 79 separate tone changes each had to cross WiFi
+         * on time, and twice in 20 test bursts (2026-10-14) one stalled for
+         * 1.1-1.5 s: the Tab5's own timing was perfect, the radio's was not.
+         * So the whole burst goes to the relay in one frame, and the relay
+         * keys the QMX, plays every tone on the NAS's clock over its wired
+         * link and USB, sends the key-up tone and reports back (TX_DONE).
+         * The loop below still runs on the Tab5 clock - for the UI progress,
+         * the live power/SWR query, SWR protection and abort - it just doesn't
+         * send the tones. Any failure handing the burst over falls back to the
+         * old tone-by-tone path, so a burst is never lost to this. */
+        bool relay_play = false;
+#if FT8_TX_SEND_LIVE
+        if (!sim && strcmp(cat_get_transport(), "net") == 0 && radio_relay_txsched_supported()) {
+            float *fr = malloc((size_t)nn * sizeof(float));
+            if (fr) {
+                for (int i = 0; i < nn; i++)
+                    fr[i] = (float)req->audio_freq_hz + (float)req->tones[i] * tone_spacing_hz;
+                esp_err_t se = radio_relay_tx_schedule(fr, nn, (uint32_t)symbol_period_us);
+                free(fr);
+                if (se == ESP_OK) {
+                    relay_play = true;
+                    ESP_LOGI(TAG, "burst handed to the relay (%d tones, relay-timed)", nn);
+                } else {
+                    ESP_LOGW(TAG, "relay-timed burst refused (0x%x) - sending tones from the Tab5", se);
+                }
+            }
+        }
+#endif
+        if (!relay_play) tx_cmd(t0, sim, "TX;");   // key down - radio's own envelope shaping

         // Live power/SWR: fire ONE non-blocking PC;SW; once the PA has settled
         // (symbol 6 ≈ 1 s in), then read the async response a few symbols later
@@ -1347,7 +1377,7 @@ void ft8_tx_run(const ft8_tx_request_t *req)
             }
             float freq = (float)req->audio_freq_hz + (float)req->tones[i] * tone_spacing_hz;
             sleep_until(t0, (int64_t)i * symbol_period_us);
-            tx_cmd(t0, sim, "TA%.2f;", (double)freq);
+            if (!relay_play) tx_cmd(t0, sim, "TA%.2f;", (double)freq);
             // Live power/SWR mid-burst query is FT8-only timing (tuned to FT8's
             // 160 ms symbol slack at symbols 6/14) and is skipped for FT4 on
             // those grounds alone - see the note on the branch below. This
@@ -1474,10 +1504,29 @@ void ft8_tx_run(const ft8_tx_request_t *req)
             // up - otherwise we'd truncate the last tone for receivers.
             sleep_until(t0, (int64_t)nn * symbol_period_us);
         }
+        // Relay-timed: the relay sends the key-up tone itself. Tell it to stop
+        // early on an abort, then wait for its report. If no report comes (link
+        // lost, old relay), fall through and send the key-up tone from here as
+        // well - a second TA0; is harmless, a missing one is not.
+        bool relay_keyed_up = false;
+#if FT8_TX_SEND_LIVE
+        if (relay_play) {
+            if (aborted) radio_relay_tx_abort();
+            radio_relay_tx_result_t rr;
+            if (radio_relay_tx_wait_done(aborted ? 1500 : 4000, &rr)) {
+                relay_keyed_up = true;
+                ESP_LOGI(TAG, "relay burst %s: %u/%d tones, RF on-air %lu ms, latest symbol %+ld ms (NAS clock)",
+                         rr.status == 0 ? "complete" : rr.status == 1 ? "aborted" : "FAILED",
+                         (unsigned)rr.tones_sent, nn, (unsigned long)rr.duration_ms, (long)rr.latest_ms);
+            } else {
```

```
+                ESP_LOGW(TAG, "no report from the relay for the burst - sending key-up from the Tab5");
+            }
+        }
+#endif
         // Either way - whether all symbols played or we broke out early
         // on an abort request - key up immediately now. This is the part
         // that must ALWAYS run: the radio must never be left transmitting.
-        {
+        if (!relay_keyed_up) {
             char keyup[24];
             snprintf(keyup, sizeof(keyup), "TA%.0f;", (double)FT8_TX_KEYUP_TONE_HZ);
             tx_cmd_critical(t0, sim, keyup);   // drops the envelope
```

main/ui/ui.c

changes against upstream v1.18.0 (unified diff)

```
@@ -86,6 +86,7 @@ LV_FONT_DECLARE(qmx_mono_25);   /* shared with the radio-menus screen */
 #include "ft8_test.h"
 #include "esp_lcd_touch.h"
 #include "esp_system.h"        // esp_restart - the drawer's "Restart now" (#270)
+#include "radio_relay.h"       // Radio source: QMX on the Tab5 USB port or on a NAS
 #include "bt_hid_mouse.h"      // is the BLE radio actually up, and was it at boot

 static const char *TAG = "ui";
@@ -3100,7 +3101,15 @@ static bool s_drawer_swipe_vertical = false;  /* this drag went vertical */
 //   lv_obj_has_flag_any <- lv_obj_is_layout_positioned
 //
 // A comment cannot stop this on its own, so the assert below now does.
-#define N_DRAWER_SECTIONS     50
+#define N_DRAWER_SECTIONS     51
+// "Radio source" - which QMX the Tab5 talks to: the one on its own USB port,
+// or one plugged into a NAS running tools/nas-relay. Same effect as the web
+// page's POST /api/relay (settings_set_relay + radio_relay_set_target +
+// cat_reconnect), so the switch no longer needs a computer.
+// (50: upstream v1.18.0 took 47 PORTA, 48 GPSSTATS, 49 KEYER.)
+#define DRAWER_SEC_RADIOSRC   50
+// (!) IDs are 0..50 now. The next one raises N_DRAWER_SECTIONS AND gets an
+// assert below - see the 2026-10-03 note above.

 /* Catches the mistake above at COMPILE time instead of as a crash minutes into
  * a session. Every id must be a valid index; raise N_DRAWER_SECTIONS when you
@@ -3110,6 +3119,8 @@ _Static_assert(DRAWER_SEC_SDEJECT < N_DRAWER_SECTIONS, "raise N_DRAWER_SECTIONS"
 _Static_assert(DRAWER_SEC_OUTPWR  < N_DRAWER_SECTIONS, "raise N_DRAWER_SECTIONS");
 _Static_assert(DRAWER_SEC_PORTA   < N_DRAWER_SECTIONS, "raise N_DRAWER_SECTIONS");
 _Static_assert(DRAWER_SEC_KEYER   < N_DRAWER_SECTIONS, "raise N_DRAWER_SECTIONS");
+_Static_assert(DRAWER_SEC_RADIOSRC < N_DRAWER_SECTIONS, "raise N_DRAWER_SECTIONS");
+_Static_assert(N_DRAWER_SECTIONS <= 64, "drawer masks are uint64_t");
 static lv_obj_t *s_drawer_sections[N_DRAWER_SECTIONS];
 static int       s_drawer_section_y[N_DRAWER_SECTIONS];
 static int       s_drawer_section_h[N_DRAWER_SECTIONS];
@@ -3142,6 +3153,7 @@ static const drawer_item_t GRP_STATION[] = {
     { DRAWER_SEC_BPREGION, "Band-plan region", false },
 };
 static const drawer_item_t GRP_RADIO[] = {
+    { DRAWER_SEC_RADIOSRC, "Radio source (USB / NAS)", true },
     { DRAWER_SEC_QMXVOL, "QMX volume", true },
     { DRAWER_SEC_QMXRF, "RF gain", true },
     /* Above CW centre (operator, 2026-09-07). It was filed under Spectrum with
@@ -3713,6 +3725,9 @@ static void gain_resolve_stop(void);
 static void drawer_pause_btn_cb(lv_event_t *e);
 static void drawer_term_btn_cb(lv_event_t *e);
 static void drawer_gpsstats_btn_cb(lv_event_t *e);
+static void drawer_radiosrc_btn_cb(lv_event_t *e);
+static void radiosrc_label_timer_cb(lv_timer_t *t);
+static lv_obj_t *s_radiosrc_lbl;
 static void topbar_reconcile_cb(lv_timer_t *t);
 static void drawer_slider_cwtxoff_cb(lv_event_t *e);
 static void ui_set_cw_tx_offset_label(int hz);
@@ -13105,6 +13120,29 @@ static void drawer_build(void)
     //
     // Directly under the pause button on purpose: an operator reaching for
     // "let me use the QMX menus" on a radio with no front panel wanted this.
+    // "Radio source" - USB or NAS, with the link state on the button itself so
+    // the drawer answers "which radio am I on, and is it there" at a glance.
+    // The label is refreshed once a second by radiosrc_label_timer_cb.
+    {
+        lv_obj_t *sec = drawer_section(DRAWER_SEC_RADIOSRC, y, 72);
+        lv_obj_t *btn = lv_btn_create(sec);
+        lv_obj_set_size(btn, DRAWER_W - 32, 56);
+        lv_obj_align(btn, LV_ALIGN_TOP_LEFT, 0, 0);
+        lv_obj_set_style_bg_color(btn, lv_color_hex(0x2a3138), 0);
+        lv_obj_set_style_border_color(btn, lv_color_hex(UI_COLOR_PRIMARY), 0);
+        lv_obj_set_style_border_width(btn, 2, 0);
+        lv_obj_add_event_cb(btn, drawer_radiosrc_btn_cb, LV_EVENT_CLICKED, NULL);
+        s_radiosrc_lbl = lv_label_create(btn);
+        lv_label_set_text(s_radiosrc_lbl, "Radio source");
+        lv_obj_set_style_text_font(s_radiosrc_lbl, &lv_font_montserrat_24, 0);
```

```
+        lv_obj_set_style_text_color(s_radiosrc_lbl, lv_color_hex(0xffffff), 0);
+        lv_obj_center(s_radiosrc_lbl);
+        static lv_timer_t *s_radiosrc_timer;
+        if (!s_radiosrc_timer) s_radiosrc_timer = lv_timer_create(radiosrc_label_timer_cb, 1000, NULL);
+        radiosrc_label_timer_cb(NULL);
+        y += 72;
+    }
+
     {
         lv_obj_t *sec = drawer_section(DRAWER_SEC_TERM, y, 72);
         lv_obj_t *btn = lv_btn_create(sec);
@@ -16137,6 +16175,305 @@ static void drawer_bt_restart_cb(lv_event_t *e)
  * physically swap a card - which is exactly when it disappears. It also has a
  * second instruction after the swap, so it has to still be there afterwards.
  * A window waits. */
+/* ---- Radio source (USB / NAS) ------------------------------------------
+ * Built once and hidden/shown, not rebuilt per open: ui_kbd_set_buttons()
+ * keeps a bounded registry keyed by the button pointers, and a modal rebuilt
+ * on every open would use up a slot each time. */
+#define RADIOSRC_NS "radiosrc"          /* remembers the last NAS address, so
+                                         * USB -> NAS is not a retype */
+static lv_obj_t *s_radiosrc_modal;
+static lv_obj_t *s_radiosrc_ta;
+static lv_obj_t *s_radiosrc_state;
+static lv_obj_t *s_radiosrc_kb;
+static uint16_t  s_radiosrc_port = RADIO_RELAY_DEFAULT_PORT;
+
+static lv_obj_t *s_radiosrc_find_btn;
+static lv_obj_t *s_radiosrc_find_lbl;
+
+/* ---- "Find NAS": ask the network which relays are there ------------------
+ * radio_relay_discover() blocks for ~1.5 s, so it runs on a short-lived task
+ * of its own and hands the answer back under the display lock. One search at
+ * a time; the task deletes itself (internal-RAM stack, freed on delete). */
+#define RADIOSRC_FIND_MAX 4
+static radio_relay_found_t s_radiosrc_found[RADIOSRC_FIND_MAX];
+static volatile bool       s_radiosrc_finding;
+
+static void radiosrc_show_found(int n)
+{
+    if (!s_radiosrc_state || !s_radiosrc_ta) return;
+    if (s_radiosrc_find_lbl) lv_label_set_text(s_radiosrc_find_lbl, LV_SYMBOL_REFRESH "  Find NAS");
+    char st[160];
+    uint32_t col = 0xE0A040;
+    if (n < 0) {
+        snprintf(st, sizeof(st), "WiFi is not connected - cannot search. Connect WiFi, or type the address.");
+    } else if (n == 0) {
+        snprintf(st, sizeof(st), "No NAS answered. Is the relay running, and on the same network as the Tab5?");
+    } else {
+        lv_textarea_set_text(s_radiosrc_ta, s_radiosrc_found[0].ip);
+        s_radiosrc_port = s_radiosrc_found[0].port;
+        int len = snprintf(st, sizeof(st), "Found %s", s_radiosrc_found[0].ip);
+        for (int i = 1; i < n && len < (int)sizeof(st) - 20; i++)
+            len += snprintf(st + len, sizeof(st) - len, "%s%s", i == 1 ? "   (also: " : ", ",
+                            s_radiosrc_found[i].ip);
+        if (n > 1 && len < (int)sizeof(st) - 2) snprintf(st + len, sizeof(st) - len, ")");
+        col = 0x40C060;
+    }
+    lv_label_set_text(s_radiosrc_state, st);
+    lv_obj_set_style_text_color(s_radiosrc_state, lv_color_hex(col), 0);
+}
+
+static void radiosrc_find_task(void *arg)
+{
+    (void)arg;
+    int n = radio_relay_discover(s_radiosrc_found, RADIOSRC_FIND_MAX, 1500);
+    if (display_lock(2000)) {
+        radiosrc_show_found(n);
+        display_unlock();
+    }
+    s_radiosrc_finding = false;
+    vTaskDelete(NULL);
+}
+
+static void radiosrc_find_cb(lv_event_t *e)
+{
+    (void)e;
+    if (s_radiosrc_finding) return;
+    s_radiosrc_finding = true;
+    lv_label_set_text(s_radiosrc_find_lbl, "Searching...");
+    lv_label_set_text(s_radiosrc_state, "Asking the network for a NAS running the QMX relay...");
+    lv_obj_set_style_text_color(s_radiosrc_state, lv_color_hex(0xC8D0D8), 0);
+    if (xTaskCreate(radiosrc_find_task, "nas_find", 4096, NULL, 4, NULL) != pdPASS) {
+        s_radiosrc_finding = false;
+        radiosrc_show_found(-1);
+    }
+}
```

```
+
+static void radiosrc_label_timer_cb(lv_timer_t *t)
+{
+    (void)t;
+    if (!s_radiosrc_lbl) return;
+    char host[RADIO_RELAY_HOST_LEN] = "";
+    uint16_t port = 0;
+    radio_relay_get_target(host, sizeof(host), &port);
+    char txt[RADIO_RELAY_HOST_LEN + 48];
+    if (!radio_relay_enabled() || !host[0]) {
+        const bool up = (strcmp(cat_get_transport(), "usb") == 0);
+        snprintf(txt, sizeof(txt), LV_SYMBOL_USB "  Radio: USB%s", up ? "" : " (no QMX)");
+    } else {
+        snprintf(txt, sizeof(txt), LV_SYMBOL_WIFI "  NAS %s%s", host,
+                 radio_relay_connected() ? "  " LV_SYMBOL_OK : " (offline)");
+    }
+    const char *cur = lv_label_get_text(s_radiosrc_lbl);
+    if (!cur || strcmp(cur, txt) != 0) lv_label_set_text(s_radiosrc_lbl, txt);
+}
+
+static void radiosrc_last_get(char *buf, size_t cap)
+{
+    buf[0] = '\0';
+    nvs_handle_t h;
+    if (nvs_open(RADIOSRC_NS, NVS_READONLY, &h) != ESP_OK) return;
+    size_t n = cap;
+    if (nvs_get_str(h, "last", buf, &n) != ESP_OK) buf[0] = '\0';
+    nvs_close(h);
+}
+
+static void radiosrc_last_set(const char *host)
+{
+    nvs_handle_t h;
+    if (nvs_open(RADIOSRC_NS, NVS_READWRITE, &h) != ESP_OK) return;
+    nvs_set_str(h, "last", host);
+    nvs_commit(h);
+    nvs_close(h);
+}
+
+static void radiosrc_close(void)
+{
+    if (s_radiosrc_ta) lv_obj_clear_state(s_radiosrc_ta, LV_STATE_FOCUSED);
+    if (s_radiosrc_modal) lv_obj_add_flag(s_radiosrc_modal, LV_OBJ_FLAG_HIDDEN);
+}
+
+/* The same three calls POST /api/relay makes (webserver.c relay_post_handler):
+ * persist, retarget, and drop the current link so cat's link_task reopens on
+ * the new source. Empty host = USB. */
+static void radiosrc_apply(const char *host)
+{
+    settings_set_relay(host, s_radiosrc_port);
+    radio_relay_set_target(host, s_radiosrc_port);
+    cat_reconnect();
+    radiosrc_label_timer_cb(NULL);
+    radiosrc_close();
+    drawer_close();
+    char msg[RADIO_RELAY_HOST_LEN + 48];
+    if (host[0]) snprintf(msg, sizeof(msg), "Radio source: NAS %s - connecting...", host);
+    else         snprintf(msg, sizeof(msg), "Radio source: USB - plug the QMX into the Tab5");
+    ui_toast_ms(msg, 3000);
+}
+
+static void radiosrc_use_nas_cb(lv_event_t *e)
+{
+    (void)e;
+    char host[RADIO_RELAY_HOST_LEN];
+    const char *t = lv_textarea_get_text(s_radiosrc_ta);
+    while (*t == ' ') t++;
+    snprintf(host, sizeof(host), "%s", t);
+    for (int i = (int)strlen(host) - 1; i >= 0 && host[i] == ' '; i--) host[i] = '\0';
+    if (!host[0]) {
+        lv_label_set_text(s_radiosrc_state, "Type the NAS address first (e.g. 192.168.1.20).");
+        lv_obj_set_style_text_color(s_radiosrc_state, lv_color_hex(0xE0A040), 0);
+        return;
+    }
+    radiosrc_last_set(host);
+    radiosrc_apply(host);
+}
+
+static void radiosrc_use_usb_cb(lv_event_t *e)
+{
+    (void)e;
+    /* Keep what is typed as the remembered NAS, so switching back is one tap. */
+    const char *t = lv_textarea_get_text(s_radiosrc_ta);
+    if (t && t[0]) radiosrc_last_set(t);
+    radiosrc_apply("");
+}
```

```
+
+static void radiosrc_cancel_cb(lv_event_t *e) { (void)e; radiosrc_close(); }
+static void radiosrc_swallow_cb(lv_event_t *e) { (void)e; }
+
+static void radiosrc_kb_cb(lv_event_t *e)
+{
+    lv_event_code_t c = lv_event_get_code(e);
+    if (c == LV_EVENT_READY)  radiosrc_use_nas_cb(e);    /* keyboard's OK key */
+    if (c == LV_EVENT_CANCEL) radiosrc_close();
+}
+
+static lv_obj_t *radiosrc_btn(lv_obj_t *p, const char *txt, uint32_t bg, int x,
+                              lv_event_cb_t cb)
+{
+    lv_obj_t *b = lv_btn_create(p);
+    lv_obj_set_size(b, 250, 64);
+    lv_obj_align(b, LV_ALIGN_BOTTOM_MID, x, -12);
+    lv_obj_set_style_bg_color(b, lv_color_hex(bg), 0);
+    lv_obj_set_style_radius(b, 8, 0);
+    lv_obj_add_event_cb(b, cb, LV_EVENT_CLICKED, NULL);
+    lv_obj_t *l = lv_label_create(b);
+    lv_label_set_text(l, txt);
+    lv_obj_set_style_text_font(l, &lv_font_montserrat_28, 0);
+    lv_obj_set_style_text_color(l, lv_color_hex(0xffffff), 0);
+    lv_obj_center(l);
+    return b;
+}
+
+static void radiosrc_modal_build(void)
+{
+    if (s_radiosrc_modal) return;
+    s_radiosrc_modal = lv_obj_create(lv_screen_active());
+    lv_obj_set_size(s_radiosrc_modal, LV_PCT(100), LV_PCT(100));
+    lv_obj_set_pos(s_radiosrc_modal, 0, 0);
+    lv_obj_set_style_bg_color(s_radiosrc_modal, lv_color_hex(0x000000), 0);
+    lv_obj_set_style_bg_opa(s_radiosrc_modal, UI_OPA_MODAL_SCRIM, 0);
+    lv_obj_set_style_border_width(s_radiosrc_modal, 0, 0);
+    lv_obj_set_style_pad_all(s_radiosrc_modal, 0, 0);
+    lv_obj_clear_flag(s_radiosrc_modal, LV_OBJ_FLAG_SCROLLABLE);
+    lv_obj_add_flag(s_radiosrc_modal, LV_OBJ_FLAG_CLICKABLE);
+    lv_obj_add_event_cb(s_radiosrc_modal, radiosrc_swallow_cb, LV_EVENT_CLICKED, NULL);
+
+    lv_obj_t *p = lv_obj_create(s_radiosrc_modal);
+    lv_obj_set_size(p, 900, 380);
+    lv_obj_align(p, LV_ALIGN_TOP_MID, 0, 20);
+    lv_obj_set_style_bg_color(p, lv_color_hex(0x1c2128), 0);
+    lv_obj_set_style_bg_opa(p, LV_OPA_COVER, 0);
+    lv_obj_set_style_border_color(p, lv_color_hex(UI_COLOR_PRIMARY), 0);
+    lv_obj_set_style_border_width(p, 2, 0);
+    lv_obj_set_style_radius(p, 10, 0);
+    lv_obj_clear_flag(p, LV_OBJ_FLAG_SCROLLABLE);
+
+    lv_obj_t *t = lv_label_create(p);
+    lv_label_set_text(t, "Radio source");
+    lv_obj_set_style_text_font(t, &lv_font_montserrat_28, 0);
+    lv_obj_set_style_text_color(t, lv_color_hex(0xffffff), 0);
+    lv_obj_align(t, LV_ALIGN_TOP_MID, 0, 4);
+
+    s_radiosrc_state = lv_label_create(p);
+    lv_label_set_long_mode(s_radiosrc_state, LV_LABEL_LONG_WRAP);
+    lv_obj_set_width(s_radiosrc_state, 840);
+    lv_obj_set_style_text_font(s_radiosrc_state, &lv_font_montserrat_24, 0);
+    lv_obj_align(s_radiosrc_state, LV_ALIGN_TOP_LEFT, 10, 50);
+
+    lv_obj_t *hl = lv_label_create(p);
+    lv_label_set_text(hl, "NAS address:");
+    lv_obj_set_style_text_font(hl, &lv_font_montserrat_24, 0);
+    lv_obj_set_style_text_color(hl, lv_color_hex(0xE6E6E6), 0);
+    lv_obj_align(hl, LV_ALIGN_TOP_LEFT, 10, 158);
+
+    s_radiosrc_ta = lv_textarea_create(p);
+    lv_textarea_set_one_line(s_radiosrc_ta, true);
+    lv_textarea_set_max_length(s_radiosrc_ta, RADIO_RELAY_HOST_LEN - 1);
+    lv_textarea_set_placeholder_text(s_radiosrc_ta, "e.g. 192.168.1.20");
+    lv_obj_set_size(s_radiosrc_ta, 440, 60);
+    lv_obj_align(s_radiosrc_ta, LV_ALIGN_TOP_LEFT, 200, 144);
+    lv_obj_set_style_text_font(s_radiosrc_ta, &lv_font_montserrat_28, 0);
+    ui_theme_style_textarea(s_radiosrc_ta);
+
+    s_radiosrc_find_btn = lv_btn_create(p);
+    lv_obj_set_size(s_radiosrc_find_btn, 200, 60);
+    lv_obj_align(s_radiosrc_find_btn, LV_ALIGN_TOP_LEFT, 656, 144);
+    lv_obj_set_style_bg_color(s_radiosrc_find_btn, lv_color_hex(0x2a3138), 0);
+    lv_obj_set_style_border_color(s_radiosrc_find_btn, lv_color_hex(UI_COLOR_PRIMARY), 0);
+    lv_obj_set_style_border_width(s_radiosrc_find_btn, 2, 0);
+    lv_obj_set_style_radius(s_radiosrc_find_btn, 8, 0);
+    lv_obj_add_event_cb(s_radiosrc_find_btn, radiosrc_find_cb, LV_EVENT_CLICKED, NULL);
```

```
+    s_radiosrc_find_lbl = lv_label_create(s_radiosrc_find_btn);
+    lv_label_set_text(s_radiosrc_find_lbl, LV_SYMBOL_REFRESH "  Find NAS");
+    lv_obj_set_style_text_font(s_radiosrc_find_lbl, &lv_font_montserrat_24, 0);
+    lv_obj_set_style_text_color(s_radiosrc_find_lbl, lv_color_hex(0xffffff), 0);
+    lv_obj_center(s_radiosrc_find_lbl);
+
+    lv_obj_t *nas = radiosrc_btn(p, LV_SYMBOL_WIFI "  Use NAS", UI_COLOR_PRIMARY, -280, radiosrc_use_nas_cb);
+    radiosrc_btn(p, LV_SYMBOL_USB "  Use USB", 0x3a4149, 0, radiosrc_use_usb_cb);
+    lv_obj_t *cancel = radiosrc_btn(p, "Cancel", 0x3a4149, 280, radiosrc_cancel_cb);
+    ui_kbd_set_buttons(nas, cancel);       /* physical keyboard: Enter / Esc */
+
+    s_radiosrc_kb = lv_keyboard_create(s_radiosrc_modal);
+    ui_theme_style_keyboard(s_radiosrc_kb);
+    lv_keyboard_set_mode(s_radiosrc_kb, LV_KEYBOARD_MODE_NUMBER);
+    lv_obj_set_size(s_radiosrc_kb, LV_PCT(100), 280);
+    lv_obj_align(s_radiosrc_kb, LV_ALIGN_BOTTOM_MID, 0, 0);
+    lv_keyboard_set_textarea(s_radiosrc_kb, s_radiosrc_ta);
+    lv_obj_add_event_cb(s_radiosrc_kb, radiosrc_kb_cb, LV_EVENT_ALL, NULL);
+}
+
+static void radiosrc_modal_show(void)
+{
+    radiosrc_modal_build();
+    char host[RADIO_RELAY_HOST_LEN] = "";
+    uint16_t port = 0;
+    radio_relay_get_target(host, sizeof(host), &port);
+    s_radiosrc_port = port ? port : RADIO_RELAY_DEFAULT_PORT;
+    const bool nas = radio_relay_enabled() && host[0];
+
+    char st[RADIO_RELAY_HOST_LEN + 96];
+    if (nas) {
+        snprintf(st, sizeof(st), "Now: QMX on the NAS at %s, port %u - %s.", host,
+                 (unsigned)s_radiosrc_port,
+                 radio_relay_connected() ? "connected" : "not connected");
+    } else {
+        radiosrc_last_get(host, sizeof(host));
+        snprintf(st, sizeof(st), "Now: QMX on the Tab5's USB port - %s.",
+                 strcmp(cat_get_transport(), "usb") == 0 ? "connected" : "no QMX found");
+    }
+    lv_label_set_text(s_radiosrc_state, st);
+    lv_obj_set_style_text_color(s_radiosrc_state, lv_color_hex(0xC8D0D8), 0);
+    lv_textarea_set_text(s_radiosrc_ta, host);
+    lv_obj_clear_flag(s_radiosrc_modal, LV_OBJ_FLAG_HIDDEN);
+    lv_obj_move_foreground(s_radiosrc_modal);
+}
+
+static void drawer_radiosrc_btn_cb(lv_event_t *e)
+{
+    (void)e;
+    drawer_close();            /* the modal and its keyboard need the screen */
+    radiosrc_modal_show();
+}
+
 static lv_obj_t *s_sdeject_modal;

 static void sdeject_modal_close_cb(lv_event_t *e)
```

main/net/webserver.c

changes against upstream v1.18.0 (unified diff)

```
@@ -87,6 +87,7 @@
 #include <ctype.h>
 #include "net/manual_embed.h"  // manual_embed_get - /api/manual serves the built-in manual
 #include "config_io.h"         // config_io_export / config_io_import
+#include "radio_relay.h"       // network radio source (QMX on a NAS)
 #include "usb_replug.h"        // usb_replug (hidden /api/cmd recovery action)
 #include "gpio_relay.h"        // gpio_pulse - remote relay for a QMX power cycle
 #include "unit_gps/unit_gps.h" // /api/status port.gps - the PORT.A pipeline state
@@ -618,6 +619,7 @@ static esp_err_t status_handler(httpd_req_t *req)
     }

     cJSON_AddNumberToObject(root, "freq_hz",     (double)cat_get_frequency());
+    cJSON_AddStringToObject(root, "radio_link",  cat_get_transport());   // "usb" / "net" / "none"
     cJSON_AddStringToObject(root, "qmx_fw",       cat_get_qmx_fw());
     cJSON_AddStringToObject(root, "mode",         ui_get_mode_str());
     // Decoded CW, EXACTLY the line the Tab5 is drawing - same grid, same wrap
@@ -4309,6 +4311,70 @@ static esp_err_t eqsl_upload_handler(httpd_req_t *req)
     return httpd_resp_sendstr(req, "{\"status\":\"uploading\"}");
 }

+// GET  /api/relay -> {"host","port","enabled","connected","transport","iq_pairs","last_error"}
+// POST /api/relay  {"host":"192.168.1.20","port":7355}  (host "" = back to USB)
+// Network radio: the QMX plugged into a NAS running tools/nas-relay. Applies
+// immediately: persists, retargets, and drops the current link so cat.c's
+// link_task reconnects over the newly selected transport.
+static esp_err_t relay_get_handler(httpd_req_t *req)
+{
```

```
+    char host[RADIO_RELAY_HOST_LEN];
+    uint16_t port;
+    radio_relay_get_target(host, sizeof(host), &port);
+    cJSON *root = cJSON_CreateObject();
+    if (!root) return httpd_resp_send_500(req);
+    cJSON_AddStringToObject(root, "host", host);
+    cJSON_AddNumberToObject(root, "port", port);
+    cJSON_AddBoolToObject(root, "enabled", radio_relay_enabled());
+    cJSON_AddBoolToObject(root, "connected", radio_relay_connected());
+    cJSON_AddStringToObject(root, "transport", cat_get_transport());
+    cJSON_AddNumberToObject(root, "iq_pairs", (double)radio_relay_iq_pairs_total());
+    cJSON_AddStringToObject(root, "last_error", radio_relay_last_error());
+    char *out = cJSON_PrintUnformatted(root);
+    cJSON_Delete(root);
+    if (!out) return httpd_resp_send_500(req);
+    httpd_resp_set_type(req, "application/json");
+    esp_err_t err = httpd_resp_sendstr(req, out);
+    cJSON_free(out);
+    return err;
+}
+
+static esp_err_t relay_post_handler(httpd_req_t *req)
+{
+    char buf[160];
+    int len = httpd_req_recv(req, buf, sizeof(buf) - 1);
+    if (len < 0) {
+        httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "no body");
+        return ESP_FAIL;
+    }
+    buf[len] = '\0';
+    cJSON *root = cJSON_Parse(buf);
+    if (!root) {
+        httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "bad json");
+        return ESP_FAIL;
+    }
+    const char *host = cJSON_GetStringValue(cJSON_GetObjectItem(root, "host"));
+    cJSON *jp = cJSON_GetObjectItem(root, "port");
+    int port = cJSON_IsNumber(jp) ? jp->valueint : RADIO_RELAY_DEFAULT_PORT;
+    if (port <= 0 || port > 65535 || (host && strlen(host) >= RADIO_RELAY_HOST_LEN)) {
+        cJSON_Delete(root);
+        httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "bad host/port");
+        return ESP_FAIL;
+    }
+    settings_set_relay(host ? host : "", (uint16_t)port);
+    radio_relay_set_target(host ? host : "", (uint16_t)port);
+    cJSON_Delete(root);
+    cat_reconnect();
+    return relay_get_handler(req);
+}
+
+static const httpd_uri_t uri_relay_get = {
+    .uri = "/api/relay", .method = HTTP_GET, .handler = relay_get_handler,
+};
+static const httpd_uri_t uri_relay_post = {
+    .uri = "/api/relay", .method = HTTP_POST, .handler = relay_post_handler,
+};
+
 static const httpd_uri_t uri_eqsl_creds = {
     .uri = "/api/eqsl_creds", .method = HTTP_POST, .handler = eqsl_creds_handler,
 };
@@ -7814,6 +7880,8 @@ esp_err_t webserver_start(void)
     reg(&uri_help);
     reg(&uri_manual);
     reg(&uri_eqsl_creds);
+    reg(&uri_relay_get);    // network radio (QMX on a NAS)
+    reg(&uri_relay_post);
     reg(&uri_qrz_lookup_creds);
     reg(&uri_eqsl_upload);
     reg(&uri_cloudlog_creds);
```

main/net/www/index.html

changes against upstream v1.18.0 (unified diff)

```
@@ -1118,7 +1118,7 @@

   <div id="bandplan-wrap"><canvas id="bandplan" title="Whole-band plan - drag or tap to retune"></canvas></div>

-  <div id="ft8-notice"><div id="ft8-topline"><div id="ft8-status"><div id="ft8-tx" class="st-idle">--</div><div id="ft8-countdown"
  ↪ class="ft8-stat" hidden><div class="k">TX in</div><div class="v" id="ft8-countdown-v">--</div></div><div id="ft8-stats"></div><d
  ↪ iv id="ft8-webr"></div></div></div><div id="ft8-body"><div id="ft8-slotrow"><canvas id="ft8-eo" width="1000" height="72"></canva
  ↪ s><div id="ft8-slot"><div id="slot-sec"><b id="slot-n">--</b><i>sec</i></div><div id="slot-meter"><div id="slot-track"><div id="
  ↪ slot-fill"></div></div></div></div></div><div id="ft8-controls"><div id="ft8-actions"><button id="cq-restart" title="Start a fre
  ↪ sh CQ run on the Tab5">Call CQ</button><button id="cq-parity" title="Which 15-second slot your CQ goes out in. ANY uses whicheve
  ↪ r comes first; EVEN and ODD lock it.">TXCQ ANY</button><select id="ft8-preset" title="Conventional FT8/FT4 calling frequencies -
  ↪  the same list the Tab5 offers"><option value="">Band…</option></select><button id="tone-btn" title="Choose the audio tone you t
  ↪ ransmit on, against the live occupancy of the band">TX tone</button><button id="ft8opts-btn" title="FT8-specific settings: filte
  ↪ rs, robot, pileup, Field Day - the same panel the Tab5's own Options button opens">Options<span id="ft8opts-badge" class="badge-
  ↪ count"></span></button></div><div id="ft8-override" style="visibility:hidden;"><span class="ovr-label">Mid-QSO:</span><button id
  ↪ ="ovr-resend" title="Send the current message again">Re-send</button><button id="ovr-rr73" title="Jump straight to RR73">RR73</b
  ↪ utton><button id="ovr-73" title="Jump straight to 73">73</button><button id="ovr-pause" title="Skip the next transmission withou
  ↪ t ending the exchange - use the listening slot to check the frequency">Pause</button><button id="ovr-cancel" class="btn-danger"
  ↪ title="Stop transmitting and end this exchange">Cancel</button></div></div></div>
+  <div id="ft8-notice"><div id="ft8-topline"><div id="ft8-status"><div id="ft8-tx" class="st-idle">--</div><div id="ft8-countdown"
  ↪ class="ft8-stat" hidden><div class="k">TX in</div><div class="v" id="ft8-countdown-v">--</div></div><div id="ft8-stats"></div><d
  ↪ iv id="ft8-webr"></div></div></div><div id="ft8-body"><div id="ft8-slotrow"><canvas id="ft8-eo" width="1000" height="72"></canva
  ↪ s><div id="ft8-slot"><div id="slot-sec"><b id="slot-n">--</b><i>sec</i></div><div id="slot-meter"><div id="slot-track"><div id="
  ↪ slot-fill"></div></div></div></div></div><div id="ft8-controls"><div id="ft8-actions"><button id="cq-restart" title="Start a fre
  ↪ sh CQ run on the Tab5">Call CQ</button><button id="burst-test" hidden title="Transmit test: ten CQ bursts in a row (stop after 1
  ↪ 0 calls, no listening slot), then put both settings back">10-burst test</button><button id="cq-parity" title="Which 15-second sl
  ↪ ot your CQ goes out in. ANY uses whichever comes first; EVEN and ODD lock it.">TXCQ ANY</button><select id="ft8-preset" title="C
  ↪ onventional FT8/FT4 calling frequencies - the same list the Tab5 offers"><option value="">Band…</option></select><button id="ton
  ↪ e-btn" title="Choose the audio tone you transmit on, against the live occupancy of the band">TX tone</button><button id="ft8opts
  ↪ -btn" title="FT8-specific settings: filters, robot, pileup, Field Day - the same panel the Tab5's own Options button opens">Opti
  ↪ ons<span id="ft8opts-badge" class="badge-count"></span></button></div><div id="ft8-override" style="visibility:hidden;"><span cl
  ↪ ass="ovr-label">Mid-QSO:</span><button id="ovr-resend" title="Send the current message again">Re-send</button><button id="ovr-rr
  ↪ 73" title="Jump straight to RR73">RR73</button><button id="ovr-73" title="Jump straight to 73">73</button><button id="ovr-pause"
  ↪  title="Skip the next transmission without ending the exchange - use the listening slot to check the frequency">Pause</button><b
  ↪ utton id="ovr-cancel" class="btn-danger" title="Stop transmitting and end this exchange">Cancel</button></div></div></div>
     <div id="ft8-decodes" style="width:100%; max-width:1000px; margin:14px auto 0; text-align:left;">
       <div style="display:flex; align-items:baseline; gap:10px; margin-bottom:4px;">
         <strong id="dec-title" style="color:#ffa040">Decode list</strong>
@@ -1367,6 +1367,7 @@
       <button type="button" class="bot-btn" id="radio-group-btn">Radio &#9650;</button>
       <div class="bot-menu" id="radio-menu">
         <a href="#" id="screen-toggle-link" title="Switch the Tab5 between the panadapter and the FT8/FT4 screen">Switch to FT8/FT4
  ↪ </a>
+        <a href="#" id="relay-link" title="Where the QMX is: on the Tab5's own USB port, or plugged into your NAS and reached over
  ↪ the network (tools/nas-relay). Click to change.">Radio source: --</a>
         <a href="#" id="memory-btn" title="Your 32 memory channels - click one to tune to it">Memory channels</a>
         <a href="#" id="pause-btn" title="Stops the Tab5 talking to the radio, so its own menus and the Band Configuration terminal
  ↪  app behave normally. Nothing is decoded until you hand it back">Let me use the QMX menus</a>
         <a href="#" id="tune-ant-btn" style="display:none" title="Antenna Tune (QMX 1.04+): keys the radio with a steady carrier an
  ↪ d shows live power and SWR. Auto-stops after 60 s.">Antenna Tune</a>
@@ -3627,6 +3628,97 @@ async function sendCmd(payload) {
    then held disabled briefly: the device defers the request to its own 1 Hz timer,
    so a second click inside that window would do nothing visible and invite
    double-tapping. */
+/* 10-burst test (network-radio), shown only at http://qmx.local/?test.
+   One click sets up a transmit timing test:
+   CQ "Stop after" = 10 calls and no listening slot, so the ten bursts follow
+   each other with nothing skipped; switches the Tab5 to FT8 if needed; starts
+   the CQ run; counts the bursts from /api/status; and when the run ends -
+   ten sent, answered, cancelled - puts both settings back as they were.
+   The previous values are kept in localStorage so a reload mid-run still
+   restores them. Compare the result with tools/tx-test/tx_compare.py. */
+(function wireBurstTest() {
+  const btn = document.getElementById("burst-test");
+  if (!btn) return;
+  const KEY = "qmxBurstTest";
+  const N = 10;
+  let st = null;                        // { prev:{max_calls,listen_every}, sent, active, phase }
+  try { st = JSON.parse(localStorage.getItem(KEY) || "null"); } catch (e) { st = null; }
+  /* Test equipment, not an operating control: the button only appears when the
+     page is opened as http://qmx.local/?test (or while a test it started is
+     still running, so it can always be stopped). Everyone else never sees it. */
+  if (!/[?&]test\b/.test(location.search) && !st) return;
+  btn.hidden = false;
+  const save = () => { try { st ? localStorage.setItem(KEY, JSON.stringify(st)) : localStorage.removeItem(KEY); } catch (e) {} };
+  const label = (t) => { btn.textContent = t; };
+  const postCq = (cq) => fetch("/api/settings", {
+    method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ cq })
+  }).then(r => r.ok).catch(() => false);
+
+  async function finish(why) {
+    if (!st || st.phase === "finishing") return;
+    st.phase = "finishing";             // the next status poll must not finish it twice
+    const prev = st.prev, sent = st.sent;
```

```
+    label("Restoring...");
+    const ok = await postCq({ max_calls: prev.max_calls, listen_every: prev.listen_every });
+    st = null; save();
+    btn.disabled = false;
+    label("10-burst test");
+    alert("10-burst test " + why + ": " + sent + " burst(s) sent.\n" +
+          (ok ? "CQ settings restored (Stop after: " + (prev.max_calls || "never") +
+                ", listening slot: " + (prev.listen_every || "off") + ")."
+              : "Could not restore the CQ settings - set them back in Settings > CQ messages.") +
+          "\n\nNow save the Tab5 log: open /api/log (it empties the Tab5's log), " +
+          "and download txlog from the NAS.");
+  }
+
+  btn.addEventListener("click", async () => {
+    if (st) {                            // running: this click stops it
+      if (!confirm("Stop the 10-burst test now? The CQ run is cancelled and the settings restored.")) return;
+      await sendCmd({ action: "qso_override", what: "cancel" });
+      await finish("stopped");
+      return;
+    }
+    if (!confirm("Start a 10-burst test?\n\nThe Tab5 calls CQ ten times in a row (every other " +
+                 "15 s slot, about 5 minutes) and transmits each time. Use a dummy load, or make " +
+                 "sure the frequency is clear and you are licensed to transmit on it.\n\n" +
+                 "CQ settings are changed for the test and put back afterwards.")) return;
+    btn.disabled = true; label("Preparing...");
+    let cfg = null;
+    try { cfg = await (await fetch("/api/settings", { cache: "no-store" })).json(); } catch (e) {}
+    if (!cfg || !cfg.cq) { btn.disabled = false; label("10-burst test"); alert("Could not read the Tab5's settings."); return; }
+    st = { prev: { max_calls: cfg.cq.max_calls || 0, listen_every: cfg.cq.listen_every || 0 },
+           sent: 0, active: false, phase: "starting", t0: Date.now() };
+    save();
+    if (!(await postCq({ max_calls: N, listen_every: 0 }))) { await finish("could not start"); return; }
+    let screen = "";
+    try { screen = (await (await fetch("/api/status", { cache: "no-store" })).json()).screen || ""; } catch (e) {}
+    if (screen !== "ft8") {
+      await sendCmd({ action: "set_screen", screen: "ft8" });
+      await new Promise(r => setTimeout(r, 4000));      // let the FT8 engine start
+    }
+    await sendCmd({ action: "cq_start" });
+    st.phase = "running"; save();
+    btn.disabled = false; label("Burst 0 of " + N + " - stop");
+  });
+
+  /* Called from the /api/status handler once per poll. */
+  window.burstTestOnStatus = function (s) {
+    if (!st || st.phase !== "running" || !s || !s.ft8) return;
+    const f = s.ft8, now = f.st || "idle";
+    if (now === "active" && !st.active) { st.active = true; }
+    else if (now !== "active" && st.active) { st.active = false; st.sent++; save(); }
+    label((st.active ? "Burst " + (st.sent + 1) : "Burst " + st.sent) + " of " + N + " - stop");
+    const text = f.text || "";
+    const elapsed = Date.now() - (st.t0 || 0);
+    if (!st.active && (st.sent >= N || /sent - listening/.test(text))) { finish("finished"); return; }
+    // ended some other way: answered (QSO done/timeout), cancelled, or never got going
+    if (!st.active && elapsed > 60000 && (now === "idle" || now === "done" || now === "timeout")) {
+      finish(st.sent ? "ended early" : "did not start"); return;
+    }
+  };
+  if (st) { label("Burst " + st.sent + " of " + N + " - stop"); st.phase = "running"; }
+})();
+
 (function wireCqRestart() {
   const btn = document.getElementById("cq-restart");
   if (!btn) return;
@@ -6571,6 +6663,39 @@ function paintPaused(on) {
     bar.remove();
   }
 }
+/* Network radio - the QMX plugged into a NAS, reached over TCP through
+   tools/nas-relay on the NAS. Blank host = QMX on the Tab5's USB port. */
+document.getElementById("relay-link").addEventListener("click", async (e) => {
+  e.preventDefault();
+  let cur = { host: "", port: 7355, last_error: "" };
+  try { cur = await (await fetch("/api/relay", { cache: "no-store" })).json(); } catch (_) {}
+  const shown = cur.host ? (cur.port && cur.port !== 7355 ? cur.host + ":" + cur.port : cur.host) : "";
+  const msg = "Radio source: QMX on a NAS (network).\n\n" +
+              "Enter the NAS IP address (optionally IP:port, default port 7355),\n" +
+              "or leave blank to use the QMX plugged into the Tab5's USB port.\n\n" +
+              "Current: " + (cur.host ? shown + "  (" + (cur.connected ? "connected" : (cur.last_error || "not connected")) + ")" :
  ↪  "USB");
+  const v = prompt(msg, shown);
+  if (v === null) return;
+  let host = v.trim(), port = 7355;
+  const m = host.match(/^(.*):(\d{1,5})$/);
+  if (m) { host = m[1].trim(); port = parseInt(m[2], 10); }
+  if (port < 1 || port > 65535) { alert("Bad port"); return; }
+  try {
```

```
+    const r = await fetch("/api/relay", { method: "POST", body: JSON.stringify({ host, port }) });
+    if (!r.ok) throw new Error("HTTP " + r.status + " " + (await r.text()));
+  } catch (err) {
+    alert("Setting the radio source failed: " + err);
+  }
+});
+
+function updateRelayLink(link) {
+  const el = document.getElementById("relay-link");
+  if (!el) return;
+  el.textContent = link === "net" ? "Radio source: NAS (network)"
+                 : link === "usb" ? "Radio source: USB"
+                 : "Radio source: none connected";
+}
+
 document.getElementById("pause-btn").addEventListener("click", async () => {
   await sendCmd({ action: pausedNow ? "resume" : "pause" });
   paintPaused(!pausedNow);   // optimistic; the next status poll is authoritative
@@ -6829,6 +6954,7 @@ async function tick() {
      * one poll has had time to return carrying it. That window is the debounce
      * plus a round trip; anything shorter and the last snap-back still shows. */
     radioFreqHz = s.freq_hz || 0;      /* always - see its declaration */
+    updateRelayLink(s.radio_link);
     if (Date.now() >= freqHoldUntil) lastFreqHz = s.freq_hz || 0;
     lastPassbandHz = s.passband_hz || 0;
     lastCwFilterMask = (typeof s.cw_filter_mask === "number") ? s.cw_filter_mask : 0;
@@ -7081,6 +7207,7 @@ async function tick() {
     }
     }

+    if (window.burstTestOnStatus) window.burstTestOnStatus(s);
     if (screen === "ft8" && s.ft8) {
       /* (*) TWO HOMES FOR ONE STRING, decided by the state. While nothing is
          being transmitted this is a RECEIVE readout - "RX: capturing...",
```

main/storage/settings.h

changes against upstream v1.18.0 (unified diff)

```
@@ -803,6 +803,10 @@ typedef struct {
     char     lotw_state[4];     // 2-letter US state, e.g. "VA"
     char     lotw_county[40];   // county name only, e.g. "Arlington"
     uint32_t lotw_uploaded_n;   // count of ADIF records already uploaded to LoTW
+    // Network radio: IP/host of the NAS the QMX is plugged into, running
+    // tools/nas-relay. Empty = QMX on the Tab5's own USB port (default).
+    char     relay_host[64];
+    uint16_t relay_port;        // network radio TCP port (default 7355)
 } qmx_settings_t;

 // Initialise the settings module. Opens an NVS handle. Safe to call
@@ -1229,6 +1233,10 @@ void settings_set_qrz_uploaded_n(uint32_t n);
 void settings_set_eqsl_user(const char *user);
 void settings_set_eqsl_pswd(const char *pswd);

+// Network radio (QMX on a NAS via tools/nas-relay instead of USB). Persists
+// only; callers also call radio_relay_set_target() + cat_reconnect().
+void settings_set_relay(const char *host, uint16_t port);
+
 // QRZ.com Callbook (XML) lookup credentials, set via the web UI (debounced
 // flush, one field per call like the eQSL setters above). Pass NULL or empty
 // to clear. See the struct field comment in this header for why this is
```

main/storage/settings.c

changes against upstream v1.18.0 (unified diff)

```
@@ -81,6 +81,8 @@ static const char *TAG = "settings";
 #define KEY_QRZ_UPLOADED "qrz_upl_n"
 #define KEY_EQSL_USER    "eqsl_user"
 #define KEY_EQSL_PSWD    "eqsl_pswd"
+#define KEY_RELAY_HOST   "relay_host"
+#define KEY_RELAY_PORT   "relay_port"
 #define KEY_QRZ_LU_USER  "qrz_lu_user"
 #define KEY_QRZ_LU_PASS  "qrz_lu_pass"
 #define KEY_EQSL_UPLOADED "eqsl_upl_n"
@@ -239,7 +241,7 @@ static const char *TAG = "settings";
  * taken every one of them while the comment still said 62. Bumped to 5 words
  * rather than land the next person on a full bitmap behind a wrong number.
  * Cost is 4 bytes. Recount before trusting this figure again. */
-#define DIRTY_WORDS      5                      /* 160 bits; highest used 138, 21 spare */
+#define DIRTY_WORDS      5                      /* 160 bits; highest used 140, 19 spare */
 #define DIRTY_BITS_MAX   (DIRTY_WORDS * 32)

 typedef struct { uint32_t w[DIRTY_WORDS]; } dirty_t;
@@ -442,7 +444,8 @@ static inline bool dirty_test_any(const dirty_t *d, const uint8_t *bits, size_t
 #define DIRTY_PORT_A_MODE     135  /* PORT.A owner: relay | unit_gps (#15) */
 #define DIRTY_GNSS_MBUS       136  /* M-Bus GNSS receiver enable */
 #define DIRTY_RX_SOURCE       137  /* RX input source: qmx_usb | line_in */
-#define DIRTY_LINEIN_GAIN     138  /* line-in analogue gain, dB; 21 spare */
+#define DIRTY_LINEIN_GAIN     138  /* line-in analogue gain, dB */
+#define DIRTY_RELAY           140  /* network radio: relay_host + relay_port (139 is DIRTY_PA_CEILING); 19 spare */

 // Bits that actually affect config_io_export()'s output (storage/config_io.c).
 // Bookkeeping bits like DIRTY_LAST_TIME (rewritten by the continuous time-sync
@@ -465,6 +468,7 @@ static const uint8_t s_config_export_bits[] = {
     DIRTY_CQ_MSG2, DIRTY_CQ_SEL, DIRTY_ONBOARDED, DIRTY_FT8_FILT,
     DIRTY_WIFI_ENABLED, DIRTY_QMX_GPS, DIRTY_FREQ_KP_CALC,
     DIRTY_QRZ_KEY, DIRTY_QRZ_LU_USER, DIRTY_QRZ_LU_PASS, DIRTY_EQSL_USER, DIRTY_EQSL_PSWD,
+    DIRTY_RELAY,
     DIRTY_CL_URL, DIRTY_CL_KEY, DIRTY_CL_STATION,
     DIRTY_WF_BLACK, DIRTY_WF_CONTRAST, DIRTY_WF_BLEND, DIRTY_WF_WINDOW,
     DIRTY_DISP_FLIP, DIRTY_QMX_VOL, DIRTY_RX_AUD_VOL, DIRTY_CHARGE_LIM_EN,
@@ -636,6 +640,10 @@ static void flush_task(void *arg)
         if (dirty_test(&dirty_local, DIRTY_QRZ_UPLOADED)) nvs_set_u32(s_nvs, KEY_QRZ_UPLOADED, snap.qrz_uploaded_n);
         if (dirty_test(&dirty_local, DIRTY_EQSL_USER))     nvs_set_str(s_nvs, KEY_EQSL_USER, snap.eqsl_user);
         if (dirty_test(&dirty_local, DIRTY_EQSL_PSWD))     nvs_set_str(s_nvs, KEY_EQSL_PSWD, snap.eqsl_pswd);
+        if (dirty_test(&dirty_local, DIRTY_RELAY)) {
+            nvs_set_str(s_nvs, KEY_RELAY_HOST, snap.relay_host);
+            nvs_set_u16(s_nvs, KEY_RELAY_PORT, snap.relay_port);
+        }
         if (dirty_test(&dirty_local, DIRTY_QRZ_LU_USER))   nvs_set_str(s_nvs, KEY_QRZ_LU_USER, snap.qrz_lookup_user);
         if (dirty_test(&dirty_local, DIRTY_QRZ_LU_PASS))   nvs_set_str(s_nvs, KEY_QRZ_LU_PASS, snap.qrz_lookup_pass);
         if (dirty_test(&dirty_local, DIRTY_EQSL_UPLOADED)) nvs_set_u32(s_nvs, KEY_EQSL_UPLOADED, snap.eqsl_uploaded_n);
@@ -912,6 +920,8 @@ static void load_from_nvs(qmx_settings_t *out)
     out->qrz_lookup_pass[0] = '\0';
     out->eqsl_user[0] = '\0';
     out->eqsl_pswd[0] = '\0';
+    out->relay_host[0] = '\0';
+    out->relay_port = 7355;
     out->eqsl_uploaded_n = 0;
     out->cloudlog_url[0] = '\0';
     out->cloudlog_key[0] = '\0';
@@ -1129,6 +1139,12 @@ static void load_from_nvs(qmx_settings_t *out)
     out->eqsl_pswd[0] = '\0';
     sz = sizeof(out->eqsl_pswd);
     nvs_get_str(s_nvs, KEY_EQSL_PSWD, out->eqsl_pswd, &sz);
+    {
+        size_t rsz = sizeof(out->relay_host);
+        if (nvs_get_str(s_nvs, KEY_RELAY_HOST, out->relay_host, &rsz) != ESP_OK) out->relay_host[0] = '\0';
+        uint16_t rp = 0;
+        if (nvs_get_u16(s_nvs, KEY_RELAY_PORT, &rp) == ESP_OK && rp != 0) out->relay_port = rp;
+    }
     nvs_get_u32(s_nvs, KEY_EQSL_UPLOADED, &out->eqsl_uploaded_n);
     out->cloudlog_url[0] = '\0';
     sz = sizeof(out->cloudlog_url);
@@ -3875,3 +3891,13 @@ void settings_set_gps_grid_auto(const char *grid)
     nvs_commit(s_nvs);
     xSemaphoreGive(s_mutex);
 }
+
+void settings_set_relay(const char *host, uint16_t port)
+{
+    if (!s_ready) return;
+    xSemaphoreTake(s_mutex, portMAX_DELAY);
+    snprintf(s_pending.relay_host, sizeof(s_pending.relay_host), "%s", host ? host : "");
+    s_pending.relay_port = port ? port : 7355;
+    xSemaphoreGive(s_mutex);
+    mark_dirty(DIRTY_RELAY);
+}
```

main/storage/config_io.c

changes against upstream v1.18.0 (unified diff)

```
@@ -135,6 +135,8 @@ char *config_io_export(size_t *out_len)
     APP("qrz_lookup_pass    = %s\n", c.qrz_lookup_pass);
     APP("eqsl_user          = %s\n", c.eqsl_user);
     APP("eqsl_pass          = %s\n", c.eqsl_pswd);
+    APP("relay_host         = %s\n", c.relay_host);   // QMX on a NAS (tools/nas-relay); empty = QMX on USB
+    APP("relay_port         = %u\n", (unsigned)c.relay_port);
     APP("cloudlog_url       = %s\n", c.cloudlog_url);
     APP("cloudlog_key       = %s\n", c.cloudlog_key);
     APP("cloudlog_station   = %s\n", c.cloudlog_station);
@@ -498,6 +500,15 @@ int config_io_import(char *text)
             else if (!strcasecmp(key, "qrz_lookup_pass"))   settings_set_qrz_lookup_pass(val);
             else if (!strcasecmp(key, "eqsl_user"))         settings_set_eqsl_user(val);
             else if (!strcasecmp(key, "eqsl_pass"))         settings_set_eqsl_pswd(val);
+            else if (!strcasecmp(key, "relay_host") || !strcasecmp(key, "relay_port")) {
+                qmx_settings_t *rc = heap_caps_malloc(sizeof(*rc), MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT);
+                if (rc) {
+                    settings_load_all(rc);
+                    if (!strcasecmp(key, "relay_host")) settings_set_relay(val, rc->relay_port);
+                    else                                settings_set_relay(rc->relay_host, (uint16_t)atoi(val));
+                    heap_caps_free(rc);
+                }
+            }
             else if (!strcasecmp(key, "cloudlog_url"))      settings_set_cloudlog_url(val);
             else if (!strcasecmp(key, "cloudlog_key"))      settings_set_cloudlog_key(val);
             else if (!strcasecmp(key, "cloudlog_station"))  settings_set_cloudlog_station(val);
```

main/main.c

changes against upstream v1.18.0 (unified diff)

```
@@ -19,6 +19,7 @@
 #include "gpio_relay.h"
 #include "unit_gps.h"      // PORT.A's other owner - see the branch before time_sync_init()
 #include "cat.h"
+#include "radio_relay.h"
 #include "cw_decode.h"
 #include "audio.h"
 #include "line_in.h"   // the 3.5 mm jack as a receive source
@@ -601,6 +602,15 @@ void app_main(void)
      * link task retrying a device that cannot appear, and the UI showing a
      * radio that is not there. */
     if (!rx_src_real) {
+        /* Network radio: if a NAS relay host is configured, cat.c/audio.c use
+           the QMX plugged into the NAS (tools/nas-relay) instead of the Tab5's
+           USB port. Heap, not stack - same reasoning as the cfg block above. */
+        qmx_settings_t *rs = heap_caps_malloc(sizeof(*rs), MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT);
+        if (rs) {
+            settings_load_all(rs);
+            radio_relay_set_target(rs->relay_host, rs->relay_port);
+            heap_caps_free(rs);
+        }
         ESP_ERROR_CHECK(cat_init());
         MEM_LEDGER("cat_init (CDC)");
     }
```

CMakeLists.txt

changes against upstream v1.18.0 (unified diff)

```
@@ -13,4 +13,16 @@ include($ENV{IDF_PATH}/tools/cmake/project.cmake)
 # build/compile_commands.json for lv_mem_core_builtin.c).
 idf_build_set_property(COMPILE_DEFINITIONS "-DLV_ATTRIBUTE_LARGE_RAM_ARRAY=EXT_RAM_BSS_ATTR" APPEND)

+# Network-radio build: report the release it is based on plus "-NAS"
+# (e.g. v1.16.11-NAS) instead of git describe's v1.16.11-3-g<hash>-dirty, which
+# is too long for the UI. The update check compares only the numbers before the
+# first '-', so v1.16.11-NAS counts as v1.16.11.
+execute_process(COMMAND git describe --tags --abbrev=0
+                WORKING_DIRECTORY ${CMAKE_CURRENT_SOURCE_DIR}
+                OUTPUT_VARIABLE QMX_BASE_TAG OUTPUT_STRIP_TRAILING_WHITESPACE ERROR_QUIET)
+if(NOT QMX_BASE_TAG)
+    set(QMX_BASE_TAG "v1.16.11")
+endif()
+set(PROJECT_VER "${QMX_BASE_TAG}-NAS")
+
 project(qmx_panadapter)
```

Synology relay: code, installation, operation and maintenance scripts

tools/nas-relay/qmx_nas_relay.py

new file, 2259 lines

```
#!/usr/bin/env python3
"""
qmx_nas_relay.py - serve a QMX/QMX+ plugged into a Synology NAS to the
QMX-Panadapter (M5Stack Tab5) over the network.

    QMX --USB--> Synology NAS (this relay, run natively or in Docker)
                     |  QMXR/1 framed TCP, port 7355
                     v
              Tab5 QMX-Panadapter  (Radio source: NAS)

The relay owns the QMX directly from userspace, so it needs no DSM kernel
drivers (no cdc_acm / snd-usb-audio). It uses libusb-1.0 (python-libusb1)
when that loads, and otherwise talks to Linux usbfs (/dev/bus/usb) itself
through ioctl() - which is what runs on NAS models without libusb-1.0, such
as the DS416j (QMX_BACKEND=libusb|usbfs forces one):

  * CAT:  CDC-ACM data interface, bulk IN/OUT endpoints (38400 8N1 set for
          completeness - it's a virtual COM port).
  * I/Q:  UAC streaming interface, isochronous IN, 48 kHz stereo 24-bit.
          Converted to int16 exactly like the firmware's USB path (s24 >> 8).
  * Terminal: the QMX's optional SECOND serial port (radio set to "USB serial
          ports" = 2 or 3; on USB it is interface 5), opened on request for the
          Tab5's "Radio menus" and carried as its own stream, never mixed
          with CAT - exactly why the firmware uses port 2 on USB.
  * Port 3: with "USB serial ports" = 3 the QMX's THIRD serial port is served
          as a plain TCP serial port (default 7356) for PC software - WSJT-X,
          rigctld, a terminal - so a PC can control the radio at the same
          time as the Tab5, on its own port.

Wire format (QMXR/1): every frame = 4-byte header + payload
     u8  type   1=HELLO 2=IQ 3=CAT 4=PING
                5=TERM_OPEN 6=TERM_STATUS 7=TERM_DATA 8=TERM_CLOSE
                9=TX_SCHED 10=TX_ABORT 11=TX_DONE
     u8  flags  0
     u16 len    payload length, little-endian
  Tab5 -> relay : HELLO "QMXR/1 ...", then CAT bytes
  relay -> Tab5 : HELLO "QMXR/1 rate=48000 ch=2 fmt=s16le term=1 txsched=1", then IQ / CAT / PING
  Terminal (second serial port), Tab5-initiated:
    Tab5 -> relay : TERM_OPEN            relay -> Tab5 : TERM_STATUS u8 ok [+ reason]
    both ways     : TERM_DATA bytes      Tab5 -> relay : TERM_CLOSE
    relay -> Tab5 : TERM_STATUS 0 unsolicited if the port goes away while open
  Relay-timed transmit bursts ("txsched=1"):
    Tab5 -> relay : TX_SCHED u8 ver=1, u8 flags=0, u16 n, u32 symbol_us,
                    n x u32 tone frequency in centi-Hz (all little-endian)
                    The relay keys the QMX (TX;) at once, sends tone i at
                    key down + i x symbol_us and the key-up tone (TA0;) at
                    key down + n x symbol_us, all on its own clock.
    Tab5 -> relay : TX_ABORT             stop now: key-up tone at once
    relay -> Tab5 : TX_DONE u8 status (0 complete, 1 aborted, 2 failed),
                    u16 tones sent, u32 ms TX; -> key-up, i32 ms latest symbol
    The Tab5 still sends RX; (after its power/SWR query) and its own CAT.

The relay only answers the Tab5's HELLO after it has opened the QMX, and it
closes the connection as soon as the QMX goes away, so for the Tab5 an open
session means "radio present" - exactly like a USB plug/unplug.

One Tab5 at a time and one port-3 client at a time; a new connection replaces
an old one (e.g. after a reboot left a half-dead socket). The QMX is opened when
the first of them connects and released when the last one leaves.
Requirements: Python 3.8+. Nothing else on Linux (usbfs backend); on other
systems libusb-1.0 + `pip install libusb1` (the Dockerfile installs both).
"""

from __future__ import annotations

import argparse
import collections
import ctypes
import errno
import logging
import math
import os
import queue
import re
import select
import socket
import struct
import sys
import threading
import time
log = logging.getLogger("qmx-nas-relay")
```

```
PROTO_HELLO, PROTO_IQ, PROTO_CAT, PROTO_PING = 1, 2, 3, 4
PROTO_TERM_OPEN, PROTO_TERM_STATUS, PROTO_TERM_DATA, PROTO_TERM_CLOSE = 5, 6, 7, 8
PROTO_TX_SCHED, PROTO_TX_ABORT, PROTO_TX_DONE = 9, 10, 11
TX_SCHED_HDR = struct.Struct("<BBHI")
TX_DONE_FMT = struct.Struct("<BHIi")
HDR = struct.Struct("<BBH")
DEFAULT_PORT = 7355
DEFAULT_SERIAL_PORT = 7356     # QMX port 3 as raw TCP serial for PCs
QMX_VID, QMX_PID = 0x0483, 0xA34C
SAMPLE_RATE = 48000
FRAME_PAIRS = 480            # 10 ms of I/Q per IQ frame (1920 bytes)
IQ_QUEUE_MAX = 3             # I/Q frames (30 ms) waiting to be sent; beyond that drop I/Q, never CAT
TAB5_SNDBUF = 8192           # kernel send buffer per Tab5 (Linux doubles it): keeps the
                             # backlog - and so the CAT reply delay - under ~0.2 s
ISO_TRANSFERS = 8            # in flight
ISO_PACKETS = 8              # 1 ms each -> 8 ms per transfer
CAT_BAUD = 38400


def frame(ftype: int, payload: bytes = b"") -> bytes:
    if len(payload) > 0xFFFF:
        raise ValueError("payload too large")
    return HDR.pack(ftype, 0, len(payload)) + payload


def recv_exact(sock: socket.socket, n: int) -> bytes:
    buf = bytearray()
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed")
        buf += chunk
    return bytes(buf)


def to_s16(raw: bytes, subframe: int) -> bytes:
    """Interleaved PCM -> interleaved int16 LE, matching the firmware's USB
    path: 24-bit samples keep (s24 >> 8) = the upper two bytes; 32-bit keep
    (s32 >> 16); 16-bit pass through."""
    if subframe == 2:
        return bytes(raw[: len(raw) // 2 * 2])
    n = len(raw) // subframe
    out = bytearray(n * 2)
    out[0::2] = raw[subframe - 2: n * subframe: subframe]
    out[1::2] = raw[subframe - 1: n * subframe: subframe]
    return bytes(out)


# ---------------------------------------------------------------------------
# Radio backends
# ---------------------------------------------------------------------------

class RadioGone(Exception):
    pass


class QmxUsb:
    """The real QMX over libusb. on_iq(bytes s16) / on_cat(bytes) are called
    from the libusb event thread; on_gone() once if the device disappears."""

    def __init__(self, on_iq, on_cat, on_gone, vid=QMX_VID, pid=QMX_PID):
        import usb1  # imported here so --fake works without libusb
        _load_libusb(usb1)
        self.usb1 = usb1
        self.on_iq, self.on_cat, self.on_gone = on_iq, on_cat, on_gone
        self.vid, self.pid = vid, pid
        self.ctx = usb1.USBContext()
        self.h = None
        self.running = False
        self.claimed: list[int] = []
        self.transfers = []
        self.gone_reported = False
        self.evt_thread = None
        self.cdc_ports = []          # [(comm_if, data_if, bulk_in, bulk_out)] in interface order
        # Extra serial ports, by index into cdc_ports: 1 = port 2 (terminal), 2 = port 3 (PC).
        self.extra: dict[int, dict] = {}

    # -- descriptor discovery ------------------------------------------
    def _discover(self, dev):
        return discover_qmx(dev[0])

    # -- open / close --------------------------------------------------
    def open(self):
        usb1 = self.usb1
        h = self.ctx.openByVendorIDAndProductID(self.vid, self.pid, skip_on_error=True)
        if h is None:
            raise RadioGone("QMX (%04x:%04x) not found on the NAS USB bus" % (self.vid, self.pid))
```

```
        self.h = h
        dev = h.getDevice()
        self.cdc_ports, uac2, aud = self._discover(dev)
        if not self.cdc_ports or aud is None:
            self.close()
            raise RadioGone("QMX descriptors not recognised (CAT=%s audio=%s)" % (self.cdc_ports, aud))
        cdc_ctrl, cdc_data, self.bulk_in, self.bulk_out = self.cdc_ports[0]
        as_if, alt, self.iso_ep, self.iso_mps, ch, self.subframe, bits, rates = aud
        log.info("QMX found: CAT if%d (IN 0x%02x OUT 0x%02x), I/Q if%d alt%d EP 0x%02x mps=%d %dch %d-bit/%dB %s%s",
                 cdc_data, self.bulk_in, self.bulk_out, as_if, alt, self.iso_ep, self.iso_mps,
                 ch, bits, self.subframe, rates or "", " UAC2" if uac2 else "")
        n = len(self.cdc_ports)
        log.info("QMX USB serial ports: %d (%s)", n, ", ".join(
            "port %d = if%s/if%d" % (i + 1, c, d) for i, (c, d, _, _) in enumerate(self.cdc_ports)))
        if n < 2:
            log.info("Radio menus need 'USB serial ports' = 2 or 3 on the radio")
        if n < 3:
            log.info("PC serial (port 3) needs 'USB serial ports' = 3 on the radio")
        if ch != 2:
            log.warning("audio stream has %d channels, expected 2 (I/Q)", ch)
        try:
            h.setAutoDetachKernelDriver(True)   # if DSM bound cdc_acm / snd-usb-audio
        except Exception:
            pass
        try:
            for i in [x for x in (cdc_ctrl, cdc_data, as_if) if x is not None]:
                h.claimInterface(i)
                self.claimed.append(i)
        except usb1.USBErrorBusy:
            self.close()
            raise RadioGone("QMX is in use by another program on the NAS - stop it, then reconnect")
        except usb1.USBErrorAccess:
            self.close()
            raise RadioGone("no permission for USB - run the container privileged")

        # CAT: line coding 38400 8N1 + DTR/RTS (same as the firmware's USB path).
        if cdc_ctrl is not None:
            try:
                h.controlWrite(0x21, 0x20, 0, cdc_ctrl, struct.pack("<IBBB", CAT_BAUD, 0, 0, 8), timeout=500)
                h.controlWrite(0x21, 0x22, 0x0003, cdc_ctrl, b"", timeout=500)
            except usb1.USBError as e:
                log.warning("CDC line setup: %s (continuing)", e)

        # I/Q: select the streaming alt setting, ask for 48 kHz (UAC1 endpoint request).
        h.setInterfaceAltSetting(as_if, alt)
        self.as_if = as_if
        if not uac2:
            try:
                h.controlWrite(0x22, 0x01, 0x0100, self.iso_ep, SAMPLE_RATE.to_bytes(3, "little"), timeout=500)
            except usb1.USBError as e:
                log.info("sample-rate SET_CUR not accepted (%s) - device default used", e)

        self.running = True
        self.gone_reported = False
        # Isochronous I/Q transfers.
        for _ in range(ISO_TRANSFERS):
            t = h.getTransfer(iso_packets=ISO_PACKETS)
            t.setIsochronous(self.iso_ep, ISO_PACKETS * self.iso_mps, callback=self._iso_cb)
            t.submit()
            self.transfers.append(t)
        # CAT bulk IN.
        for _ in range(2):
            t = h.getTransfer()
            t.setBulk(self.bulk_in, 64, callback=self._bulk_cb)
            t.submit()
            self.transfers.append(t)
        self.evt_thread = threading.Thread(target=self._events, name="usb-events", daemon=True)
        self.evt_thread.start()
    def _events(self):
        while self.running:
            try:
                self.ctx.handleEventsTimeout(0.1)
            except self.usb1.USBErrorInterrupted:
                continue
            except Exception as e:
                log.warning("libusb event loop: %s", e)
                self._gone()
                return

    def _gone(self):
        if not self.gone_reported:
            self.gone_reported = True
            log.warning("QMX unplugged / powered off")
            self.running = False
            self.on_gone()

    def _resubmit_or_gone(self, t):
```

```
        if not self.running:
            return
        try:
            t.submit()
        except self.usb1.USBError as e:
            log.warning("USB resubmit failed: %s", e)
            self._gone()

    def _iso_cb(self, t):
        usb1 = self.usb1
        st = t.getStatus()
        if st == usb1.TRANSFER_COMPLETED:
            raw = bytearray()
            for pst, buf in t.iterISO():
                if pst == usb1.TRANSFER_COMPLETED and buf:
                    raw += buf
            if raw:
                self.on_iq(to_s16(raw, self.subframe))
        elif st == usb1.TRANSFER_NO_DEVICE:
            self._gone()
            return
        elif st == usb1.TRANSFER_CANCELLED:
            return
        self._resubmit_or_gone(t)

    def _bulk_cb(self, t):
        usb1 = self.usb1
        st = t.getStatus()
        if st == usb1.TRANSFER_COMPLETED:
            n = t.getActualLength()
            if n:
                self.on_cat(bytes(t.getBuffer()[:n]))
        elif st == usb1.TRANSFER_NO_DEVICE:
            self._gone()
            return
        elif st == usb1.TRANSFER_CANCELLED:
            return
        self._resubmit_or_gone(t)

    # -- extra serial ports (index 1 = port 2 terminal, index 2 = port 3 PC) --
    def serial_count(self) -> int:
        return len(self.cdc_ports)

    def serial_open(self, idx, on_data):
        """Claim the QMX's serial port number idx+1 and start reading it.
        Returns (ok, reason)."""
        usb1 = self.usb1
        name = "port %d" % (idx + 1)
        st = self.extra.get(idx)
        if st and st["cb"]:
            st["cb"] = on_data
            return True, ""
        if len(self.cdc_ports) <= idx:
            return False, ("the radio has %d USB serial port%s - set 'USB serial ports' to %d"
                           % (len(self.cdc_ports), "" if len(self.cdc_ports) == 1 else "s", idx + 1))
        comm, data, bin_, bout = self.cdc_ports[idx]
        st = {"ifs": [], "transfers": [], "out": bout, "cb": None}
        self.extra[idx] = st
        try:
            for i in [x for x in (comm, data) if x is not None]:
                self.h.claimInterface(i)
                st["ifs"].append(i)
        except usb1.USBErrorBusy:
            self._serial_release(idx)
            return False, "%s is in use by another program on the NAS" % name
        except usb1.USBError as e:
            self._serial_release(idx)
            return False, "%s claim failed: %s" % (name, e)
        if comm is not None:
            try:
                self.h.controlWrite(0x21, 0x20, 0, comm, struct.pack("<IBBB", CAT_BAUD, 0, 0, 8), timeout=500)
                self.h.controlWrite(0x21, 0x22, 0x0003, comm, b"", timeout=500)
            except usb1.USBError as e:
                log.warning("%s line setup: %s (continuing)", name, e)
        st["cb"] = on_data
        for _ in range(2):
            t = self.h.getTransfer()
            t.setBulk(bin_, 512, callback=self._serial_cb, user_data=idx)
            t.submit()
            st["transfers"].append(t)
        log.info("QMX %s open (if%s/if%d)", name, comm, data)
        return True, ""

    def _serial_cb(self, t):
        usb1 = self.usb1
        idx = t.getUserData()
        st = self.extra.get(idx)
```

```
        status = t.getStatus()
        if status == usb1.TRANSFER_COMPLETED:
            n = t.getActualLength()
            cb = st["cb"] if st else None
            if n and cb:
                cb(bytes(t.getBuffer()[:n]))
        elif status == usb1.TRANSFER_NO_DEVICE:
            self._gone()
            return
        elif status == usb1.TRANSFER_CANCELLED:
            return
        if st and st["cb"]:
            self._resubmit_or_gone(t)
    def serial_send(self, idx, data: bytes):
        st = self.extra.get(idx)
        if not self.h or not st or not st["cb"]:
            return
        try:
            self.h.bulkWrite(st["out"], data, timeout=500)
        except self.usb1.USBErrorNoDevice:
            self._gone()
            raise RadioGone("QMX gone")
        except self.usb1.USBError as e:
            log.warning("port %d write: %s", idx + 1, e)

    def serial_close(self, idx):
        st = self.extra.get(idx)
        if not st:
            return
        st["cb"] = None
        for t in st["transfers"]:
            try:
                if t.isSubmitted():
                    t.cancel()
            except Exception:
                pass
        deadline = time.time() + 1.0
        while time.time() < deadline and any(_submitted(t) for t in st["transfers"]):
            time.sleep(0.02)          # the event thread completes the cancellations
        st["transfers"] = []
        self._serial_release(idx)
        log.info("QMX port %d closed", idx + 1)

    def _serial_release(self, idx):
        st = self.extra.pop(idx, None)
        if not st:
            return
        for i in st["ifs"]:
            try:
                self.h.releaseInterface(i)
            except Exception:
                pass

    # Terminal = port 2 (kept as names the Tab5 session uses).
    def term_open(self, on_data):
        return self.serial_open(1, on_data)

    def term_send(self, data: bytes):
        self.serial_send(1, data)

    def term_close(self):
        self.serial_close(1)

    def send_cat(self, data: bytes):
        if not self.h:
            raise RadioGone("closed")
        try:
            self.h.bulkWrite(self.bulk_out, data, timeout=500)
        except self.usb1.USBErrorNoDevice:
            self._gone()
            raise RadioGone("QMX gone")

    def close(self):
        self.running = False
        for st in self.extra.values():
            st["cb"] = None
            self.transfers += st["transfers"]
            self.claimed += st["ifs"]
        self.extra = {}
        for t in self.transfers:
            try:
                if t.isSubmitted():
                    t.cancel()
            except Exception:
                pass
        # Let cancellations complete before freeing.
        deadline = time.time() + 1.0
```

```
        while time.time() < deadline and any(_submitted(t) for t in self.transfers):
            try:
                self.ctx.handleEventsTimeout(0.05)
            except Exception:
                break
        if self.evt_thread and self.evt_thread is not threading.current_thread():
            self.evt_thread.join(timeout=1.0)
        self.transfers = []
        if self.h:
            try:
                if getattr(self, "as_if", None) is not None:
                    self.h.setInterfaceAltSetting(self.as_if, 0)
            except Exception:
                pass
            for i in self.claimed:
                try:
                    self.h.releaseInterface(i)
                except Exception:
                    pass
            self.claimed = []
            try:
                self.h.close()
            except Exception:
                pass
            self.h = None


def _load_libusb(usb1):
    """Use QMX_LIBUSB=/path/libusb-1.0.so.0 if set (e.g. a NAS whose libusb is
    not on the default search path); otherwise python-libusb1's own lookup."""
    path = os.environ.get("QMX_LIBUSB")
    if path and not getattr(usb1, "_qmx_loaded", False):
        usb1.loadLibrary(path)
        usb1._qmx_loaded = True


def _submitted(t) -> bool:
    try:
        return t.isSubmitted()
    except Exception:
        return False


def _split_descs(extra) -> list[bytes]:
    """getExtra() may return one blob or a list of descriptor blobs."""
    blobs = extra if isinstance(extra, (list, tuple)) else [extra]
    out = []
    for b in blobs:
        b = bytes(b)
        i = 0
        while i + 2 <= len(b) and b[i] >= 2:
            out.append(b[i:i + b[i]])
            i += b[i]
    return out


def discover_qmx(config):
    """config = list of interfaces, each a list of alt settings (python-libusb1 or
    usbfs shim objects). Returns (cdc_ports, uac2, best audio-in candidate)."""
    comms, datas = [], []
    uac2 = False
    best = None   # (iface, alt, ep, mps, channels, subframe, bits, rates)
    for iface in config:
        for s in iface:
            cls, sub, num, alt = s.getClass(), s.getSubClass(), s.getNumber(), s.getAlternateSetting()
            if cls == 0x02 and sub == 0x02 and alt == 0:
                comms.append(num)
            elif cls == 0x0A and alt == 0:
                bin_ = bout = None
                for ep in s:
                    a, attr = ep.getAddress(), ep.getAttributes() & 3
                    if attr == 2:
                        if a & 0x80:
                            bin_ = a
                        else:
                            bout = a
                if bin_ and bout:
                    datas.append((num, bin_, bout))
            elif cls == 0x01 and sub == 0x01 and s.getProtocol() == 0x20:
                uac2 = True
            elif cls == 0x01 and sub == 0x02 and alt > 0:
                iso_in = [ep for ep in s
                          if (ep.getAddress() & 0x80) and (ep.getAttributes() & 3) == 1]
                if not iso_in:
                    continue
                ch, subframe, bits, rates = 2, 3, 24, []
                for d in _split_descs(s.getExtra()):
```

```
                    # UAC1 Format Type I: len, 0x24, 0x02, 0x01, nCh, subframe, bits, nFreq, freqs...
                    if len(d) >= 8 and d[1] == 0x24 and d[2] == 0x02 and d[3] == 0x01:
                        ch, subframe, bits, nf = d[4], d[5], d[6], d[7]
                        rates = [d[8 + 3 * i] | d[9 + 3 * i] << 8 | d[10 + 3 * i] << 16
                                 for i in range(nf) if 10 + 3 * i < len(d)]
                    # UAC2 Format Type I: len, 0x24, 0x02, 0x01, subslot, bits
                    elif len(d) == 6 and d[1] == 0x24 and d[2] == 0x02 and d[3] == 0x01:
                        subframe, bits = d[4], d[5]
                    # UAC2 AS_GENERAL carries bNrChannels at offset 10
                    elif len(d) >= 11 and d[1] == 0x24 and d[2] == 0x01 and d[0] == 16:
                        ch = d[10]
                ep = iso_in[0]
                cand = (num, alt, ep.getAddress(), ep.getMaxPacketSize() & 0x7FF, ch, subframe, bits, rates)
                # Prefer stereo; the QMX's RX/I-Q stream is the stereo IN one.
                if best is None or (ch == 2 and best[4] != 2):
                    best = cand
    # Pair each CDC comm interface with the data interface that follows it.
    # The QMX's functions are NOT contiguous (CAT 0-1, audio 2-4, port 2 at
    # 5-6), so pair by order, not by number+1.
    comms.sort()
    datas.sort()
    ports = []
    for i, (dnum, bin_, bout) in enumerate(datas):
        ports.append((comms[i] if i < len(comms) else None, dnum, bin_, bout))
    return ports, uac2, best


# ---------------------------------------------------------------------------
# Linux usbfs backend - no libusb needed (DSM ships only libusb-0.1)
# Structures and ioctl numbers follow <linux/usbdevice_fs.h>; ioctl numbers
# use the generic _IOC encoding of x86, ARM and arm64.
# ---------------------------------------------------------------------------

c_u8, c_u16, c_u32, c_int, c_void_p = ctypes.c_uint8, ctypes.c_uint16, ctypes.c_uint32, ctypes.c_int, ctypes.c_void_p


# ---- <linux/usbdevice_fs.h> ------------------------------------------------

class CtrlTransfer(ctypes.Structure):
    _fields_ = [("bRequestType", c_u8), ("bRequest", c_u8), ("wValue", c_u16), ("wIndex", c_u16),
                ("wLength", c_u16), ("timeout", c_u32), ("data", c_void_p)]


class BulkTransfer(ctypes.Structure):
    _fields_ = [("ep", c_u32), ("len", c_u32), ("timeout", c_u32), ("data", c_void_p)]


class SetInterface(ctypes.Structure):
    _fields_ = [("interface", c_u32), ("altsetting", c_u32)]


class IsoPacketDesc(ctypes.Structure):
    _fields_ = [("length", c_u32), ("actual_length", c_u32), ("status", c_u32)]


_URB_FIELDS = [("type", c_u8), ("endpoint", c_u8), ("status", c_int), ("flags", c_u32),
               ("buffer", c_void_p), ("buffer_length", c_int), ("actual_length", c_int),
               ("start_frame", c_int), ("number_of_packets", c_int), ("error_count", c_int),
               ("signr", c_u32), ("usercontext", c_void_p)]


class Urb(ctypes.Structure):
    _fields_ = _URB_FIELDS


def iso_urb_type(npackets):
    class IsoUrb(ctypes.Structure):
        _fields_ = _URB_FIELDS + [("iso_frame_desc", IsoPacketDesc * npackets)]
    return IsoUrb


class UsbdevfsIoctl(ctypes.Structure):
    _fields_ = [("ifno", c_int), ("ioctl_code", c_int), ("data", c_void_p)]


def _IOC(d, t, nr, size):
    return (d << 30) | (size << 16) | (ord(t) << 8) | nr
_R, _W = 2, 1
USBDEVFS_CONTROL = _IOC(_R | _W, "U", 0, ctypes.sizeof(CtrlTransfer))
USBDEVFS_BULK = _IOC(_R | _W, "U", 2, ctypes.sizeof(BulkTransfer))
USBDEVFS_SETINTERFACE = _IOC(_R, "U", 4, ctypes.sizeof(SetInterface))
USBDEVFS_SUBMITURB = _IOC(_R, "U", 10, ctypes.sizeof(Urb))
USBDEVFS_DISCARDURB = _IOC(0, "U", 11, 0)
USBDEVFS_REAPURBNDELAY = _IOC(_W, "U", 13, ctypes.sizeof(c_void_p))
USBDEVFS_CLAIMINTERFACE = _IOC(_R, "U", 15, ctypes.sizeof(ctypes.c_uint))
USBDEVFS_RELEASEINTERFACE = _IOC(_R, "U", 16, ctypes.sizeof(ctypes.c_uint))
USBDEVFS_IOCTL = _IOC(_R | _W, "U", 18, ctypes.sizeof(UsbdevfsIoctl))
```

```
USBDEVFS_DISCONNECT = _IOC(0, "U", 22, 0)
USBDEVFS_CONNECT = _IOC(0, "U", 23, 0)

URB_TYPE_ISO, URB_TYPE_BULK = 0, 3
URB_ISO_ASAP = 0x02

_libc = ctypes.CDLL(None, use_errno=True)
_ioctl = _libc.ioctl
_ioctl.argtypes = [c_int, ctypes.c_ulong, c_void_p]
_ioctl.restype = c_int


class UsbfsError(OSError):
    pass


def ioctl(fd, req, arg):
    """ioctl with a real pointer (fcntl.ioctl copies small buffers, which
    breaks async URBs whose address the kernel keeps)."""
    ptr = None if arg is None else (arg if isinstance(arg, int) else ctypes.addressof(arg))
    r = _ioctl(fd, req, ptr)
    if r < 0:
        e = ctypes.get_errno()
        raise UsbfsError(e, os.strerror(e))
    return r


# ---- device discovery from sysfs + raw descriptors --------------------------

SYS_USB = "/sys/bus/usb/devices"


def usbfs_find_device(vid, pid):
    """Return (sysfs name, /dev/bus/usb path) of the first matching device."""
    try:
        names = sorted(os.listdir(SYS_USB))
    except OSError:
        return None
    for name in names:
        d = os.path.join(SYS_USB, name)
        try:
            v = int(open(os.path.join(d, "idVendor")).read(), 16)
            p = int(open(os.path.join(d, "idProduct")).read(), 16)
            if (v, p) != (vid, pid):
                continue
            bus = int(open(os.path.join(d, "busnum")).read())
            dev = int(open(os.path.join(d, "devnum")).read())
        except (OSError, ValueError):
            continue
        return name, "/dev/bus/usb/%03d/%03d" % (bus, dev)
    return None


def usbfs_list_devices():
    out = []
    try:
        names = sorted(os.listdir(SYS_USB))
    except OSError:
        return out
    for name in names:
        d = os.path.join(SYS_USB, name)
        try:
            v = int(open(os.path.join(d, "idVendor")).read(), 16)
            p = int(open(os.path.join(d, "idProduct")).read(), 16)
            bus = int(open(os.path.join(d, "busnum")).read())
            dev = int(open(os.path.join(d, "devnum")).read())
        except (OSError, ValueError):
            continue
        out.append((bus, dev, v, p, name))
    return out


class UsbfsEndpoint:
    def __init__(self, addr, attrs, mps):
        self.a, self.attrs, self.mps = addr, attrs, mps

    def getAddress(self):
        return self.a

    def getAttributes(self):
        return self.attrs

    def getMaxPacketSize(self):
        return self.mps


class UsbfsSetting(list):
```

```
    """An interface alternate setting; iterates over its endpoints. Same
    accessor names as python-libusb1, so the relay's discovery code is shared."""

    def __init__(self, num, alt, cls, sub, proto):
        super().__init__()
        self.v = (num, alt, cls, sub, proto)
        self.extra = b""

    def getNumber(self):
        return self.v[0]

    def getAlternateSetting(self):
        return self.v[1]

    def getClass(self):
        return self.v[2]

    def getSubClass(self):
        return self.v[3]

    def getProtocol(self):
        return self.v[4]
    def getExtra(self):
        return [self.extra] if self.extra else []


def usbfs_parse_config(raw: bytes):
    """Raw descriptors (device descriptor, then the first configuration) ->
    list of interfaces, each a list of Settings."""
    i = 0
    # skip the device descriptor
    if len(raw) >= 2 and raw[1] == 1:
        i = raw[0]
    ifaces: dict = {}
    cur = None
    seen_config = False
    while i + 2 <= len(raw):
        ln, typ = raw[i], raw[i + 1]
        if ln < 2:
            break
        d = raw[i:i + ln]
        if typ == 2:                        # CONFIGURATION
            if seen_config:
                break                       # only the first configuration
            seen_config = True
        elif typ == 4 and ln >= 9:          # INTERFACE
            cur = UsbfsSetting(d[2], d[3], d[5], d[6], d[7])
            ifaces.setdefault(d[2], []).append(cur)
        elif typ == 5 and ln >= 7 and cur is not None:   # ENDPOINT
            cur.append(UsbfsEndpoint(d[2], d[3], d[4] | d[5] << 8))
        elif cur is not None and typ in (0x24, 0x25):    # class-specific
            cur.extra += bytes(d)
        i += ln
    return [ifaces[k] for k in sorted(ifaces)]


# ---- the device -----------------------------------------------------------

class UsbfsDevice:
    """Minimal async usbfs device: control/bulk sync calls plus bulk/iso URBs
    completed on a reaper thread that calls each URB's callback."""

    def __init__(self, path):
        self.path = path
        self.fd = os.open(path, os.O_RDWR)
        self.lock = threading.Lock()
        self.pending: dict = {}          # urb address -> (urb, buffer, callback)
        self.running = True
        self.gone = False
        self.on_gone = None
        self.reaper = threading.Thread(target=self._reap_loop, name="usbfs-reaper", daemon=True)
        self.reaper.start()

    def read_descriptors(self) -> bytes:
        os.lseek(self.fd, 0, os.SEEK_SET)
        chunks = []
        while True:
            b = os.read(self.fd, 4096)
            if not b:
                break
            chunks.append(b)
        return b"".join(chunks)
    # interfaces
    def detach_kernel_driver(self, ifno):
        req = UsbdevfsIoctl(ifno, USBDEVFS_DISCONNECT, None)
        try:
            ioctl(self.fd, USBDEVFS_IOCTL, req)
```

```
        except UsbfsError as e:
            if e.errno not in (errno.ENODATA, errno.ENOENT, errno.EINVAL):
                raise

    def attach_kernel_driver(self, ifno):
        req = UsbdevfsIoctl(ifno, USBDEVFS_CONNECT, None)
        try:
            ioctl(self.fd, USBDEVFS_IOCTL, req)
        except UsbfsError:
            pass

    def claim(self, ifno):
        self.detach_kernel_driver(ifno)
        ioctl(self.fd, USBDEVFS_CLAIMINTERFACE, ctypes.c_uint(ifno))

    def release(self, ifno):
        try:
            ioctl(self.fd, USBDEVFS_RELEASEINTERFACE, ctypes.c_uint(ifno))
        except UsbfsError:
            pass

    def set_altsetting(self, ifno, alt):
        ioctl(self.fd, USBDEVFS_SETINTERFACE, SetInterface(ifno, alt))

    # synchronous transfers
    def control_write(self, rt, req, val, idx, data: bytes, timeout_ms=500):
        buf = ctypes.create_string_buffer(bytes(data), max(1, len(data)))
        ct = CtrlTransfer(rt, req, val, idx, len(data), timeout_ms, ctypes.addressof(buf) if data else None)
        return ioctl(self.fd, USBDEVFS_CONTROL, ct)

    def bulk_write(self, ep, data: bytes, timeout_ms=500):
        buf = ctypes.create_string_buffer(bytes(data), max(1, len(data)))
        bt = BulkTransfer(ep, len(data), timeout_ms, ctypes.addressof(buf))
        return ioctl(self.fd, USBDEVFS_BULK, bt)

    # asynchronous URBs
    def submit_bulk_in(self, ep, length, callback):
        buf = ctypes.create_string_buffer(length)
        urb = Urb()
        urb.type, urb.endpoint, urb.buffer, urb.buffer_length = URB_TYPE_BULK, ep, ctypes.addressof(buf), length
        self._submit(urb, buf, callback)

    def submit_iso_in(self, ep, npackets, mps, callback):
        T = iso_urb_type(npackets)
        buf = ctypes.create_string_buffer(npackets * mps)
        urb = T()
        urb.type, urb.endpoint, urb.flags = URB_TYPE_ISO, ep, URB_ISO_ASAP
        urb.buffer, urb.buffer_length, urb.number_of_packets = ctypes.addressof(buf), npackets * mps, npackets
        for k in range(npackets):
            urb.iso_frame_desc[k].length = mps
        self._submit(urb, buf, callback)

    def _submit(self, urb, buf, callback):
        addr = ctypes.addressof(urb)
        with self.lock:
            self.pending[addr] = (urb, buf, callback)
        try:
            ioctl(self.fd, USBDEVFS_SUBMITURB, urb)
        except UsbfsError:
            with self.lock:
                self.pending.pop(addr, None)
            raise

    def resubmit(self, urb, buf, callback):
        if self.running and not self.gone:
            # reset per-transfer fields before reuse
            urb.status = 0
            urb.actual_length = 0
            urb.error_count = 0
            if urb.type == URB_TYPE_ISO:
                for k in range(urb.number_of_packets):
                    urb.iso_frame_desc[k].actual_length = 0
                    urb.iso_frame_desc[k].status = 0
            self._submit(urb, buf, callback)

    def discard_all(self, callbacks=None):
        """Cancel pending URBs (all, or only those with the given callbacks)."""
        with self.lock:
            items = [(a, u) for a, (u, _b, cb) in self.pending.items() if callbacks is None or cb in callbacks]
        for _a, u in items:
            try:
                ioctl(self.fd, USBDEVFS_DISCARDURB, u)
            except UsbfsError:
                pass   # already completed: the reaper will collect it

    def outstanding(self, callbacks=None):
        with self.lock:
```

```
            return sum(1 for (_u, _b, cb) in self.pending.values() if callbacks is None or cb in callbacks)

    def _reap_loop(self):
        p = select.poll()
        p.register(self.fd, select.POLLOUT | select.POLLERR | select.POLLHUP)
        ptr = c_void_p()
        while self.running or self.outstanding():
            try:
                ev = p.poll(100)
            except InterruptedError:
                continue
            if not ev:
                if not self.running and self.gone:
                    break
                continue
            while True:
                try:
                    ioctl(self.fd, USBDEVFS_REAPURBNDELAY, ptr)
                except UsbfsError as e:
                    if e.errno == errno.EAGAIN:
                        break
                    if e.errno == errno.ENODEV:
                        self._mark_gone()
                        with self.lock:
                            self.pending.clear()
                        return
                    break
                with self.lock:
                    entry = self.pending.pop(ptr.value, None)
                if entry:
                    urb, buf, cb = entry
                    try:
                        cb(urb, buf)
                    except Exception:
                        pass
            if any(e & (select.POLLERR | select.POLLHUP) for _fd, e in ev):
                self._mark_gone()
                with self.lock:
                    self.pending.clear()
                return

    def _mark_gone(self):
        if not self.gone:
            self.gone = True
            cb = self.on_gone
            if cb:
                cb()

    def close(self):
        self.running = False
        self.discard_all()
        if self.reaper is not threading.current_thread():
            self.reaper.join(timeout=2.0)
        try:
            os.close(self.fd)
        except OSError:
            pass


class UsbfsQmx:
    """The real QMX through Linux usbfs directly (no libusb). Same interface
    and behaviour as QmxUsb; used where libusb-1.0 isn't installed (DSM)."""

    def __init__(self, on_iq, on_cat, on_gone, vid=QMX_VID, pid=QMX_PID):
        self.on_iq, self.on_cat, self.on_gone = on_iq, on_cat, on_gone
        self.vid, self.pid = vid, pid
        self.dev = None
        self.running = False
        self.gone_reported = False
        self.claimed: list[int] = []
        self.cdc_ports = []
        self.extra: dict[int, dict] = {}
        self.as_if = None

    def open(self):
        found = usbfs_find_device(self.vid, self.pid)
        if not found:
            raise RadioGone("QMX (%04x:%04x) not found on the NAS USB bus" % (self.vid, self.pid))
        name, path = found
        try:
            self.dev = UsbfsDevice(path)
        except PermissionError:
            raise RadioGone("no permission for %s - run the relay as root" % path)
        except OSError as e:
            raise RadioGone("cannot open %s: %s" % (path, e))
        self.dev.on_gone = self._gone
        cfg = usbfs_parse_config(self.dev.read_descriptors())
```

```
        self.cdc_ports, uac2, aud = discover_qmx(cfg)
        if not self.cdc_ports or aud is None:
            self.close()
            raise RadioGone("QMX descriptors not recognised (CAT=%s audio=%s)" % (self.cdc_ports, aud))
        cdc_ctrl, cdc_data, self.bulk_in, self.bulk_out = self.cdc_ports[0]
        as_if, alt, self.iso_ep, self.iso_mps, ch, self.subframe, bits, rates = aud
        log.info("QMX found via usbfs %s (%s): CAT if%d (IN 0x%02x OUT 0x%02x), I/Q if%d alt%d EP 0x%02x "
                 "mps=%d %dch %d-bit/%dB %s%s", path, name, cdc_data, self.bulk_in, self.bulk_out, as_if, alt,
                 self.iso_ep, self.iso_mps, ch, bits, self.subframe, rates or "", " UAC2" if uac2 else "")
        n = len(self.cdc_ports)
        log.info("QMX USB serial ports: %d (%s)", n, ", ".join(
            "port %d = if%s/if%d" % (i + 1, c, d) for i, (c, d, _, _) in enumerate(self.cdc_ports)))
        try:
            for i in [x for x in (cdc_ctrl, cdc_data, as_if) if x is not None]:
                self.dev.claim(i)
                self.claimed.append(i)
        except UsbfsError as e:
            self.close()
            if e.errno == errno.EBUSY:
                raise RadioGone("QMX is in use by another program on the NAS - stop it, then reconnect")
            raise RadioGone("claiming the QMX failed: %s" % e)
        if cdc_ctrl is not None:
            try:
                self.dev.control_write(0x21, 0x20, 0, cdc_ctrl, struct.pack("<IBBB", CAT_BAUD, 0, 0, 8))
                self.dev.control_write(0x21, 0x22, 0x0003, cdc_ctrl, b"")
            except UsbfsError as e:
                log.warning("CDC line setup: %s (continuing)", e)
        self.dev.set_altsetting(as_if, alt)
        self.as_if = as_if
        if not uac2:
            try:
                self.dev.control_write(0x22, 0x01, 0x0100, self.iso_ep, SAMPLE_RATE.to_bytes(3, "little"))
            except UsbfsError as e:
                log.info("sample-rate SET_CUR not accepted (%s) - device default used", e)
        self.running = True
        self.gone_reported = False
        for _ in range(ISO_TRANSFERS):
            self.dev.submit_iso_in(self.iso_ep, ISO_PACKETS, self.iso_mps, self._iso_cb)
        for _ in range(2):
            self.dev.submit_bulk_in(self.bulk_in, 64, self._bulk_cb)

    # -- completions (reaper thread) ----------------------------------------
    def _gone(self):
        if not self.gone_reported:
            self.gone_reported = True
            log.warning("QMX unplugged / powered off")
            self.running = False
            self.on_gone()

    def _status_ok(self, urb):
        st = urb.status
        if st in (0, -errno.EXDEV, -errno.EREMOTEIO):   # EXDEV: some iso packets missed; still usable
            return True
        if st in (-errno.ENODEV, -errno.ESHUTDOWN, -errno.EPROTO) and self.running:
            self._gone()                                # only while running: closing kills URBs too
        return False                                    # ENOENT / ECONNRESET = discarded

    def _resubmit(self, urb, buf, cb):
        if not self.running:
            return
        try:
            self.dev.resubmit(urb, buf, cb)
        except UsbfsError as e:
            log.warning("USB resubmit failed: %s", e)
            self._gone()

    def _iso_cb(self, urb, buf):
        if not self._status_ok(urb):
            return
        raw = bytearray()
        mps = self.iso_mps
        for k in range(urb.number_of_packets):
            d = urb.iso_frame_desc[k]
            if d.status == 0 and d.actual_length:
                raw += buf.raw[k * mps: k * mps + d.actual_length]
        if raw:
            self.on_iq(to_s16(raw, self.subframe))
        self._resubmit(urb, buf, self._iso_cb)

    def _bulk_cb(self, urb, buf):
        if not self._status_ok(urb):
            return
        if urb.actual_length:
            self.on_cat(buf.raw[:urb.actual_length])
        self._resubmit(urb, buf, self._bulk_cb)

    # -- extra serial ports -------------------------------------------------
```

```
    def serial_count(self) -> int:
        return len(self.cdc_ports)

    def serial_open(self, idx, on_data):
        name = "port %d" % (idx + 1)
        st = self.extra.get(idx)
        if st and st["cb"]:
            st["cb"] = on_data
            return True, ""
        if len(self.cdc_ports) <= idx:
            return False, ("the radio has %d USB serial port%s - set 'USB serial ports' to %d"
                           % (len(self.cdc_ports), "" if len(self.cdc_ports) == 1 else "s", idx + 1))
        comm, data, bin_, bout = self.cdc_ports[idx]
        st = {"ifs": [], "out": bout, "cb": None, "rx": None}
        self.extra[idx] = st
        try:
            for i in [x for x in (comm, data) if x is not None]:
                self.dev.claim(i)
                st["ifs"].append(i)
        except UsbfsError as e:
            self._serial_release(idx)
            return False, ("%s is in use by another program on the NAS" % name if e.errno == errno.EBUSY
                           else "%s claim failed: %s" % (name, e))
        if comm is not None:
            try:
                self.dev.control_write(0x21, 0x20, 0, comm, struct.pack("<IBBB", CAT_BAUD, 0, 0, 8))
                self.dev.control_write(0x21, 0x22, 0x0003, comm, b"")
            except UsbfsError as e:
                log.warning("%s line setup: %s (continuing)", name, e)
        st["cb"] = on_data

        def rx(urb, buf, idx=idx):
            s = self.extra.get(idx)
            if not self._status_ok(urb) or not s or not s["cb"]:
                return
            if urb.actual_length:
                s["cb"](buf.raw[:urb.actual_length])
            self._resubmit(urb, buf, s["rx"])
        st["rx"] = rx
        for _ in range(2):
            self.dev.submit_bulk_in(bin_, 512, rx)
        log.info("QMX %s open (if%s/if%d)", name, comm, data)
        return True, ""

    def serial_send(self, idx, data: bytes):
        st = self.extra.get(idx)
        if not self.dev or not st or not st["cb"]:
            return
        try:
            self.dev.bulk_write(st["out"], data)
        except UsbfsError as e:
            if e.errno == errno.ENODEV:
                self._gone()
                raise RadioGone("QMX gone")
            log.warning("port %d write: %s", idx + 1, e)

    def serial_close(self, idx):
        st = self.extra.get(idx)
        if not st:
            return
        st["cb"] = None
        if st["rx"] and self.dev:
            self.dev.discard_all({st["rx"]})
            deadline = time.time() + 1.0
            while time.time() < deadline and self.dev.outstanding({st["rx"]}):
                time.sleep(0.02)
        self._serial_release(idx)
        log.info("QMX port %d closed", idx + 1)

    def _serial_release(self, idx):
        st = self.extra.pop(idx, None)
        if st and self.dev:
            for i in st["ifs"]:
                self.dev.release(i)

    def term_open(self, on_data):
        return self.serial_open(1, on_data)

    def term_send(self, data: bytes):
        self.serial_send(1, data)

    def term_close(self):
        self.serial_close(1)

    def send_cat(self, data: bytes):
        if not self.dev:
            raise RadioGone("closed")
```

```
        try:
            self.dev.bulk_write(self.bulk_out, data)
        except UsbfsError as e:
            if e.errno == errno.ENODEV:
                self._gone()
                raise RadioGone("QMX gone")
            log.warning("CAT write: %s", e)

    def close(self):
        self.running = False
        dev, self.dev = self.dev, None
        if not dev:
            return
        for st in self.extra.values():
            st["cb"] = None
            self.claimed += st["ifs"]
        self.extra = {}
        dev.on_gone = None                    # an orderly close is not an unplug
        dev.discard_all()
        deadline = time.time() + 1.0
        while time.time() < deadline and dev.outstanding():
            time.sleep(0.02)
        if self.as_if is not None and not dev.gone:
            try:
                dev.set_altsetting(self.as_if, 0)   # stop the isochronous stream
            except UsbfsError:
                pass
        dev.close()          # waits for the reaper, closes the fd (releases interfaces)
        self.claimed = []


def libusb_available() -> bool:
    try:
        from usb1 import _libusb1
        path = os.environ.get("QMX_LIBUSB")
        _libusb1.loadLibrary(path) if path else _libusb1.loadLibrary()
        return True
    except Exception:
        return False


def pick_backend():
    """libusb if it loads (any OS), else Linux usbfs. QMX_BACKEND=libusb|usbfs forces one."""
    want = os.environ.get("QMX_BACKEND", "").lower()
    if want == "usbfs" or (want != "libusb" and not libusb_available()):
        if not sys.platform.startswith("linux"):
            raise RadioGone("libusb-1.0 not found (and usbfs is Linux-only)")
        return UsbfsQmx
    return QmxUsb


class FakeRadio:
    """Bench-test stand-in: a +3 kHz tone and a tiny Kenwood CAT emulator."""

    def __init__(self, on_iq, on_cat, on_gone):
        self.on_iq, self.on_cat, self.on_gone = on_iq, on_cat, on_gone
        self.running = False
        self.freq, self.mode = 14074000, "2"
        self.bufs = {0: b"", 2: b""}
        self.cbs = {}            # idx -> data callback for extra ports
        self.sel = 0

    def open(self):
        self.running = True
        threading.Thread(target=self._gen, daemon=True).start()
        log.info("FAKE radio: synthetic I/Q tone at +3000 Hz, CAT emulator")
    def _gen(self):
        ph, step = 0.0, 2 * math.pi * 3000 / SAMPLE_RATE
        nxt = time.monotonic()
        while self.running:
            out = bytearray()
            for _ in range(FRAME_PAIRS):
                out += struct.pack("<hh", int(8000 * math.cos(ph)), int(8000 * math.sin(ph)))
                ph = (ph + step) % (2 * math.pi)
            self.on_iq(bytes(out))
            nxt += FRAME_PAIRS / SAMPLE_RATE
            time.sleep(max(0.0, nxt - time.monotonic()))

    def serial_count(self) -> int:
        n = getattr(self.args_ns, "fake_ports", 3)
        return 1 if getattr(self.args_ns, "fake_one_port", False) else n

    def send_cat(self, data: bytes, idx: int = 0, reply=None):
        reply = reply or self.on_cat
        self.bufs[idx] += data
        while b";" in self.bufs[idx]:
            cmd, self.bufs[idx] = self.bufs[idx].split(b";", 1)
```

```
            c = cmd.decode(errors="replace")
            r = None
            if c == "ID":
                r = "ID020;"
            elif c == "FA":
                r = "FA%011d;" % self.freq
            elif c.startswith("FA") and c[2:].isdigit():
                self.freq = int(c[2:])
            elif c == "MD":
                r = "MD%s;" % self.mode
            elif c.startswith("MD") and len(c) == 3:
                self.mode = c[2]
            elif c == "FW":
                r = "FW2700;"
            elif c == "VN":
                r = "VN1_03_002QMX;"
            elif c in ("Q9", "Q9 1"):
                r = "Q91;"
            elif c.startswith("MM"):
                r = "MM;"
            if r:
                reply(r.encode())

    # Fake extra ports: port 2 answers CR with a tiny ANSI menu (arrows move);
    # port 3 is a second, independent CAT port.
    def serial_open(self, idx, on_data):
        n = self.serial_count()
        if n <= idx:
            return False, ("the radio has %d USB serial port%s - set 'USB serial ports' to %d"
                           % (n, "" if n == 1 else "s", idx + 1))
        self.cbs[idx] = on_data
        if idx == 1:
            self.sel = 0
        return True, ""

    def _paint(self):
        items = ["Band config.", "Mode config.", "System config", "Exit terminal"]
        out = "\x1b[2J\x1b[H QMX menu\r\n"
        for i, it in enumerate(items):
            out += ("\x1b[7m %s \x1b[0m" if i == self.sel else " %s ") % it + "\r\n"
        self.cbs[1](out.encode())

    def serial_send(self, idx, data: bytes):
        cb = self.cbs.get(idx)
        if not cb:
            return
        if idx == 2:
            self.send_cat(data, idx=2, reply=cb)
        elif data in (b"\r", b"\x1b[B", b"\x1b[A"):
            if data == b"\x1b[B":
                self.sel = (self.sel + 1) % 4
            elif data == b"\x1b[A":
                self.sel = (self.sel - 1) % 4
            self._paint()

    def serial_close(self, idx):
        self.cbs.pop(idx, None)

    def term_open(self, on_data):
        return self.serial_open(1, on_data)

    def term_send(self, data: bytes):
        self.serial_send(1, data)

    def term_close(self):
        self.serial_close(1)

    def close(self):
        self.running = False
        self.cbs = {}


# ---------------------------------------------------------------------------
# Shared radio: opened by the first user (Tab5 session or port-3 client),
# released by the last.
# ---------------------------------------------------------------------------

class RadioManager:
    def __init__(self, args):
        self.args = args
        self.lock = threading.Lock()
        self.radio = None
        self.dead = False
        self.users: set = set()
        self.session = None          # the current Tab5 session gets I/Q and CAT
        self.txlog = None            # TxLog, unless --txlog-dir ''
        self.catlog = None           # CatLog (see catlog.off)
```

```
        self.flags = Flags(RELAY_DIR)

    def acquire(self, user):
        with self.lock:
            if self.radio is not None and self.dead:
                self._close_locked()            # a previous QMX connection died; start fresh
            if self.radio is None:
                cls = FakeRadio if self.args.fake else pick_backend()
                r = cls(self._on_iq, self._on_cat, self._on_gone)
                r.args_ns = self.args
                r.open()                        # raises RadioGone
                self.radio, self.dead = r, False
            self.users.add(user)
            return self.radio

    def release(self, user, radio):
        with self.lock:
            self.users.discard(user)
            if self.session is user:
                self.session = None
            if radio is self.radio and not self.users:
                self._close_locked()

    def _close_locked(self):
        r, self.radio = self.radio, None
        if r:
            r.close()
            log.info("QMX released")

    # radio callbacks (libusb / fake thread)
    def _on_iq(self, b):
        s = self.session
        if s:
            s._on_iq(b)

    def _on_cat(self, b):
        s = self.session
        if s:
            s._on_cat(b)

    def _on_gone(self):
        self.dead = True
        for u in list(self.users):
            u.radio_gone()


# ---------------------------------------------------------------------------
# CAT log and the power-off guard.
#
# CatLog writes every command that reaches the QMX, and every reply, to
# catlog/catlog-<date>.log, one line each, stamped with the NAS's clock:
#     11:26:03.214  T5>QMX   FA;
#     11:26:03.236  QMX>T5   FA00007025450;
#     11:26:05.001  PC>QMX   PS0;   BLOCKED (power off)
# "T5" is the Tab5 session, "PC" the port-3 client (WSJT-X, rigctld ...),
# "T5 menu" the keystrokes the Tab5's Radio menus send to port 2. Lines are
# queued and written by a thread once a second, never on the USB or socket
# threads. A day's file stops at CATLOG_MAX_BYTES with a note saying so.
#
# Flag files in the relay folder, like restart.request, checked every 2 s:
#     catlog.off   present = no CAT log
#     allow-ps0    present = PS0; (power off) is passed to the radio.
#                  Absent (the default) = it is dropped and logged as BLOCKED.
#     iq-during-tx present = keep streaming I/Q to the Tab5 while it transmits.
#                  Absent (the default) = I/Q is paused from TX; to RX; - see
#                  Session.tx_hold below.
# ---------------------------------------------------------------------------

CATLOG_MAX_BYTES = 40 * 1024 * 1024
TX_IQ_HOLD_MAX_S = 180      # longest I/Q pause if RX; never comes (WSPR burst ~111 s)
RELAY_DIR = os.path.dirname(os.path.abspath(__file__))
class Flags:
    def __init__(self, directory: str):
        self.dir, self._t, self._v = directory, 0.0, {}

    def has(self, name: str) -> bool:
        now = time.monotonic()
        if now - self._t > 2.0:
            self._t = now
            self._v = {n: os.path.exists(os.path.join(self.dir, n)) for n in ("catlog.off", "allow-ps0", "iq-during-tx", "tab5-timed
  ↪ -tx")}
        return self._v.get(name, False)


class CatLog:
    def __init__(self, directory: str, flags: Flags):
        self.dir, self.flags = directory, flags
```

```
        self.q: "queue.Queue[str]" = queue.Queue()
        threading.Thread(target=self._writer, daemon=True, name="catlog").start()

    def note(self, who: str, data: bytes, extra: str = ""):
        if self.flags.has("catlog.off"):
            return
        t = time.time()
        stamp = time.strftime("%H:%M:%S", time.localtime(t)) + ".%03d" % int((t % 1) * 1000)
        text = data.decode("ascii", errors="backslashreplace").replace("\r", "\\r").replace("\n", "\\n")
        self.q.put("%s  %-8s %s%s\n" % (stamp, who, text, ("   " + extra) if extra else ""))

    def _writer(self):
        full_for = None
        while True:
            lines = [self.q.get()]
            time.sleep(1.0)
            while True:
                try:
                    lines.append(self.q.get_nowait())
                except queue.Empty:
                    break
            day = time.strftime("%Y-%m-%d")
            path = os.path.join(self.dir, "catlog-%s.log" % day)
            try:
                os.makedirs(self.dir, exist_ok=True)
                size = os.path.getsize(path) if os.path.exists(path) else 0
                if size >= CATLOG_MAX_BYTES:
                    if full_for != day:
                        full_for = day
                        with open(path, "a") as f:
                            f.write("--- %s: %d MB reached, CAT log paused until tomorrow ---\n"
                                    % (time.strftime("%H:%M:%S"), CATLOG_MAX_BYTES // 1048576))
                    continue
                with open(path, "a") as f:
                    f.writelines(lines)
            except OSError as e:
                log.warning("catlog: cannot write %s: %s", self.dir, e)


POWER_OFF = re.compile(rb"^\s*PS\s*0\s*$", re.I)
def guard_cat(data: bytes, carry: bytearray, who: str, mgr) -> bytes:
    """Pass complete CAT commands through, dropping PS0; unless allow-ps0 is
    present; log each command. An incomplete tail waits in `carry` for the
    rest of it (every CAT command ends with ';')."""
    carry += data
    out = bytearray()
    while True:
        i = carry.find(b";")
        if i < 0:
            break
        cmd = bytes(carry[:i + 1])
        del carry[:i + 1]
        if POWER_OFF.match(cmd[:-1]) and not mgr.flags.has("allow-ps0"):
            log.warning("%s sent %r (power off) - BLOCKED; put a file named allow-ps0 in %s to allow it",
                        who, cmd, RELAY_DIR)
            if mgr.catlog:
                mgr.catlog.note(who + ">QMX", cmd, "BLOCKED (power off)")
            continue
        if mgr.catlog:
            mgr.catlog.note(who + ">QMX", cmd)
        out += cmd
    if len(carry) > 256:                  # not CAT (no ';' for 256 bytes): pass it on as-is
        if mgr.catlog:
            mgr.catlog.note(who + ">QMX", bytes(carry), "(no ';')")
        out += carry
        del carry[:]
    return bytes(out)


# ---------------------------------------------------------------------------
# Transmit log: what the relay actually handed to the QMX during a TX burst,
# timed on THIS host's clock only. Every interval in it (burst length, gap
# between tone changes, lateness of a symbol) is the difference of two
# readings of time.monotonic() on the NAS, so it never mixes clocks. The
# NAS's wall clock is used for one thing: naming the FT8/FT4 slot, so a burst
# here can be paired with the same burst in the Tab5's console log
# (tools/tx-test/tx_compare.py). Nothing is written to disk during a burst;
# the whole burst is queued to a writer thread when RX; has gone out.
# ---------------------------------------------------------------------------

class TxLog:
    MAX_CMDS = 400           # a burst is ~85 commands; anything beyond is noise

    def __init__(self, directory: str):
        self.dir = directory
        self.q: "queue.Queue[str]" = queue.Queue()
        self.burst = None        # list of (t_in_ns, t_out_ns, cmd) while keyed
```

```
        self.wall0 = 0.0         # NAS wall clock at key down (slot naming only)
        self.count = 0
        self.carry = b""
        self.flags = None            # Flags, set by main(): records whether I/Q was paused
        self.iq_paused = None
        self.timed_by = "Tab5"       # who scheduled the tones: the Tab5 or this relay (TX_SCHED)
        threading.Thread(target=self._writer, daemon=True, name="txlog").start()

    # called on the Tab5 session thread, around radio.send_cat()
    def cat(self, payload: bytes, t_in: int, t_out: int, timed_by: str = "Tab5"):
        data = self.carry + payload
        cmds = data.split(b";")
        self.carry = cmds.pop()[-64:]             # incomplete tail, if any
        for c in cmds:
            c = c.strip().decode(errors="replace")
            if not c:
                continue
            if c.startswith("TX"):
                if self.burst:
                    self._finish(complete=False)
                self.burst = [(t_in, t_out, c)]
                self.wall0 = time.time() - (time.monotonic_ns() - t_out) / 1e9
                self.iq_paused = (not self.flags.has("iq-during-tx")) if self.flags else None
                self.timed_by = timed_by
            elif self.burst is not None:
                if len(self.burst) < self.MAX_CMDS:
                    self.burst.append((t_in, t_out, c))
                if c.startswith("RX"):
                    self._finish(complete=True)

    @staticmethod
    def _ta_hz(c: str):
        try:
            return float(c[2:]) if c.startswith("TA") else None
        except ValueError:
            return None

    def _finish(self, complete: bool):
        b, self.burst = self.burst, None
        self.count += 1
        try:
            self.q.put(self._format(b, self.wall0, complete))
        except Exception as e:                       # never let logging hurt the relay
            log.warning("txlog: %s", e)

    def _format(self, b, wall0: float, complete: bool) -> str:
        t_key = b[0][1]
        ms = lambda t: (t - t_key) / 1e6
        tones = [(ti, to, c) for ti, to, c in b if (self._ta_hz(c) or 0) >= 10]
        keyup = next((to for ti, to, c in b if self._ta_hz(c) is not None and self._ta_hz(c) < 10), None)
        rx = next((to for ti, to, c in b if c.startswith("RX")), None)
        n = len(tones)
        # FT8 = 79 tones at 160 ms, FT4 = 105 at 48 ms; anything else: no ideal
        sym = 48.0 if 90 <= n <= 110 else (160.0 if 60 <= n < 90 else 0.0)
        period = 7.5 if sym == 48.0 else 15.0
        slot0 = math.floor(wall0 / period) * period
        key_into = (wall0 - slot0) * 1000
        utc = time.strftime("%Y-%m-%dT%H:%M:%S", time.gmtime(slot0)) + (".5" if slot0 % 1 else "") + "Z"
        gaps = [(tones[i][1] - tones[i - 1][1]) / 1e6 for i in range(1, n)]
        # symbol i is due i*sym after key down (the Tab5 sends TX; and the
        # first tone together, at t0); + = reached the radio late, - = early
        late = [ms(tones[i][1]) - i * sym for i in range(n)] if sym else []
        hold = max((to - ti) / 1e6 for ti, to, c in b)
        out = ["# burst %d  slot %s  (%s)" % (self.count, utc, "complete" if complete else "NO RX; - next TX; came first")]
        out.append("RELAY key down at +%.0f ms into the slot (NAS clock)" % key_into)
        parts = ["RELAY TX burst: %d tones" % n]
        if keyup is not None:
            parts.append("RF on-air %.0f ms (TX; -> key-up TA0;)" % ms(keyup))
        if rx is not None:
            parts.append("TX; -> RX; %.0f ms" % ms(rx))
        if gaps:
            off = sum(1 for g in gaps if sym and abs(g - sym) > 20)
            parts.append("tone gaps min/avg/max %.0f/%.1f/%.0f ms" % (min(gaps), sum(gaps) / len(gaps), max(gaps)))
            if sym:
                parts.append("gaps off by >20 ms: %d" % off)
        if late:
            parts.append("symbol timing vs schedule %+d..%+d ms" % (round(min(late)), round(max(late))))
        parts.append("relay USB write max %.1f ms" % hold)
        if self.iq_paused is not None:
            parts.append("I/Q to Tab5 %s" % ("paused" if self.iq_paused else "streamed"))
        parts.append("tones timed by %s" % self.timed_by)
        out.append("; ".join(parts))
        for ti, to, c in b:
            out.append("  %9.1f ms  %s" % (ms(to), c))
        return "\n".join(out) + "\n"

    def _writer(self):
```

```
        while True:
            text = self.q.get()
            try:
                os.makedirs(self.dir, exist_ok=True)
                path = os.path.join(self.dir, time.strftime("txlog-%Y-%m-%d.log"))
                with open(path, "a") as f:
                    f.write(text)
                log.info("txlog: burst %d written to %s", self.count, path)
            except OSError as e:
                log.warning("txlog: cannot write %s: %s", self.dir, e)


# ---------------------------------------------------------------------------
# One Tab5 session
# ---------------------------------------------------------------------------

class Session:
    def __init__(self, sock: socket.socket, peer, args, mgr: RadioManager):
        self.sock, self.peer, self.args, self.mgr = sock, peer, args, mgr
        # Two queues: control frames (CAT, terminal, status, ping) always go
        # out before queued I/Q, so a slow WiFi link delays the spectrum, not
        # the radio's replies. The I/Q queue is short and drops when full.
        self.ctl: collections.deque[bytes] = collections.deque()
        self.iq: collections.deque[bytes] = collections.deque()
        self.wake = threading.Condition()
        self.iq_acc = bytearray()
        self.alive = True
        self.dropped = 0
        self.iq_bytes = 0
        self.cat_carry = bytearray()
        # I/Q pause during transmit. From the Tab5's TX; until its RX; the QMX
        # is transmitting and its I/Q is of no use, yet it is ~190 KB/s of
        # traffic INTO the Tab5 - the direction that overflows its WiFi
        # chip's SDIO link ("SDIO RX oversize - draining"), and on 2026-10-14
        # one of those stalled a burst's last symbols by 1.5 s. Pausing it
        # leaves the link nearly idle while timing matters. PINGs continue, so
        # the Tab5's 3 s idle timeout is not tripped. TX_IQ_HOLD_MAX_S caps a
        # pause whose RX; never arrives (WSPR bursts are ~111 s).
        self.tx_hold = False
        self.tx_hold_t0 = 0.0
        self.tx_held_frames = 0
        # Relay-timed bursts (TX_SCHED). cat_lock keeps the playout thread's
        # tone writes and the Tab5's own CAT (its mid-burst PC;SW;, RX;) from
        # interleaving inside one USB write and keeps the logs in order.
        self.cat_lock = threading.Lock()
        self.play: threading.Thread | None = None
        self.play_abort = threading.Event()
        self.txsched = False

    def _put_ctl(self, data: bytes):
        with self.wake:
            self.ctl.append(data)
            self.wake.notify()

    def _tx_state(self, payload: bytes):
        """Follow TX;/RX; in the Tab5's CAT stream to pause I/Q while keyed."""
        for c in payload.split(b";"):
            c = c.strip()
            if c.startswith(b"TX"):
                if not self.tx_hold and not self.mgr.flags.has("iq-during-tx"):
                    self.tx_hold, self.tx_hold_t0, self.tx_held_frames = True, time.monotonic(), 0
            elif c.startswith(b"RX") and self.tx_hold:
                self.tx_hold = False
                log.info("TX: I/Q to the Tab5 paused %.1f s (%d frames not sent)",
                         time.monotonic() - self.tx_hold_t0, self.tx_held_frames)

    # radio callbacks (libusb thread)
    def _on_iq(self, s16: bytes):
        if self.tx_hold:
            if time.monotonic() - self.tx_hold_t0 < TX_IQ_HOLD_MAX_S:
                self.tx_held_frames += 1
                self.iq_acc.clear()          # restart on a clean frame when RX; comes
                return
            self.tx_hold = False
            log.warning("TX: no RX; after %d s - I/Q to the Tab5 resumed anyway", TX_IQ_HOLD_MAX_S)
        self.iq_acc += s16
        fb = FRAME_PAIRS * 4
        while len(self.iq_acc) >= fb:
            chunk = bytes(self.iq_acc[:fb])
            del self.iq_acc[:fb]
            with self.wake:
                if len(self.iq) >= IQ_QUEUE_MAX:
                    self.dropped += 1      # Tab5/WiFi can't keep up: drop I/Q, keep latency low
                    continue
                self.iq_bytes += len(chunk)
                self.iq.append(frame(PROTO_IQ, chunk))
                self.wake.notify()
```

```
    # ---- relay-timed transmit burst -----------------------------------
    def _send_cat(self, radio, data: bytes, who: str, timed_by: str = "Tab5") -> int:
        """One CAT write to the QMX, logged; returns the time it completed."""
        with self.cat_lock:
            t_in = time.monotonic_ns()
            if who and self.mgr.catlog:
                self.mgr.catlog.note(who, data)
            self._tx_state(data)
            radio.send_cat(data)
            t_out = time.monotonic_ns()
            if self.mgr.txlog:
                self.mgr.txlog.cat(data, t_in, t_out, timed_by)
            return t_out

    def _tx_sched(self, radio, payload: bytes):
        try:
            ver, _flags, n, sym_us = TX_SCHED_HDR.unpack_from(payload)
        except struct.error:
            ver = n = sym_us = 0
        if ver != 1 or not 1 <= n <= 400 or not 10_000 <= sym_us <= 1_000_000 \
                or len(payload) < TX_SCHED_HDR.size + 4 * n:
            log.warning("TX_SCHED: bad request (ver %d, %d tones, %d us, %d bytes) - refused",
                        ver, n, sym_us, len(payload))
            self._put_ctl(frame(PROTO_TX_DONE, TX_DONE_FMT.pack(2, 0, 0, 0)))
            return
        hz = struct.unpack_from("<%dI" % n, payload, TX_SCHED_HDR.size)
        self._stop_playout()                        # a new burst replaces one still playing
        self.play_abort.clear()
        self.play = threading.Thread(target=self._playout, args=(radio, hz, sym_us),
                                     daemon=True, name="tx-playout")
        self.play.start()

    def _playout(self, radio, centi_hz, sym_us: int):
        n, sym_ns = len(centi_hz), sym_us * 1000
        status, sent, latest_ns, t_key, t_up = 2, 0, 0, None, None
        try:
            t_key = self._send_cat(radio, b"TX;", "RELAY>QMX", "relay")
            for i, c in enumerate(centi_hz):
                due = t_key + i * sym_ns
                wait = (due - time.monotonic_ns()) / 1e9
                if wait > 0 and self.play_abort.wait(wait):
                    break
                if self.play_abort.is_set():
                    break
                t = self._send_cat(radio, b"TA%d.%02d;" % (c // 100, c % 100), "RELAY>QMX", "relay")
                latest_ns = max(latest_ns, t - due)
                sent += 1
            else:
                wait = (t_key + n * sym_ns - time.monotonic_ns()) / 1e9
                if wait > 0:
                    self.play_abort.wait(wait)
                status = 1 if self.play_abort.is_set() else 0
        except Exception as e:                      # RadioGone, USB error: still try to key up
            log.warning("TX playout: %s", e)
        finally:
            try:
                t_up = self._send_cat(radio, b"TA0;", "RELAY>QMX", "relay")
            except Exception as e:
                log.error("TX playout: key-up tone failed: %s", e)
                status = 2
        dur = int((t_up - t_key) / 1e6) if (t_key and t_up) else 0
        log.info("TX playout %s: %d/%d tones, RF on-air %d ms, latest symbol %+.1f ms",
                 ("complete", "aborted", "FAILED")[status], sent, n, dur, latest_ns / 1e6)
        self._put_ctl(frame(PROTO_TX_DONE, TX_DONE_FMT.pack(status, sent, dur, int(latest_ns / 1e6))))

    def _stop_playout(self, timeout: float = 2.0) -> bool:
        """Abort a running playout and wait for its key-up. True if one was running."""
        th = self.play
        if not th or not th.is_alive():
            return False
        self.play_abort.set()
        th.join(timeout)
        return True

    def _on_cat(self, data: bytes):
        log.debug("QMX -> Tab5 CAT %r", data)
        if self.mgr.catlog:
            self.mgr.catlog.note("QMX>T5", data)
        self._put_ctl(frame(PROTO_CAT, data))

    def _on_term(self, data: bytes):
        self._put_ctl(frame(PROTO_TERM_DATA, data))

    def radio_gone(self):
        self.stop("QMX disconnected")

    def stop(self, why: str):
```

```
        if self.alive:
            self.alive = False
            with self.wake:
                self.wake.notify()
            log.info("Tab5 session %s:%d ending: %s", self.peer[0], self.peer[1], why)
            try:
                self.sock.shutdown(socket.SHUT_RDWR)
            except OSError:
                pass

    def run(self):
        sock = self.sock
        sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
        sock.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
        # Small kernel send buffer: otherwise Linux autotunes it to megabytes,
        # seconds of I/Q pile up there instead of being dropped above, and every
        # CAT reply waits behind them (the Tab5 then times out IQ-mode setup).
        try:
            sock.setsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF, TAB5_SNDBUF)
        except OSError:
            pass
        radio = None
        try:
            sock.settimeout(5)
            t, _, n = HDR.unpack(recv_exact(sock, 4))
            hello = recv_exact(sock, n) if n else b""
            if t != PROTO_HELLO or not hello.startswith(b"QMXR/1"):
                raise ConnectionError("not a QMXR/1 client")
            log.info("Tab5 %s:%d hello: %s", self.peer[0], self.peer[1], hello.decode(errors="replace"))

            radio = self.mgr.acquire(self)    # raises RadioGone -> we close without answering
            self.mgr.session = self

            # txsched=1: this relay times transmit bursts itself (TX_SCHED). A
            # file named tab5-timed-tx in the relay folder leaves it out, so the
            # Tab5 times them as before - for comparing the two.
            self.txsched = not self.mgr.flags.has("tab5-timed-tx")
            sock.sendall(frame(PROTO_HELLO, b"QMXR/1 rate=%d ch=2 fmt=s16le src=nas term=1%s"
                               % (SAMPLE_RATE, b" txsched=1" if self.txsched else b"")))
            log.info("transmit bursts timed by %s", "this relay" if self.txsched else "the Tab5 (tab5-timed-tx)")
            sock.settimeout(None)
            sender = threading.Thread(target=self._send_loop, daemon=True)
            sender.start()
            while self.alive:
                t, _, n = HDR.unpack(recv_exact(sock, 4))
                payload = recv_exact(sock, n) if n else b""
                if t == PROTO_CAT and payload:
                    log.debug("Tab5 -> QMX CAT %r", payload)
                    payload = guard_cat(payload, self.cat_carry, "T5", self.mgr)
                    if not payload:
                        continue
                    self._send_cat(radio, payload, "")     # guard_cat already logged it
                elif t == PROTO_TX_SCHED and self.txsched:
                    self._tx_sched(radio, payload)
                elif t == PROTO_TX_ABORT:
                    if self.play and self.play.is_alive():
                        log.info("TX_ABORT from the Tab5")
                        self.play_abort.set()
                elif t == PROTO_TERM_DATA and payload:
                    if self.mgr.catlog:
                        self.mgr.catlog.note("T5 menu", payload)
                    radio.term_send(payload)
                elif t == PROTO_TERM_OPEN:
                    ok, why = radio.term_open(self._on_term)
                    if not ok:
                        log.warning("terminal: %s", why)
                    self._put_ctl(frame(PROTO_TERM_STATUS, (b"\x01" if ok else b"\x00" + why.encode())[:200]))
                elif t == PROTO_TERM_CLOSE:
                    radio.term_close()
        except RadioGone as e:
            log.warning("%s", e)
        except (ConnectionError, OSError, socket.timeout) as e:
            if self.alive:
                log.info("Tab5 %s:%d disconnected: %s", self.peer[0], self.peer[1], e)
        finally:
            self.alive = False
            # The Tab5 is gone - never leave the QMX keyed on its behalf.
            if radio and (self._stop_playout() or self.tx_hold):
                try:
                    self._send_cat(radio, b"TA0;RX;", "RELAY>QMX")
                    log.warning("Tab5 left while transmitting - QMX put back in receive")
                except Exception as e:
                    log.error("could not put the QMX back in receive: %s", e)
            if radio:
                try:
                    radio.term_close()
                except Exception:
```

```
                    pass
                if self.dropped:
                    log.info("%d I/Q frames were dropped for a slow link", self.dropped)
                self.mgr.release(self, radio)
            try:
                sock.close()
            except OSError:
                pass

    def _send_loop(self):
        last_ping = time.monotonic()
        while self.alive:
            with self.wake:
                if not self.ctl and not self.iq:
                    self.wake.wait(0.25)
                # all pending control frames first, then at most one I/Q frame,
                # so a CAT reply never waits behind more than one I/Q frame here
                out = list(self.ctl)
                self.ctl.clear()
                if self.iq:
                    out.append(self.iq.popleft())
            if time.monotonic() - last_ping >= 1.0:
                out.insert(0, frame(PROTO_PING))
                last_ping = time.monotonic()
            try:
                for data in out:
                    self.sock.sendall(data)
            except OSError as e:
                self.stop("send failed: %s" % e)
                return


# ---------------------------------------------------------------------------
# QMX port 3 as raw TCP serial for PC software
# ---------------------------------------------------------------------------

PORT3 = 2   # index into the QMX's serial ports (0 = CAT, 1 = terminal, 2 = port 3)


class SerialClient:
    """One PC on the QMX's third serial port. Bytes are passed through
    unchanged in both directions - no framing, like a serial cable."""

    def __init__(self, sock: socket.socket, peer, mgr: RadioManager):
        self.sock, self.peer, self.mgr = sock, peer, mgr
        self.q: queue.Queue[bytes] = queue.Queue()
        self.alive = True

    def radio_gone(self):
        self.stop("QMX disconnected")

    def stop(self, why: str):
        if self.alive:
            self.alive = False
            log.info("port 3 client %s:%d ending: %s", self.peer[0], self.peer[1], why)
            try:
                self.sock.shutdown(socket.SHUT_RDWR)
            except OSError:
                pass

    def _on_data(self, data: bytes):
        if self.mgr.catlog:
            self.mgr.catlog.note("QMX>PC", data)
        self.q.put(data)               # libusb thread: never block on the PC's socket

    def _send_loop(self):
        while self.alive:
            try:
                data = self.q.get(timeout=0.25)
            except queue.Empty:
                continue
            try:
                self.sock.sendall(data)
            except OSError as e:
                self.stop("send failed: %s" % e)
                return

    def run(self):
        sock = self.sock
        sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
        sock.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
        radio = opened = None
        try:
            radio = self.mgr.acquire(self)
            ok, why = radio.serial_open(PORT3, self._on_data)
            if not ok:
                log.warning("port 3 client %s refused: %s", self.peer[0], why)
```

```
                return
            opened = True
            log.info("port 3 client %s:%d connected", self.peer[0], self.peer[1])
            threading.Thread(target=self._send_loop, daemon=True).start()
            carry = bytearray()
            while self.alive:
                data = sock.recv(4096)
                if not data:
                    log.info("port 3 client %s:%d disconnected", self.peer[0], self.peer[1])
                    break
                data = guard_cat(data, carry, "PC", self.mgr)
                if data:
                    radio.serial_send(PORT3, data)
        except RadioGone as e:
            log.warning("port 3: %s", e)
        except OSError as e:
            if self.alive:
                log.info("port 3 client %s:%d disconnected: %s", self.peer[0], self.peer[1], e)
        finally:
            self.alive = False
            if radio:
                if opened:
                    try:
                        radio.serial_close(PORT3)
                    except Exception:
                        pass
                self.mgr.release(self, radio)
            try:
                sock.close()
            except OSError:
                pass


def serve(listen, port, make, label):
    """Accept loop: one client at a time; a new connection replaces the old one."""
    srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind((listen, port))
    srv.listen(2)
    log.info("listening on %s:%d (%s)", listen, port, label)
    current: list = [None, None]   # [client, thread]
    while True:
        sock, peer = srv.accept()
        old, th = current
        if old and old.alive:
            old.stop("replaced by new connection from %s" % peer[0])
            th.join(timeout=3)     # let it release its port before the new one opens it
        c = make(sock, peer)
        t = threading.Thread(target=c.run, daemon=True)
        current[:] = [c, t]
        t.start()


# ---------------------------------------------------------------------------
# Discovery: a Tab5 broadcasts "QMXR?" on UDP (same port number as the TCP
# service, 7355) and every relay on the network answers it directly with
# "QMXR! <its IP address> <TCP port>". Nothing is sent unless asked.
# ---------------------------------------------------------------------------

DISCOVERY_QUERY = b"QMXR?"
DISCOVERY_REPLY = b"QMXR!"


def local_ip_for(peer_ip: str) -> str:
    """The address of THIS host on the interface that reaches peer_ip (a NAS can
    have several). A connected UDP socket sends nothing; it only picks a route."""
    u = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    try:
        u.connect((peer_ip, 9))
        return u.getsockname()[0]
    except OSError:
        return "0.0.0.0"
    finally:
        u.close()


def discovery_responder(listen: str, udp_port: int, tcp_port: int):
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((listen, udp_port))
    except OSError as e:
        log.warning("discovery disabled - cannot listen on UDP %d: %s", udp_port, e)
        return
    log.info("discovery: answering QMXR? on UDP %s:%d", listen, udp_port)
    last_logged: dict = {}
    while True:
```

```
        try:
            data, peer = s.recvfrom(256)
            if not data.startswith(DISCOVERY_QUERY):
                continue
            ip = local_ip_for(peer[0])
            if ip == "0.0.0.0":
                continue
            s.sendto(b"%s %s %d" % (DISCOVERY_REPLY, ip.encode(), tcp_port), peer)
            now = time.monotonic()
            if now - last_logged.get(peer[0], -1e9) > 60:      # a search sends 3 queries
                last_logged[peer[0]] = now
                log.info("discovery: %s asked, answered %s:%d", peer[0], ip, tcp_port)
        except Exception as e:                                  # never take the relay down
            log.warning("discovery: %s", e)
            time.sleep(1)


# ---------------------------------------------------------------------------
# Self-test against the real radio (read-only: never keys the transmitter)
# ---------------------------------------------------------------------------

class _SelfTestUser:
    def __init__(self):
        self.pairs = 0
        self.sumsq = 0
        self.nsq = 0
        self.peak = 0
        self.cat = bytearray()
        self.gone = False
    def radio_gone(self):
        self.gone = True

    def _on_iq(self, b: bytes):
        from array import array
        a = array("h")
        a.frombytes(b[: len(b) // 2 * 2])
        self.pairs += len(a) // 2
        sub = a[::8]
        self.sumsq += sum(x * x for x in sub)
        self.nsq += len(sub)
        m = max((abs(x) for x in sub), default=0)
        if m > self.peak:
            self.peak = m

    def _on_cat(self, b: bytes):
        self.cat += b


def selftest(mgr) -> bool:
    """Open the QMX, measure the I/Q stream, ask read-only CAT questions on
    port 1, check ports 2 and 3, then release it. Results go to the log."""
    log.info("SELFTEST: starting (read-only - nothing is transmitted)")
    u = _SelfTestUser()
    try:
        radio = mgr.acquire(u)
    except RadioGone as e:
        log.error("SELFTEST FAIL: %s", e)
        return False
    ok_all = True
    mgr.session = u
    try:
        # 1. I/Q stream
        t0 = time.monotonic()
        time.sleep(2.0)
        dt = time.monotonic() - t0
        rate = u.pairs / dt if dt else 0
        rms = math.sqrt(u.sumsq / u.nsq) if u.nsq else 0.0
        dbfs = 20 * math.log10(rms / 32768) if rms > 0 else float("-inf")
        iq_ok = 40000 <= rate <= 56000
        ok_all &= iq_ok
        log.info("SELFTEST %s: I/Q %.0f pairs/s (expect ~48000), level %.1f dBFS rms, peak %d",
                 "PASS" if iq_ok else "FAIL", rate, dbfs, u.peak)
        if iq_ok and u.peak == 0:
            log.warning("SELFTEST note: I/Q is all zeros - is the radio receiving? (IQ mode is set by the Tab5)")

        # 2. CAT on port 1 - queries only
        for q in (b"ID;", b"FA;", b"MD;", b"VN;"):
            u.cat.clear()
            radio.send_cat(q)
            deadline = time.monotonic() + 1.0
            while time.monotonic() < deadline and b";" not in u.cat:
                time.sleep(0.02)
            time.sleep(0.05)
            reply = bytes(u.cat).decode(errors="replace").strip()
            good = bool(reply) and reply.endswith(";")
            ok_all &= good if q == b"ID;" else True
            log.info("SELFTEST %s: port 1 CAT %s -> %s", "PASS" if good else "FAIL",
```

```
                    q.decode(), reply or "(no reply)")
        # 3. serial ports 2 and 3
        n = radio.serial_count()
        log.info("SELFTEST: radio reports %d USB serial port(s)", n)
        if n >= 2:
            ok, why = radio.serial_open(1, lambda b: None)   # open/close only: no bytes sent,
            if ok:                                           # so the radio never enters terminal mode
                radio.serial_close(1)
            ok_all &= ok
            log.info("SELFTEST %s: port 2 (Radio menus) %s", "PASS" if ok else "FAIL", "opens" if ok else why)
        else:
            log.info("SELFTEST skip: port 2 - set 'USB serial ports' to 2 or 3 for Radio menus")
        if n >= 3:
            got = bytearray()
            ok, why = radio.serial_open(2, lambda b: got.extend(b))
            reply = ""
            if ok:
                radio.serial_send(2, b"FA;")
                deadline = time.monotonic() + 1.0
                while time.monotonic() < deadline and b";" not in got:
                    time.sleep(0.02)
                radio.serial_close(2)
                reply = bytes(got).decode(errors="replace").strip()
                ok = bool(reply)
            ok_all &= ok
            log.info("SELFTEST %s: port 3 (PC serial) FA; -> %s", "PASS" if ok else "FAIL",
                     reply or why or "(no reply)")
        else:
            log.info("SELFTEST skip: port 3 - set 'USB serial ports' to 3 for PC software on TCP 7356")
        if u.gone:
            ok_all = False
            log.error("SELFTEST FAIL: the QMX disconnected during the test")
    except RadioGone as e:
        ok_all = False
        log.error("SELFTEST FAIL: %s", e)
    finally:
        if mgr.session is u:
            mgr.session = None
        mgr.release(u, radio)
    log.info("SELFTEST %s", "PASSED" if ok_all else "FAILED - see the lines above")
    return ok_all


def probe_usbfs():
    print("(usbfs backend - libusb-1.0 not available)")
    found = False
    for bus, devn, vid, pid, name in usbfs_list_devices():
        mark = "  <== QMX" if (vid, pid) == (QMX_VID, QMX_PID) else ""
        print("%03d/%03d %04x:%04x %s%s" % (bus, devn, vid, pid, name, mark))
        if mark:
            found = True
            with open("/dev/bus/usb/%03d/%03d" % (bus, devn), "rb") as f:
                cfg = usbfs_parse_config(f.read())
            for iface in cfg:
                for st in iface:
                    eps = ", ".join("0x%02x/%s/%d" % (e.getAddress(), "CIBI"[e.getAttributes() & 3],
                                                      e.getMaxPacketSize()) for e in st)
                    print("    if%d alt%d class %02x/%02x/%02x  eps: %s" % (
                        st.getNumber(), st.getAlternateSetting(), st.getClass(), st.getSubClass(),
                        st.getProtocol(), eps or "-"))
            ports, _uac2, aud = discover_qmx(cfg)
            print("    USB serial ports: %d  (Radio menus need 2+, PC serial on port 3 needs 3)" % len(ports))
            print("    I/Q input: %s" % (aud,))
    if not found:
        print("QMX not found. Is it plugged into the NAS and powered on?")
    return 0 if found else 1


def probe(args):
    if os.environ.get("QMX_BACKEND", "").lower() == "usbfs" or not libusb_available():
        return probe_usbfs()
    import usb1
    _load_libusb(usb1)
    with usb1.USBContext() as ctx:
        found = False
        for dev in ctx.getDeviceIterator(skip_on_error=True):
            vid, pid = dev.getVendorID(), dev.getProductID()
            mark = "  <== QMX" if (vid, pid) == (QMX_VID, QMX_PID) else ""
            print("%03d/%03d %04x:%04x%s" % (dev.getBusNumber(), dev.getDeviceAddress(), vid, pid, mark))
            if mark:
                found = True
                for iface in dev[0]:
                    for s in iface:
                        eps = ", ".join("0x%02x/%s/%d" % (e.getAddress(), "CIBI"[e.getAttributes() & 3],
                                                          e.getMaxPacketSize()) for e in s)
                        print("    if%d alt%d class %02x/%02x/%02x  eps: %s" % (
                            s.getNumber(), s.getAlternateSetting(), s.getClass(), s.getSubClass(),
```

```
                            s.getProtocol(), eps or "-"))
                ncdc = sum(1 for iface in dev[0] for s in iface
                           if s.getClass() == 0x0A and s.getAlternateSetting() == 0)
                print("    USB serial ports: %d  (Radio menus need 2+, PC serial on port 3 needs 3)" % ncdc)
        if not found:
            print("QMX not found. Is it plugged into the NAS and powered on?")
        return 0 if found else 1


def main():
    ap = argparse.ArgumentParser(description=__doc__.split("\n\n")[0],
                                 formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--listen", default="0.0.0.0", help="bind address (default all)")
    ap.add_argument("--port", type=int, default=DEFAULT_PORT, help="Tab5 TCP port (default 7355)")
    ap.add_argument("--serial-port", type=int, default=DEFAULT_SERIAL_PORT,
                    help="TCP port for the QMX's 3rd serial port, for PC software "
                         "(default 7356, 0 = off; needs 'USB serial ports' = 3 on the radio)")
    ap.add_argument("--fake", action="store_true", help="synthetic radio for bench tests (no QMX needed)")
    ap.add_argument("--fake-one-port", action="store_true", help=argparse.SUPPRESS)
    ap.add_argument("--fake-ports", type=int, default=3, choices=(1, 2, 3), help=argparse.SUPPRESS)
    ap.add_argument("--probe", action="store_true", help="list USB devices / QMX interfaces and exit")
    ap.add_argument("--selftest", action="store_true",
                    help="at startup, test the QMX (I/Q, read-only CAT, ports 2/3), log the result, then serve")
    ap.add_argument("--selftest-only", action="store_true", help="run the self-test and exit (0 = passed)")
    ap.add_argument("--discovery-port", type=int, default=None,
                    help="UDP port answering Tab5 'QMXR?' searches (default: same as --port; 0 = off)")
    ap.add_argument("--catlog-dir", default=os.path.join(RELAY_DIR, "catlog"),
                    help="where to log every CAT command and reply (default: catlog/ next to this "
                         "script; '' = off; a file named catlog.off in the relay folder pauses it)")
    ap.add_argument("--txlog-dir", default=os.path.join(os.path.dirname(os.path.abspath(__file__)), "txlog"),
                    help="where to log each transmit burst as the relay sent it to the QMX "
                         "(default: txlog/ next to this script; '' = off)")
    ap.add_argument("-v", "--verbose", action="store_true")
    args = ap.parse_args()
    logging.basicConfig(level=logging.DEBUG if args.verbose else logging.INFO,
                        format="%(asctime)s %(levelname)s %(message)s")
    if args.probe:
        return probe(args)

    mgr = RadioManager(args)
    if args.catlog_dir:
        mgr.catlog = CatLog(args.catlog_dir, mgr.flags)
        log.info("CAT commands are logged to %s/catlog-<date>.log (pause: file catlog.off in %s)",
                 args.catlog_dir, RELAY_DIR)
    log.info("PS0; (power off) is %s (flag file allow-ps0 in %s)",
             "PASSED" if mgr.flags.has("allow-ps0") else "BLOCKED", RELAY_DIR)
    log.info("I/Q to the Tab5 during transmit: %s (flag file iq-during-tx in %s)",
             "STREAMED" if mgr.flags.has("iq-during-tx") else "PAUSED", RELAY_DIR)
    if args.txlog_dir:
        mgr.txlog = TxLog(args.txlog_dir)
        mgr.txlog.flags = mgr.flags
        log.info("transmit bursts are logged to %s/txlog-<date>.log", args.txlog_dir)
    if args.selftest or args.selftest_only:
        passed = selftest(mgr)
        if args.selftest_only:
            return 0 if passed else 1
    if args.fake:
        log.info("FAKE radio with %d USB serial port(s)", 1 if args.fake_one_port else args.fake_ports)
    if args.serial_port:
        threading.Thread(target=serve, daemon=True, args=(
            args.listen, args.serial_port, lambda s, p: SerialClient(s, p, mgr),
            "QMX port 3, raw serial for PC software")).start()
    disc = args.port if args.discovery_port is None else args.discovery_port
    if disc:
        threading.Thread(target=discovery_responder, daemon=True, name="discovery",
                         args=(args.listen, disc, args.port)).start()
    serve(args.listen, args.port, lambda s, p: Session(s, p, args, mgr), "QMXR/1 for the Tab5")


if __name__ == "__main__":
    try:
        sys.exit(main())
    except KeyboardInterrupt:
        pass
```

tools/nas-relay/start-relay.sh

new file, 26 lines

```
#!/bin/sh
# Start the QMX relay in the background and keep it running (restarts if it
# exits). DSM: Task Scheduler > Create > Triggered Task > User-defined script,
# user root, event "Boot-up". Log: qmx-relay.log in this folder.
. "$(cd "$(dirname "$0")" && pwd)/env.sh"
LOG="$DIR/qmx-relay.log"
PIDF="$DIR/relay.pid"
[ -n "$PY" ] || { echo "$(date) no python3 found" >>"$LOG"; exit 1; }
# python-libusb1 is optional: without libusb-1.0 the relay uses Linux usbfs.
"$PY" "$DIR/get_libusb1.py" >>"$LOG" 2>&1 || echo "$(date) python-libusb1 not installed - using usbfs" >>"$LOG"
if [ -f "$PIDF" ] && kill -0 "$(cat "$PIDF")" 2>/dev/null; then
  echo "$(date) already running (pid $(cat "$PIDF"))" >>"$LOG"; exit 0
fi
(
  trap 'kill $CHILD 2>/dev/null; exit 0' TERM INT
  while :; do
    if [ -f "$LOG" ] && [ "$(wc -c <"$LOG")" -gt 2000000 ]; then mv -f "$LOG" "$LOG.1"; fi
    "$PY" -u "$DIR/qmx_nas_relay.py" --selftest >>"$LOG" 2>&1 &
    CHILD=$!
    wait $CHILD
    echo "$(date) relay exited (code $?), restarting in 5 s" >>"$LOG"
    sleep 5
  done
) </dev/null >/dev/null 2>&1 &
echo $! >"$PIDF"
echo "$(date) started (supervisor pid $!)" >>"$LOG"
```

tools/nas-relay/stop-relay.sh

new file, 7 lines

```
#!/bin/sh
# Stop the QMX relay started by start-relay.sh.
. "$(cd "$(dirname "$0")" && pwd)/env.sh"
PIDF="$DIR/relay.pid"
[ -f "$PIDF" ] && kill "$(cat "$PIDF")" 2>/dev/null
rm -f "$PIDF"
echo "$(date) stopped" >>"$DIR/qmx-relay.log"
```

tools/nas-relay/check.sh

new file, 43 lines

```
#!/bin/sh
# One-off check: environment + QMX probe + read-only self-test (or, when the
# relay is running, a network test against it from the NAS itself).
# DSM: Control Panel > Task Scheduler > Create > Scheduled Task > User-defined
# script, user root, then "Run". Result: check.log in this folder.
. "$(cd "$(dirname "$0")" && pwd)/env.sh"
OUT="$DIR/check.log"
# Restart on request: put a file named "restart.request" in this folder and run
# this task. The relay process is stopped; start-relay.sh's supervisor starts the
# (updated) relay again 5 s later. Avoids re-creating root tasks after an update.
if [ -f "$DIR/restart.request" ]; then
  rm -f "$DIR/restart.request"
  # /proc rather than ps: DSM's "ps w" only lists processes on the current
  # terminal, and a Task Scheduler job has none.
  for d in /proc/[0-9]*; do
    if tr '\0' ' ' < "$d/cmdline" 2>/dev/null | grep -q 'qmx_nas_relay\.py'; then kill "${d#/proc/}" 2>/dev/null; fi
  done
  echo "$(date) restart requested via check.sh" >> "$DIR/qmx-relay.log"
  sleep 12
fi
{
  echo "=== QMX relay check $(date)"
  echo "--- system";  uname -a; cat /etc.defaults/VERSION 2>/dev/null | grep -E 'productversion|buildnumber'
  echo "--- python";  echo "PY=$PY"; [ -n "$PY" ] && "$PY" -V 2>&1
  echo "--- libusb";  ls -l /lib/libusb* /usr/lib/libusb* /lib64/libusb* /usr/lib64/libusb* /usr/local/lib/libusb* 2>/dev/null; echo
  ↪  "QMX_LIBUSB=$QMX_LIBUSB"
  echo "--- usb devices"; for d in /sys/bus/usb/devices/*; do [ -f "$d/idVendor" ] && echo "$(basename $d) $(cat $d/idVendor):$(cat
  ↪ $d/idProduct) $(cat $d/product 2>/dev/null)"; done
  echo "--- kernel drivers bound to the QMX"; for i in /sys/bus/usb/devices/*:*; do [ -L "$i/driver" ] && case "$(cat $i/../idVendor
  ↪  2>/dev/null)" in 0483) echo "$(basename $i) -> $(basename $(readlink $i/driver))";; esac; done
  echo "--- relay already running?"; for d in /proc/[0-9]*; do tr '\0' ' ' < "$d/cmdline" 2>/dev/null | grep -q 'qmx_nas_relay\.py'
  ↪ && echo "yes, pid ${d#/proc/}"; done
  if [ -n "$PY" ]; then
    echo "--- libusb1 (python)"; "$PY" "$DIR/get_libusb1.py" 2>&1
    echo "--- probe";    "$PY" -u "$DIR/qmx_nas_relay.py" --probe 2>&1
    if [ -f "$DIR/relay.pid" ] && kill -0 "$(cat "$DIR/relay.pid")" 2>/dev/null; then
      echo "--- relay is running: network test (as Tab5 on 7355 and PC on 7356)"
      "$PY" -u "$DIR/nettest.py" 127.0.0.1 2>&1; echo "nettest exit code: $?"
      echo "--- relay log (last 25 lines)"; tail -25 "$DIR/qmx-relay.log"
    else
      echo "--- selftest"; "$PY" -u "$DIR/qmx_nas_relay.py" --selftest-only 2>&1; echo "selftest exit code: $?"
    fi
  else
    echo "NO PYTHON 3 FOUND"
  fi
  echo "=== done $(date)"
} > "$OUT" 2>&1
```

tools/nas-relay/env.sh

new file, 11 lines

```
# Shared settings for the QMX relay scripts (sourced, not run).
DIR="$(cd "$(dirname "$0")" && pwd)"
export PYTHONPATH="$DIR/lib"
PY=""
for c in python3 /usr/bin/python3 /bin/python3 /usr/local/bin/python3 \
         /var/packages/Python3.9/target/usr/bin/python3.9 /var/packages/Python3/target/usr/bin/python3; do
  if command -v "$c" >/dev/null 2>&1; then PY="$(command -v "$c")"; break; fi
done
# If libusb is not found automatically, set its full path here (or put
# libusb-1.0.so.0 into this folder):
if [ -z "$QMX_LIBUSB" ] && [ -f "$DIR/libusb-1.0.so.0" ]; then export QMX_LIBUSB="$DIR/libusb-1.0.so.0"; fi
```

tools/nas-relay/nettest.py

new file, 133 lines

```
#!/usr/bin/env python3
"""End-to-end network test of a running relay, from the NAS itself.
Acts as the Tab5 on TCP 7355 (QMXR/1) and as a PC on TCP 7356 (port 3).
Read-only: CAT queries only, the terminal port is opened and closed without
sending anything, so the radio never transmits or enters terminal mode."""
import socket, struct, sys, time

HOST = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
HDR = struct.Struct("<BBH")
ok_all = True


def say(ok, msg):
    global ok_all
    ok_all &= ok
    print("NETTEST %s: %s" % ("PASS" if ok else "FAIL", msg), flush=True)


def frame(t, p=b""):
    return HDR.pack(t, 0, len(p)) + p


def rx(s, n):
    b = b""
    while len(b) < n:
        c = s.recv(n - len(b))
        if not c:
            raise ConnectionError("closed by relay")
        b += c
    return b


def read_frames(s, secs, want=None):
    """Collect frames for `secs` seconds (or until `want(type, payload)` is true)."""
    out, end = [], time.monotonic() + secs
    while time.monotonic() < end:
        s.settimeout(max(0.05, end - time.monotonic()))
        try:
            t, _f, n = HDR.unpack(rx(s, 4))
            p = rx(s, n) if n else b""
        except socket.timeout:
            break
        out.append((t, p))
        if want and want(t, p):
            break
    return out


def discovery(dest, label, required):
    """Ask like a Tab5 does: 'QMXR?' on UDP 7355, expect 'QMXR! <ip> <port>'."""
    u = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    u.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
    u.settimeout(1.0)
    got = None
    try:
        for _ in range(3):
            try:
                u.sendto(b"QMXR?", (dest, 7355))
                data, peer = u.recvfrom(256)
            except socket.timeout:
                continue
            except OSError as e:            # e.g. no route for a broadcast
                got = None
                print("NETTEST INFO: discovery %s: %s" % (label, e), flush=True)
                break
            parts = data.decode(errors="replace").split()
            if len(parts) == 3 and parts[0] == "QMXR!" and parts[2].isdigit():
                got = "%s port %s (from %s)" % (parts[1], parts[2], peer[0])
                break
    finally:
        u.close()
    if required or got:
        say(got is not None, "discovery %s: %s" % (label, got or "no answer"))
    else:
        print("NETTEST INFO: discovery %s: no answer (a host may not hear its own "
              "broadcast; the Tab5 is a different host)" % label, flush=True)


try:
    discovery(HOST, "to %s" % HOST, True)
    discovery("255.255.255.255", "by broadcast", False)
    tab5 = socket.create_connection((HOST, 7355), 5)
    tab5.sendall(frame(1, b"QMXR/1 nettest"))
    tab5.settimeout(8)
```

```
    t, _f, n = HDR.unpack(rx(tab5, 4))
    hello = rx(tab5, n)
    say(t == 1 and hello.startswith(b"QMXR/1"), "Tab5 handshake on 7355: %s" % hello.decode(errors="replace"))

    fr = read_frames(tab5, 3.0)
    iq = sum(len(p) // 4 for t, p in fr if t == 2)
    pings = sum(1 for t, _ in fr if t == 4)
    say(40000 * 3 <= iq <= 56000 * 3, "I/Q over the network: %.0f pairs/s, %d pings" % (iq / 3.0, pings))

    for q in (b"ID;", b"FA;", b"VN;"):
        tab5.sendall(frame(3, q))
        cat = b""
        for t, p in read_frames(tab5, 2.0, lambda t, p: t == 3 and p.endswith(b";")):
            if t == 3:
                cat += p
        say(cat.endswith(b";"), "Tab5 CAT %s -> %s" % (q.decode(), cat.decode(errors="replace") or "(nothing)"))

    tab5.sendall(frame(5))                      # TERM_OPEN (Radio menus port) - nothing is sent on it
    status = [p for t, p in read_frames(tab5, 3.0, lambda t, p: t == 6) if t == 6]
    if status:
        okt = status[0][:1] == b"\x01"
        say(okt, "Radio menus port over the network: %s" % ("opens" if okt else status[0][1:].decode(errors="replace")))
    else:
        say(False, "Radio menus port: no TERM_STATUS reply")
    tab5.sendall(frame(8))                      # TERM_CLOSE
    time.sleep(0.3)

    pc = socket.create_connection((HOST, 7356), 5)
    pc.settimeout(3)
    pc.sendall(b"FA;")
    reply = b""
    end = time.monotonic() + 2
    while time.monotonic() < end and not reply.endswith(b";"):
        try:
            reply += pc.recv(256)
        except socket.timeout:
            break
    say(reply.endswith(b";"), "PC on port 3 (7356) while the Tab5 is connected: FA; -> %s" % reply.decode(errors="replace"))
    pc.close()

    tab5.sendall(frame(3, b"FA;"))
    cat = b"".join(p for t, p in read_frames(tab5, 2.0, lambda t, p: t == 3 and p.endswith(b";")) if t == 3)
    say(cat.endswith(b";"), "Tab5 still answered after the PC left: %s" % cat.decode(errors="replace"))
    tab5.close()
except Exception as e:
    say(False, "error: %r" % (e,))

print("NETTEST %s" % ("PASSED" if ok_all else "FAILED"), flush=True)
sys.exit(0 if ok_all else 1)
```

tools/nas-relay/get_libusb1.py

new file, 36 lines

```
#!/usr/bin/env python3
"""Fetch python-libusb1 3.4.0 (pure Python) from PyPI into ./lib if it isn't
there yet. The download is checked against its published SHA-256, so the
result is the exact file PyPI lists, even if TLS verification has to fall
back on a NAS without a CA bundle."""
import hashlib, io, os, ssl, sys, urllib.request, zipfile

URL = ("https://files.pythonhosted.org/packages/d4/64/d4b59444e4d3b6979aa5eb58840634465a24b41a9ab03dcf8434c9b89551/"
       "libusb1-3.4.0-py3-none-any.whl")
SHA256 = "e83d034e44c3efe1c4599c6281d34bca50a38c12cab3b7b6217d583161a01ffd"
LIB = os.path.join(os.path.dirname(os.path.abspath(__file__)), "lib")

if os.path.isfile(os.path.join(LIB, "usb1", "__init__.py")):
    print("libusb1: already present in", LIB)
    sys.exit(0)
try:
    data = urllib.request.urlopen(URL, timeout=60).read()
except ssl.SSLError as e:
    print("libusb1: TLS verification failed (%s); retrying unverified - the SHA-256 check still applies" % e)
    data = urllib.request.urlopen(URL, timeout=60, context=ssl._create_unverified_context()).read()
except Exception as e:
    if "CERTIFICATE_VERIFY_FAILED" not in str(e):
        raise
    print("libusb1: TLS verification failed; retrying unverified - the SHA-256 check still applies")
    data = urllib.request.urlopen(URL, timeout=60, context=ssl._create_unverified_context()).read()
digest = hashlib.sha256(data).hexdigest()
if digest != SHA256:
    sys.exit("libusb1: SHA-256 mismatch (%s) - not installed" % digest)
with zipfile.ZipFile(io.BytesIO(data)) as z:
    for n in z.namelist():
        if n.startswith("usb1/") or n == "libusb1.py" or "/licenses/" in n:
            target = os.path.join(LIB, "usb1", os.path.basename(n)) if "/licenses/" in n else os.path.join(LIB, n)
            os.makedirs(os.path.dirname(target), exist_ok=True)
            with open(target, "wb") as f:
                f.write(z.read(n))
print("libusb1: installed 3.4.0 into", LIB)
```

tools/nas-relay/Dockerfile

new file, 8 lines

```
# QMX network relay for a Synology NAS (Container Manager / Docker).
# Talks to the QMX with libusb in userspace - no DSM kernel drivers needed.
FROM python:3.12-alpine
RUN apk add --no-cache libusb \
 && pip install --no-cache-dir libusb1
COPY qmx_nas_relay.py /app/qmx_nas_relay.py
EXPOSE 7355
ENTRYPOINT ["python", "-u", "/app/qmx_nas_relay.py"]
```

tools/nas-relay/docker-compose.yml

new file, 15 lines

```
# Synology: Container Manager > Project > Create, path = the folder holding
# this file (e.g. /volume1/docker/qmx-relay). Or over SSH: sudo docker compose up -d --build
services:
  qmx-relay:
    build: .
    image: qmx-nas-relay:latest
    container_name: qmx-relay
    restart: unless-stopped
    network_mode: host          # Tab5 -> <NAS IP>:7355, PC software -> <NAS IP>:7356 (QMX port 3)
    privileged: true            # raw USB access, incl. the QMX being power-cycled/replugged
    volumes:
      - /dev/bus/usb:/dev/bus/usb
    command: ["--selftest"]     # test the QMX at startup (read-only) and log the result, then serve
    # all options: ["--selftest", "--port", "7355", "--serial-port", "7356", "-v"]
    #   --serial-port 0 = off; -v logs every CAT exchange
```

Raspberry Pi relay: installation, operation and maintenance scripts

tools/pi-relay/install-pi.sh

new file, 134 lines

```
#!/bin/bash
# install-pi.sh - install the QMX relay as a service on a Raspberry Pi 4 or 5
# (Raspberry Pi OS Bookworm or any Debian/Ubuntu with systemd, 64- or 32-bit).
#
#   sudo bash install-pi.sh             install and start the relay now
#   sudo bash install-pi.sh --no-start  install only; it starts 60 s after the next boot
#
# What it sets up:
#   /opt/qmx-relay/                   the relay (tools/nas-relay/qmx_nas_relay.py, unchanged), nettest.py,
#                                     qmx-relay-event.sh, qmx-relay-ctl, README.md
#   /usr/local/bin/qmx-relay-ctl      status / log / test / restart / stop / start
#   /etc/default/qmx-relay            relay arguments (kept if it already exists)
#   qmx-relay.timer                   starts the relay 60 s after every boot
#   qmx-relay.service                 runs it; restarts it up to 3 times if it fails
#   qmx-relay-boot.service            logs each boot
#   qmx-relay-gaveup.service          logs when the 3 restarts are used up
#   /etc/udev/rules.d/70-qmx-relay.rules  keeps ModemManager / PipeWire off the QMX, no USB autosuspend
#   /etc/logrotate.d/qmx-relay        weekly log rotation
#   /var/log/qmx-relay/qmx-relay.log  the log: service events and the relay's own output
set -euo pipefail

START=1
for a in "$@"; do
    case "$a" in
        --no-start) START=0 ;;
        -h|--help) sed -n '2,24p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
        *) echo "unknown option: $a"; exit 1 ;;
    esac
done

if [ "$(id -u)" != 0 ]; then
    exec sudo bash "$0" "$@"
fi

HERE="$(cd "$(dirname "$0")" && pwd)"
RELAY_SRC="$HERE/../nas-relay"
OPT=/opt/qmx-relay
LOGDIR=/var/log/qmx-relay
UNITDIR=/etc/systemd/system

say() { echo "==> $*"; }
die() { echo "ERROR: $*" >&2; exit 1; }

# ---- checks ------------------------------------------------------------
[ "$(uname -s)" = Linux ] || die "this installer is for Linux (Raspberry Pi OS)."
command -v systemctl >/dev/null || die "systemd is needed (systemctl not found)."
[ -d /run/systemd/system ] || die "systemd is not running as the init system."
command -v python3 >/dev/null || die "python3 is needed: sudo apt install python3"
python3 -c 'import sys; sys.exit(0 if sys.version_info >= (3, 8) else 1)' \
    || die "python3 3.8 or newer is needed (found $(python3 -V 2>&1))."
[ -f "$RELAY_SRC/qmx_nas_relay.py" ] || die "relay not found at $RELAY_SRC/qmx_nas_relay.py (run this from tools/pi-relay in the pro
  ↪ ject)."
[ -f "$HERE/systemd/qmx-relay.service" ] || die "systemd unit files missing next to this script."
if ! grep -q "discovery_responder" "$RELAY_SRC/qmx_nas_relay.py"; then
    echo "WARNING: this copy of the relay predates Find NAS (it won't answer QMXR? searches)."
    echo "         Install from the current NAS build instead: ~/src/QMX-Panadapter-NAS/tools/pi-relay"
fi

model=""
[ -r /proc/device-tree/model ] && model="$(tr -d '\0' </proc/device-tree/model)"
say "Installing the QMX relay on ${model:-$(uname -n)} ($(uname -m), $(python3 -V 2>&1))"
case "$model" in
    *"Raspberry Pi 4"*|*"Raspberry Pi 5"*) ;;
    "") say "note: not a Raspberry Pi as far as I can tell - continuing anyway" ;;
    *)  say "note: made for a Raspberry Pi 4/5; this is '$model' - continuing anyway" ;;
esac
for p in 7355 7356; do
    if command -v ss >/dev/null && ss -ltnH "sport = :$p" 2>/dev/null | grep -q . \
       && ! systemctl is-active --quiet qmx-relay.service; then
        die "TCP port $p is already in use by another program (see: sudo ss -ltnp 'sport = :$p')."
    fi
done

# ---- files -------------------------------------------------------------
was_active=0
systemctl is-active --quiet qmx-relay.service && was_active=1
if [ $was_active = 1 ]; then
    say "Stopping the running relay for the update"
    "$HERE/qmx-relay-event.sh" note "installer: stopping the relay to update it." || true
    systemctl stop qmx-relay.service
fi

say "Copying the relay to $OPT"
install -d -m 755 "$OPT" "$LOGDIR"
```

```
install -m 644 "$RELAY_SRC/qmx_nas_relay.py" "$RELAY_SRC/nettest.py" "$OPT/"
install -m 755 "$HERE/qmx-relay-event.sh" "$HERE/qmx-relay-ctl" "$OPT/"
install -m 644 "$HERE/README.md" "$OPT/" 2>/dev/null || true
ln -sf "$OPT/qmx-relay-ctl" /usr/local/bin/qmx-relay-ctl

if [ -f /etc/default/qmx-relay ]; then
    say "Keeping the existing /etc/default/qmx-relay"
else
    install -m 644 "$HERE/qmx-relay.default" /etc/default/qmx-relay
fi

say "Installing the systemd units, udev rule and log rotation"
for u in qmx-relay.service qmx-relay.timer qmx-relay-boot.service qmx-relay-gaveup.service; do
    install -D -m 644 "$HERE/systemd/$u" "$UNITDIR/$u"
done
install -D -m 644 "$HERE/udev/70-qmx-relay.rules" /etc/udev/rules.d/70-qmx-relay.rules
install -D -m 644 "$HERE/logrotate/qmx-relay" /etc/logrotate.d/qmx-relay
touch "$LOGDIR/qmx-relay.log"

if command -v udevadm >/dev/null; then
    udevadm control --reload-rules || true
    udevadm trigger --subsystem-match=usb --attr-match=idVendor=0483 --attr-match=idProduct=a34c || true
fi

systemctl daemon-reload
systemctl enable qmx-relay.timer qmx-relay-boot.service >/dev/null
"$HERE/qmx-relay-event.sh" note "installed by install-pi.sh: relay starts 60 s after each boot, restarts up to 3 times."

# ---- start -----------------------------------------------------------------
if [ $START = 1 ]; then
    say "Starting the relay now (after a reboot it waits 60 s)"
    "$HERE/qmx-relay-event.sh" note "installer: starting the relay now."
    systemctl reset-failed qmx-relay.service 2>/dev/null || true
    systemctl start qmx-relay.service
    sleep 4
else
    say "Not started now; it starts 60 s after the next boot (or: sudo qmx-relay-ctl start)"
fi

echo
"$OPT/qmx-relay-ctl" status || true
echo
cat <<EOF
Installed.
  Status / log : qmx-relay-ctl status     qmx-relay-ctl follow
  Network test : qmx-relay-ctl test        (read-only; needs the QMX plugged in)
  Log file     : $LOGDIR/qmx-relay.log
  Settings     : /etc/default/qmx-relay   then: sudo qmx-relay-ctl restart
  Tab5         : Radio source -> Find NAS (or type this Pi's address: $(hostname -I 2>/dev/null | awk '{print $1}'))
  Remove       : sudo bash $HERE/uninstall-pi.sh
EOF
```

tools/pi-relay/uninstall-pi.sh

new file, 26 lines

```
#!/bin/bash
# uninstall-pi.sh - remove the QMX relay service from this Raspberry Pi.
#   sudo bash uninstall-pi.sh            keeps the log (/var/log/qmx-relay) and /etc/default/qmx-relay
#   sudo bash uninstall-pi.sh --purge    removes those too
set -uo pipefail
PURGE=0
[ "${1:-}" = "--purge" ] && PURGE=1
if [ "$(id -u)" != 0 ]; then exec sudo bash "$0" "$@"; fi

[ -x /opt/qmx-relay/qmx-relay-event.sh ] && /opt/qmx-relay/qmx-relay-event.sh note "uninstall-pi.sh: removing the relay service."
systemctl disable --now qmx-relay.timer qmx-relay-boot.service 2>/dev/null
systemctl stop qmx-relay.service 2>/dev/null
for u in qmx-relay.service qmx-relay.timer qmx-relay-boot.service qmx-relay-gaveup.service; do
    rm -f "/etc/systemd/system/$u"
done
systemctl daemon-reload
systemctl reset-failed qmx-relay.service qmx-relay-gaveup.service 2>/dev/null
rm -f /etc/udev/rules.d/70-qmx-relay.rules /etc/logrotate.d/qmx-relay /usr/local/bin/qmx-relay-ctl
command -v udevadm >/dev/null && udevadm control --reload-rules
rm -rf /opt/qmx-relay
if [ $PURGE = 1 ]; then
    rm -rf /var/log/qmx-relay /etc/default/qmx-relay
    echo "Removed, including the log and settings."
else
    echo "Removed. Kept: /var/log/qmx-relay (log) and /etc/default/qmx-relay (settings); --purge removes them."
fi
```

tools/pi-relay/qmx-relay-ctl

new file, 91 lines

```
#!/bin/sh
# qmx-relay-ctl - look after the QMX relay service on this Raspberry Pi.
#
#   qmx-relay-ctl status          service, timer, restarts used, ports, QMX on USB, last log lines
#   qmx-relay-ctl log [N]         last N log lines (default 50)
#   qmx-relay-ctl follow          follow the log as it is written (Ctrl-C to stop)
#   qmx-relay-ctl test            network test against the running relay (read-only, never transmits)
#   qmx-relay-ctl probe           list the USB devices and the QMX's interfaces
#   sudo qmx-relay-ctl restart    start again now, with a fresh set of 3 restarts
#   sudo qmx-relay-ctl stop       stop it (it starts again 60 s after the next boot)
#   sudo qmx-relay-ctl start      start it now if stopped

OPT=/opt/qmx-relay
LOG=/var/log/qmx-relay/qmx-relay.log
UNIT=qmx-relay.service
EVENT="$OPT/qmx-relay-event.sh"

need_root() {
    [ "$(id -u)" = 0 ] || { echo "Run this with sudo: sudo qmx-relay-ctl $1"; exit 1; }
}

who_did() { echo "${SUDO_USER:-$(id -un)}"; }

qmx_on_usb() {
    for d in /sys/bus/usb/devices/*; do
        [ -f "$d/idVendor" ] || continue
        if [ "$(cat "$d/idVendor")" = "0483" ] && [ "$(cat "$d/idProduct")" = "a34c" ]; then
            echo "yes ($(cat "$d/product" 2>/dev/null || echo QMX), bus $(cat "$d/busnum") device $(cat "$d/devnum"))"
            return
        fi
    done
    echo "NO - is the QMX switched on and plugged in with a data cable?"
}

case "${1:-status}" in
status)
    echo "QMX relay on $(uname -n)"
    echo "  service : $(systemctl is-active $UNIT 2>/dev/null)  (restarts used: $(systemctl show -p NRestarts --value $UNIT 2>/dev/n
  ↪ ull) of 3)"
    res="$(systemctl show -p Result --value $UNIT 2>/dev/null)"
    [ -n "$res" ] && [ "$res" != "success" ] && echo "  result  : $res  -> 'sudo qmx-relay-ctl restart' starts it again"
    echo "  timer   : $(systemctl is-enabled qmx-relay.timer 2>/dev/null) (starts the relay 60 s after boot)"
    echo "  uptime  : $(cut -d. -f1 /proc/uptime) s"
    echo "  QMX USB : $(qmx_on_usb)"
    if command -v ss >/dev/null 2>&1; then
        echo "  ports   :"
        ss -ltnu 2>/dev/null | awk 'NR > 1 && $5 ~ /:735[56]$/ { print "            " $1 " " $5 }'
    fi
    echo "  log     : $LOG"
    echo
    tail -n 15 "$LOG" 2>/dev/null
    ;;
log)
    tail -n "${2:-50}" "$LOG"
    ;;
follow)
    tail -n 20 -F "$LOG"
    ;;
test)
    python3 "$OPT/nettest.py" 127.0.0.1
    ;;
probe)
    need_root probe
    QMX_BACKEND="${QMX_BACKEND:-usbfs}" python3 "$OPT/qmx_nas_relay.py" --probe
    ;;
restart)
    need_root restart
    "$EVENT" note "manual restart by $(who_did): restart count and limit reset."
    systemctl reset-failed $UNIT 2>/dev/null
    systemctl restart $UNIT
    sleep 2
    systemctl is-active $UNIT
    ;;
start)
    need_root start
    "$EVENT" note "manual start by $(who_did)."
    systemctl reset-failed $UNIT 2>/dev/null
    systemctl start $UNIT
    sleep 2
    systemctl is-active $UNIT
    ;;
stop)
    need_root stop
    "$EVENT" note "manual stop by $(who_did) (it starts again 60 s after the next boot)."
```

```
    systemctl stop $UNIT
    systemctl is-active $UNIT
    ;;
*)
    sed -n '2,13p' "$0" | sed 's/^# \{0,1\}//'
    exit 1
    ;;
esac
```

tools/pi-relay/qmx-relay-event.sh

new file, 90 lines

```
#!/bin/sh
# qmx-relay-event.sh - write what happens to the QMX relay service into its log,
# /var/log/qmx-relay/qmx-relay.log (the relay's own output goes to the same file).
#
# Called by systemd:
#   boot    qmx-relay-boot.service    the system booted; the relay starts 60 s later
#   start   ExecStartPre              the relay is starting (first start, restart n of 3, or by hand)
#   stop    ExecStopPost              the relay stopped: why, and whether it will be restarted
#   gaveup  OnFailure                 the 3 restarts are used up; it stays stopped
# and by qmx-relay-ctl:
#   note "text"                       a line of its own (manual start/stop/restart)
#
# It never fails, so it can never stop the relay from starting.

LOGDIR=/var/log/qmx-relay
LOG="$LOGDIR/qmx-relay.log"
UNIT=qmx-relay.service
MAX_RESTARTS=3          # keep in step with StartLimitBurst=4 in qmx-relay.service
RESTART_SEC=10          # keep in step with RestartSec= in qmx-relay.service
BOOT_DELAY=60           # keep in step with OnBootSec= in qmx-relay.timer

mkdir -p "$LOGDIR" 2>/dev/null

say() {
    printf '%s SERVICE %s\n' "$(date '+%Y-%m-%d %H:%M:%S')" "$*" >>"$LOG" 2>/dev/null
}

uptime_s() { cut -d. -f1 /proc/uptime 2>/dev/null || echo "?"; }

nrestarts() {
    n="$(systemctl show -p NRestarts --value "$UNIT" 2>/dev/null)"
    case "$n" in ''|*[!0-9]*) echo 0 ;; *) echo "$n" ;; esac
}

case "$1" in
boot)
    say "---- system booted (uptime $(uptime_s) s, $(uname -n), kernel $(uname -r))."
    say "relay will be started by qmx-relay.timer ${BOOT_DELAY} s after boot."
    ;;
start)
    n="$(nrestarts)"
    up="$(uptime_s)"
    if [ "$n" -gt 0 ] 2>/dev/null; then
        say "starting the relay: restart $n of $MAX_RESTARTS (uptime $up s)."
    elif [ "$up" != "?" ] && [ "$up" -lt $((BOOT_DELAY + 60)) ]; then
        say "starting the relay: ${BOOT_DELAY} s after boot, by qmx-relay.timer (uptime $up s)."
    else
        say "starting the relay (uptime $up s)."
    fi
    say "command: python3 qmx_nas_relay.py ${RELAY_ARGS:-}   backend: ${QMX_BACKEND:-auto}"
    ;;
stop)
    n="$(nrestarts)"
    res="${SERVICE_RESULT:-unknown}"; code="${EXIT_CODE:-?}"; st="${EXIT_STATUS:-?}"
    case "$res" in
    success)
        if [ "$code" = "exited" ]; then
            say "relay exited normally (status $st); not restarted."
        else
            say "relay stopped on request ($code $st)."
        fi
        ;;
    *)
        case "$code" in
            exited) why="exited with status $st" ;;
            killed) why="was killed by signal $st" ;;
            dumped) why="crashed (signal $st, core dumped)" ;;
            *)      why="stopped ($code $st)" ;;
        esac
        if [ "$n" -lt "$MAX_RESTARTS" ] 2>/dev/null; then
            say "relay FAILED: it $why (result: $res). Restart $((n + 1)) of $MAX_RESTARTS in ${RESTART_SEC} s."
        else
            say "relay FAILED: it $why (result: $res). All $MAX_RESTARTS restarts used - not restarting."
        fi
        ;;
```

```
    esac
    ;;
gaveup)
    say "GAVE UP: the relay failed and its $MAX_RESTARTS restarts are used up. It stays stopped"
    say "until the next reboot or 'sudo qmx-relay-ctl restart'. See the lines above for why."
    ;;
note)
    shift
    say "$*"
    ;;
*)
    say "qmx-relay-event.sh: unknown event '$1'"
    ;;
esac
exit 0
```

tools/pi-relay/qmx-relay.default

new file, 14 lines

```
# Settings for the QMX relay service (qmx-relay.service). After a change:
#   sudo qmx-relay-ctl restart
#
# Arguments for qmx_nas_relay.py:
#   --selftest            test the QMX at start-up (read-only) and log the result
#   --port 7355           Tab5 port (QMXR/1); the Find NAS search answers on UDP 7355 too
#   --serial-port 7356    QMX serial port 3 for PC software (0 = off)
#   --discovery-port N    UDP port for the QMXR? search (0 = off)
#   -v                    log every CAT exchange
RELAY_ARGS="--selftest"

# USB access: usbfs (built in, the backend tested on the hardware) or libusb
# (needs python3-usb1).
QMX_BACKEND=usbfs
```

tools/pi-relay/systemd/qmx-relay.service

new file, 31 lines

```
[Unit]
Description=QMX network relay (QMX on USB -> Tab5 on TCP 7355, PC software on TCP 7356)
Documentation=file:///opt/qmx-relay/README.md
Wants=network-online.target
After=network-online.target
# Restart at most 3 times: the first start plus 3 restarts = 4 starts. A 5th
# start is refused for the rest of the window (a year, so in practice until the
# next reboot or "sudo qmx-relay-ctl restart"), and qmx-relay-gaveup.service
# writes that to the log.
StartLimitIntervalSec=365d
StartLimitBurst=4
OnFailure=qmx-relay-gaveup.service

[Service]
Type=simple
EnvironmentFile=-/etc/default/qmx-relay
Environment=PYTHONUNBUFFERED=1
ExecStartPre=/opt/qmx-relay/qmx-relay-event.sh start
ExecStart=/usr/bin/python3 -u /opt/qmx-relay/qmx_nas_relay.py $RELAY_ARGS
ExecStopPost=/opt/qmx-relay/qmx-relay-event.sh stop
Restart=on-failure
RestartSec=10
StandardOutput=append:/var/log/qmx-relay/qmx-relay.log
StandardError=inherit
# The relay opens the QMX through /dev/bus/usb (usbfs) and detaches the
# kernel's cdc_acm / snd-usb-audio drivers from it, which needs root.
User=root
NoNewPrivileges=yes
ProtectSystem=full
ProtectHome=yes
PrivateTmp=yes
```

tools/pi-relay/systemd/qmx-relay.timer

new file, 13 lines

```
[Unit]
Description=Start the QMX relay 60 seconds after boot
Documentation=file:///opt/qmx-relay/README.md

[Timer]
# The relay is started by this timer only, 60 s after the system boots, so
# USB, the network and the QMX have settled. A manual start is immediate.
OnBootSec=60s
AccuracySec=1s
Unit=qmx-relay.service

[Install]
WantedBy=timers.target
```

tools/pi-relay/systemd/qmx-relay-boot.service

new file, 10 lines

```
[Unit]
Description=Log the boot for the QMX relay (it starts 60 s later)
After=local-fs.target

[Service]
Type=oneshot
ExecStart=/opt/qmx-relay/qmx-relay-event.sh boot

[Install]
WantedBy=multi-user.target
```

tools/pi-relay/systemd/qmx-relay-gaveup.service

new file, 6 lines

```
[Unit]
Description=Log that the QMX relay used up its 3 restarts

[Service]
Type=oneshot
ExecStart=/opt/qmx-relay/qmx-relay-event.sh gaveup
```

tools/pi-relay/udev/70-qmx-relay.rules

new file, 7 lines

```
# QRP Labs QMX / QMX+ (USB 0483:a34c) is served to the network by the QMX relay.
# Keep other programs off it: ModemManager would probe its serial ports with
# AT commands, and PipeWire/PulseAudio would open its audio. Also keep it out
# of USB autosuspend so the I/Q stream never pauses.
ACTION=="add|change", SUBSYSTEM=="usb", ATTR{idVendor}=="0483", ATTR{idProduct}=="a34c", ENV{ID_MM_DEVICE_IGNORE}="1", ATTR{power/co
  ↪ ntrol}="on"
ACTION=="add|change", SUBSYSTEM=="tty", ATTRS{idVendor}=="0483", ATTRS{idProduct}=="a34c", ENV{ID_MM_DEVICE_IGNORE}="1"
ACTION=="add|change", SUBSYSTEM=="sound", ATTRS{idVendor}=="0483", ATTRS{idProduct}=="a34c", ENV{PULSE_IGNORE}="1", ENV{ACP_IGNORE}=
  ↪ "1"
```

tools/pi-relay/logrotate/qmx-relay

new file, 10 lines

```
/var/log/qmx-relay/qmx-relay.log {
    weekly
    rotate 8
    maxsize 5M
    compress
    delaycompress
    missingok
    notifempty
    copytruncate
}
```

Mac tools

tools/mac-serial-bridge/qmx_serial_bridge.py

new file, 473 lines

```
#!/usr/bin/env python3
"""
qmx_serial_bridge.py - make the QMX on the Synology NAS show up on a Mac as a
serial port, so WSJT-X, Hamlib/rigctld, fldigi, loggers or a terminal can use
it as if the radio were plugged into the Mac's USB.

    QMX --USB--> NAS (qmx_nas_relay.py)  --TCP 7356 (QMX serial port 3)-->
        this program --> serial device on the Mac  (~/cu.QMX-NAS by default)

Finding the NAS: unless --host is given, it asks the network the same way the
Tab5's Find NAS does - broadcasts "QMXR?" on UDP 7355 and the relay answers
"QMXR! <its IP address> <TCP port>". The address is taken from that answer.
The port in the answer is the Tab5's QMXR/1 service (7355); this program uses
the relay's port 3 service on the same NAS, TCP 7356 (--port).
If the NAS stops answering at the remembered address, it searches again.

It creates a pseudo-terminal (the same kind of device a USB serial adapter
gives you) and links it at a fixed path. Point the program at that path the
same way you would for a USB-connected QMX (any baud rate works; the setting
is ignored).

  * Connects to the relay only when a program actually talks to the port, and
    lets go after --idle seconds of silence, so the radio's port 3 is only
    held while something is using it (--keep-connected holds it all the time).
  * If the QMX is off or the relay is down, what the program sends is dropped
    and it retries on the next command, like a radio that is switched off.
  * Bytes pass through unchanged in both directions - no framing.
  * If the NAS is not found it falls back to --fallback-host, if given.

Needs only the Python 3 that comes with macOS (Xcode command line tools).
Also runs on Linux.
```

```
    python3 qmx_serial_bridge.py                      # find the NAS, ~/cu.QMX-NAS
    python3 qmx_serial_bridge.py --discover           # list the relays that answer, then exit
    python3 qmx_serial_bridge.py --host 10.0.0.137    # fixed address, no search
    python3 qmx_serial_bridge.py --probe              # one-off check: ID; FA; MD;
"""

from __future__ import annotations

import argparse
import errno
import logging
import os
import select
import signal
import re
import socket
import subprocess
import sys
import termios
import time
import tty

log = logging.getLogger("qmx-serial-bridge")

DEFAULT_PORT = 7356          # relay: QMX serial port 3, raw bytes
DISCOVERY_PORT = 7355        # relay: answers "QMXR?" on UDP (same number as its QMXR/1 TCP port)
DISCOVERY_TIME = 1.5         # seconds; "QMXR?" is sent 3 times in this window, like the Tab5
DISCOVERY_QUERY = b"QMXR?"
DISCOVERY_REPLY = b"QMXR!"
DEFAULT_LINK = "~/cu.QMX-NAS"
DEFAULT_IDLE = 120          # seconds without traffic before the relay port is released
CONNECT_TIMEOUT = 3.0
RETRY_AFTER = 3.0           # after a failed connect, drop commands for this long
QUICK_CLOSE = 1.5           # relay closing this fast after connect = port 3 refused


def fmt(b: bytes, n: int = 60) -> str:
    s = b.decode("ascii", "replace")
    return s if len(s) <= n else s[:n] + "..."


def broadcast_addresses() -> list[str]:
    """255.255.255.255 plus each interface's own broadcast address. A Mac sends
    255.255.255.255 out of one interface only, so with Wi-Fi and Ethernet both
    up the directed addresses make sure the NAS's network is asked too."""
    out = ["255.255.255.255"]
    try:
        txt = subprocess.run(["ifconfig"], capture_output=True, text=True, timeout=3).stdout
        for b in re.findall(r"\bbroadcast (\d+\.\d+\.\d+\.\d+)", txt) + \
                 re.findall(r"\bBcast:(\d+\.\d+\.\d+\.\d+)", txt):
            if b not in out:
                out.append(b)
    except (OSError, subprocess.SubprocessError):
        pass
    return out


def discover(timeout: float = DISCOVERY_TIME, udp_port: int = DISCOVERY_PORT,
             targets: list[str] | None = None) -> list[tuple[str, int]]:
    """Ask the network for QMX relays: send "QMXR?" three times over `timeout`
    seconds and collect the distinct "QMXR! <ip> <tcp port>" answers, in the
    order they arrive. The answer's own address is used; if it doesn't parse,
    the address the answer came from. Returns [(ip, advertised tcp port), ...]."""
    targets = targets or broadcast_addresses()
    found: list[tuple[str, int]] = []
    u = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    try:
        u.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
        u.settimeout(0.1)
        start, sent = time.monotonic(), 0
        while time.monotonic() - start < timeout:
            if sent < 3 and time.monotonic() - start >= sent * timeout / 3:
                for t in targets:
                    try:
                        u.sendto(DISCOVERY_QUERY, (t, udp_port))
                    except OSError as e:
                        log.debug("discovery: send to %s failed: %s", t, e)
                sent += 1
            try:
                data, peer = u.recvfrom(256)
            except socket.timeout:
                continue
            except OSError as e:
                log.debug("discovery: %s", e)
                continue
            parts = data.decode("ascii", "replace").split()
            if not parts or parts[0] != DISCOVERY_REPLY.decode():
```

```
                continue
            ip = parts[1] if len(parts) > 1 else ""
            try:
                socket.inet_aton(ip)
                if ip.count(".") != 3:
                    raise OSError
            except OSError:
                ip = peer[0]
            port = int(parts[2]) if len(parts) > 2 and parts[2].isdigit() else DISCOVERY_PORT
            if not 0 < port < 65536:
                port = DISCOVERY_PORT
            if (ip, port) not in found:
                found.append((ip, port))
    finally:
        u.close()
    return found


class Bridge:
    def __init__(self, args):
        self.args = args
        self.link = os.path.expanduser(args.link)
        self.master = self.slave = -1
        self.sock: socket.socket | None = None
        self.connected_at = 0.0
        self.got_reply = False
        self.last_traffic = 0.0
        self.retry_at = 0.0
        self.dropped_note = 0.0
        self.running = True
        self.host = args.host            # None = find it with "QMXR?"
        self.searched_at = -1e9

    # ---- finding the NAS ----------------------------------------------------

    def find_nas(self) -> bool:
        """Ask the network where the relay is (not more than every RETRY_AFTER s)."""
        now = time.monotonic()
        if now - self.searched_at < RETRY_AFTER:
            return self.host is not None
        self.searched_at = now
        found = discover(udp_port=self.args.discovery_port)
        if found:
            ip, adv = found[0]
            if ip != self.host:
                log.info("found the NAS relay: QMXR! %s %d - using %s:%d for QMX port 3",
                         ip, adv, ip, self.args.port)
            if len(found) > 1:
                log.info("other relays answered too (using the first): %s",
                         ", ".join("%s:%d" % f for f in found[1:]))
            self.host = ip
            return True
        if self.args.fallback_host:
            if self.host != self.args.fallback_host:
                log.warning("no relay answered QMXR? on UDP %d; using --fallback-host %s",
                            self.args.discovery_port, self.args.fallback_host)
            self.host = self.args.fallback_host
            return True
        log.warning("no relay answered QMXR? on UDP %d (is the relay running, and the Mac "
                    "on the same network as the NAS?)", self.args.discovery_port)
        return self.host is not None
    # ---- the serial device --------------------------------------------------

    def open_pty(self):
        self.master, self.slave = os.openpty()
        # Raw 8-bit, no echo: an echo would send the radio's replies back to it.
        tty.setraw(self.slave)
        self.no_echo()
        os.set_blocking(self.master, False)
        name = os.ttyname(self.slave)
        # We keep our own handle on the device open (and never read it), so the
        # device stays put while programs open and close it.
        d = os.path.dirname(self.link)
        if d:
            os.makedirs(d, exist_ok=True)
        tmp = "%s.%d.tmp" % (self.link, os.getpid())
        os.symlink(name, tmp)
        os.replace(tmp, self.link)
        log.info("serial port ready: %s  (device %s)", self.link, name)

    def no_echo(self):
        """Programs may change the port settings; never allow echo, which would
        loop the radio's replies straight back into the radio."""
        try:
            a = termios.tcgetattr(self.slave)
            bad = termios.ECHO | termios.ECHONL
            if a[3] & bad:
```

```
            a[3] &= ~bad
            termios.tcsetattr(self.slave, termios.TCSANOW, a)
            log.info("turned echo off on the serial port")
        except termios.error:
            pass

    def flush_stale(self):
        """Throw away radio bytes no program has read (e.g. a reply that came
        after the program closed the port), so the next program doesn't see them."""
        try:
            termios.tcflush(self.slave, termios.TCIFLUSH)
        except termios.error:
            pass

    def to_port(self, data: bytes):
        self.no_echo()
        view = memoryview(data)
        while view:
            try:
                n = os.write(self.master, view)
                view = view[n:]
            except BlockingIOError:
                log.warning("serial port full (nothing reading it): dropped %d bytes", len(view))
                return
            except OSError as e:
                log.warning("serial port write failed: %s", e)
                return

    # ---- the relay ----------------------------------------------------------

    def connect(self) -> bool:
        a = self.args
        searching = a.host is None
        if self.host is None and not (searching and self.find_nas()):
            self.retry_at = time.monotonic() + RETRY_AFTER
            return False
        try:
            s = socket.create_connection((self.host, a.port), CONNECT_TIMEOUT)
        except OSError as e:
            log.warning("can't reach the relay at %s:%d: %s", self.host, a.port, e)
            if e.errno in (errno.EHOSTUNREACH, errno.ENETUNREACH, errno.EPERM):
                log.warning("if this Mac is on the NAS's network, check System Settings -> "
                            "Privacy & Security -> Local Network allows python3")
            old = self.host
            if searching and self.find_nas() and self.host != old:
                return self.connect()        # the NAS moved: try its new address once
            self.retry_at = time.monotonic() + RETRY_AFTER
            return False
        s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
        s.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
        s.setblocking(False)
        self.sock = s
        self.connected_at = self.last_traffic = time.monotonic()
        self.got_reply = False
        self.flush_stale()
        log.info("connected to the relay %s:%d (QMX serial port 3)", self.host, a.port)
        return True

    def disconnect(self, why: str):
        if self.sock:
            try:
                self.sock.close()
            except OSError:
                pass
            self.sock = None
            log.info("released the relay: %s", why)

    def to_relay(self, data: bytes):
        try:
            self.sock.setblocking(True)
            self.sock.settimeout(CONNECT_TIMEOUT)
            self.sock.sendall(data)
        except OSError as e:
            self.disconnect("send failed: %s" % e)
        finally:
            if self.sock:
                self.sock.setblocking(False)

    # ---- main loop ----------------------------------------------------------

    def on_port_data(self):
        try:
            data = os.read(self.master, 4096)
        except BlockingIOError:
            return
        except OSError as e:
            if e.errno == errno.EIO:      # no program has the port open
```

```
                time.sleep(0.05)
                return
            raise
        if not data:
            return
        log.debug("port -> radio: %s", fmt(data))
        now = time.monotonic()
        if not self.sock:
            if now < self.retry_at or not self.connect():
                if now - self.dropped_note > 30:
                    log.info("radio not reachable: dropping commands (retrying)")
                    self.dropped_note = now
                return
        self.last_traffic = now
        self.to_relay(data)

    def on_relay_data(self):
        try:
            data = self.sock.recv(4096)
        except BlockingIOError:
            return
        except OSError as e:
            if not self.quick_close():
                self.disconnect("connection lost: %s" % e)
            return
        if not data:
            if not self.quick_close():
                self.disconnect("relay closed the connection (QMX off or relay restarted)")
            return
        self.got_reply = True
        self.last_traffic = time.monotonic()
        log.debug("radio -> port: %s", fmt(data))
        self.to_port(data)

    def quick_close(self) -> bool:
        """The relay drops port 3 at once when the radio can't serve it."""
        if not self.got_reply and time.monotonic() - self.connected_at < QUICK_CLOSE:
            log.warning("the relay closed port 3 straight away. Is the QMX on, and is "
                        "'USB serial ports' set to 3 on the radio? (see the relay's log)")
            self.retry_at = time.monotonic() + RETRY_AFTER
            self.disconnect("port 3 refused")
            return True
        return False

    def run(self):
        self.open_pty()
        a = self.args
        if a.host is None:
            self.find_nas()
        log.info("relay: %s:%d   %s", self.host or "(not found yet)", a.port,
                 "kept connected" if a.keep_connected else
                 "connects on demand, releases after %ds idle" % a.idle if a.idle else
                 "connects on demand, stays connected")
        try:
            while self.running:
                now = time.monotonic()
                if a.keep_connected and not self.sock and now >= self.retry_at:
                    self.connect()
                if (self.sock and a.idle and not a.keep_connected
                        and now - self.last_traffic > a.idle):
                    self.disconnect("idle for %ds" % a.idle)
                    self.flush_stale()
                rl = [self.master] + ([self.sock] if self.sock else [])
                try:
                    r, _, _ = select.select(rl, [], [], 1.0)
                except InterruptedError:
                    continue
                if self.sock and self.sock in r:
                    self.on_relay_data()
                if self.master in r:
                    self.on_port_data()
        finally:
            self.cleanup()

    def cleanup(self):
        self.disconnect("stopping")
        try:
            if os.path.islink(self.link) and self.slave >= 0 and \
                    os.readlink(self.link) == os.ttyname(self.slave):
                os.unlink(self.link)
        except OSError:
            pass
        for fd in (self.slave, self.master):
            if fd >= 0:
                try:
                    os.close(fd)
                except OSError:
```

```
                    pass


def probe(args) -> int:
    """One-off check through the relay: read-only CAT queries, never transmits."""
    host = args.host
    if host is None:
        found = discover(udp_port=args.discovery_port)
        if found:
            print("ok  : QMXR? -> %s" % ", ".join("QMXR! %s %d" % f for f in found))
            host = found[0][0]
        elif args.fallback_host:
            print("note: no relay answered QMXR?; using --fallback-host %s" % args.fallback_host)
            host = args.fallback_host
        else:
            print("FAIL: no relay answered QMXR? on UDP %d" % args.discovery_port)
            return 1
    try:
        s = socket.create_connection((host, args.port), CONNECT_TIMEOUT)
    except OSError as e:
        print("FAIL: can't reach the relay at %s:%d: %s" % (host, args.port, e))
        return 1
    print("ok  : connected to %s:%d (QMX port 3)" % (host, args.port))
    s.settimeout(3)
    ok = True
    for q in (b"ID;", b"FA;", b"MD;"):
        s.sendall(q)
        reply, end = b"", time.monotonic() + 3
        while time.monotonic() < end and not reply.endswith(b";"):
            try:
                c = s.recv(256)
            except socket.timeout:
                break
            if not c:
                print("FAIL: the relay closed port 3 - is the QMX on, with "
                      "'USB serial ports' = 3?")
                return 1
            reply += c
        good = reply.endswith(b";")
        ok &= good
        print("%s: %s -> %s" % ("ok  " if good else "FAIL", q.decode(), fmt(reply) or "(no reply)"))
    s.close()
    print("PROBE %s" % ("PASSED" if ok else "FAILED"))
    return 0 if ok else 1


def main():
    p = argparse.ArgumentParser(description="QMX on the NAS as a serial port on this Mac")
    p.add_argument("--host", default=os.environ.get("QMX_NAS_HOST") or None,
                   help="fixed NAS address; without it the NAS is found by asking QMXR?")
    p.add_argument("--fallback-host", default=None,
                   help="address to use when no relay answers the search")
    p.add_argument("--discovery-port", type=int, default=DISCOVERY_PORT,
                   help="UDP port the relay answers QMXR? on (default %(default)s)")
    p.add_argument("--discover", action="store_true",
                   help="list the relays that answer QMXR? and exit")
    p.add_argument("--port", type=int, default=DEFAULT_PORT,
                   help="relay port for QMX serial port 3 (default %(default)s)")
    p.add_argument("--link", default=DEFAULT_LINK,
                   help="path of the serial port to create (default %(default)s)")
    p.add_argument("--idle", type=int, default=DEFAULT_IDLE,
                   help="release the relay after this many idle seconds, 0 = never (default %(default)s)")
    p.add_argument("--keep-connected", action="store_true",
                   help="stay connected to the relay all the time")
    p.add_argument("--probe", action="store_true",
                   help="check the relay once (ID; FA; MD;) and exit")
    p.add_argument("-v", "--verbose", action="store_true", help="log every byte")
    args = p.parse_args()
    logging.basicConfig(level=logging.DEBUG if args.verbose else logging.INFO,
                        format="%(asctime)s %(levelname)s %(message)s", stream=sys.stdout)
    if args.discover:
        found = discover(udp_port=args.discovery_port)
        for ip, port in found:
            print("QMXR! %s %d   -> serial port 3 at %s:%d" % (ip, port, ip, args.port))
        if not found:
            print("no relay answered QMXR? on UDP %d" % args.discovery_port)
        sys.exit(0 if found else 1)
    if args.probe:
        sys.exit(probe(args))
    b = Bridge(args)

    def stop(*_):
        b.running = False
    signal.signal(signal.SIGTERM, stop)
    signal.signal(signal.SIGINT, stop)
    signal.signal(signal.SIGHUP, stop)
    b.run()
```

```
    log.info("stopped")


if __name__ == "__main__":
    main()
```

tools/mac-serial-bridge/install-mac.sh

new file, 57 lines

```
#!/bin/bash
# Install the QMX serial bridge as a login item (LaunchAgent) on this Mac.
#   bash install-mac.sh                 # find the NAS by asking QMXR? (falls back to 10.0.0.137)
#   bash install-mac.sh 192.168.1.20    # fixed NAS address, no search
#   EXTRA="--keep-connected" bash install-mac.sh
set -euo pipefail
HOST="${1:-}"
FALLBACK="${FALLBACK:-10.0.0.137}"
LABEL="com.qmx.serial-bridge"
SRC="$(cd "$(dirname "$0")" && pwd)"
DEST="$HOME/Library/Application Support/QMX-Serial-Bridge"
PLIST="$HOME/Library/LaunchAgents/$LABEL.plist"
LOG="$HOME/Library/Logs/qmx-serial-bridge.log"
PY=/usr/bin/python3

if ! xcode-select -p >/dev/null 2>&1; then
  echo "macOS's Python 3 needs the Xcode command line tools: run 'xcode-select --install' first."
  exit 1
fi

mkdir -p "$DEST" "$HOME/Library/LaunchAgents" "$HOME/Library/Logs"
cp "$SRC/qmx_serial_bridge.py" "$DEST/"

ARGS="<string>$PY</string><string>$DEST/qmx_serial_bridge.py</string>"
if [ -n "$HOST" ]; then
  ARGS="$ARGS<string>--host</string><string>$HOST</string>"; WHERE="NAS $HOST (fixed)"; PROBE="--host $HOST"
else
  ARGS="$ARGS<string>--fallback-host</string><string>$FALLBACK</string>"
  WHERE="NAS found by QMXR? search (fallback $FALLBACK)"; PROBE="--fallback-host $FALLBACK"
fi
for a in ${EXTRA:-}; do ARGS="$ARGS<string>$a</string>"; done

cat > "$PLIST" <<EOF
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>Label</key><string>$LABEL</string>
  <key>ProgramArguments</key><array>$ARGS</array>
  <key>RunAtLoad</key><true/>
  <key>KeepAlive</key><true/>
  <key>ThrottleInterval</key><integer>5</integer>
  <key>StandardOutPath</key><string>$LOG</string>
  <key>StandardErrorPath</key><string>$LOG</string>
</dict>
</plist>
EOF

launchctl bootout "gui/$(id -u)/$LABEL" 2>/dev/null || true
launchctl bootstrap "gui/$(id -u)" "$PLIST"
sleep 1
echo "Installed. Serial port: $HOME/cu.QMX-NAS   ($WHERE, relay port 7356)"
echo "Log: $LOG"
tail -n 3 "$LOG" 2>/dev/null || true
echo
echo "Checking the relay:"
"$PY" "$DEST/qmx_serial_bridge.py" $PROBE --probe || true
```

tools/mac-serial-bridge/uninstall-mac.sh

new file, 8 lines

```
#!/bin/bash
# Remove the QMX serial bridge login item.
LABEL="com.qmx.serial-bridge"
launchctl bootout "gui/$(id -u)/$LABEL" 2>/dev/null || true
rm -f "$HOME/Library/LaunchAgents/$LABEL.plist"
rm -rf "$HOME/Library/Application Support/QMX-Serial-Bridge"
rm -f "$HOME/cu.QMX-NAS"
echo "Removed. (The log stays at ~/Library/Logs/qmx-serial-bridge.log)"
```

tools/mac-serial-bridge/test/fake_port3_relay.py

new file, 23 lines

```
#!/usr/bin/env python3
# Bench test: stand-in for qmx_nas_relay.py port 3 (TCP 17356) with a tiny CAT emulator.
# Usage: python3 fake_port3_relay.py [ok|refuse]   (refuse = act like a radio without port 3)
import socket, sys, threading
mode = sys.argv[1] if len(sys.argv) > 1 else "ok"
srv = socket.socket(); srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind(("127.0.0.1", 17356)); srv.listen(2)
freq = [14074000]
def handle(c):
    if mode == "refuse": c.close(); print("refused", flush=True); return
    print("client", flush=True); buf = b""
    while True:
        d = c.recv(4096)
        if not d: print("client gone", flush=True); return
        print("got", d, flush=True); buf += d
        while b";" in buf:
            cmd, buf = buf.split(b";", 1)
            if cmd == b"FA": c.sendall(b"FA%011d;" % freq[0])
            elif cmd.startswith(b"FA"): freq[0] = int(cmd[2:])
            elif cmd == b"ID": c.sendall(b"ID020;")
            elif cmd == b"MD": c.sendall(b"MD3;")
while True:
    c, _ = srv.accept(); threading.Thread(target=handle, args=(c,), daemon=True).start()
```

tools/mac-serial-bridge/test/port_client.py

new file, 14 lines

```
#!/usr/bin/env python3
# Bench test: open the bridge serial port like an application and send CAT commands.
# Usage: python3 port_client.py <port path> "FA;" ["ID;" ...]   (ECHO=1 turns echo on first)
import os, sys, termios, tty, time, select
path = os.path.expanduser(sys.argv[1]); cmds = sys.argv[2:]
fd = os.open(path, os.O_RDWR | os.O_NOCTTY)
if os.environ.get("ECHO"):   # simulate a sloppy app that turns echo on
    a = termios.tcgetattr(fd); a[3] |= termios.ECHO; termios.tcsetattr(fd, termios.TCSANOW, a)
for c in cmds:
    os.write(fd, c.encode()); r = b""; end = time.time() + 5
    while time.time() < end and not r.endswith(b";"):
        if select.select([fd], [], [], 0.2)[0]: r += os.read(fd, 256)
    print(c, "->", r.decode() or "(none)")
os.close(fd)
```

tools/mac-build-flash.sh

new file, 110 lines

```
#!/bin/bash
# Build this firmware on a Mac and flash it onto a Tab5 over USB.
#
#   bash ~/src/qmx-panadapter-v1.16/tools/mac-build-flash.sh          build + flash
#   bash ~/src/qmx-panadapter-v1.16/tools/mac-build-flash.sh --build  build only
#
# Uses the ESP-IDF in $IDF_PATH or ~/esp/esp-idf, and an installed PowerShell
# (pwsh, or set PWSH=/path/to/pwsh), because the project's standing patches are
# PowerShell scripts (tools/patches/apply_*.ps1); tools/check_patches.py refuses
# the build if any patch is missing. Flashing is a NORMAL flash: settings,
# WiFi, memories and the QSO log are kept. Everything is logged to
# build/mac-build-flash.log.

REPO="$(cd "$(dirname "$0")/.." && pwd)"
mkdir -p "$REPO/build"
LOG="$REPO/build/mac-build-flash.log"
exec > >(tee "$LOG") 2>&1
step() { echo; echo "=== $*"; }
fail() { echo; echo "FAILED: $*"; echo "(log: $LOG)"; exit 1; }
fwver() { sed -n 's/.*"project_version": *"\([^"]*\)".*/\1/p' "$REPO/build/project_description.json" | head -1; }
BUILD_ONLY=0; [ "$1" = "--build" ] && BUILD_ONLY=1

step "Mac and project"
date; sw_vers 2>/dev/null | tr '\n' ' '; echo; uname -m
cd "$REPO" || fail "no project folder"
echo "project: $REPO"
echo "branch:  $(git rev-parse --abbrev-ref HEAD)   version: $(git describe --tags --dirty)"

step "PowerShell (for the standing patches)"
# Use the PowerShell that's already installed. Look on PATH, in the usual install
# places, and in your login shell's PATH (Terminal's zsh may set it up there).
PWSH="${PWSH:-}"
for c in pwsh pwsh-preview /usr/local/bin/pwsh /opt/homebrew/bin/pwsh /usr/local/bin/pwsh-preview \
         /opt/homebrew/bin/pwsh-preview /usr/local/microsoft/powershell/7/pwsh \
         /usr/local/microsoft/powershell/7-preview/pwsh "$HOME/.dotnet/tools/pwsh" \
         /Applications/PowerShell.app/Contents/MacOS/pwsh; do
  [ -n "$PWSH" ] && break
  command -v "$c" >/dev/null 2>&1 && PWSH="$(command -v "$c")"
done
[ -n "$PWSH" ] || PWSH="$(zsh -lic 'command -v pwsh || command -v pwsh-preview' 2>/dev/null | tail -1)"
```

```
[ -n "$PWSH" ] && [ -x "$PWSH" ] || fail "can't find PowerShell (pwsh). In a PowerShell window run  (Get-Process -Id \$PID).Path  an
  ↪ d re-run this as:  PWSH=<that path> bash $0"
echo "PowerShell: $PWSH"
"$PWSH" -NoProfile -Command '$PSVersionTable.PSVersion.ToString()' || fail "pwsh does not run"

step "ESP-IDF"
export IDF_PATH="${IDF_PATH:-$HOME/esp/esp-idf}"
[ -f "$IDF_PATH/export.sh" ] || fail "ESP-IDF not found at $IDF_PATH"
echo "IDF tree: $(git -C "$IDF_PATH" describe --tags --dirty 2>/dev/null)"
# PlatformIO puts its own python3 first on PATH; ESP-IDF must not pick that one.
PATH="$(echo "$PATH" | tr ':' '\n' | grep -v '/.platformio/penv' | paste -sd: -)"
# Use the ESP-IDF Python environment that already exists for this IDF (made by
# install.sh with whichever Python was current then), and that Python itself.
if [ -z "$IDF_PYTHON_ENV_PATH" ]; then
  IDF_PYTHON_ENV_PATH="$(ls -d "$HOME/.espressif/python_env/idf5.4_py"*_env 2>/dev/null | sort -V | tail -1)"
fi
[ -x "$IDF_PYTHON_ENV_PATH/bin/python" ] || fail "no ESP-IDF Python environment in ~/.espressif/python_env - run $IDF_PATH/install.s
  ↪ h esp32p4"
export IDF_PYTHON_ENV_PATH
ENVPY="$(cd "$IDF_PYTHON_ENV_PATH/bin" && pwd -P)/python"
BASEPY="$(readlink -f "$IDF_PYTHON_ENV_PATH/bin/python" 2>/dev/null)"
[ -x "$BASEPY" ] && "$BASEPY" -V >/dev/null 2>&1 || fail "the Python behind $IDF_PYTHON_ENV_PATH is gone ($BASEPY) - run $IDF_PATH/i
  ↪ nstall.sh esp32p4 once to rebuild it"
mkdir -p build/pyshim && ln -sf "$BASEPY" build/pyshim/python3 && PATH="$REPO/build/pyshim:$PATH"
echo "ESP-IDF Python env: $IDF_PYTHON_ENV_PATH"
echo "python3 for export: $(command -v python3) ($(python3 --version 2>&1))"
. "$IDF_PATH/export.sh" > build/idf-export.log 2>&1
RC=$?
if [ $RC -ne 0 ] || ! command -v idf.py >/dev/null 2>&1; then
  echo "--- ESP-IDF export.sh output (exit code $RC):"; cat build/idf-export.log
  fail "ESP-IDF export.sh did not set up idf.py"
fi
tail -3 build/idf-export.log
idf.py --version
case "$(idf.py --version)" in *v5.4*) ;; *) echo "WARNING: the project and its patches are written for ESP-IDF v5.4.4";; esac

step "Fetch components (idf.py reconfigure)"
idf.py reconfigure > build/reconfigure.log 2>&1 || { tail -40 build/reconfigure.log; fail "idf.py reconfigure"; }
tail -3 build/reconfigure.log
ls managed_components

step "Apply the standing patches"
for s in tools/patches/apply_*.ps1; do
  echo "--- $(basename "$s")"
  "$PWSH" -NoProfile -ExecutionPolicy Bypass -File "$s" || fail "$(basename "$s")"
done
step "Check the standing patches"
python tools/check_patches.py || fail "check_patches.py"

step "Build"
idf.py build > build/build.log 2>&1 || { grep -E "error|Error|FAILED" build/build.log | head -40; tail -30 build/build.log; fail "id
  ↪ f.py build"; }
tail -8 build/build.log
echo "Firmware version: $(fwver)"
ls -l build/*.bin
[ "$BUILD_ONLY" = 1 ] && { step "Done (build only)"; exit 0; }

step "Flash the Tab5"
PORTS=( $(ls /dev/cu.usbmodem* 2>/dev/null) )
echo "USB serial ports: ${PORTS[*]:-none}"
[ ${#PORTS[@]} -gt 0 ] || fail "no Tab5 found - plug it in with a USB-C DATA cable and run this again"
PORT="${PORTS[0]}"
if [ ${#PORTS[@]} -gt 1 ]; then
  echo "More than one USB serial device is plugged in:"
  select PORT in "${PORTS[@]}"; do [ -n "$PORT" ] && break; done
fi
echo
echo "About to flash the Tab5 on $PORT (normal flash - settings are kept)."
echo "Nothing should be plugged into the Tab5's own USB port: a flash wedges an attached QMX."
read -r -p "Press Enter to flash, or Ctrl+C to cancel... "
idf.py -p "$PORT" -b 460800 flash || fail "flash (try another USB-C cable, or reboot the Tab5 and run again)"

step "SUCCESS"
echo "Flashed $(fwver). The Tab5 is restarting."
```

tools/mac-capture-serial.sh

new file, 32 lines

```
#!/bin/bash
# Record the Tab5's USB serial output (boot messages, crash reasons) for a while.
#   bash ~/src/qmx-panadapter-v1.16/tools/mac-capture-serial.sh [seconds]   (default 90)
# Output: build/serial-capture.log in this project. Uses ESP-IDF's Python (pyserial).
# Note: on macOS opening the port restarts the Tab5 (USB-serial-JTAG reset), so
# the capture always starts from a fresh boot. Setting DTR/RTS first does not help.
REPO="$(cd "$(dirname "$0")/.." && pwd)"; mkdir -p "$REPO/build"
OUT="$REPO/build/serial-capture.log"; SECS="${1:-90}"
PY="$(ls -d "$HOME/.espressif/python_env/idf5.4_py"*_env 2>/dev/null | sort -V | tail -1)/bin/python"
[ -x "$PY" ] || { echo "ESP-IDF Python not found"; exit 1; }
PORT="$(ls /dev/cu.usbmodem* 2>/dev/null | head -1)"
[ -n "$PORT" ] || { echo "No Tab5 USB serial port found - is it plugged in with a data cable?"; exit 1; }
echo "Recording $PORT for $SECS s into $OUT ..."
"$PY" - "$PORT" "$SECS" "$OUT" <<'PYEOF'
import sys, time, serial
port, secs, out = sys.argv[1], float(sys.argv[2]), sys.argv[3]
end = time.time() + secs
with open(out, "wb") as f:
    f.write(("# capture of %s started %s\n" % (port, time.ctime())).encode())
    while time.time() < end:
        try:
            s = serial.Serial(port, 115200, timeout=0.5)
            s.dtr = False; s.rts = False          # never reset the board ourselves
            while time.time() < end:
                d = s.read(4096)
                if d: f.write(d); f.flush(); sys.stdout.buffer.write(d); sys.stdout.flush()
        except (serial.SerialException, OSError) as e:
            # the port disappears while the board reboots - wait and reopen
            f.write(("\n# [port gone: %s] %s\n" % (e.__class__.__name__, time.ctime())).encode()); f.flush()
            time.sleep(0.3)
print("\nSaved:", out)
PYEOF
```

Transmit timing test

tools/tx-test/tx_compare.py

new file, 251 lines

```
#!/usr/bin/env python3
"""
tx_compare.py - compare transmit bursts as the Tab5 timed them with the same
bursts as the NAS relay handed them to the QMX.

    python3 tools/tx-test/tx_compare.py TAB5_CONSOLE.log RELAY_TXLOG.log [...]
    python3 tools/tx-test/tx_compare.py --usb TAB5_USB.log      (Tab5 only, e.g. a direct-USB baseline)

Inputs
  TAB5_CONSOLE.log  from tools/tx-test/capture-tab5-console.sh (each line
                    stamped with the Mac's clock)
  RELAY_TXLOG.log   txlog/txlog-<date>.log from the relay folder on the NAS

The rule this script is built on: every number it compares is an interval
whose two ends were read from ONE clock.
  Tab5 side  "key down at +N ms into the slot"  and "(N ms on-air)" - both
             measured by the firmware on the Tab5's own clock.
  Relay side "RF on-air", tone gaps and symbol timing - differences of the
             NAS's monotonic clock, around the actual USB writes to the QMX.
The Mac's and the NAS's wall clocks are used only to say which 15 s (FT8) or
7.5 s (FT4) slot a burst was in, so the same burst can be found in both logs.
They are never subtracted from each other. Two clocks only need to agree to
within a few seconds for that, and a burst that cannot be placed is listed as
unpaired rather than guessed.
"""
from __future__ import annotations

import argparse
import calendar
import re
import statistics
import sys
import time

ANSI = re.compile(r"\x1b\[[0-9;]*m")
STAMP = re.compile(r"^(\d{4}-\d\d-\d\d \d\d:\d\d:\d\d\.\d{3}) \| (.*)$")
KEY = re.compile(r"key down at \+(-?\d+) ms into the slot")
START = re.compile(r"TX burst starting \((FT4|FT8)\)")
DONE = re.compile(r"TX burst (\[SIM\] )?(complete|ABORTED): '(.*)' \((\d+) ms on-air\)")
R_HEAD = re.compile(r"^# burst (\d+)\s+slot (\S+)\s+\((.*)\)")
R_KEY = re.compile(r"^RELAY key down at \+(-?\d+) ms into the slot")
R_SUM = re.compile(r"^RELAY TX burst: (\d+) tones")

FT8_ON_AIR_MS = 79 * 160          # 12640: the burst length if every symbol is on time
def slot_key(t: float, period: float) -> int:
```

```
    """Index of the slot that started nearest to t (seconds since the epoch)."""
    return int(round(t / period))


ESPLINE = re.compile(r"^[IWEDV] \((\d+)\) [^:]+: (.*)$")
UTC_ANCHOR = re.compile(r"(?:SNTP sync: UTC|online; UTC) (\d{4}-\d\d-\d\d \d\d:\d\d:\d\d)")


def parse_tab5(path: str):
    """Two input forms:
      - Mac capture: "<Mac date time.ms> | <console line>"  (capture-tab5-console.sh)
      - the Tab5's own log over WiFi (web page log download, /api/log):
        "I (<ms since boot>) tag: text". Its slot is placed from the latest
        "SNTP sync: UTC ..." / "online; UTC ..." line on the Tab5 itself.
    Either way the wall time only picks the slot; the compared numbers are
    the firmware's own "key down at +N ms" and "(N ms on-air)"."""
    bursts, cur, proto = [], None, "FT8"
    anchor = None                       # (boot_ms, utc_epoch) from the Tab5's own log
    last_boot_ms = -1
    with open(path, "rb") as f:
        for raw in f:
            line = ANSI.sub("", raw.decode(errors="replace").rstrip("\r\n"))
            wall = None
            m = STAMP.match(line)
            if m:
                wall = time.mktime(time.strptime(m.group(1)[:19], "%Y-%m-%d %H:%M:%S")) + int(m.group(1)[20:]) / 1000
                line = m.group(2)
            e = ESPLINE.match(line)
            if e:
                boot_ms = int(e.group(1))
                if boot_ms < last_boot_ms:          # the Tab5 rebooted: old anchor is void
                    anchor, cur = None, None
                last_boot_ms = boot_ms
                if (u := UTC_ANCHOR.search(line)):
                    anchor = (boot_ms, calendar.timegm(time.strptime(u.group(1), "%Y-%m-%d %H:%M:%S")))
                if wall is None and anchor:
                    wall = anchor[1] + (boot_ms - anchor[0]) / 1000
            text = line
            if (s := START.search(text)):
                proto = s.group(1)
            elif (k := KEY.search(text)):
                cur = {"proto": proto, "key_ms": int(k.group(1)), "wall": wall}
            elif (d := DONE.search(text)) and cur:
                cur.update(sim=bool(d.group(1)), status=d.group(2), text=d.group(3), on_air=int(d.group(4)))
                period = 7.5 if cur["proto"] == "FT4" else 15.0
                cur["period"] = period
                if cur["wall"] is None:
                    cur["slot"] = None                # no clock anchor: cannot be placed
                else:
                    # the slot started key_ms before the key-down line was logged
                    cur["slot"] = slot_key(cur["wall"] - cur["key_ms"] / 1000, period)
                bursts.append(cur)
                cur = None
    return bursts


def parse_relay(paths):
    bursts, cur = [], None
    for path in paths:
        with open(path) as f:
            for line in f:
                line = line.rstrip("\n")
                if (h := R_HEAD.match(line)):
                    utc = h.group(2).rstrip("Z")
                    frac = 0.5 if utc.endswith(".5") else 0.0
                    t = calendar.timegm(time.strptime(utc[:19], "%Y-%m-%dT%H:%M:%S")) + frac
                    cur = {"slot_t": t, "status": h.group(3)}
                    bursts.append(cur)
                elif cur is not None and (k := R_KEY.match(line)):
                    cur["key_ms"] = int(k.group(1))
                elif cur is not None and (s := R_SUM.match(line)):
                    cur["tones"] = int(s.group(1))
                    for key, pat in (("rf_on_air", r"RF on-air (\d+) ms"), ("tx_rx", r"TX; -> RX; (\d+) ms"),
                                     ("gap_max", r"min/avg/max \d+/[\d.]+/(\d+) ms"),
                                     ("gaps_off", r"off by >20 ms: (\d+)"),
                                     ("late_min", r"vs schedule ([+-]\d+)\.\."), ("late_max", r"\.\.([+-]\d+) ms"),
                                     ("usb_max", r"USB write max ([\d.]+) ms")):
                        m = re.search(pat, line)
                        if m:
                            cur[key] = float(m.group(1))
                    m = re.search(r"I/Q to Tab5 (paused|streamed)", line)
                    cur["iq"] = m.group(1) if m else ""
                    m = re.search(r"tones timed by (\w+)", line)
                    cur["timed"] = m.group(1) if m else "Tab5"
    for b in bursts:
        period = 7.5 if 90 <= b.get("tones", 0) <= 110 else 15.0
        b["period"] = period
```

```
        b["slot"] = slot_key(b["slot_t"], period)
    return bursts


def fmt(v, w=6, plus=False):
    if v is None:
        return "-".rjust(w)
    return (("%+d" if plus else "%d") % round(v)).rjust(w)


def stats(name, vals):
    vals = [v for v in vals if v is not None]
    if not vals:
        return "  %-34s -" % name
    med = statistics.median(vals)
    return "  %-34s n=%-3d min %6.0f  median %6.0f  max %6.0f" % (name, len(vals), min(vals), med, max(vals))


def main():
    ap = argparse.ArgumentParser(description=__doc__.split("\n\n")[0])
    ap.add_argument("tab5", help="Tab5 console capture")
    ap.add_argument("relay", nargs="*", help="relay txlog file(s); none = Tab5-only summary")
    ap.add_argument("--usb", action="store_true", help="the Tab5 log is a direct-USB run (no relay expected)")
    a = ap.parse_args()

    t5 = [b for b in parse_tab5(a.tab5) if not b.get("sim")]
    rl = parse_relay(a.relay) if a.relay else []
    by_slot = {}
    for r in rl:
        by_slot.setdefault((r["period"], r["slot"]), []).append(r)

    print("Tab5 log : %s  (%d transmit bursts)" % (a.tab5, len(t5)))
    if a.relay:
        print("Relay log: %s  (%d bursts)" % (", ".join(a.relay), len(rl)))
    print()
    hdr = "%-20s %5s | %6s %7s | %6s %7s %7s %6s %6s %13s" % (
        "slot (local time)", "mode", "T5 key", "T5 on", "R key", "R RF on", "R TX-RX", "R gap", ">20ms", "R sym timing")
    print(hdr)
    print("-" * len(hdr))
    paired, used = [], set()
    for b in t5:
        rs = by_slot.get((b["period"], b["slot"]), [])
        r = rs[0] if len(rs) == 1 else None
        if r:
            used.add(id(r))
            paired.append((b, r))
        slot_txt = ("(no UTC in log)" if b["slot"] is None
                    else time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(b["slot"] * b["period"])))
        timing = "-"
        if r and r.get("late_min") is not None:
            timing = "%+d..%+d" % (r["late_min"], r["late_max"])
        if r and r.get("iq"):
            timing += "  IQ " + ("off" if r["iq"] == "paused" else "on")
        if r and r.get("timed") == "relay":
            timing += "  relay-timed"
        flag = "" if r or not a.relay else ("  (no relay burst in this slot)" if not rs else "  (2+ relay bursts in this slot)")
        if b["status"] != "complete":
            flag += "  ABORTED"
        print("%-20s %5s | %6s %7s | %6s %7s %7s %6s %6s %13s%s" % (
            slot_txt, b["proto"], fmt(b["key_ms"], plus=True), fmt(b["on_air"]),
            fmt(r and r.get("key_ms"), plus=True), fmt(r and r.get("rf_on_air")), fmt(r and r.get("tx_rx")),
            fmt(r and r.get("gap_max")), fmt(r and r.get("gaps_off")), timing, flag))
    for r in rl:
        if id(r) not in used:
            print("%-20s %5s | %6s %7s | %6s %7s %7s   relay burst with no Tab5 burst in this slot" % (
                time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(r["slot_t"])), "", "", "",
                fmt(r.get("key_ms"), plus=True), fmt(r.get("rf_on_air")), fmt(r.get("tx_rx"))))

    ft8 = [b for b in t5 if b["proto"] == "FT8" and b["status"] == "complete"]
    print()
    print("Columns: T5 = the Tab5's own numbers, R = the relay's (NAS clock). All in ms.")
    print("  key     key down, ms after the slot started (each on its own clock vs UTC via NTP)")
    print("  T5 on   Tab5 'on-air': key down -> RX; sent, including the post-burst power/SWR")
    print("          query, which waits for the radio's reply (one network round trip on NAS)")
    print("  R RF on what the QMX got: TX; -> key-up tone TA0; (FT8 ideal %d)" % FT8_ON_AIR_MS)
    print("  R gap   longest gap between two tone changes reaching the radio (FT8 ideal 160)")
    print("  >20ms   tone changes more than 20 ms off their 160 ms spacing")
    print("  R sym timing  earliest..latest symbol vs key down + i x 160 ms")
    print()
    print("Summary")
    print(stats("Tab5 key down (+ms into slot)", [b["key_ms"] for b in ft8]))
    print(stats("Tab5 on-air (ms)", [b["on_air"] for b in ft8]))
    pr = [(b, r) for b, r in paired if b["proto"] == "FT8"]
    if pr:
        print(stats("Relay RF on-air (ms)", [r.get("rf_on_air") for _, r in pr]))
        print(stats("Relay TX; -> RX; (ms)", [r.get("tx_rx") for _, r in pr]))
        print(stats("Relay longest tone gap (ms)", [r.get("gap_max") for _, r in pr]))
```

```
        print(stats("Relay latest symbol (ms late)", [r.get("late_max") for _, r in pr]))
        print(stats("Relay key down (+ms, NAS clock)", [r.get("key_ms") for _, r in pr]))
        bad = [r for _, r in pr if (r.get("gaps_off") or 0) > 0 or abs((r.get("rf_on_air") or FT8_ON_AIR_MS) - FT8_ON_AIR_MS) > 20]
        print()
        print("%d of %d paired FT8 bursts reached the radio with every tone within 20 ms of schedule"
              " and RF on-air within 20 ms of %d." % (len(pr) - len(bad), len(pr), FT8_ON_AIR_MS))
        groups = {}
        for b, r in pr:
            g = "tones timed by %-5s I/Q %s" % (r.get("timed", "Tab5"), r.get("iq") or "unknown")
            groups.setdefault(g, []).append((b, r))
        if len(groups) > 1 or any("paused" in g or "relay" in g for g in groups):
            print()
            print("By mode (relay flag files tab5-timed-tx and iq-during-tx):")
            for g, items in sorted(groups.items()):
                # A relay-timed burst's Tab5 on-air includes waiting for the
                # relay's report, so only the relay's own gap counts there.
                stalls = sum(1 for b, r in items if (r.get("gap_max") or 0) > 300 or
                             (r.get("timed") != "relay" and b["on_air"] > 12750))
                print("  %-30s %3d bursts  stalls: %d   median latest symbol %+.0f ms" % (
                      g, len(items), stalls, statistics.median([r.get("late_max") or 0 for _, r in items])))
            print("  (stall = a tone gap over 300 ms at the radio, or for Tab5-timed bursts a Tab5")
            print("   on-air time over 12750 ms)")
    elif a.relay:
        print("\nNo burst could be paired - check that both logs cover the same time and that the"
              " Mac and the NAS clocks are set by NTP.")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

tools/tx-test/capture-tab5-console.sh

new file, 53 lines

```
#!/bin/bash
# Record the Tab5's USB-serial console to a file for a whole test session,
# each line stamped with this Mac's clock. Stop with Ctrl-C.
#   bash tools/tx-test/capture-tab5-console.sh [name]
# Output: build/tab5-console-<name or date-time>.log in this project.
#
# The Mac timestamps are ONLY used by tx_compare.py to tell which FT8 slot a
# burst belongs to. Every measurement it compares comes from the firmware's
# own lines ("key down at +N ms into the slot", "(N ms on-air)"), which are
# timed on the Tab5's clock alone.
#
# On macOS, opening the port restarts the Tab5 (USB-serial-JTAG reset): start
# this FIRST, let the Tab5 boot and connect, then begin transmitting.
REPO="$(cd "$(dirname "$0")/../.." && pwd)"; mkdir -p "$REPO/build"
NAME="${1:-$(date '+%Y%m%d-%H%M%S')}"
OUT="$REPO/build/tab5-console-$NAME.log"
PY="$(ls -d "$HOME/.espressif/python_env/idf5.4_py"*_env 2>/dev/null | sort -V | tail -1)/bin/python"
[ -x "$PY" ] || PY="$(command -v python3)"
"$PY" -c "import serial" 2>/dev/null || { echo "pyserial missing: $PY -m pip install pyserial"; exit 1; }
PORT="$(ls /dev/cu.usbmodem* 2>/dev/null | head -1)"
[ -n "$PORT" ] || { echo "No Tab5 USB serial port found - is it plugged in with a data cable?"; exit 1; }
echo "Recording $PORT into $OUT - Ctrl-C to stop"
"$PY" - "$PORT" "$OUT" <<'PYEOF'
import sys, time, serial
port, out = sys.argv[1], sys.argv[2]
def stamp():
    t = time.time()
    return time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(t)) + ".%03d | " % int((t % 1) * 1000)
buf = b""
with open(out, "ab") as f:
    f.write(("# Tab5 console %s, capture started %s (Mac clock)\n" % (port, time.strftime("%Y-%m-%d %H:%M:%S %z"))).encode())
    try:
        while True:
            try:
                s = serial.Serial(port, 115200, timeout=0.2)
                s.dtr = False; s.rts = False          # never reset the board ourselves
                while True:
                    d = s.read(4096)
                    if not d:
                        continue
                    buf += d
                    while b"\n" in buf:
                        line, buf = buf.split(b"\n", 1)
                        rec = stamp().encode() + line.rstrip(b"\r") + b"\n"
                        f.write(rec); f.flush()
                        sys.stdout.buffer.write(rec); sys.stdout.flush()
            except (serial.SerialException, OSError) as e:
                f.write(("%s# [port gone: %s]\n" % (stamp(), e.__class__.__name__)).encode()); f.flush()
                time.sleep(0.3)
    except KeyboardInterrupt:
        pass
print("\nSaved:", out)
PYEOF
```

Update kit

tools/nas-kit/make-qmx-panadapter-nas.sh

update kit (in ~/src/qmx-panadapter-v1.16)

```
#!/bin/bash
# make-qmx-panadapter-nas.sh - build the NAS ("network radio") version of
# QMX-Panadapter from the upstream repository plus this kit's patch series.
#
#   bash make-qmx-panadapter-nas.sh                newest upstream release -> ~/src/QMX-Panadapter-NAS
#   bash make-qmx-panadapter-nas.sh --ref v1.18.0   a specific upstream tag (or branch/commit)
#   bash make-qmx-panadapter-nas.sh --force         rebuild even if NAS is already on that release
#   bash make-qmx-panadapter-nas.sh --mark-tested   after flashing and testing NAS on the hardware:
#                                                   copy it to ~/src/QMX-Panadapter-Shared
#   bash make-qmx-panadapter-nas.sh --status        what is in NAS and Shared
#
# The two folders:
#   ~/src/QMX-Panadapter-NAS     newest upstream release + the patches. Build, flash
#                                and test it from here (bash tools/mac-build-flash.sh).
#   ~/src/QMX-Panadapter-Shared  the last version that was built AND tested on the
#                                hardware. Only --mark-tested (or the very first run,
#                                see below) changes it, so a new upstream release
#                                that does not work never replaces a working one.
#
# Nothing is deleted. A NAS or Shared folder that is replaced is moved to
# ~/src/.qmx-panadapter-archive/<name>-<version>-<date>.
#
# The patches in ./patches are the NAS commits ported to upstream v1.18.0 on
# 2026-10-14, exported with
# `git format-patch --binary v1.18.0..<branch> -- . ':(exclude)tools/nas-kit'`.
# The earlier series are kept for reference: ./patches-v1.16.12 (tested on the
# hardware 2026-10-09) and ./patches-v1.16.11 (tested 2026-10-04). On a newer
# upstream release the patches are applied with git's 3-way merge; if upstream
# changed the same lines, the run stops, says which patch did not apply, and
# leaves NAS and Shared untouched.
#
# First run: if Shared does not exist yet and the release is TESTED_BASE (the
# one this patch series was tested on), the result is the tested code, so it
# is also copied to Shared and marked tested. The v1.18.0 series has not been
# hardware-tested yet, so TESTED_BASE is empty until it has.
#
# Needs: git, and network access to github.com. Works with macOS's bash 3.2.

set -u
UPSTREAM_URL="${UPSTREAM_URL:-https://github.com/SteffenLav/qmx-panadapter}"
SRC="${QMX_SRC_DIR:-$HOME/src}"
NAS="$SRC/QMX-Panadapter-NAS"
SHARED="$SRC/QMX-Panadapter-Shared"
ARCHIVE="$SRC/.qmx-panadapter-archive"
KIT="$(cd "$(dirname "$0")" && pwd)"
PATCHES="$KIT/patches"
TESTED_BASE=""                  # release this patch series was hardware-tested on ("" = not yet)
STAMP=".qmx-nas-build"          # in each tree: what it was built from
TESTED=".qmx-nas-tested"        # in each tree: when it was marked tested
BRANCH="network-radio"

say()  { printf '%s\n' "$*"; }
die()  { printf 'FAILED: %s\n' "$*" >&2; exit 1; }
now()  { date '+%Y-%m-%d %H:%M'; }
stampval() { [ -f "$1/$STAMP" ] && sed -n "s/^$2=//p" "$1/$STAMP" | head -1; }

patch_set_id() {   # one checksum over the whole patch series
    if command -v shasum >/dev/null 2>&1; then cat "$PATCHES"/*.patch | shasum -a 256 | cut -c1-12
    else cat "$PATCHES"/*.patch | sha256sum | cut -c1-12; fi
}
archive_move() {   # archive_move <dir> <label>   - move aside, never delete
    [ -d "$1" ] || return 0
    mkdir -p "$ARCHIVE" || die "cannot create $ARCHIVE"
    local base; base="$(stampval "$1" base)"; [ -n "$base" ] || base="unknown"
    local dest="$ARCHIVE/$2-$base-$(date '+%Y%m%d-%H%M%S')"
    mv "$1" "$dest" || die "cannot move $1 to $dest"
    say "  moved the previous $(basename "$1") to $dest"
}

hide_markers() {   # keep the marker files out of `git status`
    local ex="$1/.git/info/exclude"
    for f in "$STAMP" "$TESTED"; do
        grep -qx "/$f" "$ex" 2>/dev/null || printf '/%s\n' "$f" >> "$ex"
    done
}

latest_release() {   # newest vX.Y.Z tag upstream, compared numerically
    git ls-remote --tags --refs "$UPSTREAM_URL" 2>/dev/null \
      | sed -n 's#.*refs/tags/\(v[0-9][0-9]*\.[0-9][0-9]*\.[0-9][0-9]*\)$#\1#p' \
      | sed 's/^v//' | sort -t. -k1,1n -k2,2n -k3,3n | tail -1 | sed 's/^/v/'
}
```

```
copy_to_shared() {   # copy_to_shared <tree>
    archive_move "$SHARED" "Shared"
    cp -Rp "$1" "$SHARED" || die "cannot copy $1 to $SHARED"
    say "  copied to $SHARED"
}

status() {
    for d in "$NAS" "$SHARED"; do
        say "$(basename "$d"):"
        if [ ! -d "$d" ]; then say "  (not created yet)"; continue; fi
        say "  upstream release : $(stampval "$d" base)  ($(stampval "$d" upstream_commit))"
        say "  patch series     : $(stampval "$d" patches)  (kit now: $(patch_set_id))"
        say "  built            : $(stampval "$d" built)"
        if [ -f "$d/$TESTED" ]; then say "  tested           : $(cat "$d/$TESTED")"
        else say "  tested           : NO - flash it, test it, then run --mark-tested"; fi
    done
}

mark_tested() {
    [ -d "$NAS/.git" ] || die "$NAS does not exist - build it first"
    printf '%s (marked by %s)\n' "$(now)" "$(id -un)" > "$NAS/$TESTED"
    say "Marked $NAS as tested."
    copy_to_shared "$NAS"
    say "Shared now holds $(stampval "$NAS" base) + patch series $(stampval "$NAS" patches)."
}

build() {
    local ref="$1" force="$2"
    command -v git >/dev/null 2>&1 || die "git is not installed"
    ls "$PATCHES"/*.patch >/dev/null 2>&1 || die "no patches in $PATCHES"
    mkdir -p "$SRC" || die "cannot create $SRC"

    if [ -z "$ref" ]; then
        say "Asking $UPSTREAM_URL for its newest release ..."
        ref="$(latest_release)"
        [ -n "$ref" ] || die "could not read the release tags (network? URL?)"
    fi
    local pid; pid="$(patch_set_id)"
    say "Upstream release: $ref    patch series: $pid ($(ls "$PATCHES"/*.patch | wc -l | tr -d ' ') patches)"

    if [ "$force" = 0 ] && [ -d "$NAS" ] && [ "$(stampval "$NAS" base)" = "$ref" ] \
       && [ "$(stampval "$NAS" patches)" = "$pid" ]; then
        say "$NAS is already $ref with this patch series - nothing to do (--force rebuilds)."
        status; return 0
    fi

    local tmp="$SRC/.QMX-Panadapter-NAS.new"
    [ -e "$tmp" ] && archive_move "$tmp" "incomplete"
    say "Cloning upstream ..."
    git clone --quiet "$UPSTREAM_URL" "$tmp" || die "git clone failed"
    ( cd "$tmp" && git checkout --quiet -b "$BRANCH" "$ref" ) || die "upstream has no $ref"
    local commit; commit="$(cd "$tmp" && git rev-parse --short HEAD)"
    printf 'base=%s\n' "$ref" > "$tmp/$STAMP"     # so a failed attempt is archived by name

    say "Applying the NAS patches on $ref ($commit) ..."
    local name email
    name="$(git config --global user.name 2>/dev/null)";  [ -n "$name" ]  || name="QMX NAS kit"
    email="$(git config --global user.email 2>/dev/null)"; [ -n "$email" ] || email="qmx-nas-kit@localhost"
    if ! ( cd "$tmp" && git -c user.name="$name" -c user.email="$email" \
             am --quiet --3way --committer-date-is-author-date "$PATCHES"/*.patch ); then
        local bad; bad="$(cd "$tmp" && git am --show-current-patch 2>/dev/null | sed -n 's/^Subject: //p' | head -1)"
        ( cd "$tmp" && git am --abort >/dev/null 2>&1 )
        say ""
        say "The patch series does not apply cleanly to $ref."
        [ -n "$bad" ] && say "  First patch that failed: $bad"
        say "  Upstream changed the same code the NAS changes touch. $NAS and"
        say "  $SHARED were NOT changed. The failed attempt is kept for inspection."
        archive_move "$tmp" "failed"
        exit 2
    fi

    # Sanity: the files the NAS build depends on are there.
    local f
    for f in main/net/radio_relay.c tools/nas-relay/qmx_nas_relay.py tools/mac-build-flash.sh \
             tools/patches/apply_esp_hosted_sdio_split_psram.ps1 QMX-Panadapter-Shared.pdf \
             tools/mac-serial-bridge/qmx_serial_bridge.py tools/pi-relay/install-pi.sh; do
        [ -f "$tmp/$f" ] || die "applied, but $f is missing - not replacing $NAS"
    done

    {
        printf 'base=%s\n' "$ref"
        printf 'upstream_commit=%s\n' "$commit"
        printf 'upstream_url=%s\n' "$UPSTREAM_URL"
        printf 'patches=%s\n' "$pid"
        printf 'built=%s\n' "$(now)"
    } > "$tmp/$STAMP"
    hide_markers "$tmp"
```

```
    archive_move "$NAS" "NAS"
    mv "$tmp" "$NAS" || die "cannot move the new tree to $NAS"
    say "Created $NAS  ($ref + NAS patches, branch $BRANCH)"

    if [ ! -d "$SHARED" ] && [ -n "$TESTED_BASE" ] && [ "$ref" = "$TESTED_BASE" ]; then
        printf '%s (patch series hardware-tested on %s)\n' "$(now)" "$TESTED_BASE" > "$NAS/$TESTED"
        say "First run on the tested release $TESTED_BASE - this is the tested code:"
        copy_to_shared "$NAS"
    fi
    say ""
    say "Next: cd \"$NAS\" && bash tools/mac-build-flash.sh"
    say "      (power the Tab5 off and on after flashing; then test touch, NAS stream, Radio source)"
    if [ ! -f "$NAS/$TESTED" ]; then
        say "      When it works: bash \"$KIT/$(basename "$0")\" --mark-tested"
    fi
}

ref=""; force=0; action="build"
while [ $# -gt 0 ]; do
    case "$1" in
        --ref)          ref="${2:-}"; [ -n "$ref" ] || die "--ref needs a tag"; shift ;;
        --force)        force=1 ;;
        --mark-tested)  action="mark" ;;
        --status)       action="status" ;;
        -h|--help)      sed -n '2,36p' "$0"; exit 0 ;;
        *)              die "unknown option $1 (see --help)" ;;
    esac
    shift
done

case "$action" in
    build)  build "$ref" "$force" ;;
    mark)   mark_tested ;;
    status) status ;;
esac
```

tools/nas-kit/push-to-github.sh

update kit (in ~/src/qmx-panadapter-v1.16)

```
#!/bin/bash
# push-to-github.sh - publish ~/src/QMX-Panadapter-NAS and ~/src/QMX-Panadapter-Shared
# to https://github.com/jpwatters/QMX-PanadapterShared, side by side, with the
# NAS folder's README.md as the repository's front page.
#
#   bash ~/src/qmx-panadapter-v1.16/tools/nas-kit/push-to-github.sh
#
# What it does:
#   1. Clones the GitHub repository into ~/src/QMX-PanadapterShared (or updates
#      that clone if it is already there).
#   2. Replaces QMX-Panadapter-NAS/ and QMX-Panadapter-Shared/ in it with a
#      snapshot of each folder's committed files (`git archive HEAD`): no .git,
#      no build output, nothing uncommitted.
#   3. Copies QMX-Panadapter-NAS/README.md to the top as README.md, with its
#      relative links and images pointed into QMX-Panadapter-NAS/ so they work.
#   4. Commits (only if something changed) and pushes.
# Run it again after each new tested version; it updates the same repository.
#
# Sign-in: GitHub must accept a push from this Mac - e.g. run `gh auth login`
# once, or use an SSH key and REPO_URL=git@github.com:jpwatters/QMX-PanadapterShared.git
set -euo pipefail

SRC="${QMX_SRC_DIR:-$HOME/src}"
REPO_URL="${REPO_URL:-https://github.com/jpwatters/QMX-PanadapterShared.git}"
WORK="${WORK:-$SRC/QMX-PanadapterShared}"
BRANCH="${BRANCH:-main}"
FOLDERS="QMX-Panadapter-NAS QMX-Panadapter-Shared"

say() { printf '%s\n' "$*"; }
die() { printf 'FAILED: %s\n' "$*" >&2; exit 1; }

for f in $FOLDERS; do
    [ -d "$SRC/$f/.git" ] || die "$SRC/$f is not there (or is not a git folder)"
    if [ -n "$(git -C "$SRC/$f" status --porcelain --untracked-files=no)" ]; then
        die "$SRC/$f has uncommitted changes - commit them first (only committed files are published)"
    fi
done

# 1. the local clone of the GitHub repository
if [ ! -d "$WORK/.git" ]; then
    say "Cloning $REPO_URL into $WORK ..."
    git clone "$REPO_URL" "$WORK" || die "git clone failed - does the repository exist, and can this Mac sign in to GitHub?"
fi
cd "$WORK"
if git rev-parse --verify -q "origin/$BRANCH" >/dev/null 2>&1 || git ls-remote --exit-code --heads origin "$BRANCH" >/dev/null 2>&1;
  ↪  then
    git fetch -q origin || die "cannot reach $REPO_URL - check the network and that this Mac can sign in to GitHub"
    git checkout -q -B "$BRANCH" "origin/$BRANCH"
```

```
else
    git checkout -q -B "$BRANCH" 2>/dev/null || true    # empty repository: first push
fi

# 2. fresh snapshots of both folders
msg_lines=""
for f in $FOLDERS; do
    rm -rf "$WORK/$f"
    mkdir -p "$WORK/$f"
    git -C "$SRC/$f" archive HEAD | tar -x -C "$WORK/$f"
    base="$(sed -n 's/^base=//p' "$SRC/$f/.qmx-nas-build" 2>/dev/null | head -1)"
    tested="$(head -1 "$SRC/$f/.qmx-nas-tested" 2>/dev/null | cut -c1-16)"
    head="$(git -C "$SRC/$f" log -1 --format='%h %s' | cut -c1-72)"
    msg_lines="$msg_lines
$f: upstream ${base:-?}, commit $head${tested:+, tested $tested}"
    say "  $f: snapshot of ${head%% *} (${base:-?})"
done

# 3. front page = the NAS folder's README, links pointed into its folder
python3 - "$WORK/QMX-Panadapter-NAS/README.md" "$WORK/README.md" <<'PY'
import re, sys
src, dst = sys.argv[1], sys.argv[2]
text = open(src, encoding="utf-8").read()
def fix(m):
    lead, url = m.group(1), m.group(2)
    if re.match(r"^([a-z][a-z0-9+.-]*:|#|/)", url, re.I):
        return m.group(0)                  # web links, anchors, absolute paths stay
    return f"{lead}QMX-Panadapter-NAS/{url}"
text = re.sub(r"(\]\()\s*([^)\s]+)", fix, text)                       # [text](path) and ![img](path)
text = re.sub(r'((?:src|href)=")([^"]+)', fix, text)                  # <img src="..."> / <a href="...">
open(dst, "w", encoding="utf-8").write(text)
PY
[ -f "$WORK/.gitignore" ] || printf '.DS_Store\n' > "$WORK/.gitignore"

# 4. commit if anything changed, then push. --force: upstream tracks a few files
# its own .gitignore excludes (scratchpad/*.ps1, test/*.exe); the snapshot only
# ever holds committed files, so forcing adds exactly those and nothing else.
git add -A --force
if git diff --cached --quiet; then
    say "Nothing changed since the last push."
else
    git commit -q -m "Update QMX-Panadapter-NAS and QMX-Panadapter-Shared$(printf '\n%s' "$msg_lines")"
    say "Committed: $(git log -1 --format='%h')"
fi
say "Pushing to $REPO_URL ($BRANCH) ..."
if ! git push -u origin "$BRANCH"; then
    say ""
    say "The push failed. Two usual causes:"
    say "  - not signed in to GitHub: run 'gh auth login' (or use an SSH key), then run this again;"
    say "  - the upload timed out (HTTP 408 / 'Operation timed out' / 'remote end hung up'):"
    say "    a large first upload on a slow or unstable link. Push it in smaller pieces -"
    say "    see 'Publishing to GitHub' in $(dirname "$0")/README.md - then use this script again."
    die "git push failed"
fi
say "Done: https://github.com/jpwatters/QMX-PanadapterShared"
```